

OpenStitch Studio

Numérisation open source pour broderie machine

Documentation utilisateur et technique

Version du logiciel : 0.1.0 · Version de la documentation : 1.0

Générée le 2026-08-17

Licence : Apache-2.0

Nom temporaire du projet (remplaçable, ADR-001)

Informations éditoriales

Élément	Valeur
Statut du document	Version initiale, générée automatiquement à partir des sources
Version de la documentation	1.0
Version du logiciel documentée	0.1.0 (constante <code>kAppVersion</code> , <code>libs/core/include/openstitch/core/app_info.hpp</code>)
Date de génération	2026-08-17
Licence du logiciel	Apache-2.0 (LICENSE)
Licence de la documentation	Apache-2.0 (même dépôt)
Dépôt	Dépôt Git local ; aucun remote public déclaré à ce jour

Historique des révisions

Version	Date	Changements
1.0	2026-08-17	Première édition complète : audit du dépôt, guides utilisateur et technique, référence des modules, format DST, format projet.

Signaler une erreur

La documentation est générée depuis les fichiers Markdown de `docs/source/`. Pour corriger une erreur, modifiez le fichier source concerné puis régénérez le PDF (voir *Système de génération de la documentation* dans `docs/README.md`). Les affirmations techniques sont tracées vers le code réel dans chaque section « Implémentation associée ».

Table des matières

Les titres sont cliquables et les numéros de page indiqués. Les lecteurs PDF affichent aussi les signets (panneau latéral) pour une navigation détaillée par section.

4	À propos de ce document
6	Introduction et présentation générale
8	Guide de prise en main
10	Installation et premier démarrage
12	Guide utilisateur détaillé
18	Traitement d'image
20	Segmentation
22	Vectorisation
24	Objets de broderie
26	Génération de points — point droit et fondations
29	Moteur de génération de points
40	Colonne satin
105	Remplissage tatami
109	Retouche des points et de la géométrie
112	Palettes et fils
113	Simulation de couture
114	Analyse et validation
116	Format DST (Tajima)
	Format de projet `.osp`
121	Architecture logicielle
123	Référence des modules
125	Modèle de données
127	Algorithmes
129	Compilation et développement
131	Guide du développeur
132	Tests
142	Guide de contribution
143	Dépannage
145	Limitations et état des fonctionnalités
150	Roadmap
152	Glossaire
153	Licences
155	Annexe — Audit du dépôt

Partie I

Présentation et manuel utilisateur

À propos de ce document

Cette documentation décrit **OpenStitch Studio**, une application de bureau libre de numérisation pour broderie machine. Elle couvre l'usage du logiciel, les concepts de broderie qu'il met en œuvre, son architecture logicielle, ses formats de fichiers et sa compilation.

Le document s'adresse à quatre publics : l'**utilisateur débutant**, l'**utilisateur avancé en broderie**, le **développeur contributeur** et le **mainteneur**. Chaque chapitre indique clairement à qui il s'adresse en priorité.

Principe de véracité

Les descriptions de fonctionnalités ont été **rapprochées du code et des tests** présents dans le dépôt (lire le code confirme qu'une fonction existe et ce qu'elle fait, mais ne garantit pas que son résultat est de qualité — d'où la distinction des niveaux de validation, voir *Limitations*). Lorsqu'une capacité était prévue mais n'est pas encore disponible, elle est marquée explicitement :

- **Implémenté** : disponible et testé ;
- **Partiellement implémenté** : présent mais incomplet ;
- **Expérimental** : présent, non stabilisé ;
- **Prévu** : conçu dans l'architecture, non implémenté ;
- **Non implémenté** : absent.

L'annexe *Audit du dépôt* recense l'inventaire factuel qui sert de base à ce document. Chaque chapitre technique se termine par une section **Implémentation associée** listant les fichiers, classes, fonctions et tests réels.

Comment lire ce document

- Vous voulez **utiliser** le logiciel : lisez *Introduction*, *Installation*, *Guide de prise en main*, puis *Guide utilisateur détaillé*.

- Vous voulez **comprendre la broderie** telle qu'implémentée : lisez *Concepts* (points, satin, tatami) et *Pipeline image vers broderie*.
- Vous voulez **contribuer** : lisez *Architecture*, *Référence des modules*, *Modèle de données*, *Algorithmes*, *Compilation et développement*, *Tests*.
- Vous voulez **maintenir** : ajoutez *Limitations et roadmap*, *Licences* et le *Guide de contribution*.

Note : Ce document est généré automatiquement à partir des fichiers Markdown de `docs/source/`. Ne le modifiez pas directement dans le PDF.

Introduction et présentation générale

Vision du projet

OpenStitch Studio vise à offrir, **gratuitement et en logiciel libre**, la chaîne de travail qui va d'une **image matricielle** jusqu'à un **fichier de broderie machine** au format DST (Tajima). Le projet est écrit principalement en C++ moderne, fonctionne localement (sans compte, sans cloud, sans télémétrie) et sépare strictement son cœur métier de son interface graphique.

Le nom « OpenStitch Studio » est temporaire et remplaçable (décision ADR-001) : il n'apparaît en dur qu'à un seul endroit du code (`kAppName`).

Cas d'usage

- Transformer un logo ou une illustration simple en motif brodable.
- Segmenter une image en zones de couleur et les convertir en objets de broderie éditables (contours, remplissages, colonnes satin).
- Générer, prévisualiser et analyser les points avant de produire un fichier DST.
- Relire un fichier DST existant et l'inspecter.

La chaîne de travail en un coup d'œil

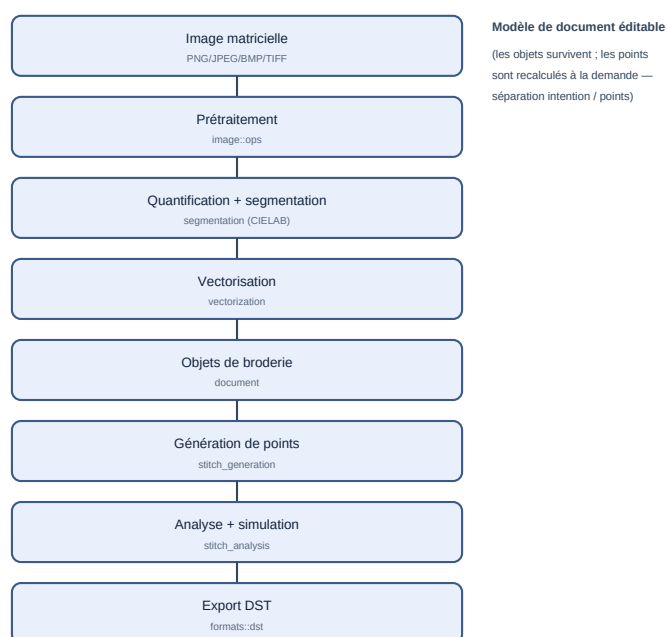


Figure 1 — Le pipeline image → broderie et les modules responsables.

L'idée centrale est la **séparation entre l'intention et le résultat** : la géométrie éditable (objets vectoriels et de broderie) est conservée dans un modèle de document ; les **points** sont recalculés à la demande à partir de cette géométrie. Modifier un paramètre régénère les points sans détruire l'objet source.

Quatre représentations distinctes

Il est essentiel de distinguer quatre choses que le logiciel manipule :

Représentation	Ce que c'est	Type/Module
Image	pixels RGBA	image::Image
Région	zone de couleur connue issue de la segmentation	segmentation::Region
Objet vectoriel	contours (ordinaire + trous), latérales	document::VectorObject
Objet de broderie	intention de couture (type de point + paramètres)	document::EmbroideryObject
Séquence de points	commandes machine (stitchJump, ...)	stitch::StitchSequence

Un DST, lui, ne contient **que** la séquence de points : il ne conserve **pas** les objets éditables (voir le chapitre *Format DST*).

Formats

- **Entrée image** : PNG, JPEG, BMP, TIFF.
- **Broderie** : DST en import et export.
- **Diagnostic** : SVG en export.
- **Projet** : `.osp` (archive contenant tout le document éditable).

Limites générales

Le logiciel constitue un **prototype fonctionnel** couvrant l'ensemble du pipeline principal. Ses composants sont **testés logiciellement**, mais la **qualité de broderie produite n'a pas encore été validée sur une machine à broder réelle**. Les heuristiques physiques (compensation de tirage, densité) sont paramétrables mais approximatives, le satin est **expérimental** et le tatami **partiel** (voir *Limitations et roadmap*). Certaines conventions DST varient aussi selon les machines.

Limitation : Certaines fonctionnalités décrites dans l'étude de cadrage initiale (palette de fils réelle, remplissages courbes, profils machine avancés, édition manuelle des points) sont **prévues mais non implémentées**. Elles sont signalées comme telles tout au long du document.

Philosophie open source

Le projet est sous licence **Apache-2.0** (permissive avec clause de brevets). Pour conserver cette distribution, aucun code sous licence copyleft incompatible (par exemple la GPL d'Ink/Stitch) ne doit être intégré directement — cela imposerait vraisemblablement une redistribution sous licence copyleft. Les idées algorithmiques restent réimplémentables indépendamment à partir de sources publiques. Voir *Licences* pour la formulation complète et les précautions.

Implémentation associée

- `libs/core/include/openstitch/core/app_info.hpp` — nom et version.
- `docs/phase0/` — étude de cadrage et 14 décisions d'architecture (ADR).
- `README.md` — résumé du projet (établi à un jalon antérieur).

Guide de prise en main

Public : utilisateur débutant. Ce tutoriel suit un flux réaliste, **adapté à ce qui est réellement implémenté**. Les étapes non disponibles dans l'interface sont signalées.

Vue d'ensemble

Le flux typique est : importer une image → la préparer → la segmenter en régions → vectoriser → créer des objets de broderie (ou laisser l'auto-numérisation le faire) → régler les points → organiser l'ordre → analyser → exporter en DST.

1. Ouvrir une image

Fichier → **Ouvrir une image...** (Ctrl+O). Choisissez un PNG, JPEG, BMP ou TIFF. Un logo simple à quelques couleurs franches donne les meilleurs résultats.

2. Choisir les dimensions physiques

Un dialogue d'import demande la **taille physique** en millimètres (largeur et hauteur), avec l'option « conserver les proportions ». La valeur par défaut suppose 96 dpi. C'est ici que se fixe la résolution de travail (mm par pixel) — elle ne changera plus ensuite.

Conseil : visez une taille qui tient dans le cadre affiché (100 × 100 mm par défaut) ; sinon le motif dépassera du tambour.

3. Préparer l'image

Menu **Image** : niveaux de gris, luminosité/contraste (avec aperçu), débruitage, symétries, rotations 90°, recadrage (par sélection au rectangle) et quantification des couleurs. Toutes ces opérations sont **non destructives** : l'original est conservé, chaque opération s'annule (Ctrl+Z).

4. Réduire les couleurs et segmenter

Segmentation → **Segmenter l'image...** : choisissez le nombre maximal de couleurs et la taille minimale de région. Le logiciel quantifie en espace perceptuel CIELAB puis extrait les **régions connexes**. Activez **Afficher la carte des régions** pour les visualiser.

5. Nettoyer les régions

Cliquez une région pour la sélectionner (ses infos s'affichent dans la barre d'état). Vous pouvez la **supprimer** (elle redevient du fond), la **recolorer**, ou en **fusionner** deux (« Fusionner avec... » puis clic sur la cible).

6. Vectoriser

Avec une région sélectionnée : **Segmentation** → **Convertir la région en objet vectoriel**. Le contour (et ses trous) devient un objet éditable ; ses **nœuds** apparaissent et se déplacent à la souris.

7. Créer des objets de broderie

Sur un objet vectoriel sélectionné, menu **Broderie** :

- **Créer un objet de point de contour...** (point simple, double ou triple) ;
- **Créer un remplissage tatami...** (densité, longueur, angle) ;
- **Créer une colonne satin...** (densité, compensation, sous-couche) — avec un avertissement si la colonne est trop large.

Alternative rapide : **Broderie** → **Numérisation automatique** crée des objets pour toutes les régions : **satin topologique** pour les bandes fines compatibles, **tatami** si le moteur satin refuse la forme, **contour** cousu

pour les petites. Le satin automatique naïf qui débordait reste désactivé. On change ensuite le type d'une forme par **clic droit ■ Type de points**, et on règle l'orientation d'un tatami à la souris (poignée de rotation).

Pour une colonne satin sélectionnée, **Broderie ■ Éditer les guides satin...** (G) affiche les barreaux d'orientation. Glissez une extrémité le long de son rail pour infléchir localement les points ; plusieurs barreaux pilotent des orientations successives. Cliquez sur un barreau pour le sélectionner, utilisez **Ajouter un guide satin** (Maj+G) pour partager le plus grand intervalle ou **Supprimer le guide satin sélectionné** pour l'enlever. Le logiciel conserve toujours au moins deux guides et chaque geste est annulable avec `ctrl+z`.

8. Régler les points et les couleurs

Les points se régénèrent automatiquement à chaque changement. La couleur d'un objet reprend celle de sa région.

Limitation : il n'existe pas encore de **palette de fils** réelle ni de sélection de fil par référence fabricant (voir *Palettes et fils*).

9. Organiser l'ordre de couture

Le panneau **Ordre de couture** liste les objets ; vous pouvez les monter/descendre, les verrouiller, et appliquer une stratégie automatique (par couleur, par proximité) avec une estimation de coût.

10. Simuler et analyser

La **barre de simulation** rejoue la couture point par point (lecture/pause, curseur). **Analyse → Analyser le motif** (F5) liste les problèmes détectés (points trop courts/longs, sauts trop longs, hors cadre) ; un double-clic centre la vue sur le problème.

11. Exporter en DST

Fichier → Exporter en DST... Le logiciel rappelle qu'un DST **ne conserve pas** les objets éditables : gardez aussi votre projet `.osp`.

12. Vérifier le fichier

En ligne de commande : `openstitch-cli stats motif.dst` affiche points, sauts, dimensions et longueur de fil estimée ; `openstitch-cli dst2svg motif.dst apercu.svg` produit un aperçu vectoriel.

Implémentation associée

- `apps/desktop/main_window.cpp` — tous les menus et actions ci-dessus.
- `apps/desktop/import_dialog.cpp` — dialogue de taille physique.
- `libs/autodigitize/src/autodigitize.cpp` — numérisation automatique.
- `apps/cli/main.cpp` — `stats`, `dst2svg`.

Installation et premier démarrage

Public : utilisateur débutant, développeur.

À ce jour, **aucun binaire pré-compilé n'est distribué** : l'installation se fait par **compilation depuis les sources**. Cette section décrit la mise en place minimale ; la compilation détaillée est traitée dans *Compilation et développement*.

Systèmes supportés

Système	Statut
Windows 10 / 11 (x64)	Cible principale (application complète)
Linux	Cœur + CLI uniquement (garde-fou de portabilité, vérifié en CI) — pas l'interface Qt à ce stade
macOS	Prévu, non vérifié

Prérequis (Windows)

Outil	Version	Rôle
Visual Studio 2022 ou plus récent (charge « Développement Desktop C++ »)	MSVC v143+	Compilateur
CMake	≥ 3.27	Configuration du build
Git	récent	Récupération du code et de vcpkg
vcpkg	bootstrappé	Dépendances (OpenCV, Clipper2, ...)
Qt	6.8 LTS, binaires msvc2022_64	Interface graphique
Espace disque	~10 Go	vcpkg compile OpenCV au premier configure

Mise en place

1. Installer vcpkg et définir VCPKG_ROOT :

```
git clone https://github.com/microsoft/vcpkg.git C:\dev\vcpkg
C:\dev\vcpkg\bootstrap-vcpkg.bat -disableMetrics
setx VCPKG_ROOT C:\dev\vcpkg
```

2. Installer Qt 6.8 (sans compte, via aqtinstall) et définir QT_ROOT :

```
python -m pip install --user aqtinstall
python -m aqt install-qt windows desktop 6.8.3 win64_msvc2022_64 -O C:\Qt -m qtimageformats
setx QT_ROOT C:\Qt\6.8.3\msvc2022_64
```

3. Configurer, compiler, tester (nouveau terminal pour prendre les variables) :

```
cmake --preset msvc
cmake --build --preset msvc-debug
ctest --preset msvc-debug
```

Les exécutables produits :

- `build\msvc\apps\desktop\Debug\openstitch.exe` — application graphique ;
- `build\msvc\apps\cli\Debug\openstitch-cli.exe` — outil en ligne de commande.

Démarrage

Lancez `openstitch.exe`. La fenêtre principale s'ouvre sur un **canevas** vide, avec des règles graduées en millimètres et un **cadre** de broderie de 100 × 100 mm par défaut. Utilisez **Fichier** → **Ouvrir une image...** pour commencer.

Emplacement des données, configuration, désinstallation

- **Données** : le logiciel ne stocke rien en dehors des fichiers projet `.osp` et des fichiers exportés que vous choisissez. *Information non déterminée dans le dépôt* : aucun répertoire de configuration utilisateur n'est créé (pas de persistance de préférences repérée dans le code).
- **Configuration** : les variables d'environnement `VCPKG_ROOT` et `QT_ROOT` concernent la **compilation**, pas l'exécution.
- **Désinstallation** : supprimez le dossier de build et, le cas échéant, `vcpkg` et `Qt`. Aucune entrée de registre n'est écrite par l'application.

Vérification de l'installation

- `ctest --preset msvc-debug` doit rapporter **100 % des tests réussis**.
- `openstitch-cli.exe --version` affiche la version.
- `openstitch-cli.exe info une-image.png` affiche les métadonnées d'une image.

Avertissement : au **premier** `cmake --preset msvc`, `vcpkg` compile OpenCV et d'autres dépendances, ce qui peut prendre 10 à 30 minutes. Les compilations suivantes réutilisent le cache binaire de `vcpkg`.

Implémentation associée

- `docs/build-windows.md` — guide de compilation d'origine.
- `CMakePresets.json` — presets `msvc` et `linux-core`.
- `vcpkg.json` — dépendances verrouillées par baseline.
- `apps/cli/main.cpp` — sous-commande `info`, drapeau `--version`.

Guide utilisateur détaillé

Public : utilisateur débutant et avancé. Ce chapitre documente chaque menu, outil et raccourci **réellement présents** dans `apps/desktop/`.

Disposition générale

L'interface place le **canevas au centre** et l'entoure de zones fonctionnelles :

```
Menus
Barre d'outils principale
Barre contextuelle (suit la sélection)
[Palette d'outils] [ CANEVAS ] [ Inspecteur ]
[Document / Ordre] [ Workflow ]
Barre de simulation (zone réservée)
Barre d'état
```

Tous les panneaux sont des **docks** redimensionnables, déplaçables et masquables (**Ctrl+Shift+P** = mode canevas). Leur disposition, la géométrie de la fenêtre, le **thème** et la **densité** sont mémorisés d'une session à l'autre (préférences d'interface, distinctes du projet `.osp`).

État d'accueil

Tant qu'aucun document n'est ouvert, le centre du canevas propose **Ouvrir une image**, **Ouvrir un projet** et **Importer un DST**, avec une courte explication.

Le canevas

La zone centrale est un canevas dont l'unité est le **millimètre** (origine au centre, Y vers le haut). Des **règles** graduées en mm bordent le haut et la gauche ; une **grille** adaptative et le **cadre** de broderie (rectangle rouge, défini par l'utilisateur, cf. *Taille du cadre*) sont dessinés. La position du curseur en mm apparaît dans la barre d'état.

Le **mode d'interaction** vient de la palette d'outils (à gauche) :

- **Sélection** (V) : sélectionner une région/un objet ; le glisser déplace la vue.
- **Déplacer la vue** (H) : déplacement pur (le clic ne sélectionne pas).
- **Rectangle / Recadrage** (M) : sélection rectangulaire pour recadrer l'image.
- **Zoom** : molette (ancrée sous le curseur) ou barre d'outils / menu Affichage.
- **Échap** : revient à la Sélection et annule le mode fusion en cours.

La sélection d'un objet est tracée en **double contraste** (halo clair + trait d'accent), lisible sur tout fond. Le rendu est organisé en deux couches (image/vecteurs/régions d'une part, points d'autre part) pour rester fluide sur de gros motifs.

Menu Fichier

Action	Raccourci	Effet
Ouvrir une image...	Ctrl+O	Charge PNG/JPEG/BMP/TIFF puis demande la taille physique
Enregistrer le projet...	Ctrl+S	Écrit un fichier <code>.osp</code> (tout le document)
Ouvrir un projet...	—	Recharge un <code>.osp</code>
Exporter en DST...	—	Écrit un fichier <code>.dst</code> (points uniquement)
Importer un DST...	—	Relit un <code>.dst</code> comme séquence de points
Quitter	Ctrl+Q	Ferme l'application

Import : le dialogue affiche un aperçu, les dimensions en pixels, la résolution **mm/pixel** en direct, la taille du cadre, et **alerte si l'image dépasse le cadre**. **Export DST** : un **résumé** (dimensions, points, sauts, coupes, changements de couleur, fil estimé, cadre, dépassement éventuel) est présenté avant écriture, avec un rappel que le DST ne conserve pas les objets.

Le titre de la fenêtre affiche un indicateur **modifié (*)** ; quitter avec des modifications non enregistrées propose **Enregistrer / Ignorer / Annuler**.

Menu Édition

Action	Raccourci	Effet
Annuler	Ctrl+Z	Défait la dernière opération (le libellé nomme l'action)
Rétablir	Ctrl+Y	Refait l'opération annulée

L'annulation couvre les opérations d'image, la segmentation (segmenter, fusionner, supprimer, recolorer), la création et le déplacement de nœuds vectoriels, la création d'objets de broderie, le **changement de type**, l'**orientation** d'un remplissage, la conversion satin→tatami et le réordonnancement.

Menu Image

Toutes ces opérations sont non destructives (pile de transformations sur l'original) :

- Niveaux de gris ;
- Luminosité/contraste... (aperçu en direct) ;
- Débruitage léger / moyen (médian) ;
- Quantifier les couleurs... (k-means, nombre de couleurs 2–64) ;
- Symétrie horizontale / verticale ;
- Rotation 90° horaire / antihoraire ;
- Recadrer (sélection) — dessinez un rectangle sur le canevas.

Menu Segmentation

- Segmenter l'image... (nombre de couleurs, taille min de région) ;
- Afficher la carte des régions (basculer) ;
- Fusionner avec... (puis clic sur la région cible) ;
- Supprimer la région sélectionnée (Suppr) — la région redevient du fond ;
- Recolorer la région sélectionnée... ;
- Convertir la région en objet vectoriel.

Sélection : cliquez une région ; ses statistiques (pixels, mm², RGB) s'affichent.

Menu Broderie

- Numérisation automatique — crée des objets pour toutes les régions. Les zones remplissables deviennent des **tatami** (le satin automatique naïf, qui débordait, est désactivé par défaut) ;
- Créer un objet de point de contour... (longueur, type simple/double/triple) ;
- Créer un remplissage tatami... (espacement, longueur, angle) ;
- Créer une colonne satin... (densité, compensation, sous-couche centrale) ;
- **Orientation du remplissage...** — change l'angle des fils du tatami sélectionné (aussi réglable à la souris, voir *poignée de rotation* plus bas) ;
- **Convertir les satins auto en tatami** — répare un projet dont les satins automatiques débordent ;

- Statistiques... (points, sauts, coupes, changements de couleur, dimensions, longueur de fil).

Avertissement : si une colonne satin dépasse la largeur recommandée, un dialogue propose de continuer ou de préférer un remplissage tatami.

Menu Affichage

Le sous-menu **Calques** regroupe des interrupteurs indépendants :

Calque	Effet
Image	Affiche/masque l'image de travail
Carte des régions	Affiche/masque la segmentation
Vecteurs	Affiche/masque les objets vectoriels
Broderie (points)	Affiche/masque les points générés

Action	Raccourci	Effet
Zoom avant	Ctrl++	Agrandit
Zoom arrière	Ctrl+-	Réduit
Ajuster au canevas	Ctrl+0 ou F	Cadre la vue sur le canevas
Taille du cadre...	—	Définit la zone physique de broderie (voir ci-dessous)
Thème	—	Clair / Sombre
Densité	—	Confortable / Compact
Masquer les panneaux	Ctrl+Shift+P	Mode canevas (masque puis restaure les docks)

Taille du cadre : largeur/hauteur en mm (10–500). Le cadre est une donnée du projet (persistée dans le .osp, annulable) ; l'analyse « hors cadre » et le rectangle rouge s'y réfèrent. Accessible aussi via le bouton « **Cadre : W×H mm...** » de la barre contextuelle quand rien n'est sélectionné.

Les points cousus sont tracés **dans la couleur de fil de chaque objet** (un fil très clair est légèrement assombri pour rester visible) ; les **sauts** en pointillés orange ; des pastilles marquent les pénétrations (masquées au-delà de 4000 points pour la fluidité, et pendant la simulation).

Menu contextuel (clic droit)

Un **clic droit** sur une forme ouvre un menu :

- **Type de points** ■ Contour cousu / Remplissage tatami / Colonne satin (le type courant est coché) — bascule instantanée et annulable ;
- **Orientation du remplissage...** (si la forme est un tatami) ;
- **Calques** — les mêmes interrupteurs que le menu Affichage.

Poignée de rotation (orientation à la souris)

Quand un remplissage tatami est sélectionné, un **axe bleu avec une poignée** apparaît en son centre. Faites glisser la poignée pour tourner l'orientation des fils ; l'angle est appliqué au relâchement (annulable). C'est l'équivalent visuel de *Orientation du remplissage*....

Panneau Filtres d'affichage

Dock (à droite, dès qu'il existe des objets) pour **isoler** ce qu'on regarde :

- **Types de points** : cases Contour / Tatami / Satin ;
- **Taille min. des zones** : masque les zones dont l'aire source est sous le seuil (mm²) — utile pour cacher les micro-régions ;
- **Couleurs de fil** : une case par couleur distincte, pour n'afficher que certains fils.

Ces filtres n'affectent que **l'affichage** (pas l'export ni les points générés).

Barre d'outils principale

Actions fréquentes, icônes monochromes avec infobulle : ouvrir image/projet, enregistrer, annuler/rétablir, zoom –/ajuster/+, analyser, aperçu des points, exporter DST. Les actions indisponibles dans le contexte courant sont désactivées.

Barre d'outils contextuelle

Sous la barre principale, son contenu **suit la sélection** :

- **objet de broderie** : bascule rapide du type (Contour/Tatami/Satin) + *Orientation...* si tatami ;
- **objet vectoriel** : boutons de création rapide (Contour/Tatami/Satin) ;
- **région** : aire + *Fusionner / Supprimer / Vectoriser* ;
- **aucune sélection** : résumé du motif + bouton *Cadre*....

Inspecteur de propriétés

Dock (droite) affichant l'élément sélectionné. Pour un **objet de broderie**, il expose ses **paramètres de couture éditables après création** (contour : longueur/min/passages ; tatami : espacement/longueur/angle/retrait/décalage ; satin : densité/compensation/sous-couche). Chaque changement passe par une commande **annulable** et régénère les points. Une région ou un objet vectoriel y affiche ses informations en lecture seule.

Panneau Document

Dock (gauche) à deux onglets, **Objets** et **Régions**, tabifié avec l'**Ordre de couture**. Sélectionner une ligne met l'élément en évidence au canevas et dans l'inspecteur ; une sélection au canevas surligne la ligne correspondante (synchronisation bidirectionnelle).

Indicateur de workflow

Bandeau (sous Document) listant les étapes **Image** → **Régions** → **Vecteurs** → **Broderie** → **Vérification** → **Export**. L'état de chaque étape (à faire / disponible / en cours / terminé / attention) est déduit du document et rendu par pastille + libellé + mot d'état. Un clic rappelle en barre d'état l'action à faire.

Menu Analyse et panneau

Analyser le motif (F5) remplit le panneau *Analyse* (dock) avec les problèmes détectés, triés par gravité. Un double-clic sur un problème centre la vue sur sa localisation.

Panneau Ordre de couture

Onglet du panneau Document listant les objets de broderie dans l'ordre. Vous pouvez monter/descendre un objet, le verrouiller (il ne bougera plus lors de l'optimisation), et appliquer une stratégie (ordre du document, par couleur, par proximité, couleur puis proximité). Un libellé affiche le coût estimé.

Barre de simulation

Boutons de lecture/pause et un curseur qui révèle la couture jusqu'à un index de point, avec un repère d'aiguille. La barre occupe une **zone réservée** (toujours visible, contrôles grisés tant qu'aucune séquence n'existe) pour ne pas faire sauter la mise en page.

Erreurs et messages

Les opérations impossibles (recadrage hors image, satin non constructible, export d'une séquence vide...) affichent un message clair et **n'appliquent rien**. Les erreurs connues et les entrées invalides **couvertes par les tests** renvoient une erreur structurée au lieu de provoquer un arrêt du programme. Aucun plantage n'a été observé dans le corpus de tests actuel — ce qui ne constitue pas une garantie absolue en version 0.1.0.

Menu Aide

- **Raccourcis clavier...** : la liste ci-dessous.
- **À propos** : nom, version, licence.

Raccourcis

Raccourci	Action
Ctrl+O / Ctrl+S	Ouvrir une image / Enregistrer le projet
Ctrl+Z / Ctrl+Y	Annuler / Rétablir
Suppr	Supprimer la région sélectionnée
Ctrl++ / Ctrl+- / Ctrl+0	Zoom avant / arrière / ajuster
F	Ajuster au canevas
F5	Analyser le motif
V / H / M	Outils : Sélection / Déplacer la vue / Rectangle
Échap	Revenir à la Sélection (annule la fusion)
Ctrl+Shift+P	Masquer / afficher les panneaux
Ctrl+Q	Quitter

Les raccourcis standard proviennent des séquences Qt ; Ctrl+0, F5, F, V/H/M, Ctrl+Shift+P sont définis explicitement, sans conflit avec la navigation clavier (Tab reste réservé au parcours des contrôles).

Implémentation associée

- `apps/desktop/main_window.cpp/.hpp` — menus, barres, docks, sélection, cadre.
- `apps/desktop/app_theme.*`, `design_tokens.*` — thème et tokens.
- `apps/desktop/properties_panel.*`, `document_panel.*`, `workflow_panel.*`, `empty_state_widget.*` — panneaux.

- `apps/desktop/tools.hpp`, `ui_icons.*` — modes d'interaction et icônes.
- `apps/desktop/canvas_view.cpp` — zoom, déplacement, grille, cadre, rendu points.
- `apps/desktop/ruler.cpp` — règles en mm.
- `apps/desktop/import_dialog.cpp`, `brightness_dialog.cpp` — dialogues.

Traitement d'image

Public : utilisateur avancé, développeur. État : **Implémenté** (non destructif).

Principe

L'image importée est convertie en une **image de travail RGBA 8 bits** (`image::Image`). L'original n'est **jamais** modifié : le document conserve l'original et une **pile d'opérations** (`std::vector<ImageOp>`) rejouée à la demande par `apply_pipeline`. C'est ce qui rend toutes les retouches annulables.

Chargement

`load_image` lit le fichier via OpenCV (`cv::imdecode` sur les octets, ce qui gère les chemins Windows non-ASCII), gère 8 et 16 bits, niveaux de gris + alpha, et normalise tout en RGBA. `read_image_info` renvoie largeur, hauteur, canaux, présence d'alpha et format.

Opérations disponibles

Chaque opération est une alternative du variant `image::ImageOp` :

Opération	Type	Paramètres	Effet
Recadrage	<code>CropOp</code>	x, y, largeur, hauteur (px)	Découpe (borné à l'image)
Symétrie	<code>FlipOp</code>	horizontal/vertical	Miroir
Rotation 90°	<code>Rotate90Op</code>	quarts de tour 1–3	Rotation
Niveaux de gris	<code>GrayscaleOp</code>	—	Gris (alpha conservé)
Luminosité/contraste	<code>BrightnessContrastOp</code>	–100..100	Ajuste (alpha conservé)
Débruitage	<code>MedianDenoiseOp</code>	force 1–2 (noyau 3/5)	Médian
Quantification	<code>QuantizeOp</code>	2–64 couleurs	k-means déterministe (RGB)

Note : Le **rééchantillonnage** (redimensionnement de la résolution de travail) est volontairement absent de cette pile : il changerait le rapport mm/pixel. La taille physique se fixe à l'import (voir *Modèle physique*).

Déterminisme

La quantification fixe la graine du générateur aléatoire d'OpenCV (`cv::setRNGSeed`), donc deux exécutions donnent le même résultat — prérequis des tests et de la reproductibilité.

Aperçu et annulation

La luminosité/contraste offre un **aperçu en direct** (le dialogue émet un signal, la fenêtre recalcule l'affichage sans modifier le document tant que l'utilisateur n'a pas validé). Toute opération validée passe par une commande d'annulation (`AppendImageOpCommand`).

Erreurs possibles

- Recadrage hors de l'image → message, aucune modification.
- Nombre de couleurs hors bornes → refus.
- Fichier illisible/corrompu → erreur `InvalidFile`, pas de crash.

Implémentation associée

- `libs/image/include/openstitch/image/ops.hpp` — le variant `ImageOp`.
- `libs/image/src/ops.cpp` — `apply_op`, `apply_pipeline` (OpenCV encapsulé).
- `libs/image/src/load.cpp` — `load_image`, `read_image_info`, `encode_png`, `decode_image`.
- `libs/commands/include/openstitch/commands/project_commands.hpp` — `AppendImageOpCommand`.
- Tests : `tests/unit/image/test_ops.cpp`, `test_image_load.cpp`.

Segmentation

Public : utilisateur avancé, développeur. État : **Implémenté**.

But

Réduire l'image de travail en **régions de couleur connexes**, sélectionnables et éditables, qui serviront de base à la vectorisation puis aux objets de broderie.

Algorithme

1. Les pixels d'alpha < 128 sont traités comme **fond**.
2. Les pixels opaques sont convertis en **CIELAB** (espace perceptuel) ;
3. un **k-means** déterministe (graine fixe) sur un échantillon de pixels calcule jusqu'à N couleurs représentatives ;
4. chaque pixel est affecté au centre Lab le plus proche ;
5. par couleur, les **composantes connexes** (4-connexité, `cv::connectedComponents`) deviennent des **régions**. Deux régions de même couleur mais disjointes restent distinctes ;
6. les régions plus petites que la taille minimale sont **absorbées** par leur voisine majoritaire (nettoyage).

Identifiants stables

Chaque région a un `RegionId` = index de slot + 1. Les slots ne sont **jamais** réutilisés : un identifiant reste valide pour toute la vie de la segmentation (important pour l'undo/redo). La carte est stockée comme `labels[y*w+x]` (0 = fond) et `region_slots[]` (la région ou `nullopt` si supprimée/fusionnée).

Éditions

Action	Fonction	Effet
Sélection	<code>region_at(x, y)</code>	Renvoie la région sous un pixel
Fusion	<code>merge_regions(keep, absorb)</code>	Réétiquette absorb en keep
Suppression	<code>remove_region(id)</code>	Renvoie les pixels au fond (pas de fusion)
Recoloration	<code>recolor_region(id, rgb)</code>	Change la couleur représentative
Carte	<code>render_map(highlight)</code>	Image RGBA (fond transparent, sélection éclaircie)

Note : « Supprimer » fait **disparaître** la région (retour au fond) et ne la fusionne pas avec une voisine — corrigé après un retour utilisateur (voir *Limitations*). Pour transférer des pixels à une autre région, utilisez la **fusion** explicite.

Undo/redo

Les commandes `SetSegmentationCommand`, `MergeRegionsCommand`, `RemoveRegionCommand` et `RecolorRegionCommand` mémorisent les **indices de pixels** réétiquetés (pas une copie de la carte entière), ce qui rend l'annulation exacte et peu coûteuse.

Erreurs

- Image entièrement transparente → erreur `UserInput`.

- Nombre de couleurs hors bornes → refus.

Implémentation associée

- `libs/segmentation/include/openstitch/segmentation/segmentation.hpp`
- `libs/segmentation/src/segmentation.cpp` — `segment`, `merge_regions`, `remove_region`, `recolor_region`, `render_map`.
- `libs/commands/.../project_commands.hpp` — commandes de segmentation.
- Tests : `tests/unit/segmentation/test_segmentation.cpp`, `tests/unit/commands/test_undo_stack.cpp`.

Vectorisation

Public : utilisateur avancé, développeur. État : **Implémenté** (édition de nœuds limitée au déplacement).

But

Transformer une région (masque de pixels) en **contours vectoriels propres** : contour extérieur et trous, en coordonnées physiques (µm), simplifiés et nettoyés.

Algorithme

1. Construction du **masque binaire** de la région.
2. Extraction des contours et trous par `cv::findContours` (mode `RETR_CCOMP`, approximation `CHAIN_APPROX_SIMPLE`) — algorithme de Suzuki-Abe.
3. Conversion des points en **coordonnées physiques** (µm, origine au centre de l'image, Y vers le haut).
4. **Simplification** Douglas-Peucker par contour, avec une tolérance **adaptative** bornée à `périmètre / 16` : les petits trous (mât, cordage traversant une voile) ne sont pas aplatis au point de disparaître.
5. **Nettoyage** par union Clipper2 (`clean_to_path_sets`) qui reconstruit la hiérarchie extérieur/trous (règle pair-impair) et corrige les auto-intersections. Une région en plusieurs morceaux produit plusieurs `PathSet`.

Résultat

Un document `::VectorObject` contient un ou plusieurs `geometry::PathSet` (`outer + holes`), garde la couleur de la région et un lien vers la `RegionId` d'origine.

Formes dessinées à la main

État : *Présent · Testé (QTest headless) · Validé visuellement : non (capture d'écran non disponible dans cet environnement — voir Limitations)*. Activé par défaut, aucun réglage.

Second chemin de création d'un `VectorObject`, indépendant de toute image : avant cette fonctionnalité, la seule façon d'obtenir un objet vectoriel était de vectoriser une région segmentée depuis une image importée — un logiciel de digitalisation classique (Hatch et équivalents) permet aussi de dessiner directement une forme de base sur le canevas. Quatre outils dans la palette (barre d'outils gauche) :

- **Rectangle** (R) : glisser un cadre élastique → rectangle à 4 coins droits.
- **Ellipse** (0, Maj = cercle) : glisser un cadre élastique englobant → ellipse à 4 nœuds **lisses** (tangentes de Bézier, constante de Kappa ≈ 0,5523 — approximation standard, erreur radiale relative maximale ~0,027 %). Maj enfoncée pendant le glisser : le côté le plus petit du cadre contraint l'autre → cercle.
- **Polygone** (P) : clics successifs posent les sommets (aperçu élastique jusqu'au curseur) ; double-clic pour fermer (minimum 3 sommets, sinon le tracé est abandonné sans rien créer) ; Échap annule le tracé en cours à tout moment, y compris en changeant d'outil.
- **Forme libre / lasso** (L) : glisser en continu → un point par évènement de déplacement souris pendant que le bouton reste enfoncé (aperçu élastique mis à jour à chaque point) ; au relâchement, le tracé brut est **simplifié** (Douglas-Peucker, tolérance 0,3 mm — cf. *Vectorisation*, même algorithme que la simplification des contours vectorisés) pour ne garder qu'un contour à angles droits exploitable, sans lissage. Moins de 3 sommets après simplification (tracé trop court, ou entièrement colinéaire) → rien n'est créé, message en barre de statut.

La forme créée est un `VectorObject` **sans région source** (`source_region = std::nullopt` — le champ est optionnel précisément pour ce cas), automatiquement sélectionné : l'utilisateur enchaîne avec les actions **Créer un...** existantes (menu Broderie) pour la convertir en objet de broderie, exactement comme pour une forme vectorisée depuis une image. Aucun nouveau chemin de création côté broderie : le pipeline aval (génération, routage, export DST) est intégralement réutilisé.

Géométrie construite par `libs/geometry/src/primitives.cpp` (`rectangle_path/ellipse_path/polygon_path/freeform_path`, testés indépendamment de Qt) ; `MainWindow` ne fait que router le geste souris (glisser pour rectangle/ellipse — mécanique de cadre élastique déjà utilisée pour le recadrage image, généralisée ; clics successifs pour le polygone ; glisser continu pour la forme libre) vers ces fonctions puis vers `AddVectorObjectCommand` (annulable, comme la vectorisation). Un cadre trop petit (glisser quasi nul) ne crée rien.

Correctif de revue — Maj = cercle, désormais testable et fiable. La détection de Maj (contrainte cercle sur l'ellipse) interrogeait `QGuiApplication::keyboardModifiers()` (état clavier **global**) au moment de traiter le signal de fin de glisser — fonctionnellement correct en usage réel (même thread, traitement synchrone), mais impossible à couvrir par un test `QTest offscreen` (`QTest::keyPress` sur une fenêtre non affichée ne met pas à jour cet état de façon fiable), donc jamais réellement vérifié bout en bout. Corrigé : les modificateurs sont désormais capturés directement sur l'évènement Qt de relâchement qui termine le glisser (`CanvasView::mouseReleaseEvent`) et transmis par le signal `boxDrawnMm` — plus de dépendance à un état global, testable en passant directement `Qt::ShiftModifier`.

Édition de nœuds

Les nœuds de l'objet sélectionné s'affichent (poignées de taille constante) et se **déplacent** à la souris ; chaque déplacement passe par `MoveNodeCommand` (annulable).

Limitation : l'**ajout/suppression de nœuds**, la **conversion anguleux/lisse** et l'édition de tangentes de Bézier ne sont **pas** exposés dans l'interface (le modèle les prévoit : `PathNode` porte `tan_in/tan_out` et un `NodeType`).

Robustesse

La correction de la tolérance de simplification (adaptative) évite un défaut observé : des trous trop simplifiés faisaient déborder les remplissages (voir *Tatami* et *Limitations*).

Implémentation associée

- `libs/vectorization/src/vectorize.cpp` — `vectorize_region`.
- `libs/geometry/src/simplify.cpp` — Douglas-Peucker, aire signée.
- `libs/geometry/src/clean.cpp` — `clean_to_path_sets` (Clipper2 encapsulé).
- `libs/document/include/openstitch/document/vector_object.hpp` — `VectorObject`, `NodeRef`.
- `libs/commands/.../project_commands.hpp` — `AddVectorObjectCommand`, `MoveNodeCommand`.
- `libs/geometry/include/openstitch/geometry/primitives.hpp` + `src/primitives.cpp` — `rectangle_path`, `ellipse_path`, `polygon_path`, `freeform_path` (formes dessinées à la main).
- `apps/desktop/main_window.cpp` — `addVectorPrimitive`, `onBoxDrawn`, `onCanvasDoubleClicked`, `updatePolygonPreview/finishPolygon/ cancelPolygonDraw`, `onFreeformPointAdded/finishFreeform/ cancelFreeformDraw` ; `apps/desktop/canvas_view.hpp/.cpp` — modes `setBoxDrawMode/setPolygonDrawMode/setFreeformDrawMode` (généralisation du cadre élastique de recadrage) ; `apps/desktop/ui_icons.hpp/.cpp` — `icons::freeform`.
- Tests : `tests/unit/vectorization/test_vectorize.cpp`, `tests/unit/geometry/test_simplify.cpp`, `test_clean.cpp`, `test_primitives.cpp` ; `tests/unit/desktop/test_main_window.cpp` (création, undo/redo, cadre dégénéré, tracé de polygone, annulation).

Objets de broderie

Public : utilisateur avancé, développeur. État : **Implémenté** (trois types).

Définition

Un document `::EmbroideryObject` porte l'**intention de couture** : un type de point et ses paramètres, une couleur de fil, un lien vers l'objet vectoriel source, un état de visibilité et de verrouillage. Ses **points ne sont pas stockés** dans l'objet : ils sont régénérés à la demande (séparation intention/points).

Le variant `stitchParams`

Le type de point est un `std::variant` à trois alternatives :

Objet	Params	Géométrie suivie	Sous-couche
Point de contour	<code>RunningStitchParams</code>	contours de l'objet vectoriel	non
Remplissage tatami	<code>TatamiParams</code>	régions pleines (avec trous)	prévue, non générée
Colonne satin	<code>SatinParams</code>	deux rails portés par l'objet	centrale (point droit)

Le satin est particulier : il **porte sa propre géométrie** (deux rails éditables), car une colonne ne se déduit pas d'un simple contour.

Tableau récapitulatif

Objet	Générateur	Paramètres clés	Statut recommandé
Running (simple/double/triple)	<code>run_stitch + apply_repeats</code>	<code>stitch_length</code> , <code>min_length</code> , <code>repeats</code>	Implémenté
Tatami	<code>fill_tatami</code>	<code>row_spacing</code> , <code>stitch_length</code> , <code>angle</code> , <code>inset</code> , <code>stagger</code>	Partiel
Satin	<code>fill_satin</code>	<code>density</code> , <code>pull_compensation</code> , <code>center_underlay</code>	Expérimental

Le tableau reflète la **qualité métier**, pas seulement la présence de code : le running stitch est solide ; le tatami est fonctionnel mais sans sous-couche ni underpath caché ; le satin est une génération simple à deux rails (voir les chapitres dédiés). Aucun n'est validé sur machine réelle.

Création

Depuis l'interface (menu Broderie) sur un objet vectoriel sélectionné, ou en lot par la **classification automatique expérimentale des régions** (voir *Limitations*) qui choisit le type selon la forme :

- bande fine compatible → une ou plusieurs sections de **satin topologique** ;
- autre zone **remplissable** → **tatami** ;
- petite forme → **contour** cousu (point triple) ;

- satin **automatique** naïf : désactivé par défaut car il débordait sur les formes concaves/branchues (`AutoOptions::use_naive_satin` pour le réactiver).

Le **type se change ensuite** à tout moment (clic droit sur la forme ■ *Type de points* ■ contour / tatami / satin) via `SetStitchTypeCommand` (annulable) ; l'action **Convertir les satins auto en tatami** répare en lot un projet importé.

Limitation : cette classification est une heuristique simple (largeur moyenne = $2 \cdot \text{aire} / \text{périmètre total}$) autour d'un moteur géométrique ; ce n'est pas encore un moteur d'auto-numérisation robuste comparable à un logiciel commercial. Elle produit des objets **éditables** qu'il faut vérifier et retoucher.

Régénération

`generate_sequence(project)` parcourt les objets **visibles** dans l'ordre, insère un changement de couleur entre deux objets de couleurs différentes, et appelle le générateur adapté à chaque type via `std::visit`. Le résultat est une `StitchSequence`. Chaque commande porte l'`ObjectId` de sa source : l'interface s'en sert pour afficher la broderie **dans la couleur de chaque fil** et pour **filtrer** l'affichage (par type, couleur ou taille de zone).

Implémentation associée

- `libs/document/include/openstitch/document/embroidery_object.hpp` — `EmbroideryObject`, `StitchParams`, `RunningStitchParams`, `TatamiParams`, `SatinParams`.
- `libs/stitch_generation/src/generate.cpp` — `generate_sequence`, `dispatch`.
- `libs/autodigitize/src/autodigitize.cpp` — choix automatique du type.
- Tests : `tests/unit/stitch/test_generate.cpp`.

Génération de points — point droit et fondations

Public : utilisateur avancé, développeur.

État : Présent dans le code : oui · Tests unitaires : oui · Tests visuels : oui (SVG golden) · Import/export DST : oui · Test sur machine réelle : **non** · **Statut recommandé : implémenté** — reconstruit sur des fondations de longueur d'arc, c'est le générateur le plus solide du projet.

Séquence — génération des points

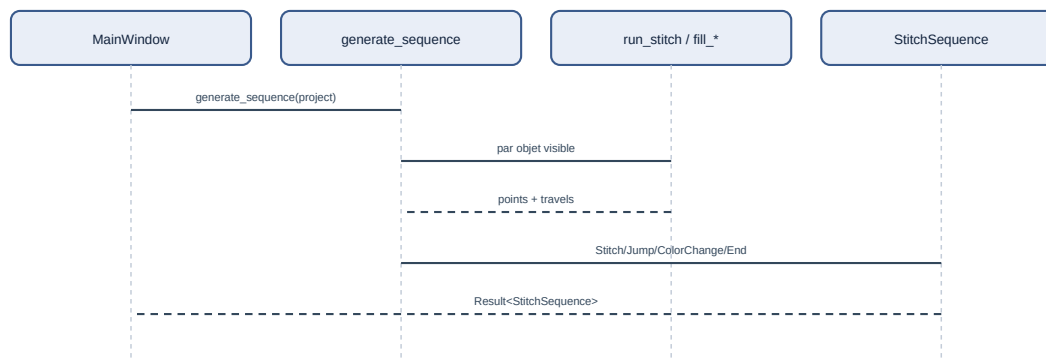


Figure — Génération de la séquence de points à partir du document.

Fondations géométriques

Avant tout type de point, deux primitives (dans `geometry`) :

- **Aplatissage de Bézier adaptatif** (`flatten`) : les segments à tangentes sont subdivisés récursivement (De Casteljau) jusqu'à ce que l'écart à la corde passe sous une tolérance. On n'échantillonne **jamais** une Bézier par pas uniforme du paramètre `t`.
- **Paramétrisation par longueur d'arc** (`cumulative_lengths`, `point_at_length`) et **ré-échantillonnage équilibré** (`resample_run`) : un tronçon de longueur `L` est divisé en $n = \text{ceil}(L / \text{cible})$ parts **égales**, donc chaque point est $\leq$ la longueur cible et il n'y a pas de segment résiduel minuscule.

Sémantique du paramètre : avec `ceil`, le champ (`RunningConfig::target_length`) est traité comme une **longueur maximale** (les points ne la dépassent jamais, mais peuvent être plus courts). Si on voulait une longueur *souhaitée* dont l'espacement colle au plus près de la consigne, `round(L / cible)` serait plus adapté. Ce choix (`ceil` plutôt que `round`) est **assumé explicitement** dans le code (`libs/geometry/src/polyline.cpp`, commentaire de `resample_run` : « Écart assumé (...) ; `ceil` garantit en plus le respect strict de la longueur maximale. ») — ce n'est plus une ambiguïté ouverte, `target_length` se comporte délibérément comme un maximum, pas une cible exacte.

Point droit (running stitch)

`run_stitch(path, config)` :

1. aplatit le chemin (courbes comprises) ;
2. détecte les **coins vifs** (angle de rotation > seuil, $\sim 35^\circ$ par défaut) ;
3. découpe le chemin en tronçons entre coins ;
4. ré-échantillonne **chaque tronçon par longueur d'arc**.

Résultat : les coins sont des pénétrations **exactes** ; les portions lisses ont un espacement **régulier**. Un cercle finement facetté ne produit donc plus un point par facette, mais des points espacés de la longueur cible.

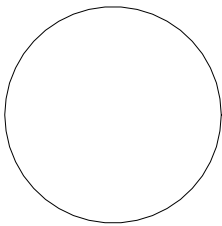


Figure — Trajectoire réelle produite par le moteur (SVG de diagnostic) : cercle de rayon 20 mm, longueur cible 3 mm. Trait = couture, orange pointillé = sauts.

Le résultat est structuré : `RunningResult { points, warnings, stats }`. Il ne plante jamais et renvoie un avertissement (chemin vide, trop court, point trop long...).

Chemins fermés, sens, phase, départ

`RunningConfig` expose : `reverse` (sens), `phase` (décalage curviligne des boucles lisses) et `start` (point de départ projeté sur une boucle fermée). Une boucle fermée revient à son point de départ sans doublon minuscule.

Répétitions

`apply_repeat_mode` implémente des modes **explicites** :

Mode	Effet
SinglePass	un passage
BackAndForth	aller complet puis retour (termine au départ)
BeanStitch	chaque intervalle cousu n fois (impair) — point triple
Backstitch	progression avec recouvrement

Les mouvements nuls sont supprimés ; le nombre exact de traversées est testé.

Traitement des points courts

Le running stitch **ne supprime pas** aveuglément les points sous une longueur minimale (cela déplacerait un sommet ou détruirait un détail). À la place, les **vrais coins** (angle de rotation > seuil) sont préservés comme pénétrations exactes, et chaque tronçon lisse est ré-échantillonné indépendamment par longueur d'arc — ce qui évite les rafales de micro-points sur les courbes finement facettées. Un point plus court que `min_length` peut subsister à un **coin serré** (deux coins proches) ; il est alors **signalé** par un avertissement, pas supprimé.

Limitation : ce traitement fin s'applique au running stitch. Aux **fin de rangée** du tatami et surtout aux **extrémités pointues** d'un satin, des pénétrations peuvent se rapprocher voire s'empiler ; la gestion dédiée (retrait, redistribution, terminaisons) n'est **pas** implémentée. C'est une des raisons du statut *expérimental* du satin.

Outil de diagnostic

`openstitch-cli stitchdebug --shape circle|line|corner|bezier|star|ring --length 3 --output-svg out.svg` génère les points sur une forme de référence, affiche les statistiques (points, longueur, segment min/max) et un SVG. Utilisé pour valider le moteur (voir *Tests*).

Implémentation associée

- `libs/geometry/src/polyline.cpp` — `flatten`, longueur d'arc, `resample_run`.

- `libs/stitch_generation/src/running_stitch.cpp` — `run_stitch`, `apply_repeat_mode`, `apply_repeats`, `sample_path`.
- `apps/cli/main.cpp` — sous-commande `stitchdebug`.
- Docs de conception : `docs/stitch-engine-audit.md`, `docs/stitch-engine-research.md`.
- Tests : `tests/unit/geometry/test_polyline.cpp`,
`tests/unit/stitch/test_running_stitch.cpp`.

Moteur de génération de points

Public : développeur, utilisateur avancé.

État : Présent dans le code : oui · Tests unitaires : oui (running/tatami/ satin/routage/retouches) · Tests visuels : partiels (SVG de diagnostic, golden) · Import/export DST : oui · Test sur machine réelle : **non** · **Statut recommandé : hétérogène** — la fondation commune (stitch, l'orchestration de generate_sequence, le contrat effective_sequence) est solide et testée ; le point droit est implémenté ; le tatami est partiel ; le satin reste expérimental (voir *Colonne satin* pour le détail). Ce chapitre documente le **mécanisme**, pas la qualité du résultat cousu — voir *Limitations* pour la synthèse par capacité.

Ce chapitre est la référence unifiée du moteur qui transforme un document::Project en std::vector<stitch::StitchCommand> exportable en DST. Il couvre l'intégralité de libs/stitch_generation/ (module stitch_generation, cf. *Référence des modules*) et son vocabulaire commun dans libs/stitch/. Les chapitres *Génération de points — point droit et fondations*, *Remplissage tatami* et *Colonne satin* restent les références narratives (audits, historique des correctifs, chiffres mesurés) pour chaque type de point ; ce chapitre les cite plutôt que de les dupliquer, et se concentre sur ce qu'aucun des trois ne couvre en un seul endroit : le contrat d'entrée (generate_sequence vs effective_sequence), le vocabulaire commun des commandes/passes, l'algorithme exact qui transforme des rails + barreaux en points cousus (Lots 2 à 6 du satin, jamais rassemblés ailleurs), et la place de la génération dans le pipeline complet (retouches, ordre, export).

1. Vue d'ensemble : de l'objet à l'export

```
document::Project
■ optimization::optimize_order()      réordonne project.embroidery_objects
  (document, AVANT génération)        (commande annulable ReorderEmbroideryCommand)
■ stitch_generation::generate_sequence(project)  -- séquence BRUTE
  ■ pour chaque EmbroideryObject visible, dans l'ordre du document :
    RunningStitchParams -> generate_running (run_stitch + apply_repeats)
    TatamiParams        -> generate_tatami  (tatami_underlay + fill_tatami)
    SatinParams         -> generate_satin ou generate_satin_group (routage)
  ■ ColorChange entre deux objets de couleurs différentes, End final
■ stitch_generation::apply_manual_overrides(sequence, project)  -- retouches Lot 8
= stitch_generation::effective_sequence(project)  -- SEUL point d'entrée de production
  ■ apps/desktop : aperçu (renderStitches), simulation, statistiques (stitch::compute_stats)
  ■ formats::write_dst_file  -- export DST (voir *Format DST*)
```

Deux fonctions pures cohabitent, et la distinction est le contrat le plus important du module :

- **generate_sequence(project)** (generate.hpp/generate.cpp) parcourt le document et produit la séquence **brute** : uniquement ce que la géométrie actuelle des objets dicte, sans aucune retouche manuelle.
- **apply_manual_overrides(sequence, project)** (overrides.hpp/overrides.cpp) patche **en place** une séquence déjà générée avec les retouches (EmbroideryObject::overrides, Lot 8/ADR-014 — déplacement de point, Stitch↔Jump, coupe de fil) des objets dans l'état ManuallyEdited.
- **effective_sequence(project)** enchaîne les deux, dans cet ordre, et rien d'autre — sa signature est volontairement identique à generate_sequence pour qu'un appelant existant n'ait qu'à substituer l'appel.

Le contrat : personne ne contourne les retouches

CLAUDE.md et le code sont explicites : **tout consommateur de production** (aperçu desktop, export DST, simulation, statistiques, CLI) doit appeler effective_sequence, jamais generate_sequence seul — sans quoi les retouches manuelles d'un utilisateur disparaîtraient silencieusement de l'aperçu ou de l'export. Ce contrat n'est pas qu'une convention documentée : il est **imposé structurellement** par tests/check_no_raw_sequence_bypass.cmake, un script CMake exécuté comme test CTest qui :

1. parcourt tous les `.cpp/.hpp` de `apps/` et `libs/` ;
2. ignore `libs/stitch_generation/` (implémentation interne légitime : `effective_sequence` doit bien appeler `generate_sequence`) et `tests/` (fixtures synthétiques, jamais un projet utilisateur réel) ;
3. échoue avec le fichier:ligne exact si un site d'appel restant invoque `generate_sequence`(sans qu'un commentaire `raw-sequence-ok: <raison>` apparaisse sur la même ligne ou la ligne précédente.

Deux sites — et seulement deux, à ce jour — portent cette annotation, tous deux dans `apps/cli/main.cpp` : `run_stitchdebug` (ligne 181, "Projet synthétique construit ici même, jamais chargé/sauvegardé, aucune retouche manuelle possible") et le générateur de debug du routage satin (ligne 534, même justification). Les deux construisent un document : `Project` synthétique en mémoire, uniquement pour visualiser un algorithme sur une forme de référence — un projet qui, par construction, ne peut porter aucun `StitchOverride`. C'est la seule catégorie d'exception prévue par le mécanisme : un vrai projet utilisateur (chargé d'un `.osp`, édité dans l'interface) ne doit jamais transiter par `generate_sequence` seul.

2. Vocabulaire commun (`libs/stitch/`)

`stitch::StitchCommand` (`libs/stitch/include/openstitch/stitch/sequence.hpp`) est l'unité atomique de toute séquence, qu'elle soit générée, importée d'un DST ou retouchée :

```
struct StitchCommand {
    Vec2um pos;
    CommandType type;    // Stitch, Jump, Trim, ColorChange, Stop, End
    ObjectId source;      // 0 = manuel/importé
    StitchPass pass;      // Underlay, TopStitch, Travel, Lock, Manual
};
```

`CommandType` est la sémantique **machine** (ce qui finit dans le DST, cf. *Format DST*) ; `StitchPass` est une métadonnée **logique**, ajoutée par ce moteur et **jamais sérialisée** (la séquence est toujours régénérée, ADR-014 — le DST l'ignore complètement). Elle permet d'afficher, filtrer et analyser les sous-couches indépendamment de la couche supérieure :

Passe	Origine	Exemples
Underlay	sous-couches, émises avant la couche supérieure	<code>tatami_underlay</code> , sous-couches satin (<code>center/edge/zigzag</code>)
TopStitch	couche visible finale	<code>running stitch</code> , <code>fill_tatami</code> hors sauts, <code>zigzag satin</code> principal
Travel	déplacement — cousu et caché, ou saut	pénétrations intermédiaires d'un <code>underpath</code> (<code>tatami §15</code> , satin routé Lot 6), et le <code>Jump</code> de tête de chaque polyligne (<code>emit_polyline</code>)
Lock	point de fixation	<code>lock_stitches</code> (satin, Lot 5)
Manual	modifié par une retouche	<code>apply_manual_overrides</code> : position déplacée ou type forcé

Point précis souvent mal lu : `Travel` couvre **deux réalités opposées** — un `Jump` (aiguille levée, aucune couture) et une pénétration `Stitch` cousue et cachée sous la couche supérieure (`underpath`). Distinguer les deux exige de regarder `CommandType`, pas seulement `StitchPass` : un point `{Travel, Jump}` est un vrai saut, un point `{Travel, Stitch}` est cousu. L'outil de diagnostic desktop (`MainWindow::buildDebugDump`, menu contextuel *Diagnostic*) imprime justement `pass=<Passe> type=<Type> pos=<x,y>` pour chaque commande d'un objet, ainsi qu'un histogramme par passe (`Underlay=...` `TopStitch=...` `Travel=...` `Lock=...`).

Manual=...) — c'est la source du vocabulaire `pass=Underlay/pass=TopStitch/...` utilisé informellement ailleurs dans le projet.

`stitch::compute_stats`

`libs/stitch/src/stats.cpp` dérive `StitchStats` (points, sauts, coupes, changements de couleur, longueur de fil, boîte englobante) en un seul passage séquentiel :

- `stitches/jumps/trims/color_changes` : compteurs directs par `CommandType`.
- `thread_length_um` : somme de `length_um(cmd.pos - prev)`, ajoutée **uniquement** sur une commande `Stitch`, où `prev` est la position de la commande précédente **quel que soit son type** (`Stitch` ou `Jump` — `prev` est mis à jour dans les deux cas). Un saut ne contribue donc jamais directement à `thread_length_um`, mais la première `Stitch` qui le suit ajoute la distance depuis le point d'atterrissage du saut. En pratique cette distance est **nulle** : `emit_polyline` saute vers `points.front()` puis coud `points[0]`, qui **est** `points.front()` — le saut et la première couture partagent exactement la même position. `thread_length_um` mesure donc bien la longueur de fil réellement cousu, jamais la distance des sauts eux-mêmes, mais ce n'est vrai que parce que chaque émetteur respecte cette convention (aucune garantie structurelle au niveau de `compute_stats` lui-même).
- `bounds` : boîte englobante des positions `Stitch` **uniquement** — un saut loin du motif (retour au point d'origine avant un `Trim`, par exemple) n'élargit pas le cadre rapporté.

3. Orchestration : `generate_sequence`

`generate_sequence` (`libs/stitch_generation/src/generate.cpp`) itère sur `project.embroidery_objects` dans l'ordre du document (l'ordre qu'*Ordre de couture* — §7 ci-dessous — a éventuellement déjà réarrangé en amont, au niveau du document, pas ici) :

1. les objets `!visible` sont ignorés (index avancé, rien émis) ;
2. pour un type non-satin, l'objet vectoriel source (`source_vector`) est recherché dans `project.vector_objects` ; absent, `generate_sequence` renvoie une erreur `ErrorCategory::Internal` nommant l'objet — la seule voie d'échec de la génération en dehors d'un document totalement vide ;
3. un `ColorChange` est inséré si l'objet précédent existait et avait une couleur RGB différente ;
4. **routage satin (§6.5)** : si l'objet est un satin *routable* (`is_routable_satin` — satin dont `SatinParams::rungs.size() >= 2`, donc issu de l'auto-satin ou d'un satin manuel avec barreaux par défaut), le moteur cherche le plus long groupe **contigu** d'objets satin routables suivants qui partagent **exactement** la même couleur (`rgb`) et le même `source_vector`. Un groupe de taille ≥ 2 part dans `generate_satin_group` (§6.5) ; un groupe de taille 1 (satin routable isolé, ou dernier de sa série) suit le chemin normal `generate_satin`. Un satin manuel/legacy sans barreaux n'est jamais groupé, quelle que soit sa couleur ;
5. sinon, `std::visit` sur `object.params` dispatch vers `generate_running`, `generate_tatami` ou `generate_satin` (§4, §5, §6).

Une fois tous les objets traités, si la séquence reste vide (aucun objet visible n'a rien produit — cas trivial : document vide, ou uniquement des objets masqués), l'erreur `ErrorCategory::UserInput` "Aucun objet de broderie visible : rien à générer" est renvoyée. Sinon, une commande `End` est ajoutée à la position de la dernière commande.

Les trois émetteurs de polylignes (vocabulaire interne de `generate.cpp`)

Trois fonctions statiques traduisent une géométrie déjà calculée en commandes, partagées par les trois générateurs :

- `emit_polyline(seq, points, source, pass)` : un `Jump` vers `points.front()`, puis chaque point en `Stitch`. Le saut est **omis** si la toute dernière commande est déjà un `Stitch` à la même position (enchaînement direct depuis une passe précédente du même objet — sous-couche → lock → satin) ; un

saut est toujours émis après un `ColorChange`, un autre `Jump`, ou entre deux objets différents, même à position identique.

- `emit_polyline_with_breaks(seq, points, jump_before, source, pass)` : identique, mais certains points internes (indices listés dans `jump_before`, triés) sont eux aussi atteints par un `Jump` plutôt qu'enchaînés — c'est le mécanisme qui matérialise `SatinResult::jump_before` (§6.4) dans la séquence machine.
- `emit_fill(seq, fill, source)` : consomme un `std::vector<FillStitch>` (tatami, §5) — `fs.jump` devient `Jump/Travel`, `fs.travel` devient `Stitch/Travel` (underpath caché), sinon `Stitch/TopStitch`. Le tout premier point est toujours un `Jump`, même si `fs.jump` vaut `false` (!started force la branche `Jump`).

4. Point droit (running stitch) — synthèse

Couvert en détail par *Génération de points — point droit et fondations* ; voici la synthèse tendue nécessaire pour situer ce point dans le moteur, avec deux corrections vérifiées contre le code actuel.

`stitch_generation::run_stitch(path, RunningConfig)`
 (libs/stitch_generation/src/running_stitch.cpp) : aplatit la Bézier (`geometry::flatten`, tolérance `flatten_tolerance = 100 μm` par défaut) → détecte les coins vifs (angle de rotation > `corner_threshold`, $\approx 0,6108652 \text{ rad} = 35^\circ$ par défaut) → découpe en tronçons entre coins → ré-échantillonne **chaque tronçon** par longueur d'arc (`geometry::resample_run`, dans `libs/geometry/src/polyline.cpp`).
`generate_running` (`generate.cpp`) appelle `sample_path` (délègue à `run_stitch`, un seul passage) puis `apply_repeats` pour chaque contour extérieur et chaque trou de l'objet vectoriel source.

Correction apportée à Génération de points... : ce chapitre présente le comportement "cible traitée comme un maximum" (`ceil(L/cible)` plutôt que `round`) comme *"un point d'ambiguïté à clarifier dans une future version (distinguer `target_length` de `maximum_length`)"*. À la lecture du code actuel, cette clarification a déjà eu lieu **partiellement** : le champ s'appelle désormais `RunningConfig::target_length` (plus `stitch_length`), et des champs `min_length`/`max_length` distincts existent bel et bien. Mais le choix `ceil` reste, et le commentaire à côté de `geometry::resample_run` (`libs/geometry/src/polyline.cpp`) le qualifie explicitement d'**assumé**, pas d'ouvert : *"Écart assumé (...); `ceil` garantit en plus le respect strict de la longueur maximale."* Le comportement observable — les points ne dépassent jamais `target_length` mais peuvent être plus courts — est donc un choix de conception délibéré et documenté dans le code, pas une ambiguïté résiduelle ; seul le nom du champ dans la doc narrative (`stitch_length`) est à mettre à jour vers `target_length`.

`apply_repeat_mode` (`RepeatMode::SinglePass/BackAndForth/BeanStitch/Backstitch`) et son alias de compatibilité `apply_repeats(points, int)` restent inchangés par rapport à la description existante.

5. Remplissage tatami — synthèse

Couvert en détail par *Remplissage tatami* ; synthèse tendue, vérifiée sans divergence notable contre `libs/stitch_generation/src/tatami.cpp`.

`fill_tatami(region, TatamiParams)` : rotation virtuelle de `-angle` → balayage de rangées horizontales espacées de `row_spacing` (400 μm par défaut) → intersections pair-impair par rangée → pénétrations sur une grille de pas `stitch_length` (3 mm par défaut), déphasée d'une rangée à l'autre selon `stagger` (2 par défaut) → les segments de rangées voisines qui se chevauchent en x sont les arêtes d'un graphe, parcouru par un algorithme glouton (`current` → voisin non visité le plus proche, sinon saut vers le segment non visité le plus proche) → rotation inverse. Chaque liaison cousue (dans une rangée ou entre rangées) est validée géométriquement par `connector_invalid` : le segment est découpé à **chaque** intersection paramétrique avec une arête du contour ou d'un trou (y compris un contact dégénéré par un sommet), et le milieu de chaque sous-segment doit rester dans la région (`in_region`, qui traite un point exactement sur une frontière comme intérieur, tolérance $\sim 0,01 \mu\text{m}$) — sans quoi la liaison devient un saut. Ce mécanisme est ce qui garantit qu'aucune couture ne traverse un trou.

Tatami avancé (Lot 7, tout désactivé par défaut) : `tatami_underlay` produit un contour rentré (`underlay_edge`, retrait `underlay_inset` = 600 µm — jamais sur les bords de trous, politique sûre si le retrait échoue) et des rangées perpendiculaires espacées (`underlay_parallel`, pas `underlay_spacing` = 2 mm) ; `hidden_underpath` remplace un saut par un trajet cousu caché (passe Travel) — direct s'il reste court (`seglen(prev, rp) <= underpathCap`, avec `underpathCap` = $\max(6 \cdot \text{row_spacing}, 8 \text{ mm})$), sinon le long du contour extérieur rentré (`routeHighway`, qui essaie les deux sens de parcours du contour et garde le plus court sous le même plafond), toujours revalidé par `connector_invalid` segment par segment.

Aucune divergence factuelle trouvée entre *Remplissage tatami* et le code actuel — la chaîne d'implémentation `fill_tatami/tatami_underlay/connector_invalid/in_region` correspond exactement à ce que ce chapitre décrit, chiffres compris.

6. Colonne satin : des rails aux points cousus (Lots 2 à 6)

Colonne satin documente en détail l'**appariement géométrique** (`ladder_correspondence`, l'algorithme "ladder" qui remplace l'ancienne correspondance par fraction d'abscisse) et tout le moteur `auto_satin` qui produit des rails et des barreaux à partir d'un squelette. Ce chapitre-ci ne reprend pas cette histoire — il documente précisément ce qui se passe **une fois que SatinParams (rails + barreaux) est connu**, jusqu'aux commandes machine : c'est le contenu des Lots 2 à 6 de `libs/stitch_generation/src/satin.cpp`, `lock.cpp` et `routing.cpp`, jamais rassemblé en un seul endroit ailleurs dans la documentation.

6.1. Deux chemins de génération, un seul point d'entrée

`generate_satin` (`generate.cpp`) construit un `SatinConfig` à partir de `document::SatinParams` (copie champ à champ des Lots 3/4, cf. tableau §6.6) puis choisit :

- `fill_satin_columns(rail_a, rail_b, rungs, config)` si `params.rungs.size() >= 2` — c'est le chemin qui implémente l'**intégralité** de `SatinConfig` (terminaisons, split, sous-couches de bord/zigzag, compensation push/pull asymétrique). Depuis `default_rungs` (voir *Colonne satin §Barreaux par défaut*), c'est le chemin emprunté par la quasi-totalité des satins de l'application, y compris manuels.
- `fill_satin(rail_a, rail_b, config)` sinon (satin legacy sans barreaux, compatibilité ascendante uniquement) — n'implémente qu'un **sous-ensemble** : densité, compensation pull **symétrique**, sous-couche centrale. Tout autre réglage exposé par l'inspecteur (terminaisons, split, sous-couches de bord/ zigzag, push, compensation asymétrique) reste silencieusement **sans effet** sur ce chemin — limitation déjà documentée dans *Colonne satin*, reconfirmée ici en lisant `fill_satin` : sa boucle d'émission ne référence ni `cap_start/cap_end`, ni `split_stitch`, ni `underlay_edge/underlay_zigzag`, ni `pull_left/pull_right/push_start/push_end`.

6.2. satin_stations : la correspondance structurelle

`fill_satin_columns` délègue d'abord à `satin_stations(rail_a, rail_b, rungs, density)` (exposée publiquement dans `satin.hpp`, réutilisée par `satin_coverage`) : aplatit les deux rails (`geometry::flatten`, tolérance `kRailFlattenTolerance` = 30 µm — sous la résolution DST de 100 µm, donc sans perte perceptible), réoriente `rail_b` si les rails sont fournis tête-bêche (`opposite_orientation`), projette chaque barreau sur les deux rails (abscisses curvilignes), **trie** les barreaux par abscisse projetée cumulée (`sa + sb`) — l'ordre du vecteur d'entrée n'est jamais signifiant — puis **fusionne** deux barreaux consécutifs après tri dont la progression sur les deux rails est inférieure à la moitié du pas de densité (`anchorMinGap = density / 2`), pour éliminer tout intervalle dégénéré.

Moins de deux barreaux après fusion → liste vide (l'appelant retombe sur `fill_satin`). Sinon, pour chaque paire de barreaux consécutifs :

- si `non_ribbon_interval(a0, a1)` est vrai (§6.4), aucune station intermédiaire n'est produite : seul le barreau d'arrivée est ajouté, marqué `jump_before = true` ;
- sinon, `ladder_correspondence` puis `resample_by_medial_spacing` (voir *Colonne satin* pour l'algorithme complet) produisent les stations intermédiaires à l'espacement `density` (400 µm par

défaut), mesuré sur la **ligne médiane** — donc approximativement perpendiculaire au fil, y compris en section inclinée ou courbe, jamais mesuré le long d'un rail.

6.3. `fill_satin_columns` : ordre exact des transformations

Une fois les stations connues, chacune devient un `Thread{a, b, anchor, jump_before}`. Les transformations s'enchaînent **dans cet ordre précis** — ordre qui a des conséquences directes sur le résultat (voir la note sur les sous-couches ci-dessous) :

1. **Terminaisons** (`cap_start/cap_end`, `SatinCapType`, §6.6) : pour chaque fil i , $f = \min(\text{cap_factor}(\text{cap_start}, i, \text{cap_length}), \text{cap_factor}(\text{cap_end}, n\text{Threads}-1-i, \text{cap_length}))$ — le **minimum** des deux facteurs, pour qu'une colonne trop courte pour loger les deux terminaisons sans chevauchement reste cohérente (jamais deux rétrécissements indépendants qui se contrediraient). Si $f < 1$, a et b sont chacun ramenés vers le milieu du fil par interpolation (`lerp(milieu, point, f)`). `cap_factor : Flat` **et Automatic** renvoient tous deux 1.0 (aucun rétrécissement) — la branche `default` : du `switch` couvre les deux ; en l'état actuel du code, **`SatinCapType : Automatic` ne se comporte pas différemment de `Flat`**, malgré son nom qui suggère un choix adaptatif. C'est une lacune non documentée ailleurs : l'inspecteur propose les deux options sans qu'aucune différence de résultat ne soit produite. `Tapered` suit une rampe linéaire de `kMin` (0,18 — jamais 0, pour ne pas empiler sur une coordonnée unique) à 1 ; `Rounded` suit un quart de sinuséide.
2. **Points courts** (`short_stitch`, §6.6), si activé : pour chaque fil $i \geq 1$ (jamais sur un barreau, `t.anchor` protégé), compare l'avance de chaque rail depuis le fil précédent (`advA, advB`) ; si le ratio `min/max < short_stitch_curvature` (0,55 par défaut) et que l'avance minimale est sous `short_stitch_min_gap` (250 μm), le virage est jugé "serré". `RemoveAndRedistribute` marque le fil `dropped` un fil sur deux (jamais deux d'affilée) ; `SingleInset/MultiLevelInset` rentrent le point du rail intérieur vers le milieu d'une fraction `short_stitch_inset` (0,35 par défaut), triangulaire sur `short_stitch_levels` (2 par défaut) niveaux en mode `MultiLevelInset`.
3. **Push** (`push_start/push_end`, §6.6) : `shiftEnd` étend/rétracte le premier et le dernier fil le long de l'axe vers leur voisin immédiat. Une rétraction (`amount < 0`) est bornée à $-(n-1)$ (n = distance au voisin) : elle ne peut jamais dépasser la station voisine, ce qui éviterait l'auto-croisement documenté dans *Colonne satin §Sous-couches et compensation*. Une extension (`amount > 0`) reste **non bornée**.
4. **Milieux** (`mids`) et longueur cumulée médiane (`cumMid`) sont calculés **après** les trois étapes précédentes — donc sur la géométrie déjà terminée/rentree/étendue.
5. **Sous-couches** (`center_underlay`, `underlay_edge`, `underlay_zigzag`, §6.6), dans l'ordre `center` → `edge` → `zigzag`, chacune une passe `Underlay` distincte : la sous-couche centrale est ré-échantillonnée à `underlay_spacing` (2 mm par défaut, **pas** à `density`, correctif détaillé dans *Colonne satin §Défaut trouvé en usage réel*) via `point_at` sur `mids/cumMid`. L'`edge walk` décale chaque point vers l'intérieur de `min(underlay_edge_inset, demi-largeur \times 0,9)`. Le `zigzag` prend un fil sur `stride = round(underlay_zigzag_spacing / density)` (indices, pas longueur d'arc — approximation valable car les fils sont déjà espacés régulièrement par construction, sauf immédiatement après un `jump_before`).
6. **Couche supérieure** : pour chaque fil non `dropped`, si `jump_before`, l'indice courant est ajouté à `result.jump_before` (§6.4) ; **la compensation pull latérale est calculée ici, pas avant** — donc les sous-couches (étape 5) voient la largeur **avant** compensation, la couche supérieure après. `split_stitch` (§6.6) fractionne toute traversée `a→b` plus longue que `max_stitch_length` (7 mm par défaut).

6.4. Rupture de ruban : `non_ribbon_interval` et `jump_before`

Entre deux barreaux consécutifs, `non_ribbon_interval(a0, a1, b0, b1)` (`libs/stitch_generation/src/satin.cpp`) compare la **direction d'avance** de chaque rail (`a1.pa - a0.pa` vs `a1.pb - a0.pb`), pas leur largeur : si les deux avances dépassent `kJumpMinSpan` (800 µm — sous ce plancher, la variation angulaire n'est pas significative) et que l'angle entre les deux directions dépasse `kJumpDegPerMm` (10°/mm) **multiplié par la distance moyenne parcourue en mm**, la colonne cesse de se comporter comme un ruban sur cet intervalle. Le seuil est un **rapport angle/distance**, pas un angle brut — le détail de calibration (deux défauts réels distincts, un seuil brut se révélant incapable de distinguer un virage légitime étalé sur ~14 mm d'un défaut concentré sur 1,5-3,7 mm) est documenté dans *Colonne satin §Seuil angle/distance plutôt qu'angle brut*.

Quand c'est le cas, `satin_stations` n'interpole **aucune** station intermédiaire sur cet intervalle et marque le barreau d'arrivée `jump_before = true`. `fill_satin_columns` reporte cet indice dans `SatinResult::jump_before`. `generate_satin` transmet ce vecteur à `emit_polyline_with_breaks` (§3), qui insère un `Jump` avant ces points précis plutôt que d'enchaîner un `Stitch` continu — le fil est levé, pas forcé à traverser la forme en diagonale.

Réorientation et `jump_before` : quand `generate_satin` inverse la colonne (§6.6, entrée/sortie), les indices de `jump_before` doivent être recalculés — un index `i` dans la séquence d'origine correspond à `n - i` après inversion (`n` = taille totale), car c'est l'**arête** (pas le point) qui doit rester marquée comme sautée, et l'arête (`i-1, i`) devient (`n-i, n-i-1`) après inversion. Le code recalcule chaque indice puis retre le vecteur (`generate.cpp`, fonction `generate_satin`) — sans ce retri, l'ordre croissant qu'`emit_polyline_with_breaks` suppose serait rompu.

6.5. Points de fixation (`lock_stitches`, Lot 5)

`lock_stitches(anchor, toward, type, length, passes)` (`libs/stitch_generation/src/lock.cpp`) construit une polyligne courte ancrée en `anchor`, orientée vers `toward` (le point voisin — pour un lock de colonne satin, c'est le point du **rail opposé**, donc la largeur locale de la colonne à cet endroit, pas un point lointain le long de la couture). La longueur effective `L` est bornée : $L = \min(\text{length}, n)$ où $n = |\text{toward} - \text{anchor}|$, sauf dans le cas dégénéré `anchor == toward` où $L = \text{length}$ sans autre référence géométrique — correctif détaillé dans *Colonne satin §Entrée/sortie et points de fixation*, qui évitait qu'un lock pique hors de la matière déjà cousue sur une colonne plus fine que `lock_length` (défaut 0,8 mm). Trois types, chacun répété `passes` fois (2 par défaut) :

- **BackAndForth** : `anchor` → `P(L, 0)` → `anchor`, répété.
- **Triangle** : `anchor` → `P(L, 0)` → `P(L·0, 5, L·0, 6)` → `anchor`, répété — un petit triangle d'ancrage.
- **MicroZigzag** : `anchor` puis 2·`passes` points alternant de côté (`P(L·0, 4· $\frac{k}{2}$ , ±L·0, 5)`), progressant le long de l'axe, puis retour à `anchor`.

`generate_satin` émet, dans cet ordre : sous-couches → lock de début (passe `lock`, entre `result.satin.front()` et `result.satin[1]`, si `lock_start != None`) → couche supérieure (avec ses éventuels `jump_before`) → lock de fin (entre `result.satin[n-1]` et `result.satin[n-2]`, si `lock_end != None`). Un seul lock par extrémité, jamais par sous-passe.

6.6. Entrée/sortie, et tableau des paramètres du satin

Si `entry_point` et/ou `exit_point` sont définis, `generate_satin` compare le coût "sens normal" (`|entry - satin.front()| + |exit - satin.back()|`) au coût "sens inversé" (`|entry - satin.back()| + |exit - satin.front()|`) ; le sens inversé l'emporte strictement en cas d'égalité non stricte du côté normal (`reversed < normal`). Inverser la séquence exige de recalculer `jump_before` (§6.4). Sans point d'entrée ni de sortie, l'orientation issue du générateur (celle des rails tels qu'orientés dans le document) est conservée telle quelle.



6.7. Routage multi-colonnes (route_columns, Lot 6)

Quand `generate_sequence` regroupe plusieurs colonnes satin routables contiguës de même couleur/source (§3 point 4), `generate_satin_group` (`generate.cpp`) construit un `RouteColumn{id, start, end, start_junction, end_junction}` par objet — `start/end` viennent de `column_endpoints(params)` (milieux des barreaux d'about, **avec le même décalage push_start/push_end que celui réellement appliqué par fill_satin_columns**, même formule de borne : reproduire un point différent de celui effectivement cousu fausserait la décision de routage, cf. le correctif détaillé dans *Colonne satin §Routage multi-colonnes*) — et `start_junction/end_junction` viennent de `SatinParams::topology` (`SatinSectionTopology::start_junction/end_junction`, un identifiant de jonction **local au réseau porté par source_vector**, absent pour une extrémité libre ou une colonne indépendante/legacy).

`route_columns(columns, origin, RoutingConfig)` (`libs/stitch_generation/src/routing.cpp`) :

1. **Ordre glouton** (`greedy_order`) : depuis la position courante, choisit l'extrémité libre la plus proche — mais une extrémité qui **partage une jonction** avec la sortie courante (même id, distance $\leq$ `underpath_max`) l'emporte systématiquement sur une simple proximité, même plus courte.
2. **Orientation optimale** (`best_orientation`) : pour l'ordre retenu, programmation dynamique sur les deux orientations possibles de chaque colonne (`std::array<OrientationCost, 2>`) — maximise d'abord le nombre de liaisons validées par une jonction commune, minimise ensuite la distance totale parcourue. Déterministe, résolu **exactement** (pas une heuristique).
3. **Amélioration 2-opt** (si `config.two_opt`, activé par défaut) : inversion de sous-segments de l'ordre tant que le coût diminue, avec une marge anti-oscillation de 1 μ m, bornée à 64 itérations de balayage complet.
4. Pour chaque étape, le **type de liaison** (`ConnectorKind`) avec la colonne précédente est décidé par la distance entre le point de sortie précédent et le point d'entrée courant : - une **jonction commune validée** (même id des deux côtés, ET distance $\leq$ `underpath_max` — 8 mm par défaut) autorise un `Underpath` jusqu'à cette même borne de 8 mm ; - en son **absence**, seul un quasi-contact sous `underpath_max_without_junction` (1,5 mm par défaut, nettement plus strict) autorise un `Underpath` ; - au-delà de la borne applicable, la liaison devient un `Jump` (avec coupe implicite — `ConnectorKind::Jump` porte déjà cette sémantique dans sa propre définition, il n'existe pas de "trim" distinct à ce niveau).

`ConnectorKind::Start` marque uniquement la toute première colonne du groupe (simple pose du fil, jamais de décision `Underpath/Jump`). Le vocabulaire diffère légèrement de celui employé par la spécification SGSD (`libs/satin_planning/`, voir *Colonne satin §Phase 9*) — "travel run" y désigne ce que ce module appelle `Underpath` — mais recouvre exactement le même mécanisme ; SGSD ne réimplémente aucune logique de routage, il construit des `RouteColumn` et délègue à ce même `route_columns`.

Émission (`generate_satin_group`) : pour chaque étape du plan, la colonne est générée normalement (`generate_satin` dans une séquence temporaire, avec `entry_point/exit_point` imposés par `step.reversed`) puis :

- si `step.connector == Underpath` et qu'une commande précédente existe déjà, un segment [`position courante` → première pénétration de la colonne] est échantillonné par `sample_path` (cible 2,5 mm, minimum 500 μ m), et **seuls les points intermédiaires** (ni le premier ni le dernier, déjà couverts respectivement par la position courante et la colonne elle-même) sont ajoutés en `Stitch/Travel` — puis la colonne est enchaînée **sans** son `Jump` de tête (`tmp.commands[1..]`) ;
- sinon (début de groupe, ou liaison trop longue), la colonne est ajoutée telle quelle, `Jump` de tête compris.

7. Ordre de couture et export — où ce moteur s'arrête

`optimization::optimize_order` (`libs/optimization/`, voir *Algorithmes §Ordre de couture*) n'est **pas** un post-traitement de la séquence de points : il réordonne la **liste d'objets du document** (`project.embroidery_objects`, via la commande annulable `ReorderEmbroideryCommand`, `MainWindow::applyOrderStrategy`), les objets verrouillés (`EmbroideryObject::locked`) restant à leur position. La génération de points est ensuite **relancée** sur ce nouvel ordre — il n'existe aucun mécanisme qui réordonnerait un `stitch::StitchSequence` déjà produit. Concrètement, dans le desktop, `applyOrderStrategy` exécute la commande de réordonnement puis appelle `refreshImage()`, qui appelle `stitch_generation::refresh_context` (donc `effective_sequence`, §1) sur le projet déjà réordonné.

L'export DST (`formats::write_dst_file`, voir *Format DST*) consomme directement la séquence effective ainsi obtenue (`MainWindow::exportDst`, membre `sequence_` rempli par `refreshImage`) — la génération de points est donc strictement **en amont** de l'ordre optimisé (qui agit sur le document avant régénération) et **en amont** de l'export (qui ne transforme plus la séquence).

8. Retouches manuelles : ce que `apply_manual_overrides` touche exactement

Détaillé pour l'utilisateur dans *Retouche des points et de la géométrie* ; ce qui suit est le contrat exact côté moteur, nécessaire pour comprendre pourquoi `effective_sequence` est incontournable (§1).

Une retouche (`document::StitchOverride{base_index, moved_to, forced_type, trim_after}`) cible un index dans la **vue brute** d'un objet (`raw_slice(sequence, object.id)`) — toutes les commandes de la séquence brute dont `source == object.id`, dans l'ordre de production, **toutes passes confondues**, qu'elles soient contiguës ou entrelacées comme dans un satin routé). Seules les entrées de passe `TopStitch` sont éligibles (`is_topstitch_entry`) — les sous-couches, trajets cachés et locks restent entièrement régénérés, jamais retouchables :

- `moved_to` exige en plus une cible `Stitch` (`is_movable_point`) — déplacer un saut n'a pas de sens, sa position est déterminée par ses deux voisins de trajet ;
- `forced_type` (basculer `Stitch↔Jump`, bidirectionnelle) et `trim_after` acceptent une cible `Stitch` **ou** `Jump`.

Chaque champ d'un `StitchOverride` est validé **indépendamment** : un champ invalide pour sa cible est ignoré silencieusement, sans rejeter les autres champs du même override. Une commande modifiée passe en passe `Manual`. Les insertions de `Trim` sont différées et appliquées après coup, triées par index décroissant, pour qu'aucune insertion ne décale un index restant à traiter.

`classify_edit_state(object, raw)` compare la taille et l'empreinte FNV-1a 64 bits (fingerprint, sérialisation little-endian explicite de position/ type/passe/source de chaque commande — reproductible bit à bit, indépendante de l'architecture hôte) de la vue brute **actuelle** contre `edited_point_count/edited_fingerprint` enregistrés lors de la dernière retouche réussie : identiques → `ManuallyEdited` (les retouches s'appliquent) ; différents → `Dirty` (la géométrie source a changé depuis, les retouches ne sont **plus jamais réappliquées** tant que l'utilisateur ne les abandonne pas explicitement) ; overrides vide → `Clean`. Rien n'est mis en cache : cet état est recalculé à chaque appel depuis le document et la séquence brute actuelle.

`edit_view`, `classify_all_edit_states` et `refresh_context` sont des compositions de ces mêmes primitives pour l'UI desktop (indicateurs `Clean/ManuallyEdited/Dirty`, poignées d'édition) — `refresh_context` en particulier consolide un **seul** appel interne à `generate_sequence` pour produire à la fois la séquence effective, les états par objet et la vue d'édition d'une cible, plutôt que jusqu'à trois régénérations indépendantes par rafraîchissement (`MainWindow::refreshImage`).

9. Limites connues (au niveau du moteur, au-delà de ce que documentent déjà satin.md/tatami.md)

- **SatinCapType::Automatic ne fait rien de différent de Flat** dans `cap_factor` (§6.3 pt 1) — la terminaison "automatique" promise par l'inspecteur n'a aucune implémentation propre à ce jour.
- **center_underlay a deux défauts différents** selon le niveau regardé : `false` dans `SatinConfig` (le type interne au moteur), `true` dans `document::SatinParams` (le type exposé par l'inspecteur) — sans conséquence pratique puisque le document fournit toujours une valeur explicite, mais un piège pour quiconque lit `SatinConfig` isolément en pensant y trouver le comportement par défaut réel d'un satin créé dans l'application.
- **Le zigzag de sous-couche satin** (`underlay_zigzag`) espace ses points par un **stride en nombre de fils** (`round(underlay_zigzag_spacing / density)`), pas par une distance réellement mesurée le long de la médiane — une approximation raisonnable tant que les fils restent régulièrement espacés, mais fausse localement juste après un `jump_before` (§6.4), où l'espacement réel entre deux fils consécutifs peut s'écarter fortement de `density`.
- **Le trajet caché d'un routage satin (underpath) reste un segment direct échantillonné**, jamais un suivi du centre d'une colonne déjà cousue — la garantie de rester "sous la broderie" vient uniquement de la borne de distance (jonction validée ou quasi-contact), pas d'un vrai calcul de trajet le long de la matière (limite déjà posée dans *Colonne satin §Routage multi-colonnes*, confirmée ici au niveau du code d'émission).
- **Le contrat raw-sequence-ok** ne protège que les appels textuellement détectables par le script CMake (`generate_sequence` sur une ligne) : un appel indirect via un alias de fonction ou un pointeur de fonction ne serait pas détecté. Aucun cas de ce genre n'existe actuellement dans le dépôt (vérifié par lecture des deux seuls sites annotés, §1), mais ce n'est pas une garantie statique complète, seulement une convention outillée.

Implémentation associée

- `libs/stitch/include/openstitch/stitch/sequence.hpp` — `StitchCommand`, `CommandType`, `StitchPass`, `StitchSequence`, `StitchStats`.
- `libs/stitch/src/stats.cpp` — `compute_stats`.
- `libs/stitch_generation/include/openstitch/stitch_generation/generate.hpp` + `src/generate.cpp` — `generate_sequence` (orchestration, dispatch, détection de groupe satin routable), `emit_polyline`, `emit_polyline_with_breaks`, `emit_fill`, `generate_running`, `generate_tatami`, `generate_satin`, `generate_satin_group`, `column_endpoints`, `is_routable_satin`.
- `libs/stitch_generation/include/openstitch/stitch_generation/overrides.hpp` + `src/overrides.cpp` — `effective_sequence`, `apply_manual_overrides`, `raw_slice`, `fingerprint`, `classify_edit_state`, `is_movable_point`, `edit_view`, `classify_all_edit_states`, `refresh_context` (Lot 8/ADR-014).
- `libs/stitch_generation/include/openstitch/stitch_generation/running_stitch.hpp`
- `src/running_stitch.cpp` — `run_stitch`, `sample_path`, `apply_repeat_mode`, `apply_repeats`, `RunningConfig`, `RunningResult`.
- `libs/stitch_generation/include/openstitch/stitch_generation/tatami.hpp` + `src/tatami.cpp` — `fill_tatami`, `tatami_underlay`, `segment_stays_in_region`, `connector_invalid`, `in_region`.
- `libs/stitch_generation/include/openstitch/stitch_generation/satin.hpp` + `src/satin.cpp` — `SatinConfig`, `SatinResult`, `SatinStation`, `satin_stations`, `fill_satin`, `fill_satin_columns`, `default_rungs`, `rails_from_contour`, `non_ribbon_interval`, `cap_factor`, `ladder_correspondence`, `resample_by_medial_spacing` (voir *Colonne satin* pour l'algorithme d'appariement).

- `libs/stitch_generation/include/openstitch/stitch_generation/lock.hpp + src/lock.cpp` — `lock_stitches`, `LockType`.
- `libs/stitch_generation/include/openstitch/stitch_generation/routing.hpp + src/routing.cpp` — `route_columns`, `RoutePlan`, `RouteColumn`, `RouteStep`, `ConnectorKind`, `RoutingConfig`, `greedy_order`, `best_orientation`.
- `libs/stitch_generation/src/satin_guides.cpp` — édition interactive des guides/barreaux (`satin_guide_junctions`, `move_satin_guide_group`, etc.), hors périmètre de la génération elle-même.
- `libs/document/include/openstitch/document/embroidery_object.hpp` — `RunningStitchParams`, `TatamiParams`, `SatinParams`, `SatinRung`, `SatinSectionTopology`, `SatinLock`, `StitchOverride`, `StitchPointType`, `EmbroideryObject`.
- `libs/optimization/include/openstitch/optimization/order.hpp + src/order.cpp` — `optimize_order`, `OrderStrategy`, `compute_cost` (réordonnancement des objets du document, en amont de la génération).
- `apps/desktop/main_window.cpp` — `refreshImage` (appel à `refresh_context`), `applyOrderStrategy` (réordonnancement puis régénération), `exportDst`, `buildDebugDump/showDebugDump` (dump `pass=...` `type=...` `pos=...` par commande, histogramme par passe).
- `apps/cli/main.cpp` — `run_stitchdebug`, générateur de debug du routage satin : les deux seuls sites annotés `raw-sequence-ok`: du dépôt.
- `tests/check_no_raw_sequence_bypass.cmake` — garde structurelle CTest du contrat `effective_sequence`.
- Tests : `tests/unit/stitch/test_generate.cpp`, `test_overrides.cpp`, `test_running_stitch.cpp`, `test_tatami.cpp`, `test_satin.cpp`, `test_satin_pairing_metrics.cpp`, `test_satin_guides.cpp`, `test_routing.cpp`, `test_stats.cpp`.
- Chapitres associés : *Génération de points — point droit et fondations*, *Remplissage tatami*, *Colonne satin*, *Retouche des points et de la géométrie*, *Format DST*, *Algorithmes*, *Objets de broderie*.

Colonne satin

Public : utilisateur avancé, développeur.

État : Présent dans le code : oui · Tests unitaires : oui · Tests visuels : partiels · Import/export DST : oui · Test sur machine réelle : **non** · **Statut recommandé : expérimental** — génération satin simple à deux rails. Plusieurs propriétés d'un générateur satin correct manquent (voir *Limitations* ci-dessous). Vérifiez impérativement le résultat avant tout passage sur machine.

Définition

Une colonne satin est un zigzag serré entre **deux rails** (bords gauche et droit) qui remplit une bande. Elle donne un effet lisse et brillant, adapté aux lettrages et aux bordures étroites.

Modèle

SatinParams porte rail_a et rail_b (deux geometry::Path), la densité (density), la compensation de tirage (pull_compensation), et l'activation d'une sous-couche centrale (center_underlay).

Génération

fill_satin(rail_a, rail_b, config) :

1. calcule une **correspondance locale monotone** entre les deux rails (ladder_correspondence, voir *Correction de l'appariement* ci-dessous) — les rails peuvent avoir des longueurs, courbures et échantillonnages différents ;
2. ré-échantillonne cette correspondance par pas fixe le long de sa **ligne médiane** ($\approx$ perpendiculaire aux fils), et produit un **zigzag alterné** d'un bord à l'autre (L0, R0, L1, R1, ...) ;
3. la **densité** fixe ce pas d'avancement, mesuré sur la ligne médiane (donc proche de la perpendiculaire aux fils, y compris en section inclinée ou courbe) ;
4. la **compensation de tirage** écarte les deux rails le long de leur médiatrice, pour compenser le resserrement du fil ;
5. la **sous-couche centrale** (optionnelle) est un point droit grossier sur l'axe, cousu avant le zigzag, dérivée de la même correspondance.

Le résultat (SatinResult) fournit les points de sous-couche, du satin, et la **largeur maximale** rencontrée (pour l'avertissement).

Paramètres

Valeurs par défaut lues dans SatinParams.

Paramètre	Unité	Défaut	Effet
density	mm	0,4	pas d'avancement le long de la colonne (plus petit = plus dense)
pull_compensation	mm	0	élargit la colonne (compense la traction du fil)
center_underlay	bool	vrai	ajoute une sous-couche centrale (point droit sur l'axe)
max_width	mm	9	seuil d'avertissement de largeur excessive

Validation physique : non effectuée. La densité satin devrait idéalement se mesurer perpendiculairement au fil (voir l'avertissement ci-dessus).

Correction de l'appariement rail gauche / rail droit (audit rails, 2026-08-01)

État : Présent · Testé numériquement (fixtures géométriques dédiées) · non validé simulateur/physique.

Défaut trouvé. L'ancien appariement (`fill_satin` sans barreaux, et l'interpolation à l'intérieur d'un intervalle entre deux barreaux dans `fill_satin_columns`) associait les deux rails par la **même fraction d'abscisse curviligne** appliquée indépendamment à chacun (ou, avec barreaux, la même fraction u sur tout l'intervalle). Dès que les deux rails divergent en longueur ou en courbure — virage, coude, largeur variable — cette hypothèse de proportionnalité est fautive : les fils cessent d'être localement perpendiculaires au ruban. Symptôme observé : orientation des points en retard sur la direction locale des rails, éventails, quasi-croisements et densité visuelle irrégulière dans un ruban courbe ou anguleux.

Correction. Remplacé par une correspondance locale **"ladder"** (triangulation de bande par la diagonale la plus courte entre deux chaînes — technique classique de géométrie algorithmique, indépendante de tout logiciel de broderie) :

1. chaque rail est sous-échantillonné entre deux ancrages exactes (barreaux, ou les deux extrémités de colonne si aucun barreau) à une résolution bornée (`maxStep`, fonction de la densité) — donne assez de détail même si les sommets d'origine sont éparés (rail dessiné à la main, contour simplifié) ;
2. à chaque pas, la correspondance avance sur le rail dont le **prochain sommet forme la diagonale la plus courte** avec le point courant de l'autre rail (`ladder_correspondence`) — un garde-fou rejette une avance qui croiserait la diagonale précédente et force l'autre côté ;
3. cette correspondance dense est ré-échantillonnée par pas fixe le long de sa **ligne médiane** (`resample_by_medial_spacing`) : l'interpolation de (s_a , s_b) se fait dans le pas LOCAL du ladder qui encadre chaque cible, jamais sur tout l'intervalle — l'erreur reste bornée par la résolution du ladder, pas par la longueur de l'intervalle.

Invariants garantis (vérifiés par fixtures, voir ci-dessous) :

- **Monotonie** : les indices/abscisses avancent sur chaque rail dans le même sens que le parcours, jamais en arrière.
- **Absence de croisement** : deux fils consécutifs (et, mesuré sur les fixtures, deux fils quelconques) ne se croisent jamais.
- **Stabilité à l'échantillonnage** : la correspondance ne dépend pas du nombre de sommets d'origine de chaque rail (rails de résolutions très différentes acceptés).
- **Déterminisme** : aucune donnée non déterministe, résultat identique à entrée identique.
- **Complexité** : $O(n_A + n_B)$ par intervalle entre deux barreaux (linéaire) — la ladder ne revisite jamais un sommet, comme la triangulation de bande classique dont elle s'inspire.

Limite assumée : à un coude **C0 franc** (angle vif sans congé), la notion de « normale locale » est elle-même discontinue ; un seul fil au sommet du coude absorbe nécessairement une déviation angulaire importante. Aucun appariement ne peut faire autrement sans mitre/congé explicite (relève de `ShortStitchMode`, hors périmètre de l'appariement de base). Ce que la correction garantit dans ce cas : la déviation reste localisée à ce fil unique (pas d'éventail étendu sur plusieurs fils) et aucun croisement n'apparaît.

Fixtures et métriques (`tests/unit/stitch/test_satin_pairing_metrics.cpp`, 13 tests, indépendants des tests de non-régression `test_satin.cpp`) : ruban droit (cas trivial), ruban en S, coude à 90° franc, largeur variable, cas combiné courbe+coude+largeur (inspiré d'un ruban capturé par l'utilisateur), plus la même correspondance via `fill_satin_columns` (barreaux), et 7 fixtures adverses (revue corrective ci-dessous). Chaque test mesure croisements (adjacents et toute paire), monotonie sur les deux rails, angle fil/normale locale (moyenne + pire cas), continuité angulaire entre fils consécutifs, régularité de la densité médiane, longueurs min/max, et déterminisme. Une réplique isolée de l'ancien algorithme (jamais utilisée en production) sert de référence comparative dans les mêmes tests. Résultats mesurés : ruban en S, angle

max fil/normale 2,7° (nouveau) contre 58,4° (ancien), angle moyen 1,2° contre 33,5° ; coude à 90°, 0 croisement (nouveau) contre 3 (ancien) ; cas combiné, angle max 4,9° contre 44,3°, 0 croisement dans les deux cas sur cette fixture précise mais éventail moyen réduit d'un facteur 12 (2,0° contre 24,0°).

Revue corrective (audit adverse, 2026-08-01)

État : Présent · Testé numériquement (fixtures adverses dédiées) · non validé simulateur/physique.

Audit ciblé de `ladder_correspondence` avec des fixtures délibérément défavorables (rails tête-bêche, échantillonnage très asymétrique avec doublons/segments nuls, longueurs très différentes + largeur quasi nulle, épingle à cheveux ~170°, barreaux désordonnés/dupliqués). Deux défauts réels trouvés et corrigés, un comportement clarifié :

- **Rails fournis tête-bêche** (bout 0 de A proche du bout N de B) : le commentaire de `fill_satin` exigeait déjà des rails « orientés dans le même sens », mais rien ne le vérifiait — un appel avec des rails inversés produisait un nœud papillon (chaque fil tend vers la diagonale opposée, croisements en $O(n^2)$) au lieu d'un ruban. Aucun chemin de production actuel ne peut produire ce cas (`rails_from_contour` et `auto_satin::build_satin_columns` garantissent déjà le même sens), mais c'est une précondition silencieuse d'une fonction de bibliothèque publique — risquée pour un futur appelant (import, script, saisie manuelle). Corrigé par une détection bon marché (`opposite_orientation` : compare la somme des distances bout-à-bout dans les deux sens possibles) et une ré-orientation interne automatique, dans `fill_satin` et `fill_satin_columns`.
- **Barreaux non triés** : `fill_satin_columns` ne gardait un barreau que si sa projection avançait sur les deux rails par rapport au **dernier barreau GARDÉ dans l'ORDRE DU VECTEUR D'ENTRÉE** — un barreau placé en tête du vecteur mais loin le long de la colonne verrouillait cette référence et faisait rejeter silencieusement tous les barreaux suivants, repliant sur `fill_satin` sans barreaux (alors que la doc promet des barreaux toujours traversés exactement). Un barreau est une station transversale, pas un élément de séquence : il n'y a aucune raison que son ordre dans le vecteur soit significatif. Corrigé par un tri par position projetée avant le filtre anti-croisement — le résultat ne dépend plus de l'ordre d'entrée des barreaux (vérifié : résultat bit-à-bit identique entre un vecteur trié et le même mélangé).
- **Barreaux dupliqués/quasi-dupliqués** : deux barreaux dont la projection n'avance que d'une quantité minuscule (mais non nulle) sur les deux rails créaient un intervalle dégénéré. Comme le garde-fou anti-croisement de `ladder_correspondence` n'agit qu'à L'INTÉRIEUR d'un intervalle (jamais entre deux intervalles voisins), les deux fils-ancres qui encadraient cet intervalle dégénéré n'étaient jamais comparés l'un à l'autre — s'il tombait près d'un virage serré, ils pouvaient se croiser (reproduit et corrigé, fixture "barreau dupliqué/quasi-dupliqué"). Corrigé en fusionnant (après tri) tout barreau dont la projection avance de moins de la moitié du pas de densité (`anchorMinGap = density / 2`) par rapport au dernier barreau gardé — élimine la cause (l'intervalle dégénéré) plutôt que de tenter de détecter le croisement a posteriori.
- **Échantillonnage très asymétrique / segments nuls / longueurs très différentes / largeur quasi nulle / épingle à cheveux serrée (~170°)** : déjà robustes sans modification — aucun croisement, monotonie et déterminisme préservés, aucune coordonnée dégénérée (vérifié par bornes larges plutôt que `isfinite` sur les micromètres entiers, qui ne peuvent pas représenter NaN).

Complexité vérifiée : le tri des barreaux ajouté est $O(m \log m)$ avec m = nombre de barreaux (petit, jamais lié à la résolution des rails) — ne change pas la complexité globale. Mesure empirique (release, ruban sinusoïdal 8 périodes, densité fixe) : temps de `fill_satin` stable (~6 ms) et nombre de fils constant pour un nombre de sommets d'entrée par rail variant de 500 à 8000 — confirme l'absence de comportement quadratique cachée dans le nombre de sommets d'origine.

Fixtures ajoutées (mêmes métriques que ci-dessus) : rails tête-bêche, longueurs très différentes + largeur quasi nulle, épingle à cheveux (mesure `crossings_total`, pas seulement `crossings_adjacent`, pour couvrir un croisement non adjacent), barreaux désordonnés (comparaison bit-à-bit avec la version triée), barreau dupliqué/quasi-dupliqué. S'ajoutent à la fixture d'échantillonnage asymétrique avec doublons/segments nuls déjà présente.

Colonne trop large

À la création, si la largeur dépasse le seuil recommandé, l'interface **prévient** et propose de préférer un remplissage tatami — la limite physique n'est jamais masquée.

Rails automatiques

`rails_from_contour(contour)` découpe un contour fermé en deux rails, coupé aux **deux sommets les plus éloignés** (les « bouts » de la colonne).

Débordement — désactivé par défaut. Cette heuristique ne connaît pas l'axe de la forme : sur un contour **concave ou branchu**, les deux rails traversent l'intérieur et les barreaux (rungs) enjambent les creux, ce qui fait **sortir les points de la région**. Mesuré sur un projet réel, jusqu'à **57 % des barreaux d'une colonne hors de sa région**. En conséquence :

- l'auto-numérisation **n'utilise plus** le satin naïf par défaut. Elle essaie d'abord `auto_satin::build_satin_columns` (`use_auto_satin`, défaut `true`) ; une forme refusée retombe sur le **tatami**. L'option `AutoOptions::use_naive_satin` (défaut `false`) ne subsiste que comme repli explicite pour essais/tests ;
- l'action **Broderie ■ Convertir les satins auto en tatami** répare un projet qui contient déjà des satins débordants ;
- le satin reste créable **manuellement** (menu Broderie, ou clic droit ■ *Type de points* ■ *Colonne satin*) — à vérifier visuellement, c'est la même heuristique de rails.

Limitation : `rails_from_contour` reste une heuristique (utilisée seulement pour la création **manuelle** de satin). La bonne méthode est le moteur géométrique ci-dessous.

Colonnes automatiques par squelette (`build_satin_columns`)

État : Présent · Testé numériquement · Validé visuellement (SVG) · non validé simulateur/physique.

`auto_satin::build_satin_columns(region, params)` construit une ou plusieurs **colonnes éditables** (`SatinColumnGeometry` : deux rails + **barreaux**) depuis une région, via le pipeline : rasterisation → transformée de distance → squelette (Zhang-Suen) → graphe élagué → satinabilité → **sections transversales** → rails → barreaux → validation.

Pour chaque branche du squelette : l'axe est lissé (Chaikin) et ré-échantillonné par longueur d'arc ; à chaque station on calcule la tangente puis la **normale**, on l'intersecte avec le contour et on retient l'**intervalle intérieur encadrant l'axe** (jamais la bounding box, jamais une association par index). Les deux extrémités donnent les rails ; la stabilité gauche/droite vient du signe de la normale (le rail A est toujours à gauche du sens de parcours). Les barreaux sont posés aux extrémités, aux virages, aux changements de largeur et à intervalle maximal. Un nettoyage anti-croisement retire les stations qui replieraient la colonne.

Extension des bouts ouverts jusqu'au bord réel

État : Présent · Testé numériquement · Validé visuellement (SVG). Activé par défaut (`SatinColumnsParameters::extend_open_ends`), rétrocompatible.

Défaut trouvé par revue (mission « auto-satin béton », audit du squelette existant confronté à la littérature — voir *Sources* ci-dessous) : un bout **ouvert** du squelette (sans jonction) s'arrête, par construction de l'amincissement (Zhang-Suen érode la forme progressivement depuis le bord), sensiblement **avant** le bord réel de la région. Démontré visuellement sur capsule (rectangle de 40 mm + deux demi-cercles de rayon 2,5 mm, longueur bout-à-bout réelle 45 mm) : avant correction, la colonne s'arrêtait à ~38,8 mm — les deux embouts arrondis (~ 11 % de la longueur totale) restaient **entièrement hors couture**, un rectangle purement rendu vide au bout de chaque colonne. Le même retrait (proportionnel à la demi-largeur locale) affecte aussi un bout **carré** : sur rectangle (bouts plats, 40 mm), le retrait mesuré est de ~2,55 mm par bout — un défaut générique de l'amincissement, pas spécifique aux formes arrondies.

Correction (`extend_tip` dans `satin_column.cpp`) : pour chaque bout **sans jonction** (un bout de jonction reste inchangé — il doit rester exactement au nœud du squelette pour la reconciliation multi-sections), on

marche depuis la dernière station le long de la tangente sortante par pas de `station_spacing`, en ré-évaluant une section transversale réelle (intersection avec le contour, pas une approximation) à chaque pas tant qu'elle continue de **rétrécir** (tolérance 5 % contre une marche qui déborderait dans une autre partie de la forme). La fermeture finale est localisée par **bissection** sur un test point-dans-région pur (robuste même quand la section transversale devient numériquement dégénérée tout près du bord), avec une marge de sûreté de quelques μm avant l'arrondi final en micromètres entiers — sans cette marge, la bissection converge si près du bord analytique que l'arrondi peut faire retomber le point de l'autre côté du polygone discrétisé (trouvé par un test en échec, corrigé avant commit). Le point de fermeture reçoit une **largeur plancher non nulle** (`tip_min_width`, 0,05 mm par défaut) plutôt qu'un barreau littéralement nul, pour rester exploitable tel quel par `fill_satin_columns`. Marche et bissection sont toutes deux **bornées** (200 pas / 24 bisections) : jamais de boucle infinie, même sur une géométrie pathologique. Les rails **se terminent désormais sur le contour réel** → plus de débordement structurel ET plus de zone non couverte.

Vérifié : la longueur bout-à-bout d'une colonne capsule passe de ~38,8 mm à 45,0 mm (± 1 mm) — la valeur géométrique exacte ; un bout carré (`rectangle`) est également corrigé (40,0 mm au lieu de ~34,9 mm) ; les bouts de jonction d'un réseau Y restent géométriquement identiques désactivé/activé (seuls les bouts ouverts s'allongent) ; aucun barreau dégénéré après extension ; extension déterministe ; le bascule `extend_open_ends=false` restaure l'ancien comportement (SVG avant/après dans `tests/golden/auto-satin/columns-*.svg`).

Sources. Ce mécanisme reprend, en le réimplémentant intégralement à partir des principes documentés (aucun code tiers consulté ni copié — projet Apache-2.0, jamais de dépendance à du code GPL) : le brevet Pulse Microsystems US6804573B2 « *Automatically generating embroidery designs from a scanned image* » (Google Patents), qui décrit l'extension du nœud terminal du squelette « dans la direction de la branche squelettique entrante », proportionnellement à l'épaisseur locale, pour que le tracé final couvre l'embout entier d'un objet fin. Le même brevet motive deux pistes non retenues dans ce lot (portée limitée à la couverture des bouts) : la classification fin/épais par statistiques de transformée de distance (max/moyenne/écart-type le long du squelette, plus riche que nos seuils actuels largeur min/max) et l'ancrage des jonctions sur les concavités réelles du contour plutôt que sur la seule topologie du squelette. Voir aussi Bai, Latecki & Liu, « *Skeleton Pruning by Contour Partitioning with Discrete Curve Evolution* », IEEE TPAMI 2007 (PDF) pour un élagage de squelette plus fidèle à la forme que notre seuil longueur/rayon actuel (`graph_cleanup.cpp`) — piste de travail future, non implémentée ici.

Ancrage des jonctions sur les sommets reflex du contour

État : Présent · Testé numériquement · Validé visuellement (SVG). Activé par défaut (`SatinColumnsParameters::anchor_junction_ends`), rétrocompatible.

Défaut trouvé par revue (suite de la mission « auto-satin béton », piste « ancrage des jonctions » du même brevet). Mesuré précisément sur y : la largeur d'une branche reste stable (~5,0 mm) loin de sa jonction, mais **dérive nettement** sur les toutes dernières stations avant le nœud du squelette — jusqu'à ~9,2 mm sur la dernière station de la branche principale, une croissance quasi linéaire station après station. Cause : la section transversale d'une branche est calculée depuis sa **seule tangente locale** ; près d'une confluence, plusieurs branches se recouvrent physiquement, et le rayon perpendiculaire balaie alors ce **bourrelet de la confluence** — pas la ceinture réelle de cette branche seule. Conséquence démontrée : les 3 sections d'un Y ne se rejoignent PAS au même point — jusqu'à ~5 mm d'écart entre deux rails censés coïncider exactement à la confluence, laissant un vide ou un repli selon les branches.

Correction (`trim_and_anchor_junction_end` dans `satin_column.cpp`) en deux temps, pour chaque bout de JUNCTION (jamais un bout ouvert — traité par l'extension des bouts ouverts, § précédente) :

1. **Amputation** de la queue instable : les stations terminales sont retirées tant que leur largeur dépasse de plus de 10 % celle de leur voisine plus intérieure — signature précise de la dérive mesurée (croissance nette et soudaine, à distinguer d'une variation progressive légitime ailleurs dans la colonne, jamais touchée par cette règle).
2. **Ancrage** indépendant de chaque rail sur le **sommet reflex** (concave) du contour le plus proche, dans un rayon borné (`junction_anchor_radius`, 6 mm par défaut) : les sommets reflex sont exactement les « encoches » où le contour réel bascule d'une branche à sa voisine. Les deux rails d'une même branche

touchent généralement **deux encoches distinctes** — jamais ancrés au même point. Si les deux rails convoient le même sommet (cas réel trouvé sur un « T » où une moitié de barre n'a qu'un seul côté avec une vraie encoche voisine), seul le plus proche le garde ; l'autre cherche son propre sommet **distinct** ou renonce — sans cette règle d'exclusion, les deux rails convergeaient par erreur vers l'unique encoche, créant un barreau de largeur nulle. Sans ancre à portée, un rail conserve sa position amputée (repli sans erreur — toutes les jonctions ne sont pas des étoiles symétriques à n encoches pour n branches).

Vérifié : sur un Y symétrique (3 branches), les 6 extrémités de rail touchant la jonction se regroupent en **exactement 3 sommets, chacun partagé par exactement 2 rails** de sections différentes (`length_um < 5` µm entre les deux) ; sur un T (topologie asymétrique, 2 encoches réelles pour 3 branches), **aucune collision** (jamais 3 rails ou plus au même sommet) et aucun barreau dégénéré ; aucune dérive de largeur résiduelle ($> 1,2 \times$ la médiane) sur aucune section ; déterminisme ; le bascule `anchor_junction_ends=false` restaure l'ancienne dérive (régression volontairement reproduite pour prouver que le bascule agit réellement).

Amas de pixels de jonction et ancrage indépendant par branche (audit jonctions branchées/concaves, 2026-08-03)

État : Présent · Testé numériquement. Correctif de revue, `skeleton_graph.cpp` + `satin_column.cpp`.

Défaut trouvé par revue (capture d'une forme branchée/concave réelle, suite de la mission « ancrage des jonctions » ci-dessus). Deux causes distinctes, une même conséquence visible : les rails venant de deux branches angulairement voisines d'une même confluence ne se raccordent pas exactement — derniers barreaux en éventail, rails terminaux qui ne partagent pas le même point, zone triangulaire mal couverte près du centre.

1. **`build_skeleton_graph` ne consolidait pas les amas de pixels de jonction.** Sur une confluence à 3+ branches — surtout asymétrique —, l'amincissement Zhang-Suen laisse souvent un petit amas de PLUSIEURS pixels adjacents ayant chacun un nombre de croisement ≥ 3 , plutôt qu'un unique pixel net. L'ancien code créait un `SkeletonNode` **par pixel** de cet amas, reliés entre eux par des micro-arêtes internes (`length_um` de quelques dizaines de µm) que `graph_cleanup.cpp` ne pouvait pas élaguer (son seuil ne s'applique qu'aux arêtes **terminales**, degré 1 — une micro-arête entre deux pixels de jonction a ses deux extrémités de degré ≥ 3). Chaque branche incidente se retrouvait donc ancrée sur un nœud légèrement différent selon le pixel de l'amas par lequel son tracé y entraît.
2. **`trim_and_anchor_junction_end` ancrant chaque rail indépendamment**, y compris après la consolidation ci-dessus : chaque rail de chaque branche cherchait pour son propre compte le sommet reflex le plus proche (`reflex_vertices`, liste globale du contour), sans aucune coordination avec les rails des branches VOISINES de la même confluence. Deux branches angulairement adjacentes pouvaient donc élire des sommets différents pour ce qui est géométriquement la MÊME encoche.

Correction, deux volets :

1. `build_skeleton_graph` regroupe désormais les pixels de jonction 8-connexes (union-find) en un seul amas logique avant de créer les `SkeletonNode` : position stable = le pixel de plus grand rayon dans le `DistanceField` au sein de l'amas (départagé par (y, x) croissants pour le déterminisme — toujours un pixel réel, jamais un barycentre flottant), toutes les branches incidentes pointent vers ce même identifiant, et toute micro-arête interne à l'amas (`from == to` après consolidation) est rejetée sans condition.
2. L'ancrage sur sommet reflex est sorti de `build_column` (qui ne fait plus que l'AMPUTATION de la queue instable, `trim_unstable_junction_tail`) et devient une résolution **globale par jonction** (`resolve_junction_anchors`, `satin_column.cpp`), appelée une fois toutes les colonnes construites : les branches incidentes à une jonction sont triées par angle autour de son centre, et chaque ESPACE ANGULAIRE entre deux branches adjacentes reçoit au plus une ancre (le sommet reflex le plus proche du centre dans ce secteur), appliquée **identiquement** aux deux rails qui se font face — le rail gauche (A) de la branche « avant » dans l'ordre angulaire et le rail droit (B) de la branche « après ». Une branche sans encoche à portée dans son secteur garde sa position brute (repli, comme avant — toutes les confluences n'ont pas autant d'encoches que de branches, cf. le T ci-dessus). Défaut annexe trouvé en

cours d'audit : la section transversale brute d'une branche rétrécit naturellement vers zéro du côté qui longe une encoche (géométrie réelle, pas une erreur) ; quand seul le côté OPPOSÉ de cette branche reçoit une ancre partagée, le côté non ancré restait à cette position quasi nulle — barreau terminal dégénéré. Corrigé par un plancher de largeur (`tip_min_width`, même paramètre que la fermeture de `extend_tip`) qui ne déplace QUE le côté non ancré, jamais le côté ancré (qui doit rester exactement coïncident avec la branche voisine).

Une validation finale par jonction (`validate_junctions`, appelée après la résolution des ancrs) refuse proprement la génération de TOUTE la région plutôt que de renvoyer un raccord incohérent : arêtes incidentes manquantes (une branche non construite sur une confluence sinon partiellement ancrée), rails terminaux qui ne coïncident pas et s'éloignent trop du centre (éventail non résolu), croisement entre barreaux terminaux de branches voisines, ou secteur angulaire anormalement large sans aucune ancre commune (trou triangulaire). `extend_tip` reste structurellement impossible côté jonction (un bout de jonction n'emprunte jamais ce chemin, inchangé par cet audit).

Vérifié : suite `test_auto_satin` complète inchangée (`y`, `t`, `cross`, `h` toujours sans collision ni barreau dégénéré) ; nouvelle fixture `trident` (grande branche verticale épaisse, branche interne pointue — triangle effilé, pas une bande à largeur constante — et branche latérale étroite, confluence délibérément non étoilée) : raccord cohérent, aucune collision à 3+ rails, aucun barreau dégénéré, aucun croisement entre les trois barreaux terminaux, tous les barreaux dans la région ; déterminisme.

Remplacement de l'ancrage par des bridges verrouillés, sans mutation de rail (2026-08-03, suite)

État : Présent · Testé numériquement. Remplacement architectural, `satin_column.cpp` + `satin_column.hpp` + `debug_export.cpp`.

Défaut trouvé en usage réel sur `trident` (confluence à 3 branches très inégales — 6 mm / 1,2 mm / pointe effilée) : `resolve_junction_anchors` (§ ci-dessus) reste correct sur les confluences symétriques ou peu contrastées (`y`, `t`, `cross`, `h`), mais sur `trident`, l'ancre trouvée pour un espace angulaire pouvait se situer à plusieurs millimètres de la dernière station réellement stable de la branche. `set_terminal_point` déplaçait alors le dernier nœud du rail **en ligne droite** jusqu'à cette ancre — une corde artificielle traversant l'intérieur de la confluence au lieu de suivre le bord réel de la forme, visible comme une grande diagonale dans le SVG de diagnostic. Le même mécanisme (`tip_min_width` plancher, repli symétrique) souffrait du même défaut de principe : déplacer un point plutôt que construire la géométrie.

Correction : suppression complète de la mutation. `resolve_junction_anchors`, `best_anchor_in_sector`, `boundary_anchor_on_sector_bisector`, `set_terminal_point` et `reflex_vertices` sont retirés. Un bout de jonction n'est plus jamais déplacé après coup : la dernière station stable que `trim_unstable_junction_tail` (inchangée dans son principe) laisse en place devient directement le **JunctionBridge** verrouillé de la colonne — `rail_a_point/rail_b_point` (déjà des points réels du contour, `cross_section` les calcule par intersection avec le bord), `axis_point`, `tangent`, `width_um`. Deux branches angulairement adjacentes ne sont plus forcées à coïncider : le petit vide géométrique qui peut subsister entre leurs bridges respectifs (`JunctionCore`, exposé dans `SatinColumnsResult::junction_cores`, jamais dissimulé) est calculé séparément et n'entraîne PAS de refus — seul un défaut structurel réel (bridge égaré, croisement) refuse encore la génération (`validate_junction_bridges`).

Deux effets de bord de la suppression de l'ancrage, trouvés en généralisant sur `trident` puis sur un réseau en T issu d'une image segmentée réelle :

1. **Le critère de stabilité de `trim_unstable_junction_tail` devait devenir plus robuste.** L'ancien plafond de distance (lié à `junction_anchor_radius`, qui ne bornait à l'origine que le rayon de recherche d'une ancre — notion disparue) arrêta parfois l'amputation alors que la largeur était encore en pleine dérive (bourrelet de `trident` dépassant 6 mm d'abscisse curviligne) ; supprimé, plus aucun plafond de distance. Le critère de largeur lui-même (décroissance stricte station à station) échouait sur un PALIER de largeur (deux stations quasi égales mais toutes deux énormes) — remplacé par une comparaison à `representative_station_width` : la médiane du **tiers médian** des stations de la branche (ni le premier

tiers, qui peut être contaminé par un bout OUVERT qui rétrécit légitimement vers une pointe — `trident`, branche interne effilée — ni le dernier, potentiellement gonflé par le bourrelet de jonction).

2. Un bridge peut légitimement croiser celui d'une branche voisine sans dérive de largeur mesurable, si l'angle entre les deux branches est proche de 90° et que l'une d'elles est nettement plus large que l'autre (la coupe transversale, perpendiculaire à la branche fine, atteint alors le bord de la branche large sans que sa PROPRE largeur ait significativement augmenté).

`resolve_and_validate_junctions` recule alors itérativement d'une station RÉELLE supplémentaire (`retract_bridge_station`, jamais un déplacement) sur les deux branches en conflit, jusqu'à ce que le croisement disparaisse ou que le plancher de stations soit atteint.

Nouvelles primitives internes (diagnostic uniquement, jamais utilisées pour construire ou déplacer un rail) : `ContourPolyline/project_to_contour/extract_contour_arc` (polygone fermé, abscisse curviligne, projection, extraction d'arc) — utilisées par `compute_junction_core` pour approximer le bord réel du noyau central entre deux bridges, et par `columns_to_svg` pour l'afficher (remplissage magenta pointillé) aux côtés des bridges terminaux (barreaux rouges épais) et des identifiants de colonne/jonction.

Vérifié : suite `test_auto_satin` complète (44 cas) ; nouveau test dédié `trident` sur la géométrie des rails (aucun segment terminal excédant $4 \times \text{station_spacing}$, progression curviligne strictement monotone, `junction_cores` non vide et borné) ; test y symétrique réécrit (l'ancienne coïncidence exacte forcée n'est plus un invariant recherché — remplacé par « bridges locaux au centre de confluence, noyau résiduel petit ») ; suite `test_autodigitize` (réseau en T issu d'image segmentée, régression trouvée et corrigée par la retraction itérative ci-dessus) ; suite d'intégration complète (`tentabrode`) ; déterminisme.

Limite connue : la décomposition d'une branche en sous-région fermée propre (rail A + bridge + rail B inversé + autre bridge, comme dans `Ink/Stitch`) et le remplissage explicite du `JunctionCore` par un objet tatami distinct ne sont pas implémentés — le noyau reste un diagnostic (aire + contour approximatif) non cousu. Les corps de branche restent construits par échantillonnage de sections transversales le long de l'axe du squelette (inchangé), pas par une paire d'arcs de contour indépendants.

Optimisation globale des bridges de jonction + correctif d'un polygone auto-croisé (suite)

État : Présent · Testé numériquement. Remplacement de `resolve_junction_anchors/retract_bridge_station` par une recherche combinatoire, `satin_column.cpp` + `satin_column.hpp` + `debug_export.cpp`.

Défaut signalé en usage réel sur `trident` : la capture indépendante « une seule candidate par branche » (§ ci-dessus) choisissait toujours la toute dernière station de chaque branche comme bridge, sans coordination avec ses voisines — sur une confluence aussi asymétrique que `trident`, ces bridges restaient parfois francs millimètres les uns des autres, et le `JunctionCore` qui en résultait apparaissait comme un grand polygone concave recouvrant toute la confluence.

Deux défauts distincts corrigés, pas un seul :

Optimisation combinatoire des bridges (la demande initiale). Chaque branche conserve désormais plusieurs sections transversales CANDIDATES (`collect_bridge_candidates`) — des stations déjà présentes dans son rail, jamais un point fabriqué — prises dans un rayon local (`junction_core_radius`, nouveau paramètre, 4 mm par défaut) autour du nœud de squelette. `optimize_junction_bridges` évalue toutes les combinaisons raisonnables (une candidate par branche, budget total borné quel que soit le degré de la jonction) et retient celle de coût minimal — aire du `JunctionCore`, distance maximale au centre, recul total le long des branches, discontinuités de largeur — parmi celles qui interdisent tout croisement bridge/bridge (`find_crossing_bridge_pair`) et toute traversée d'une branche voisine (`bridge_crosses_other_rail`). **Aucun rail n'est modifié** : seul l'index de la station retenue varie d'une combinaison à l'autre. `retract_bridge_station` (qui tronquait réellement les rails) est supprimé — l'invariant « les rails restent ceux produits par `build_column` » est désormais strict, plus seulement documenté.

Bug préexistant trouvé EN OPTIMISANT, indépendant du point 1 : l'ordre des sommets de `compute_junction_core` reliait les mauvais côtés. `rail_a (+N)` n'est PAS toujours le côté qui fait face à la branche suivante dans l'ordre angulaire : sa convention gauche/droite est relative au sens de

parcours interne du rail (`build_column`), qui va tantôt de la jonction vers le bout ouvert, tantôt l'inverse, selon l'orientation de l'arête de squelette (`atEnd`) — alors que `tangent` (`outward_tangent`) pointe toujours correctement vers l'intérieur de la branche, quel que soit `atEnd`. L'ancien code connectait systématiquement `rail_b(cur)` à `rail_a(next)` : sur une branche où `atEnd` inverse la convention (le cas des deux bras courts de `trident`), ce n'était PAS le côté qui fait réellement face au voisin. L'arc de contour « le plus court » entre deux points qui ne se font pas face part alors dans une direction non pertinente, et le polygone assemblé s'auto-croise — la formule du lacet annule alors une partie de l'aire réelle par recouvrement, produisant un nombre artificiellement PETIT (le fameux 3,24 mm² observé) pour un polygone géométriquement INVALIDE. Corrigé par un calcul explicite, `leading_point/trailing_point`, dérivé de `tangent` tourné de +90° (fiable quel que soit `atEnd`) plutôt que des labels `rail_a/rail_b` bruts.

Garde-fous ajoutés une fois le bug de connexion corrigé : `polygon_self_intersects` détecte tout polygone auto-croisé restant ; `compute_junction_core` construit désormais le noyau en DEUX PASSES — d'abord la version repli (segments droits uniquement entre bridges successifs ; si elle-même s'auto-croise, la combinaison entière est invalide) puis, pour chaque connecteur, un remplacement par l'arc de contour réel UNIQUEMENT s'il garde le polygone simple ET ne fait pas grossir son aire (sur une confluence où 3+ branches se chevauchent mutuellement, l'arc « le plus court » au sens de la longueur de contour peut rester simple tout en empruntant un grand détour convexe).

Résultat mesuré : une fois le polygone correctement calculé (simple, non auto-croisé), l'aire réelle du noyau de `trident` est de l'ordre de 22 à 23,5 mm² selon les seuils — **plus grande**, pas plus petite, que l'ancien 3,24 mm² invalide. Ceci n'est PAS une régression : `y`, `t`, `cross` et `h` donnent des noyaux carrés/triangulaires parfaitement simples de 20 à 30 mm² une fois rendus en SVG et inspectés visuellement (bandes de test larges de 5 mm, donc recouvrement mutuel substantiel aux confluences par construction), confirmant que ces ordres de grandeur sont réalistes pour ces fixtures, pas un artefact.

`JunctionCore::requires_fill` (nouveau champ, seuil `junction_core_significant_area_um2` = 0,5 mm² par défaut) signale qu'un noyau de cette taille appelle un objet de remplissage séparé plutôt qu'une zone non couturée silencieuse — la synthèse de cet objet reste hors périmètre (diagnostic seul, cf. limite connue ci-dessus).

Diagnostic enrichi : `SatinColumnsResult::junction_bridge_candidates` (nouveau, toutes les candidates évaluées par branche + laquelle est retenue) et `JunctionCore::radius_um` sont exposés et affichés par `columns_to_svg` (candidates en gris pointillé fin, bridge retenu en vert, disque de recherche/contenance en pointillé gris autour du nœud de squelette).

Vérifié : suite `test_auto_satin` complète (44 cas, déterminisme inclus) ; test `trident` étendu (rails inchangés — déterminisme + chaque nœud à moins de 0,2 mm du contour réel —, noyau local à J1 au rayon `core.radius_um` près, aire bornée à une valeur réaliste et documentée comme telle) ; inspection visuelle des SVG rendus (`y`, `cross`, `h`, `trident`) confirmant des noyaux simples, bien formés, sans auto-croisement ni grand polygone parasite.

Séparation `StableBranchEnd` / `JunctionSeparator` : le noyau n'est plus « région entière moins colonnes » (suite)

État : Présent · Testé numériquement. Remplacement de l'optimisation combinatoire de bridges (§ ci-dessus) par une partition explicite, `satin_column.cpp` + `satin_column.hpp` + `debug_export.cpp`.

Défaut architectural signalé en usage réel sur `trident` : la recherche combinatoire (§ précédente) choisit, PARMI plusieurs reculs candidats d'une même branche, celui qui minimise l'aire du `JunctionCore` — mais reculer une section transversale ne change pas où se trouve la vraie frontière géométrique entre deux branches (l'encoche réelle du contour). Sur `trident`, aucune combinaison de reculs ne pouvait donc produire un noyau petit ; le résultat mesuré (22 à 23,5 mm², quasiment toute la confluence) était réel, pas un artefact — mais architecturalement le mauvais résultat.

Correction : séparer deux notions jusqu'ici confondues.

1. **`StableBranchEnd`** — la dernière section transversale RÉELLEMENT stable d'une branche (ce qu'on appelait le « bridge »), inchangée dans son principe (`trim_unstable_junction_tail`) : point d'entrée

du traitement de jonction, jamais déplacée.

2. **JunctionSeparator** — la frontière locale PARTAGÉE entre deux branches angulairement adjacentes, construite DANS la zone de confluence (sommet reflex du contour le plus profond, ou repli sur le milieu curviligne de l'arc de contour reliant les deux branches) : ce n'est ni une section transversale, ni jamais un point sur un rail.

La région locale de jonction (disque de rayon adaptatif — $1,2 \times$ la plus éloignée des `StableBranchEnd` au nœud, jamais une constante fixe, plafonné par `junction_core_radius`) est partitionnée en un secteur par branche (bordé par ses deux `JunctionSeparator` voisins, l'arc de contour réel, et sa propre `StableBranchEnd`). Le `JunctionCore` résiduel est calculé **comme `local_region - union(secteurs)`**, jamais comme « région entière moins colonnes » : concrètement, le polygone des `JunctionSeparator` eux-mêmes, simple par construction (sommets triés par angle autour d'un centre commun = polygone en étoile).

Deux bugs trouvés en généralisant, tous deux avec un symptôme identique (noyau anormalement grand) mais des causes distinctes :

1. **Recherche de séparateur par SECTEUR ANGULAIRE au lieu de CONTIGUÏTÉ DE CONTOUR.** La première implémentation cherchait le sommet reflex d'un espace angulaire [`angle(branche i)`, `angle(branche i+1)`], basé sur l'angle du rayon squelette de chaque branche. Sur `trident`, la branche verticale (6 mm) a ses points `leading/trailing` écartés de plus de 90° autour du nœud : l'encoche réelle entre elle et sa voisine fine tombait alors, en angle pur, dans le secteur voisin — laissant DEUX des trois espaces sans aucun candidat, repliés sur un rayon lancé depuis le nœud qui pouvait heurter le mur d'une branche sans rapport (noyau mesuré : 14,4 mm², toujours trop grand). Corrigé en cherchant plutôt sur l'arc de contour qui relie réellement `leading_point(branche i)` à `trailing_point(branche i+1)` (`ContourDirection::Shortest`) : la contiguïté de contour reflète la vraie adjacence géométrique, pas l'angle depuis un centre qui peut être trompeur pour une branche large. Résultat après correction : 0,94 mm².

2. **Une `StableBranchEnd` peut être stable en LARGEUR tout en restant géométriquement dans le corridor d'une voisine.** `trim_unstable_junction_tail` ne détecte que la dérive de largeur du bourrelet — pas sa position. Sur un réseau en T très asymétrique (bras large, branche fine, régression trouvée par `test_autodigitize`), la toute première section d'un bras pouvait avoir une largeur déjà plausible (donc jamais amputée) tout en croisant le rail de la branche voisine, un défaut que le seul critère de largeur ne peut pas voir. Corrigé par une retraction itérative bornée dans `resolve_junction` (jamais dans `build_column`) : toute paire de `StableBranchEnd` dont les coupes se croisent (`find_crossing_end_pair/end_crosses_other_rail`) fait reculer les deux branches impliquées d'une station réelle supplémentaire (`make_stable_branch_end` généralisé avec un paramètre `offset`), jusqu'à ce que la confluence soit cohérente ou qu'il n'y ait plus de station disponible (refus propre, comme avant).

Diagnostic enrichi : `SatinColumnsResult` expose désormais `stable_branch_ends`, `junction_separators` et `junction_sectors` (en plus de `junction_cores`, dont les champs `radius_um` unique est remplacé par `configured_radius_um/local_radius_um/actual_max_radius_um` — le premier est un plafond de sécurité, jamais le rayon réel). `columns_to_svg` affiche les `StableBranchEnd` en rouge, les `JunctionSeparator` en vert, un secteur par branche en teinte translucide distincte, le noyau en magenta, et le disque de région locale en pointillé — avec les trois rayons étiquetés séparément pour ne plus jamais laisser croire que le plafond configuré est une mesure.

Vérifié : suite `test_auto_satin` complète (44 cas) ; test `trident` étendu (3 secteurs, 3 séparateurs partagés exactement entre secteurs adjacents, aire du noyau très inférieure à l'ancien résultat invalide, aucun point du noyau au-delà du rayon local réellement calculé) ; suite `test_autodigitize` complète (réseau en T, régression trouvée et corrigée par la retraction itérative ci-dessus) ; inspection visuelle des SVG rendus (`trident`, `y`, `t`, `cross`, `h`) : noyaux petits et centrés sur `trident/y` (encoches réelles présentes), noyaux carrés honnêtes sur `t/cross/h` (branches de même largeur se croisant à angle droit sans aucune encoche — un noyau non trivial `y` est le résultat géométriquement correct, pas un défaut).

Traçage du graphe de squelette : priorité aux nœuds, exclusion de l'origine

État : Présent · Testé numériquement. Correctif de revue, `skeleton_graph.cpp`.

Défaut trouvé par revue (deux symptômes distincts, même cause racine). Le traçage d'une arête (`build_skeleton_graph`) suit une chaîne de pixels de degré 2 depuis un pixel-nœud jusqu'au prochain pixel-nœud, en choisissant à chaque pas le **prochain voisin** parmi les 8 directions. L'ancien code s'arrêtait sur le **premier candidat rencontré** dans l'ordre fixe des 8 directions, qu'il s'agisse d'un vrai nœud ou d'un simple pixel de continuation — alors qu'un pixel juste avant une jonction a souvent, en plus de la jonction elle-même, un pixel de degré 2 d'une **autre branche** dans son 8-voisinage (les branches se recouvrent physiquement près d'une confluence, cf. section précédente).

1. **Jonction "croix" ramenée à un degré 2 au lieu de 4** (le défaut initialement identifié, stable sur toutes les résolutions testées) : deux bras opposés étaient fusionnés en une seule arête traversante quand le pixel de l'un d'eux apparaissait avant le pixel de la jonction elle-même dans l'ordre de balayage.
2. **Branche entière perdue sur un réseau "y" avec arête parasite en boucle** (défaut distinct, non documenté avant cette revue) : le pixel d'**origine** de la trace pouvait être ré-atteint par un chemin de 2 pas différent de celui emprunté au départ — seul le pixel immédiatement précédent était exclu des candidats, pas le pixel d'origine de toute la trace. La tige du "y" (une branche entière) disparaissait alors du graphe, remplacée par une arête `from == to` de quelques centaines de µm sur la jonction, sans aucune arête ni diagnostic pour l'utilisateur.

Correction : priorité **absolue** à un nœud sur tout le 8-voisinage (balayage complet des 8 directions pour un nœud avant de retomber sur un pixel de continuation, au lieu du premier candidat rencontré), et exclusion du pixel d'**origine** de la trace (pas seulement le pixel précédent) pendant toute la marche.

Vérifié : la jonction d'une `cross` a désormais un degré réel de 4 dans le graphe élagué (4 arêtes, 4 extrémités, 1 jonction) ; un réseau `y` conserve ses 3 branches (aucune arête `from == to`) ; aucune régression sur les formes déjà testées (`rectangle`, `t`, `capsule`, `ribbon`, `s`, `ring`, `wide`, `circle`).

Contrat de `graph_cleanup.cpp` : aucune suppression silencieuse

État : Présent · Testé numériquement. Correctif de revue.

Défaut trouvé par revue : `graph_cleanup.hpp` documente explicitement « ne supprime rien silencieusement (chaque suppression est diagnostiquée) », mais `prune_graph` ne réinsérait un nœud dans le graphe élagué que s'il était référencé par au moins une arête vivante — un nœud du graphe brut **sans aucune arête** (isolé dès le départ, comme le point unique auquel se réduit le squelette d'un disque plein, ou devenu orphelin après élagage d'une arête terminale courte) disparaissait donc du résultat sans jamais apparaître dans `removed`, contredisant le contrat documenté.

Correction : tout nœud du graphe d'origine non repris dans le graphe élagué (aucune arête vivante incidente, quelle qu'en soit la raison) est désormais systématiquement ajouté à `removed`, avec un motif explicite. Vérifié sur `circle` (squelette réduit à un point isolé) : le graphe élagué reste vide, mais le nœud isolé apparaît dans le diagnostic.

Arête Jonction-Jonction (pont d'un "H") convertie en colonne

État : Présent · Testé numériquement. Correctif de revue, `satin_column.cpp`.

Défaut trouvé par revue : `build_satin_columns` (cas `RequiresDecomposition`) ne convertissait en colonne que les arêtes du squelette élagué touchant au moins une **extrémité** (« une colonne par branche menant à une extrémité, bras du Y/T »). Une arête reliant **deux jonctions** — le pont horizontal d'un "H", topologie absente des formes de test précédentes (Y/T/croix n'ont qu'une seule jonction) — n'était donc jamais essayée : `r.columns` restait non vide (les bras des deux barres verticales étaient bien produits) et `r.refusal` restait vide (aucun signal d'erreur), mais le pont lui-même — une portion entière et visible de la région — ne recevait **aucun point de broderie**.

Correction : toutes les arêtes du graphe élagué sont désormais essayées, sans distinction sur le type de leurs deux extrémités — `try_edge` gère déjà chaque bout indépendamment selon son type (extension si extrémité ouverte, ancrage si jonction), aucune branche de code particulière n'était nécessaire. Vérifié sur

une fixture "H" dédiée (deux barres verticales de 40 mm reliées par un pont horizontal de 5 mm) : 5 colonnes produites (les 4 demi-barres + le pont), aucune collision aux deux jonctions, aucun barreau dégénéré.

Amputation de la queue instable : décroissance stricte, pas un seuil fixe

État : Présent · Testé numériquement. Correctif de revue, `satin_column.cpp`.

Défaut trouvé par revue, révélé par le correctif de traçage ci-dessus (la branche "y" auparavant perdue expose maintenant ce second défaut, jusque-là invisible faute d'atteindre `satin_column.cpp`). L'étape 1 de l'ancrage des jonctions (§ précédente) amputait la queue instable en comparant chaque station à sa **seule voisine immédiate** (seuil relatif fixe, +10 %) : ce critère suppose que le bourrelet de confluence retombe en un seul saut net. Mesuré sur la tige du "y" : la largeur décroît **progressivement** sur plusieurs stations (9200 → 8586 → 7576 → 6566 → 5554 → 5000 µm, stable), où **chaque écart pris isolément reste sous 10 %** (le premier, 9200 vs 8586, n'est que 7 %) mais la dérive cumulée atteint +72 % — l'ancien critère s'arrêtait dès le tout premier écart local insuffisant, laissant la quasi-totalité du bourrelet en place.

Correction : le critère devient une décroissance **stricte** (avec une tolérance de 1 µm pour le bruit flottant) au lieu d'un seuil relatif fixe — toute décroissance signale qu'on est encore dans la zone d'influence de la confluence, sans hypothèse sur l'ampleur du saut d'une station à l'autre. Vérifié : la largeur de la tige du "y" (résolution 0,1 mm, où les 3 branches survivent désormais grâce au correctif de traçage) reste sous 1,2× la médiane sur toutes les stations, y compris celles immédiatement après la jonction ; aucune régression sur les cas déjà couverts (Y symétrique à 50 µm, T).

Décision : `Suitable` → une colonne (axe principal) ; `RequiresDecomposition` (Y/T) → une colonne par branche menant à une extrémité. Un anneau fin à un trou est ouvert sur une couture déterministe puis décomposé en quatre sections cycliques ; ses barreaux sont contrôlés sur plusieurs échantillons et contre les croisements locaux/globaux. Ambiguous (quasi circulaire sans trou) et les formes trop larges/étroites ou non appariées → **refus explicite**. Déterministe. Vérifié visuellement via `openstitch-cli`
`auto-satin-debug --shape <forme> --output-svg` (SVG dans `tests/golden/auto-satin/`).

Intégration : la numérisation automatique et **Broderie ■ Convertir automatiquement en satin...** créent une `EmbroideryObject` par section. Toutes les sections d'un réseau gardent la même source/couleur ; le routage Lot 6 les traite donc comme un groupe multi-rail. Chaque objet reste un `SatinParams` historique à deux rails : aucune rupture du format. Les **barreaux sont stockés** dans `SatinParams.runs` et sérialisés (`.osp` schéma v2, rétrocompatible).

Chaque section générée porte désormais une topologie explicite et déterministe : `section_index/section_count`, plus `start_junction` et `end_junction` quand l'extrémité touche une jonction du squelette. Les anneaux forment un cycle de quatre jonctions ; les réseaux Y/T réutilisent le même identifiant au point partagé. Ces métadonnées deviennent le bloc `SatinParams.topology` optionnel dans le document et le `.osp`. Un ancien satin sans ce bloc reste une colonne isolée. Ce lot fournit une identité de jonction fiable ; il ne modifie pas encore le routage ni l'édition coordonnée des guides.

Régressions couvertes : bande simple, réseau en T (au moins trois sections), anneau rasterisé (quatre sections, trou préservé) et pipeline complet sur `tests/fixtures/tentabrode.png`, en Debug et Release. Cette validation reste logicielle ; la qualité textile et les cas très concaves demandent encore un contrôle visuel puis machine.

Génération partielle sur formes concaves : trous silencieux dans `build_column`

État : Présent · Testé numériquement. Correctif de revue, `satin_column.cpp` (audit 2026-08-03).

Défaut trouvé par revue. `build_column` échantillonne l'axe (squelette lissé) station par station et calcule à chaque point une section transversale (`cross_section`, intersection de la normale locale avec le contour). Sur une forme **concave ou très large**, cette section transversale peut légitimement échouer pour certaines stations (normale qui rase une encoche voisine, section plus large que `max_satin_width`, milieu de l'intervalle retombant hors région) — un cas déjà connu et documenté (§ *Rails automatiques* plus haut, à propos de l'heuristique naïve). Le code de `build_column`, lui, traitait cet échec par un simple `continue` : la station disparaissait de l'axe **sans aucune trace**, et les deux stations valides encadrant le trou se

retrouvaient reconnectées directement, quelle que soit la distance réelle entre elles. Le nettoyage anti-croisement qui suit (retrait d'une station dont le quadrilatère avec la précédente s'auto-intersecte) souffrait du même défaut : les stations retirées n'étaient jamais comptées ni bornées. Sur un projet réel présentant une échancrure prononcée, ceci produisait — selon la forme — de larges zones de la région sans aucun point de broderie, des rails visiblement discontinus, des éventails ou pointes artificielles à l'endroit de la reconnexion, ou une colonne partiellement construite là où un refus propre aurait été attendu.

Correction, entièrement dans `build_column` (le squelette Zhang-Suen et le graphe de squelette ne sont pas en cause — le défaut est postérieur, dans la conversion `centerline` → rails) :

1. `cross_section` retourne désormais un `std::expected<..., CrossSectionFailure>` au lieu d'un `std::optional` indifférencié : la raison exacte de l'échec (axe hors région, intersection négative/positive manquante, largeur supérieure à `max_satin_width`, intervalle invalide) est conservée et propagée jusqu'aux diagnostics (`SatinColumnsResult::warnings`).
2. Chaque échantillon d'axe garde son indice et son résultat (succès ou échec) avant tout filtrage. Un échec **isolé** (une seule station, encadrée de deux succès) est comblé par **interpolation linéaire** des rails voisins, acceptée seulement si le résultat reste géométriquement plausible (intérieur à la région, largeur non aberrante, aucun croisement avec les deux stations voisines) — sinon il est traité comme un trou irrésolu. Des échecs en tête ou en queue d'axe restent un bord légitime (comportement inchangé, absorbé silencieusement comme avant : ce n'est pas un trou interne).
3. Tout trou interne non résolu par interpolation (échecs consécutifs, ou isolé mais géométriquement invalide) **refuse la colonne entière** plutôt que de la reconnecter à travers le trou. Le nettoyage anti-croisement applique le même principe : retirer une ou quelques stations sur un virage serré reste toléré (comportement préexistant, nécessaire sur `ribbon/y`), mais si la distance d'axe réellement pontée dépasse `max_station_gap_ratio × station_spacing`, la colonne est refusée.
4. Parce qu'un trou interne irrésolu refuse systématiquement la colonne plutôt que de conserver le plus long fragment, `st.front()/st.back()` restent toujours de vrais bouts d'axe au moment d'appeler `extend_tip` : aucune extrémité artificielle issue d'une coupure interne n'est jamais étendue comme s'il s'agissait d'un vrai bout du squelette.
5. Une validation finale (sur le cœur de l'axe, **avant** extension des bouts et ancrage de jonction — ces deux étapes ont leurs propres tolérances déjà testées, notamment le saut de largeur délibéré du point de fermeture `tip_min_width`) vérifie l'absence de trou résiduel, l'absence de saut de largeur incohérent entre stations adjacentes, et une couverture minimale (`min_axis_coverage_ratio`) de la longueur d'axe réellement convertie en stations. Une dernière passe, une fois les barreaux posés, rejette tout barreau retombant hors région ou toute paire de barreaux qui se croisent.

Calibration : les trois nouveaux seuils (`max_station_gap_ratio = 5,0 × station_spacing`, `min_axis_coverage_ratio = 85 %`, `max_adjacent_width_jump_ratio = 75 %`) ont été choisis pour ne régresser aucune fixture existante — `ribbon` et `y`, sur des virages serrés, écartent normalement 1 à 3 stations d'affilée lors du nettoyage anti-croisement, et une jonction à haut degré (`cross`, `h`) peut légitimement voir sa couverture chuter autour de 80 % tout près du nœud du squelette (bourrelet de confluence, cf. *Ancrage des jonctions* plus haut).

Vérifié : suite complète `test_auto_satin` inchangée (aucune régression sur `rectangle`, `capsule`, `ribbon`, `s`, `y`, `t`, `cross`, `h`, `ring`, `wide`) ; nouvelle fixture `notch` (bande de 40 × 5 mm entaillée d'une encoche en V profonde sur un bord, resserrant la largeur locale à 1 mm) et sa variante plus sévère `pinch` (resserrement à 0,3 mm) produisent, à chaque exécution, **soit** un refus explicite (`refusal` non vide, aucune colonne), **soit** une colonne complète sans trou entre stations consécutives, avec tous les barreaux dans la région et aucun croisement entre barreaux — jamais un résultat partiel ; déterminisme vérifié (mêmes rails à chaque exécution sur les deux fixtures).

Pointes de colonne réduites à quelques dizaines de μm : plancher franchi en un seul pas dans `extend_tip`

État : Présent · Testé numériquement (unitaire + intégration sur projet réel). Correctif d'audit (2026-08-11), retour utilisateur : « le satin fait n'importe quoi » sur un logo circulaire à motif de circuit imprimé finement détaillé (nombreuses formes fines à extrémité OUVERTE — pointe de piste, plot de connecteur).

Défaut trouvé en investiguant le retour utilisateur. `extend_tip` prolonge un bout ouvert de colonne au-delà de ce que le squelette aminci couvre, en marchant par pas de `stepLen` le long de la tangente sortante et en ré-échantillonnant une section transversale à chaque pas, tant que la largeur rétrécit. La boucle **poussait la station dans `ext` avant de vérifier** si sa largeur avait déjà atteint le plancher voulu (`tip_min_width`) :

```
Station st;
st.width = width;
ext.push_back(st);          // ← poussée d'abord...
...
if (width <= tipMinWidth) {
    return ext;             // ← ...le plancher n'est vérifié qu'ensuite
}
```

Or rien ne borne la largeur **au-dessus** du plancher au moment de la poussée : si un pas de marche fait chuter la largeur de, disons, 200 μm à 34 μm en une seule itération (taper serré, pas de marche relativement grossier par rapport à la vitesse de rétrécissement — le cas courant pour une petite forme pointue), c'est cette station à 34 μm qui se retrouve poussée dans `ext`, **avant** que la condition d'arrêt n'ait la main. La bissection de fermeture qui suit (laquelle, elle, respecte correctement `tipMinWidth : half = max(tipMinWidth * 0.5, 1.0)`) n'était alors jamais atteinte pour cette extrémité — sa station de tête restait celle, sous-plancher, déjà poussée.

Un second facteur amplifiait la fréquence du défaut sans en changer la nature : `tip_min_width` valait **50 μm par défaut**, sous le pas de quantification DST lui-même (0,1 mm, cf. `formats/dst.hpp`) — un barreau à tout juste 50 μm n'aurait de toute façon jamais représenté un point cousu exploitable, même sans le défaut d'ordre de poussée ci-dessus.

Impact mesuré sur un projet réel (logo circulaire, ~340 régions segmentées, `tests/integration/test_pipeline.cpp`, diagnostic « satin pointes fines (GISTRE) ») : **178 des 190 colonnes satin (94 %)** avaient au moins un barreau sous 0,8 mm, la plupart entre 33 et 51 μm — quasiment toute colonne à extrémité ouverte était concernée, puisque `extend_tip` s'applique à chacune. Une colonne satin dont le dernier barreau mesure quelques dizaines de μm génère, selon la machine et le fil, un point quasi nul, un bourrage de fil ou une casse — exactement le symptôme rapporté (« ça fait n'importe quoi »), et son omniprésence sur ce type d'image (nombreuses petites formes en pointe) explique pourquoi il n'était pas passé inaperçu plus tôt sur des images plus simples (peu de bouts ouverts fins).

Correction, deux volets indépendants :

1. **Ordre de la boucle inversé** : la largeur est comparée au plancher *avant* de pousser la station, pas après. Une station déjà sous le plancher n'est jamais poussée — la marche s'arrête un pas plus tôt et laisse la bissection de fermeture (déjà correcte) fermer proprement au plancher exact.
2. **`tip_min_width` porté de 50 à 300 μm** : reste nettement sous `min_satin_width` (0,8 mm, le seuil « satin digne de ce nom » pour toute la colonne — la pointe reste une pointe, pas un bout à pleine largeur), tout en dépassant largement le pas DST avec une marge de sécurité ($\times 3$).

Un défaut apparenté, dans la boucle dense principale de `compute_column_stations` (partagée par `build_column` et `build_parametric_object`, donc par les deux modes de géométrie) : aucun plancher de largeur n'existait pour une station de CORPS (pas de bout), qu'elle vienne de l'axe principal ou d'une interpolation comblant un échec isolé de `cross_section` (§ *Génération partielle sur formes concaves* ci-dessus). Une région globalement fine mais localement très étroite sur une portion de son squelette pouvait ainsi passer le test d'éligibilité (`satinability.cpp` compare `max_satin_width` à la largeur

maximale relevée sur toute la région, pas à sa largeur typique) puis dégénérer en aval. Ajout d'un nouveau cas d'échec `CrossSectionFailure::TooNarrow` (largeur sous `min_satin_width`), traité par le mécanisme d'échec déjà en place (§ *Génération partielle*) : un échec isolé est comblé par interpolation si l'interpolée elle-même dépasse le plancher (`minwidth` ajouté aux critères de `interpolated_station_valid`, qui ne vérifiait jusqu'ici que `maxwidth` — sans quoi une station trop étroite se faisait réaccepter telle quelle par simple interpolation de position avec ses voisines, sans jamais revalider sa largeur) ; un échec étendu refuse la colonne entière plutôt que de produire des rails quasi confondus sur toute sa longueur.

Vérifié : suite `test_auto_satin` complète inchangée (1999 assertions, 51 cas, aucune régression sur les fixtures existantes) ; nouveau cas sur la géométrie EXACTE d'une région du projet réel en cause (`tests/unit/auto_satin/test_columns.cpp`) ; diagnostic d'intégration sur le projet réel complet (`tests/integration/test_pipeline.cpp`, § GISTRE) : 0 barreau sous 0,29 mm après correctif contre 178/190 colonnes touchées avant, pire séparation observée 299 µm (au plancher voulu, à l'arrondi µm près).

Ligne de coupe manuelle pour les jonctions satin (`geometry::cut_path_set`, 2026-08-12)

État : Présent (outil `DrawSatinCutLine`) · Complément, pas remplacement, de la détection automatique de jonctions ci-dessus.

Origine : audit de la pipeline Ink/Stitch

Sur des logos complexes réels (ex. sceau circulaire à motif de circuit imprimé), le squelette automatique (`build_satin_columns`) peine parfois sur des jonctions ambiguës — nœuds en Y/T proches, branches très courtes, squelette bruité par un contour segmenté imparfaitement. Un audit de la pipeline auto-satin d'Ink/Stitch (GPL ; documentation publique consultée, aucun code repris — implémentation ci-dessous entièrement originale) a montré que cet outil ne tente PAS de détecter les jonctions automatiquement : son "Stroke to Satin" calcule une ligne centrale puis s'appuie sur des "cut lines" tracées à la main par l'utilisateur pour séparer une forme aux endroits voulus, avant conversion en colonnes. Le remplissage tatami sur formes complexes suit le même principe ("break apart" manuel documenté comme solution officielle).

Plutôt que de renoncer à la détection automatique (qui reste supérieure sur la majorité des formes — voir toutes les sections ci-dessus), l'outil de ligne de coupe est ajouté en complément : un geste manuel simple pour les cas où l'automatique reste ambigu, sans jamais devenir la voie par défaut.

Primitive géométrique : `geometry::cut_path_set`

`libs/geometry/include/openstitch/geometry/cut.hpp` / `libs/geometry/src/cut.cpp`. Signature :

```
Result<std::vector<PathSet>> cut_path_set(const PathSet& region, Vec2um a, Vec2um b,
                                         Micrometers cut_width = Micrometers{20});
```

Retire une fine bande rectangulaire (`cut_width`, quelques dizaines de µm par défaut — négligeable une fois cousu) centrée sur le segment a-b, prolongée très au-delà de la région dans les deux sens pour garantir une traversée complète quels que soient les deux points fournis (l'utilisateur n'a donc qu'à tracer un segment qui croise la jonction visée, pas à viser précisément les bords de la forme). Implémentation via une booléenne `Clipper2 Difference` (même encapsulation par fichier que le reste de `libs/geometry` — ADR-005 : aucun type `Clipper2` ne sort de `cut.cpp`), qui garantit une vraie séparation topologique — une coupe infiniment fine laisserait parfois deux morceaux qui se retouchent en un point, toujours une seule composante connexe aux yeux d'un traitement ultérieur.

Renvoie un morceau par composante connexe résultante : 1 seul morceau (la région quasi inchangée) si la ligne ne traverse pas réellement la région ou est dégénérée (`a == b`) — à l'appelant de vérifier `size() >= 2` pour savoir si la coupe a réellement séparé quelque chose. Testé (`tests/unit/geometry/test_cut.cpp`, 6 cas / 29 assertions) : séparation d'un carré, coupe du pont d'un haltère (cas de jonction Y/T), ligne hors région sans effet, points confondus sans effet, déterminisme, largeur de bande bornée.

Outil desktop (Tool: DrawSatinCutLine)

Réutilise le mécanisme presser-glisser-relâcher déjà en place pour l'outil Bézier (`CanvasView::bezierPointDraggingMm/bezierPointCommittedMm`) — un second couple de connexions sur les mêmes signaux, sans aucune modification de `CanvasView` (les gestionnaires Bézier existants font déjà un retour anticipé hors de l'outil `DrawBezier`, donc no-op automatique quand l'outil actif est `DrawSatinCutLine`).

Séquence (`apps/desktop/main_window.cpp, createSatinObjectWithCutLine`) :

1. Requiert une forme vectorielle sélectionnée (`selectedObject_`) ; sinon, message dans la barre de statut plutôt qu'une boîte de dialogue bloquante.
2. Glisser trop court ($< 200 \mu\text{m}$, clic net) ignoré silencieusement — geste probablement involontaire.
3. `geometry::cut_path_set` sur `source->paths.front()` ; < 2 morceaux -> avertissement explicite ("tracez la ligne bien d'un bord à l'autre").
4. Boîte de dialogue densité/compensation/sous-couche, identique à celle de `createSatinObject()` (création automatique) pour cohérence d'expérience.
5. Chaque morceau est passé indépendamment à `auto_satin::build_satin_columns` (mode `Parametric`) : un morceau qui ne produit aucune colonne (trop petit/dégénéré) est simplement ignoré plutôt que d'annuler toute l'opération — la coupe a pu réussir pour la jonction visée même si un fragment marginal ne l'est pas.
6. Avertissement de largeur excessive identique à `$Colonne trop large`.
7. Un seul `commands::AddObjectBatchCommand` ("Colonne satin (ligne de coupe)", annulable en un geste comme toutes les créations de forme de cette fenêtre.

Un seul geste = une seule coupe (portée v1, pas d'accumulation multi-coupes) : succès -> retour automatique sur l'outil Sélection pour enchaîner sur la retouche du résultat ; échec -> reste sur l'outil pour retracer.

Testé (`tests/unit/desktop/test_main_window.cpp, satinCutLineToolsplitsSelectedShapeIntoTwoSatinColumns`) : glisser souris réel à travers un rectangle allongé sélectionné -> deux objets satin indépendants, undo/redo, retour automatique sur Sélection.

Objets satin paramétriques (mode `Parametric`, refonte 2026-08-04)

État : Présent, activable par `SatinColumnsParameters::geometry_mode` (CLI :

`--satin-geometry=parametric`) · Mode par défaut toujours `Legacy` · Testé numériquement (44 cas legacy inchangés + 6 nouveaux cas parametric) · Validé visuellement (SVG) sur *rectangle*, *capsule*, *y*, *t*, *cross*, *trident*.

Défaut architectural du pipeline Legacy. `build_column` convertit chaque station dense (une tous les `station_spacing = 500 \mu\text{m}`) directement en un nœud de rail `Corner` (`rail_path`) et un barreau (`SatinRung`) — alors que `geometry::PathNode` porte déjà `tan_in/tan_out` (Bézier, cf. `polyline.hpp::flatten`, De Casteljau adaptatif) et que `stitch_generation::fill_satin_columns` implémente déjà l'essentiel de l'étape « génération par longueur d'arc entre guides » (`ladder_correspondence` et `resample_by_medial_spacing`, § section suivante). Le vrai problème n'était donc pas une architecture Bézier manquante, mais que **rien ne l'utilisait** : même en y injectant des `tan_in/tan_out`, `to_points` (dans `satin.cpp`) lisait `node.pos` brut sans jamais appeler `flatten` — corrigé (§ plus bas). Les jonctions (`StableBranchEnd/JunctionSeparator/secteurs/JunctionCore`) opéraient sur ces nœuds denses avec une contrainte de partition exacte — fragile par construction : c'est cette contrainte de partition, pas le nombre de nœuds en soi, qui produisait la parade de défauts historiques (ancres centrale, diagonale, éventail, noyau énorme, bridges égarés).

Principe du mode Parametric : au lieu d'une polyligne dense, un objet satin devient `ParametricSatinObject` — deux rails Bézier ÉPARS (`geometry::Path`, quelques `PathNode` à tangentes) couplés par un petit nombre de `SatinControlPair/SatinAngleGuide`. Les points de couture ne font PAS

partie de cette représentation : ils sont dérivés à la demande par `fill_satin_columns`, qui aplatit `rail_a/rail_b` avant de les parcourir. La couche d'analyse dense (`compute_column_stations`, extraite de l'ancien `build_column` sans changement de comportement) reste PARTAGÉE par les deux modes — seule la finalisation diffère.

1. **Sélection des paires structurantes** (`select_structural_indices`) : union de critères déterministes — les deux extrémités, les extrema de largeur (`width_extrema_indices`), les sauts de largeur ADJACENTS (`adjacent_width_jump_indices` — cf. bug ci-dessous), les maxima de courbure (`curvature_maxima_indices`), et une simplification Douglas-Peucker de CHAQUE rail en réutilisant `geometry::simplify` (indices retrouvés par correspondance exacte de position, jamais une réimplémentation de l'algorithme). Fusion des indices trop proches (`minimum_control_pair_spacing`).
2. **Ajustement Bézier conjoint** (`fit_both_rails/fit_both_rails_recursive`) : un seul ensemble d'ancrages PARTAGÉ par les deux rails (jamais deux subdivisions indépendantes qui dériveraient l'une de l'autre — § étape 6, chaque paire structurante correspond exactement à une paire de points sur les deux rails). Par intervalle, `fit_cubic_segment` résout un cubique simple (Schneider, moindres carrés à tangentes fixées, sans reparamétrisation itérative) ; subdivision ADAPTATIVE (station la plus déviante, ou milieu si dépassement pur d'espacement) tant que l'erreur (distance OU tangente) ou l'espacement dépasse les seuils configurés, bornée par `minimum_control_pair_spacing` et une profondeur maximale.
3. **Lignes d'angle** (`SatinAngleGuide`) : une par paire structurante, avec son abscisse curviligne sur chaque rail APLATI (pas un paramètre Bézier brut) — consommées telles quelles par `fill_satin_columns`.

Deux bugs de fit trouvés en généralisant sur les six formes cibles, tous deux avec un symptôme identique (rail sortant de la région) mais des causes distinctes :

1. **Longueur de poignée = corde brute, pas sa PROJECTION sur la tangente.** Sur un bout OUVERT à plat (`extend_tip`, plancher `tip_min_width`), la station de fermeture saute presque uniquement en LARGEUR sur une distance d'axe minime — sa tangente fixée (avant/horizontale) est alors quasi PERPENDICULAIRE à la corde réelle. `fallbackP1 = p0 + tangente × (corde/3)` utilisait la longueur de corde totale (dominée par le saut perpendiculaire) comme échelle de poignée le long d'une direction quasi-orthogonale — la poignée partait très loin dans le mauvais axe avant de devoir rebrousser chemin, débordant largement hors la forme (démonstré : rectangle, poignée à $x=40775$ pour un bord à $x=39995$, soit $+780\ \mu\text{m}$ de débordement). Corrigé : la longueur d'échelle des poignées est la PROJECTION de la corde sur la tangente (`scale0/scale1 = |dot(p3-p0, tangente)|`), jamais la corde brute — sur un coin quasi perpendiculaire, la projection est naturellement courte, produisant une poignée courte proche d'un coin SANS code spécial pour détecter « est-ce une pointe » (§ étape 5 : « discontinuité de tangente sur un coin structurel » — la géométrie seule tranche).
2. **`width_extrema_indices` ne détecte que les pics/creux LOCAUX, pas un SAUT suivi d'un PALIER.** Une station de fermeture (largeur plancher) collée à la première vraie station de corps (largeur pleine) n'est ni un maximum ni un minimum local (elle est suivie d'un PALIER à largeur pleine, pas d'une redescente) — ce schéma passait inaperçu, laissant un seul intervalle couvrir un saut de largeur de $100\times$, avec la même conséquence que le bug précédent. Ajouté `adjacent_width_jump_indices` : détecte tout saut relatif entre deux stations ADJACENTES (pas seulement les extrema), complémentaire et nécessaire.

Robustesse de la subdivision adaptative : le point de coupure « idéal » (pire erreur, ou milieu) peut violer `minimum_control_pair_spacing` — typiquement collé au tout premier/dernier point d'un intervalle contenant un saut de largeur en bordure. Plutôt que d'abandonner toute subdivision (et garder un segment unique très imprécis, potentiellement débordant), le point valide le plus proche de la coupure idéale est recherché.

Validation (`validate_parametric_object`, § étape 12) : échantillonnage dense des rails APLATIS (jamais les seuls nœuds de contrôle) — région (tolérance dérivée de `bezier_fit_tolerance` : un rail trace le contour PAR CONSTRUCTION donc est légitimement sur le bord, `in_region` seul est ambigu pile sur une arête discrétisée), auto-intersection, croisement entre les deux rails, lignes d'angle non dégénérées et non

croisées, correspondance monotone. Un objet refusé n'écarte QUE cette branche (`warnings`), jamais la génération entière.

Jonctions : recouvrement local, sans ancre (remplace `StableBranchEnd/JunctionSeparator/secteurs/JunctionCore` en mode `Parametric`)

Une branche = un `ParametricSatinObject` INDÉPENDANT (déjà vrai structurellement : `build_satin_columns` construit une colonne par arête de squelette). Aucune ancre centrale commune, aucun sommet de bissectrice, aucun déplacement de nœud terminal, aucune diagonale, aucune rampe, aucun noyau résiduel calculé par soustraction globale — l'apparat entier `StableBranchEnd/JunctionSeparator/secteurs/JunctionCore` (toujours présent et actif en mode `Legacy`) est simplement absent du chemin `Parametric`.

À la place : après amputation de la queue instable (`trim_unstable_junction_tail`, inchangée), `extend_into_confluence` (variante de `extend_tip` sans critère de convergence — la largeur PEUT légitimement croître en entrant dans le bourrelet, c'est le recouvrement voulu) ré-échantillonne quelques sections transversales réelles au-delà du dernier point stable, jusqu'à une distance bornée (`junction_overlap_target` = fraction `junction_overlap_ratio` de la largeur locale, bornée par `junction_overlap_min/junction_overlap_max`). S'arrête dès que `cross_section` échoue (bord réel atteint, ou — cas `t/cross`, voir plus bas — section perpendiculaire qui balaie la branche voisine).

Cas `t/cross` (branches perpendiculaires de même largeur) : recouvrement mesuré = 0, et c'est correct. Rien n'est amputé par `trim_unstable_junction_tail` (aucun bourrelet à détecter sur une confluence symétrique à largeur constante), donc chaque rail atteint déjà le bord de la confluence par son propre corps — la couverture visuelle de la confluence est déjà complète sans extension. Y prolonger échouerait de toute façon dès le premier pas : une section transversale prise DANS le carré de confluence, perpendiculaire à cette branche, balaie alors la longueur de la branche perpendiculaire et dépasse `max_satin_width` immédiatement. **Cas `y/trident`** (branches à angle, largeurs différentes) : une queue instable est réellement amputée, et le recouvrement mesuré est non nul (0,3 à 2,5 mm selon la largeur locale).

Ordre de couture (`SatinJunctionPlan`, `plan_junction_stitch_order`) : heuristique déterministe — branche la plus large en premier (cousue dessous), branches plus fines ensuite par largeur décroissante (cousues dessus, masquent les transitions des précédentes). Ne modifie aucune géométrie, ordonne seulement les indices déjà construits.

Résultat mesuré sur `trident` (critère explicite : 3 objets, un par branche, aucune diagonale, aucun rail traversant la confluence, aucun éventail, recouvrements locaux visibles et bornés, déterministe) : 3 objets indépendants (17/27/48 stations denses → 5/3/10 paires structurantes), aucun croisement de rail entre branches (vérifié sur les rails APLATIS, pas les seuls nœuds de contrôle), recouvrement 0,3–1 mm sur les deux branches fines, ordre de couture [`large`, `moyenne`, `fine`]. Inspection visuelle (`trident-parametric.svg`) : rails visuellement lisses (courbure Bézier, pas une succession de petits segments), zone de recouvrement visible en jaune translucide, aucune grande diagonale, aucun grand trou central.

Branché côté apps/desktop ET `autodigitize.cpp` (audit satin, 2026-08-10, suite). Les trois chemins de création satin manuelle (`autoConvertToSatin`, `createSatinObject`, `setStitchType`) ET la numérisation automatique (`auto_digitize`, quand `AutoOptions::use_auto_satin` est actif — le défaut) demandent désormais `geometry_mode = Parametric` et lisent `SatinColumnsResult::parametric_columns` en priorité — repli automatique sur `columns` (mode `Legacy`) géré À L'INTÉRIEUR de `build_satin_columns` lui-même (anneaux, cas refusés), donc aucun cas ne perd de couverture. Un seul point de conversion vers `document::SatinParams` (`autodigitize::satin_params_from_column`, template sur `SatinColumnGeometry/ParametricSatinObject` — mêmes noms de champs, exporté depuis `libs/autodigitize` et réutilisé tel quel par `apps/desktop`) évite de dupliquer la logique entre les deux modes ET entre les deux appelants. Les commandes d'édition de rail satin existantes (`MoveSatinRailNodeCommand`, etc.) opèrent sans modification sur les rails `Parametric` : ce sont de simples `geometry::Path`, seulement plus épars.

Limites connues restantes : les anneaux (`region.holes.size()==1`) restent exclusivement Legacy même en mode Parametric demandé (`parametric_columns` vide, `columns` peuplé comme avant) ; aucune intégration éditeur DÉDIÉE (déplacer une `SatinControlPair` comme entité propre, verrouiller une `SatinAngleGuide` indépendamment du nœud de rail qu'elle partage aujourd'hui) n'est implémentée — seule l'édition de nœud de rail générique est disponible, ce qui couvre l'usage courant mais pas la manipulation fine des paires structurantes en tant que telles.

Génération par barreaux (`fill_satin_columns`, Lot 2)

État : Présent · Testé numériquement · Validé visuellement (SVG) · non validé simulateur/physique.

Quand un satin porte des **barreaux** (`SatinParams.rungs ≥ 2`), la génération utilise `fill_satin_columns` au lieu de `fill_satin` :

- **Correspondance par sections** : chaque paire de barreaux consécutifs découpe les rails en intervalles correspondants, appariés par correspondance locale **ladder** (voir *Correction de l'appariement* plus haut — rails de longueurs, nombres de nœuds et courbures différents autorisés, sans éventail). Les barreaux sont **traversés exactement**.
- **Espacement perpendiculaire** : le pas n'est plus mesuré le long d'un rail mais sur la **ligne médiane** (≈ perpendiculaire aux fils) — on ré-échantillonne la médiane de chaque intervalle par la densité demandée.
- **Séquence** L0, R0, L1, R1, ... déterministe (chaque point cousu traverse la colonne). Un satin **sans barreaux** (manuel/legacy) retombe sur `fill_satin`.

Vérifié : espacement médian régulier, barreaux exacts, rails de longueurs différentes, déterminisme (`tests/unit/stitch/test_satin.cpp`) ; zigzag superposé dans les SVG `tests/golden/auto-satin/`.

Saut plutôt qu'un point continu disproportionné entre deux barreaux non-ruban (audit lettres, 2026-08-12)

État : Présent (`SatinResult::jump_before`) · Testé numériquement (géométrie synthétique + géométrie réelle) · non validé simulateur/physique.

Origine : retour utilisateur sur un projet réel (sceau "GISTRE", texte en périphérie) — le satin automatique produisait un résultat visuellement incohérent sur les lettres, en particulier au coude intérieur d'une lettre en T. Export debug de l'objet en cause (`Satin region 301 - section 1/4`) : deux barreaux consécutifs, `rung0` quasi ponctuel (301 µm, une pointe de section) immédiatement suivi de `rung1` à 5431 µm — rail A avance presque horizontalement entre les deux, rail B presque verticalement (~88° d'écart). `fill_satin_columns` interpolait quand même un ruban continu sur cet intervalle : la ré-échantillonnage par ligne médiane (`resample_by_medial_spacing`) atteint son abscisse cible sur un rail AVANT l'autre quand les deux rails divergent autant, produisant un avant-dernier fil dont un côté est déjà au barreau cible et l'autre non — un point continu d'environ 5 mm, puis le barreau exact lui-même à la même largeur : une diagonale visible qui traverse la lettre, suivie d'un aller-retour quasi identique.

Piste écartée : un seuil de largeur absolue (`barreau > X mm ⇒ saut`). Les sections de lettre touchant une jonction ont légitimement des barreaux larges (5 à 6 mm, cf. *Jonctions : recouvrement local* plus haut — `extend_into_confluence` fait CROÎTRE la largeur en entrant dans le recouvrement, par conception) : un seuil absolu aurait déclenché un saut sur un ruban parfaitement valide, juste large.

Correctif — `non_ribbon_interval` (`libs/stitch_generation/src/satin.cpp`) : compare la DIRECTION d'avance de chaque rail entre deux barreaux consécutifs (`a1.pa - a0.pa` vs `a1.pb - a0.pb`), pas leur largeur. Au-delà de ~75° d'écart (et seulement si les deux avances dépassent 0,8 mm — sous ce plancher, le bruit sous-résolution ne justifie aucun saut), la colonne ne se comporte plus comme un ruban sur cet intervalle : aucun fil intermédiaire n'est interpolé, le barreau suivant devient un point de reprise après un SAUT (aiguille levée) plutôt qu'un point cousu. Une terminaison effilée ORDINAIRE (largeur qui décroît jusqu'à un point, rails toujours co-directionnels) garde des directions colinéaires et ne déclenche donc jamais de saut — seul un changement de direction franc le fait, jamais une simple variation de largeur. Pratique standard en broderie (logiciels commerciaux du métier : lever le fil plutôt que forcer un point disproportionné) plutôt qu'une invention propre au projet.

`SatinResult::jump_before` (indices dans satin) porte l'information jusqu'à `stitch_generation::generate.cpp`, qui émet un `Jump` (aiguille levée) au lieu d'enchaîner un `Stitch` à ces indices (`emit_polyline_with_breaks`, distincte d'`emit_polyline` pour ne pas perturber les passes sans saut — sous-couches, locks). L'orientation entrée/sortie (§ *Entrée/sortie et points de fixation*) inverse `result.satin` : les indices de `jump_before` sont recalculés (`idx_after = taille - idx_avant`, l'arête cassée reste la même arête physique, cf. commentaire dans `generate.cpp`) puis retriés.

Vérifié : `tests/unit/stitch/test_satin.cpp` (coin quasi perpendiculaire → saut, garde-fou anti-faux-positif sur une terminaison effilée ordinaire) ; `tests/unit/auto_satin/test_columns.cpp`, géométrie EXACTE de la lettre en T en cause (contour à 37 sommets) — invariant testé : aucun point cousu ne dépasse la largeur du plus large barreau RÉEL qui l'encadre (pas "aucun point large", qui serait faux pour une jonction légitimement large) ; le mécanisme de saut s'engage bien sur cette géométrie réelle (2 des 4 sections).

Seuil angle/distance plutôt qu'angle brut (deuxième cas réel, 2026-08-13)

État : Présent (`kJumpDegPerMm`) · Testé numériquement (deux cas réels distincts + garde-fous synthétiques) · non validé simulateur/physique.

Défaut trouvé en usage réel : un DEUXIÈME export debug (même sceau, `Satin region 267 - section 1/6`) montrait le même artefact — un point continu d'environ 3,3 mm faisant l'aller-retour — sur un coude à seulement ~68,5°, sous le seuil de 75° calibré sur le premier cas (~88°). Baisser le seuil brut à 50° corrigeait ce deuxième cas mais **régressait** le test « virage » existant (Lot 3, § *Sous-couches et compensation*) : un virage LÉGITIME (rail intérieur droit et court, rail extérieur en long détour) présente un angle bout-à-bout de ~55° — à peine 13° sous le vrai défaut à 68,5°. La fenêtre de seuil brut sûre entre ces deux cas était trop étroite (quelques degrés) pour être fiable face à un futur troisième cas.

Correctif — l'angle brut est remplacé par un rapport **angle/distance parcourue** (`kJumpDegPerMm = 10°/mm`) : ce qui distingue réellement un virage légitime d'un défaut n'est pas l'angle en lui-même, mais la BRUTALITÉ du changement de direction. Le virage légitime étale son virage sur ~14 mm (~4°/mm) ; les deux défauts réels observés concentrent un angle comparable sur seulement 3,7 mm (~24°/mm) et 1,5 mm (~45°/mm) — plus d'un ordre de grandeur d'écart entre le cas légitime le plus proche et le défaut le plus proche, une marge bien plus sûre qu'un seuil d'angle brut ne pouvait l'offrir. `kJumpMinSpan` (0,8 mm) inchangé.

Vérifié : nouveau cas dans `tests/unit/stitch/test_satin.cpp`, géométrie EXACTE des rails/barreaux du deuxième export debug (7 nœuds Bézier épars par rail) ; suite complète de `test_satin.cpp/test_satin_pairing_metrics.cpp` sans régression, y compris le test « virage » qui avait révélé la limite du seuil brut.

Précision a posteriori (audit complémentaire, même jour) : les largeurs proches de `max_satin_width` observées sur 2 des 4 sections dès leur première station ne sont PAS un artefact de recouvrement de jonction (`extend_into_confluence`) comme d'abord supposé — vérifié en dumpant le graphe de squelette brut de cette lettre (`SkeletonGraph`, 5 arêtes/6 nœuds) : ces deux sections sont deux branches INDÉPENDANTES (aucune jonction), et leur largeur de départ élevée reflète simplement la largeur réelle de l'empattement à cet endroit. Le squelette complet a en réalité 5 arêtes ; la 5e (~28 mm, reliant le crochet latéral gauche à la jonction du bas) est REJETÉE dans son intégralité : `compute_column_stations` y mesure une largeur locale >6 mm sur la totalité de ses ~54 stations échantillonnées (vérifié : ~8,1 mm à mi-parcours, calcul indépendant par intersection de segments) — un empattement réellement trop large pour du satin à cet endroit, pas un bug de mesure. Voir *Avertissements auto-satin remontés à l'utilisateur* ci-dessous pour la suite donnée à cette branche rejetée.

Repli tatami sur une branche auto-satin rejetée, jamais de zone sans point (2026-08-13)

État : Présent (`geometry::subtract_polygons`, `AutoResult::warnings`) · Testé numériquement (chaîne complète, géométrie réelle rasterisée, vérification de couverture géométrique de bout en bout) · non validé simulateur/physique.

Défaut trouvé en usage réel (suite directe de l'audit ci-dessus) : quand `build_satin_columns` rejette une branche de squelette (ex. trop large, comme la 5e arête de la lettre en T ci-dessus), le rejet est bien

diagnostiqué (`SatinColumnsResult::warnings`) mais `autodigitize.cpp` ne lisait jamais ce champ. Tant qu'au moins UNE branche de la même région réussit (`sectionCount > 0`), la numérisation automatique se poursuit normalement, satisfaite d'avoir produit *quelque chose* — la portion couverte par la branche rejetée, elle, ne reçoit RIEN (ni satin, ni tatami, ni contour), sans le moindre signal dans l'interface. Sur la lettre en T de l'audit : ~28 mm de squelette disparaissaient silencieusement à chaque numérisation automatique de ce logo.

Première itération (incomplète) : remonter `SatinColumnsResult::warnings` jusqu'à une boîte de dialogue utilisateur (`AutoResult::warnings`), sans générer de point de repli. Retour utilisateur explicite : *"il faut absolument que ça couvre toute la zone"* — un avertissement, aussi visible soit-il, laisse toujours un trou physique dans le motif cousu. Étendu en repli tatami automatique.

Primitive géométrique — `geometry::subtract_polygons(base, cutouts)` (nouveau, `libs/geometry`) : différence booléenne Clipper2 (règle **NonZero**, pas `EvenOdd` comme `clean.cpp/cut.cpp`) entre base et l'union de cutouts. `NonZero` choisi délibérément : des bandes de colonnes satin adjacentes qui se chevauchent légèrement (même barreau partagé) s'annuleraient localement sous `EvenOdd`, laissant un trou parasite au milieu d'une zone pourtant couverte. Comme `NonZero` est sensible à l'orientation (contrairement à `EvenOdd`), `base.outer/base.holes/chaque cutout` sont renormalisés en interne (CCW/CW) avant l'opération — l'orientation d'entrée n'est PAS garantie ailleurs dans `geometry/`, qui utilise `EvenOdd` précisément pour s'en affranchir.

Détection du besoin de repli — signal `STRUCTUREL`, jamais une correspondance de texte sur `warnings` : `network.debug.graph.edges.size() > sectionCount` (le squelette élagué compte plus d'arêtes que de sections produites). Une correspondance sur le texte des avertissements a été tentée puis abandonnée : `warnings` porte aussi des messages purement informatifs sans rapport avec une couverture manquante (ex. *"couture fermée : routage cyclique de quatre sections"* sur un anneau déjà parfaitement couvert), et un simple calcul d'aire du reliquat géométrique SANS ce garde-fou déclenchait à tort sur des réseaux satin entièrement réussis : un rail satin (surtout en mode Parametric, ajusté en Bézier épars) approxime toujours la forme source avec une légère tolérance même quand tout réussit, produisant plusieurs mm² de reliquat "naturel" aux pointes/jonctions — repérage régressé sur les fixtures existantes ("réseau en T", "anneau fin") avant d'ajouter ce garde-fou structurel.

Recouvrement délibéré, pas une couture pile au bord — `shrink_strips_for_cutout` rétrécit chaque bande satin de 0,4 mm avant de la soustraire de la région : le territoire de repli déborde ainsi À L'INTÉRIEUR de la bande satin plutôt que de s'arrêter exactement à son bord. Défaut trouvé par revue lors du premier essai (grandir la bande AVANT soustraction plutôt que la rétrécir) : l'effet est inversé — le repli recule et laisse un interstice, pas un recouvrement. Un recouvrement (léger double-point) est anodin ; un interstice ne l'est pas. Même principe que le recouvrement de jonction (`extend_into_confluence` ci-dessus) et la pratique standard du métier (chevaucher plutôt que raccorder pile, cf. audit Wilcom Hatch — Column B/miter joints : *"les trous ne sont pas des échecs, c'est de la physique attendue ; il faut faire disparaître le trou par le recouvrement, pas par la géométrie seule"*).

Mise en œuvre (`libs/autodigitize/src/autodigitize.cpp`) : pour chaque section satin réussie, une bande approximative (rail A aller + rail B retour, rails aplatis) ; la région source moins l'union rétrécie de ces bandes donne le(s) reliquat(s) ; chaque reliquat $\geq 0,5$ mm² (seuil délibérément bas et INDÉPENDANT de `min_fill_area_mm2` — ce dernier répond à "cette région entière vaut-elle un remplissage ?", pas à "ce reliquat déjà largement couvert mérite-t-il d'être comblé ?") devient un nouvel objet vectoriel + un objet de broderie tatami, nommés explicitement ("zone non couverte par le satin" / "Remplissage repli région"). `AutoResult::warnings` reste rempli (préfixé par la région source) : les avertissements expliquent maintenant POURQUOI une zone est en tatami plutôt qu'en satin comme le reste de la région, la boîte de dialogue desktop (`autoDigitize()/segmentWithAi()`) étant reformulée en conséquence.

Vérifié : `tests/unit/geometry/test_boolean.cpp` (7 cas : trou net, recouvrement toute la région, deux cutouts qui se chevauchent avec des orientations OPPOSÉES sans trou parasite, déterminisme) ; `tests/unit/autodigitize/test_autodigitize.cpp` — géométrie EXACTE de la lettre en T (37 sommets, la même que l'audit ci-dessus), **rasterisée** et passée par la VRAIE chaîne segmentation → vectorisation → auto-satin (pas `build_satin_columns` en isolation) : reconstruction indépendante de la couverture (bandes satin des sections réussies + zones de repli tatami) et vérification géométrique que la

région source moins cette couverture ne laisse aucun reliquat significatif ; suite complète (506 cas) sans régression, y compris les fixtures "réseau en T" et "anneau fin" qui auraient (à tort) déclenché un repli sans le garde-fou structurel.

Barreaux par défaut pour un satin manuel (default_rungs)

État : Présent · Testé numériquement. Correctif de revue.

Défaut trouvé par revue : un satin créé **manuellement** (deux rails seuls, `MainWindow::createSatinObject` via **Broderie ■ Colonne satin...**, cf. `rails_from_contour`) n'avait jamais de barreau, et retombait donc sur `fill_satin` — qui n'implémente qu'un **sous-ensemble** de `SatinConfig` (densité, compensation pull symétrique, sous-couche centrale). Les autres réglages Lot 3/4 (terminaisons, split, sous-couches de bord/zigzag, compensation push/pull asymétrique) restaient **silencieusement sans effet**, alors que l'inspecteur les expose et les laisse éditer sans aucune distinction ni avertissement pour ce type de satin : un utilisateur pouvait activer « Terminaison : Effilée » sur un satin manuel, voir le champ persister dans le `.osp`, sans jamais observer le moindre changement dans le résultat cousu.

Correction : `default_rungs(rail_a, rail_b, spacing)` (nouvelle fonction publique de `satin.hpp`) génère des barreaux par défaut en réutilisant **la même correspondance ladder** que `fill_satin` (aucun nouvel algorithme de correspondance, aucun risque de régression géométrique), à l'espacement `density`. `createSatinObject` les assigne à `SatinParams.rungs`, ce qui fait désormais passer **tout nouveau satin manuel** par `fill_satin_columns` — le seul chemin implémentant l'intégralité de `SatinConfig`. Portée volontairement restreinte à ce seul point d'entrée : un projet historique chargé depuis un `.osp` sans barreaux continue de suivre `fill_satin` sans changement de comportement (rétrocompatibilité), et `fill_satin` lui-même n'est pas modifié.

Vérifié : `default_rungs` produit au moins deux barreaux sur une colonne simple (vide sur des rails dégénérés) ; avec `cap_end = Tapered`, le même rail/config produit un bout à pleine largeur via `fill_satin` (réglage ignoré, ancien comportement) contre un bout réellement effilé via `fill_satin_columns + default_rungs` (réglage appliqué) — démontre que le défaut est bien corrigé, pas seulement que des barreaux existent.

Édition interactive de plusieurs guides

Sélectionner une colonne satin puis activer **Broderie ■ Éditer les guides satin...** (raccourci G) affiche chaque barreau et deux poignées. Une extrémité peut être glissée indépendamment : elle est projetée exactement sur son rail et la colonne est régénérée. Plusieurs guides successifs imposent donc plusieurs orientations locales ; `fill_satin_columns` interpole la correspondance de façon monotone entre chaque paire de guides.

Un geste est refusé s'il ferait franchir deux guides sur un seul rail ou les rapprocherait de moins d'un demi-pas de densité. Cette validation partage l'invariant du générateur, ce qui évite qu'un guide visible soit ensuite ignoré silencieusement. Chaque glisser produit une seule commande annulable **Déplacer un guide satin**. Les guides restent les `SatinParams.rungs` historiques : la persistance `.osp` demeure rétrocompatible.

Un clic sur le segment d'un guide le sélectionne et le met en évidence. La commande **Ajouter un guide satin** (Maj+G) partage automatiquement le plus grand intervalle disponible sur les deux rails ; elle est refusée si le nouveau guide serait trop proche de ses voisins. **Supprimer le guide satin sélectionné** conserve toujours au moins deux guides, seuil nécessaire à une colonne paramétrique. Ajout, suppression et déplacement utilisent trois commandes distinctes et sont entièrement annulables/rétablissables. L'insertion préserve aussi l'ordre de progression croissant ou décroissant des guides, afin que les extrémités employées par le routage textile restent exactes.

Le guide terminal qui porte une **jonction de réseau** est structurel : il reste sélectionnable et explicitement marqué comme verrouillé, mais ses poignées et l'action de suppression sont désactivées. Le générateur ne couvre que l'intervalle compris entre ses guides extrêmes ; déplacer ou supprimer ce guide pourrait donc raccourcir une section et créer un vide ou un repli à la jonction. Quand ce guide verrouillé est sélectionné, **Ajouter un guide satin** crée un guide interne dans l'intervalle directement adjacent de chacune des

sections incidentes — jamais dans un grand vide éloigné de la jonction. L'opération est globale : si le réseau est incomplet, une section dupliquée ou un intervalle inadmissible, aucun guide n'est ajouté. Une seule commande undo/redo couvre toutes les sections. Chaque guide interne reste ensuite déplaçable section par section ; ce mécanisme amorce l'orientation de toutes les branches sans déplacer la jonction structurelle. Celle-ci est reconnue par l'ordre géométrique des stations projetées sur les deux rails, et non par son index de stockage, y compris après un import aux barreaux inversés ou désordonnés.

Chaque groupe de guides internes créés ensemble par un ajout coordonné à une jonction porte un `link_id` (`SatinRung.link_id`, optionnel) — un identifiant local au réseau (`source_vector`), alloué de façon déterministe et monotone par `next_satin_guide_link_id` (échoue explicitement si les identifiants `uint32` du réseau sont épuisés plutôt que d'en réémettre un déjà utilisé). Un guide ajouté indépendamment n'a pas de `link_id` ; les projets historiques n'en ont jamais. Persisté dans le `.osp` (`rungs[]`.`linkId`, validation stricte : entier `uint32` exact, aucune troncature silencieuse d'une valeur négative/flottante/hors bornes).

Invariant central : un groupe lié est identifié par la paire (`source_vector`, `link_id`) — jamais par le seul `link_id`, qui n'est pas globalement unique entre réseaux distincts. `satin_linked_guides` énumère les membres d'un groupe (un guide par section touchée par la jonction d'origine, jamais plus), rejette la totalité si une section porte deux guides du même `link_id` (donnée corrompue), si une section déclarée manque, si les membres ne touchent pas la même jonction ou si leur progression n'est pas strictement monotone sur les deux rails, et trie le résultat par (`section_index`, `embroidery_id`, `guide_index`) — mutation et énumération restent donc déterministes indépendamment de l'ordre des objets dans le document, et un même identifiant numérique réutilisé dans un autre réseau (`source_vector` différent) n'est jamais confondu avec le groupe.

Suppression de groupe atomique. Supprimer un guide lié (Broderie ■ Supprimer le guide satin sélectionné) supprime désormais l'identité logique entière : tous les guides du groupe, dans toutes leurs sections, en une seule commande **Supprimer des guides satin coordonnés** (`RemoveSatinGuidesCommand`) annulable/rétablissable comme un tout. Refusée en bloc — aucune section n'est touchée — si une section tomberait à moins de deux guides restants.

Déplacement de groupe atomique — geste explicite. Un glisser ordinaire d'une extrémité de guide (sans modificateur) reste **local** : il ne modifie que l'angle de cette section, comme avant (édition d'angle intrinsèquement locale — deux sections d'un même réseau peuvent légitimement avoir des largeurs et orientations différentes). Pour déplacer tout le groupe de façon cohérente, l'utilisateur maintient **Maj** en glissant une extrémité d'un guide lié : `move_satin_guide_group` calcule, pour la section glissée, sa position normalisée `t` dans l'intervalle [`station de la jonction`, `station du voisin suivant`] (sur chaque rail), en déduit un delta normalisé unique à partir du point relâché, puis applique **ce même delta** à `t` dans **chaque** section du groupe — recalculée entièrement à partir de la géométrie propre de cette section (ses propres rails, sa propre densité). Aucune coordonnée ni angle brut n'est jamais recopié d'une section à l'autre ; seul le delta normalisé traverse la frontière. Le résultat est appliqué par une seule commande **Déplacer des guides satin coordonnés** (réutilise `MoveSatinGuidesCommand`), tout ou rien : si une seule section sortirait de son intervalle admissible (moins d'un demi-pas de densité de sa marge, ou guide plus adjacent à sa jonction — topologie modifiée entretemps), aucune section n'est mutée et un message explique le refus. Un Maj+clic sans déplacement est un no-op exact et ne crée pas d'entrée dans l'historique. L'infobulle du guide et le message de statut du mode documentent ce geste.

Limite actuelle : le déplacement de groupe ne s'applique qu'aux guides internes directement adjacents à leur jonction (ceux créés par l'ajout coordonné) — un guide qui aurait perdu cette adjacence (un autre guide inséré entre lui et la jonction) redevient local et refuse le geste de groupe plutôt que de deviner un intervalle arbitraire. Le guide structurel de jonction lui-même reste volontairement verrouillé (ni déplaçable ni supprimable) ; le déverrouiller exigerait un calcul garantissant simultanément la couverture, la monotonie et l'absence de repli dans toutes les sections incidentes.

Finitions : points courts, split, terminaisons (Lot 3)

État : Présent · Testé numériquement · Validé visuellement (SVG). Toutes ces options sont **désactivées par défaut** (comportement inchangé) et éditables dans l'inspecteur (section satin) ; elles sont persistées dans le `.osp`.

- **Points courts** (`ShortStitchMode`) : dans un virage serré (le rail intérieur avance beaucoup moins que l'extérieur), les pénétrations intérieures sont *rentrées* vers l'axe (`SingleInset`, `MultiLevelInset` = motif triangulaire de profondeurs) pour ne pas s'entasser sur le bord, ou allégées (`RemoveAndRedistribute` : un fil serré sur deux, jamais un barreau, jamais deux d'affilée).
- **Split** (`SplitStitchMode`) : une traversée plus longue que `max_stitch_length` reçoit des pénétrations intermédiaires. `Simple` (colinéaire régulier), `Staggered` (décalage cyclique) et `DeterministicJitter` (variation **reproductible**, graine = identifiant de l'objet) évitent une **ligne centrale** visible. Jamais d'aléa non déterministe.
- **Terminaisons** (`SatinCapType`) : `Flat` (barreau final), `Rounded` (réduction arrondie de largeur), `Tapered` (effilé vers une pointe **sans** empiler sur une coordonnée unique — largeur minimale ~18 %), `Automatic`.

Vérifié : inset modifie le rail intérieur en virage, `RemoveAndRedistribute` réduit les pénétrations, split ajoute des points décalés (`staggered` ≠ ligne centrale ; jitter déterministe), taper réduit la largeur au bout sans l'annuler (SVG tests/golden/auto-satin/lot3-*.svg : effilé 5,00 → 0,91 mm).

Sous-couches et compensation (Lot 4)

État : Présent · Testé numériquement · Validé visuellement (SVG). Toutes désactivées par défaut, éditables (inspecteur) et persistées (`.osp`).

Modèle de passes (§4) : chaque commande de la séquence porte une `StitchPass` (`Underlay`, `TopStitch`, `Travel`, `Lock`, `Manual`) — non sérialisée (la séquence est régénérée), qui permet d'**identifier, filtrer et analyser** les passes séparément. Les sous-couches sont émises **avant** la couche supérieure.

- **Center walk** : point droit sur la médiane (retrait aux extrémités).
- **Edge walk** : deux chemins internes rentrés des rails (`underlay_edge_inset`).
- **Zigzag underlay** : satin léger (largeur réduite, pas plus grand).
- Ordre : **center** → **edge** → **zigzag** → **satin** (recommandé).

Compensation (§11), appliquée à la géométrie des paires avant génération :

- **Pull latérale asymétrique** : décalage par côté = base symétrique (`pull_compensation`) + fixe gauche/droite (`pull_left/pull_right`) + proportionnel à la largeur, borné (`pull_max`).
- **Push longitudinale** : `push_start/push_end` étendent (ou rétractent) la colonne aux extrémités le long de l'axe.

Correctif de revue — rétraction bornée. Contrairement à la compensation pull (bornée par `pull_max`), `push_start/push_end` n'avaient **aucune borne** : une rétraction excessive (`push` très négatif, p. ex. une valeur saisie par erreur en mm au lieu de µm) faisait passer le premier (ou dernier) fil de la colonne **de l'autre côté** de son voisin, inversant leur ordre le long de l'axe — un point auto-croisé visible en tout début/fin de colonne, la sous-couche `mids/cumMid` étant elle-même calculée **après** ce décalage (§ sous-couches ci-dessus, potentiellement corrompue en cascade). Corrigé : une rétraction ne dépasse désormais jamais la station voisine (borne dérivée, pas un seuil arbitraire — la même logique que l'exclusion des rails à l'ancrage des jonctions plus haut). Une extension (`push` positif) reste volontairement non bornée : elle éloigne le fil sans jamais croiser un autre point. Vérifié : une rétraction demandée de -50 mm sur une colonne de 1 mm de densité reste proche de son point d'origine (tests/unit/stitch/test_satin.cpp).

Vérifié : center/edge/zigzag = 4 passes distinctes ordonnées, pull élargit **un seul côté** (asymétrique), push étend le bout, déterminisme, aller-retour `.osp` (SVG tests/golden/auto-satin/lot4-*.svg : sous-couches en vert).

Défaut trouvé en usage réel (2026-08-13) — sous-couche centrale à la densité du zigzag principal au lieu de `underlay_spacing` : sur un satin **avec barreaux** (`fill_satin_columns`, auto-satin ou satin manuel avec barreaux par défaut — le chemin réellement emprunté par la quasi-totalité des satins de l'application), la sous-couche centrale reprenait simplement le milieu (`mids`) de **chaque fil** du zigzag principal, lui-même espacé à `density` (souvent 0,4 mm) — au lieu d'être ré-échantillonnée à `underlay_spacing` (2 mm par défaut). `underlay_spacing` existe et est correctement utilisé par `fill_satin` (satin manuel/legacy, sans barreaux) ; `fill_satin_columns` l'ignorait silencieusement en réutilisant `mids` tel quel. Conséquence concrète : des points de sous-couche de 0,3-0,4 mm au lieu de ~2 mm, systématiquement signalés point-court par l'analyse (risque de casse du fil et de sur-densité) — et un nombre de points de sous-couche inutilement élevé (5x plus que nécessaire à densité 0,4 mm). Corrigé en ré-échantillonnant `mids/cumMid` (déjà calculés pour `keepEnd`) à `underlay_spacing` via `point_at` (même primitive que le reste du fichier), sur l'intervalle `[retract, totalMid - retract]` — bornes identiques à l'ancien filtre par indice `keepEnd`. Vérifié : `tests/unit/stitch/test_satin.cpp` (colonne de 20 mm, densité 0,4 mm, `underlay_spacing` 2 mm → ~10 points de sous-couche, tous espacés de plus de 0,5 mm — pas ~50 points à 0,4 mm comme avant le correctif).

Reste (lots suivants) : tatami avancé (Lot 7). Voir `docs/stitch-feature-gap-audit.md`.

Entrée/sortie et points de fixation (Lot 5)

État : Présent · Testé numériquement · Validé visuellement (SVG). Éditables (inspecteur) et persistés (.osp).

Points d'entrée/sortie (§12) : `entry_point` et `exit_point` (optionnels) orientent la couture. Le satin est **retourné** si son extrémité de départ est plus proche du point de sortie que du point d'entrée (coût = somme des distances extrémités↔points) — jamais de réordonnancement partiel, seulement le sens. Sans point, l'orientation issue du générateur est conservée.

Points de fixation (SatinLock, émis en passe Lock uniquement) : ancrent le fil au **début** et à la **fin** de l'objet, jamais par sous-passe (un lock au départ, un à l'arrivée). Chaque bout se règle indépendamment (`lock_start/lock_end`), avec longueur (`lock_length`) et nombre de passages (`lock_passes`) communs.

- **None** : aucun point de fixation.
- **BackAndForth** : aller-retour court le long du premier/dernier point.
- **Triangle** : petit triangle d'ancrage.
- **MicroZigzag** : micro-zigzag progressant dans l'axe de couture.

Toutes les formes sont **bornées** (`lock_length`, défaut 0,8 mm) et ancrées à l'extrémité exacte du satin (continuité : pas de saut parasite grâce à l'enchaînement de passes à position identique).

Correctif de revue — borne réelle, pas seulement nominale. `lock_length` n'était en réalité **jamais comparée** à la distance disponible : `toward` (le point voisin qui donne la direction du lock) est le point du rail **opposé** — donc la largeur locale du barreau, pas un point lointain le long de la couture. Sur une colonne fine (largeur < `lock_length`, cas courant pour du texte fin ou un petit motif), le lock piquait **hors de la matière déjà cousue**, dans du tissu vierge, sans qu'aucune borne ne l'en empêche. Corrigé : la longueur effective du lock ne dépasse désormais jamais la distance réelle vers `toward` (repli sur la longueur demandée uniquement dans le cas dégénéré où `toward == anchor`, sans référence géométrique fiable). Vérifié : sur une colonne de 0,3 mm de large avec `lock_length` par défaut (0,8 mm), tous les points du lock restent dans la largeur réelle, quel que soit le type (`tests/unit/stitch/test_satin.cpp`).

Vérifié : `None` → vide ; les autres progressent dans l'axe de couture et restent bornés ; le point d'entrée fait démarrer la couture à la bonne extrémité ; locks en passe Lock, exactement deux groupes (début/fin) ; aller-retour .osp des champs Lot 5 (SVG `tests/golden/auto-satin/lot5-lock-*.svg` : fixations en rouge).

Routage multi-colonnes (Lot 6)

État : Présent · Testé numériquement · Validé visuellement (SVG). Automatique à la génération ; aucun réglage persisté.

Une forme décomposée (Y, T, croix...) devient **plusieurs colonnes satin** de même couleur et même source. Sans routage, elles seraient cousues dans l'ordre du document, avec de longs sauts. §13 les ordonne et les oriente ensemble (`route_columns`) :

- **Ordre** : une jonction explicite commune et géométriquement admissible prime sur une proximité fortuite ; à égalité, glouton plus proche voisin depuis la position courante de l'aiguille, puis amélioration **2-opt**. Déterministe.
- **Orientation** : pour un ordre donné, le sens de chaque colonne est résolu **exactement** par programmation dynamique sur ses deux extrémités : maximum de liaisons par jonction, puis distance minimale (l'orientation choisie est imposée à `generate_satin` via entrée/sortie).
- **Liaisons** : une transition **justifiée par une jonction commune validée** tolère jusqu'à `underpath_max` (défaut 8 mm) en **trajet caché** (passe `Travel`, `running stitch` — pas de coupe) ; **sans** jonction, seul un quasi-contact (`underpath_max_without_junction`, défaut 1,5 mm — bien plus strict) l'autorise. Au-delà de la borne applicable, la liaison reste un **saut**. Minimise les coupes et les déplacements à découvert.

Une jonction déclarée mais séparée de plus de `underpath_max` est traitée comme incohérente et n'influence ni l'ordre ni le type de liaison. Cette garde évite qu'un `.osp` altéré force un trajet caché arbitraire. Limite actuelle : le trajet entre deux sections reste un segment échantillonné ; il ne suit pas encore le centre d'une branche déjà cousue lors d'un retour vers une jonction.

Correctif « auto-satin béton » (suite) : `route_columns` ne regardait la jonction que pour l'ORDRE et l'ORIENTATION ; le TYPE de liaison (trajet caché ou saut) ne dépendait que de la distance, sans jamais vérifier qu'elle était justifiée par cette même jonction. Deux colonnes **sans aucun lien topologique** mais distantes de 5 à 8 mm — deux lettres rapprochées, une forme en C dont les deux bouts se frôlent sans être reliés — étaient donc cousues en trajet caché à travers un espace dont rien ne garantissait qu'il était couvert de tissu. Corrigé : le type de liaison distingue désormais explicitement les deux cas (jonction validée vs proximité seule), chacun avec sa propre borne. Une jonction reste géométriquement anchrée exactement (§ ancrage des jonctions ci-dessus), donc digne de confiance jusqu'à 8 mm ; une simple proximité, sans cette garantie, ne l'est que jusqu'à 1,5 mm.

Le groupe routé est **contigu** et limité aux colonnes auto (porteuses de barreaux) de couleur et source identiques : l'ordre inter-groupes et le reste du document sont préservés. Un satin manuel isolé n'est pas réordonné.

Correctif de revue — le point de décision doit être le point réellement cousu. `column_endpoints` (extrémités représentatives d'une colonne pour le routage) utilisait le milieu **brut** des barreaux d'about, alors que la génération réelle (`fill_satin_columns`) peut **décaler** ces mêmes extrémités via `push_start/push_end` (§ sous-couches et compensation, Lot 4) avant de coudre. La décision Underpath/saut se fondait donc sur un point différent de celui effectivement cousu : un écart évalué juste sous le seuil pouvait, une fois le décalage réel appliqué, le dépasser (trajet caché accordé à travers un espace non garanti couvert — précisément le défaut visé par le correctif « auto-satin béton (suite) » ci-dessus, réintroduit par un chemin différent), ou inversement imposer un saut évitable. Corrigé : `column_endpoints` reproduit désormais le même décalage (et la même borne de rétraction, § Lot 4) que celui réellement appliqué à la génération. Vérifié : deux colonnes dont l'écart **brut** entre barreaux (900 µm, sous le seuil sans jonction de 1,5 mm) aurait autorisé un trajet caché, mais dont l'écart **réel** une fois `push_end` (rétraction de 700 µm) appliqué dépasse ce seuil (1,6 mm), produisent désormais un saut — pas un trajet caché injustifié (`tests/unit/stitch/test_routing.cpp`).

Vérifié : liste vide → plan vide ; une colonne → liaison de départ, aucun saut ; réordonnement minimisant le déplacement ; orientation par l'extrémité proche ; liaison longue → saut, liaison courte → trajet caché ; **liaison proche (5 mm) SANS jonction commune** → **désormais un saut** (défaut corrigé) ; la même liaison justifiée par une jonction reste un trajet caché ; un quasi-contact (1 mm) sans jonction reste toléré ; à la génération, un groupe adjacent n'émet qu'un saut initial (le reste cousu), un groupe éloigné **ou**

sans jonction commune conserve ses sauts (SVG `tests/golden/auto-satin/lot6-route-*.svg` : trajets cachés en bleu, sauts en rouge pointillé).

Limite : le trajet caché est un segment **direct** (échantillonné en points cousus) ; il n'est garanti *sous* la broderie que pour des colonnes reliées par une jonction (ou en quasi-contact). Un routage suivant réellement la matière viendra avec le tatami avancé (Lot 7).

Satin Coverage Analyzer : mesurer objectivement la surface couverte (2026-08-13)

État : Présent (`satin_coverage::analyze_satin_coverage`) · Testé numériquement (7 cas géométriques à résultat connu) · Observateur indépendant, n'influence encore aucune décision du générateur.

Origine

L'audit des ruptures de ruban (§ *Saut plutôt qu'un point continu...* et *Seuil angle/distance...* ci-dessus) et le repli tatami sur branche rejetée (§ précédente) partagent un point aveugle commun : aucun des deux ne répond, de façon indépendante et vérifiable, à la question « les colonnes générées couvrent-elles réellement l'intégralité de la forme d'origine ? ». Les validations existantes portent sur la cohérence des rails, du squelette, des largeurs ou des jonctions — jamais sur une comparaison de SURFACE entre la région source et ce que les colonnes balaient effectivement. Un squelette valide, des sections toutes construites sans refus, et pourtant une zone entière peut rester sans le moindre point : rien dans les validations structurelles précédentes ne le détecte directement.

Principe : comparer deux surfaces, jamais une heuristique de squelette

`satin_coverage::analyze_satin_coverage(target, columns, config)` (`libs/satin_coverage/`, nouvelle bibliothèque, dépend uniquement de `core`, `geometry`, `stitch_generation` — jamais de `auto_satin` ni `document`, pour rester un **observateur totalement indépendant du générateur**, utilisable depuis un test synthétique aussi bien que depuis `autodigitize`) calcule :

1. pour chaque colonne, la suite ordonnée de stations (L■, R■) via `stitch_generation::satin_stations` (nouvelle fonction publique, extraite sans changement de comportement de la correspondance ladder déjà utilisée par `fill_satin_columns` — cf. *Correction de l'appariement* plus haut) : la géométrie STRUCTURELLE des rails/barreaux, jamais les points de couture finaux (compensation pull, split-stitch, terminaisons), qui répondent à un besoin de couture et non à la question « les rails couvrent-ils la forme d'origine ? ». Une rupture de ruban (`SatinStation::jump_before`, § audit lettres/seuil angle-distance ci-dessus) coupe la colonne : aucun quadrilatère de couverture ne relie deux stations de part et d'autre d'un saut — le même mécanisme qui empêche un point cousu disproportionné empêche aussi un quadrilatère de couverture artificiel ;
2. chaque intervalle entre deux stations consécutives devient un quadrilatère (L■, R■, R■■■, L■■■), décomposé en DEUX TRIANGLES (diagonale L■-R■■■) — robuste à un quadrilatère non convexe, et surtout : un garde-fou défensif (`segments_cross_strict` sur L■-R■ vs L■■■-R■■■) exclut et COMPTE tout intervalle où les deux rails se croiseraient, plutôt que de laisser un quadrilatère dégénéré s'ajouter silencieusement à l'union — l'analyseur ne fait jamais confiance aveuglément à l'invariant anti-croisement de l'appariement amont, même si celui-ci est déjà testé séparément ;
3. tous les triangles de toutes les colonnes sont réunis par une union booléenne `NonZero` (`geometry::union_nonzero`) → C. Chaque triangle est explicitement réorienté CCW avant l'union : `union_nonzero`, contrairement à `intersect_polygons/difference_polygons` (ajoutés par ce lot, voir plus bas), ne renormalise pas l'orientation d'entrée — deux triangles chevauchants d'orientations opposées s'annuleraient localement sous `NonZero`, créant un trou parasite qui masquerait précisément le genre de défaut que cet outil doit révéler (piège identifié dès la conception, pas trouvé après coup) ;
4. C est comparé à la région cible P (`geometry::PathSet`, trous compris) par intersection/différence booléennes `NonZero`, trous compris DES DEUX CÔTÉS (`geometry::intersect_polygons/difference_polygons`, nouvelles primitives de `libs/geometry` — contrairement à `subtract_polygons` existant, dont les cutouts sont de simples contours sans trou,

ces deux nouvelles fonctions gèrent le cas d'une poche non couverte elle-même entourée de couverture de tous côtés, c'est-à-dire un TROU dans C) : `covered = aire(P ∩ C)`, `missing = aire(P \ C)`, `outside = aire(C \ P)`.

Composantes manquantes et profondeur des trous

Chaque composante connexe de `missing` est déjà un élément distinct du `vector<PathSet>` renvoyé par `difference_polygons` (issu directement du PolyTree64 de Clipper2) : aucun algorithme de composantes connexes séparé n'est nécessaire. Pour chacune (`MissingRegion`) : aire, part de la cible, centroïde/bbox (barycentre des sommets), et un `max_gap_radius_mm` — le rayon du plus grand disque inscrit, obtenu par **recherche binaire d'érosions successives** (`geometry::inset_path_set`, déjà présent, déjà testé) jusqu'à disparition de la région, plutôt qu'un raster + transformée de distance OpenCV : précision continue, aucune dépendance nouvelle, et cette bibliothèque reste sans dépendance OpenCV. Distingue une frange fine de plusieurs mm² le long d'un bord (rayon petit, peu grave) d'une poche ronde de même aire en plein milieu (rayon significatif, vrai défaut).

Une **couverture du cœur** (`core_coverage_ratio`) complète la couverture brute : P est érodé de `boundary_tolerance_mm` (0,1 mm par défaut) avant de recalculer le manquant — absorbe les approximations de discrétisation le long du bord sans masquer une vraie poche interne. Sur une forme trop fine pour survivre à cette érosion, repli documenté sur la couverture brute plutôt qu'une division par zéro.

Rapport et seuils explicites

`SatinCoverageReport` (aires cible/couverte/manquante/hors-forme, ratios brut et cœur, régions manquantes triées par aire décroissante, rayon de trou maximal, `passed`, message texte diagnostic prêt à logger — jamais un simple « couverture insuffisante », toujours les chiffres et la raison précise du refus) et `SatinCoverageConfig` (tous les seuils : couverture brute/cœur minimales, aire/rayon de trou maximaux, ratio hors-forme maximal, tolérance de bord, résolution de la recherche de rayon — valeurs par défaut raisonnables, explicitement PAS des constantes universelles du métier, ajustables sans toucher au code de l'analyseur).

Vérifié

Sept cas géométriques à résultat connu (`tests/unit/satin_coverage/test_coverage.cpp`, 43 assertions) : rectangle entièrement couvert (~100 %, PASS) ; rectangle couvert à moitié (~50 %, FAIL, une seule région manquante de l'aire attendue) ; trou de la forme source (anneau tuilé par quatre colonnes) jamais compté comme manquant ; vraie poche non couverte entourée de matière (FAIL, rayon de trou mesuré conforme au rectangle de la poche à 0,05 mm près) ; débordement hors cible (`outside_area > 0`, cible elle-même toujours entièrement couverte) ; deux colonnes qui se chevauchent (l'union fait foi, aucun double comptage) ; et surtout **trident** (jonction concave asymétrique à 3 branches inégales, forme historiquement problématique pour Auto-Satin, cf. audits ci-dessus) : couverture complète (toutes branches) 88,1 %, plus grande zone manquante 18,0 mm² (le noyau de jonction résiduel déjà documenté § *Optimisation globale des bridges...* — un résidu CONNU, pas un défaut de cet audit) ; en retirant la branche PRINCIPALE (simulant un rejet amont), couverture 9,4 %, plus grande zone manquante 157,8 mm² — démonstration que l'analyseur distingue nettement « toutes les branches présentes » de « une branche absente », par une mesure de surface et non un décompte de branches/stations traitées. Suite complète (514 cas ctest) sans régression, Debug et Release, y compris la suite `stitch/auto_satin/autodigitize` complète après l'extraction de `satin_stations` (comportement bit-à-bit identique à l'ancien `fill_satin_columns`).

Portée actuelle et limites assumées : observateur pur, n'influence encore aucune décision de génération. Pas de second mode « couverture des points finaux » (§16 de la demande d'origine). `degenerate_interval_count` reste à 0 sur toutes les fixtures testées à ce jour (l'invariant anti-croisement amont tient) ; le garde-fou existe pour le jour où ce ne sera plus le cas plutôt que pour un défaut déjà observé.

Non-régression sur le corpus de formes historiques

État : Présent (`tests/unit/auto_satin/test_coverage_regression.cpp`).

Photographie la couverture géométrique actuelle de dix formes du corpus `auto_satin::shapes` (`rectangle`, `capsule`, `ribbon`, `s`, `y`, `t`, `cross`, `h`, `ring`, `trident` — `circle/tiny/notch/pinch` exclus, testés ailleurs pour leur comportement de refus, pas pour leur couverture) : un plancher de couverture brute par forme, mesuré en calibrant ce test puis abaissé de quelques points de marge — un vrai plancher de non-régression, **pas** un objectif de qualité (les résidus de noyau de jonction sur `y/t/cross/h/trident`, § *Optimisation globale des bridges...* et § *Séparation StableBranchEnd/JunctionSeparator...* plus haut, sont un fait déjà documenté, pas un défaut que ce test cherche à faire disparaître). Sans surprise, `rectangle/ring` sont quasi parfaits (96 %/99,99 %) ; les formes à jonction se situent entre 85,8 % (`cross`) et 88,7 % (`t`).

`wide` (`rectangle` délibéré 100 × 20 mm, `shapes.cpp` : « un vrai cas trop large pour satin ») refuse entièrement dès l'analyse de satinabilité (`Unsuitable`, `has_wide_area` : sa largeur de 20 mm dépasse `max_satin_width`, 9 mm par défaut, `satinability.cpp:76`) — **comportement voulu par construction**, pas un défaut. Exclue du corpus de ce test pour la même raison que `circle/tiny/notch/pinch`.

Diagnostic « wide » (2026-08-13) : confirmation, pas de défaut trouvé

Investigation demandée explicitement pour vérifier l'hypothèse ci-dessus. Trace complète : `evaluate_satinability` (`satinability.cpp:76`) calcule `maxWidthUm` à partir de la transformée de distance du raster de la région (le rayon maximal du disque inscrit, doublé) ; pour `wide` (100 × 20 mm), cette largeur maximale est bien ~20 mm, très au-delà de `max_satin_width` (9 mm, `SatinColumnsParameters::analysis.thresholds`) — `has_wide_area` passe vrai, `status` devient `Unsuitable`, message « Zone trop large pour un satin (envisagez un tatami). », et `build_satin_columns` s'arrête avant toute construction de rail (`satin_column.cpp:2977`, refus immédiat sur statut `Unsuitable`). Aucune trace d'un défaut de squelette, de section transversale ou de jonction : le refus intervient AVANT que le pipeline géométrique ne s'exécute, sur un seuil de configuration simple et correctement appliqué.
Conclusion : wide fonctionne exactement comme conçu (rediriger une forme trop large vers un remplissage tatami, cf. § *Colonne trop large* plus haut) ; la note précédente (« surprenant, piste non investiguée ») provenait d'une lecture incomplète de `shapes.cpp` avant d'avoir vérifié le seuil réellement en cause — corrigée ici plutôt que laissée en l'état.

Diagnostic « trident »/pixel_size (2026-08-13) : pas de perte de couverture réelle, fragilité Legacy documentée

Investigation demandée explicitement, en plusieurs passes (deux hypothèses de correctif tentées et invalidées par la mesure avant la conclusion ci-dessous — aucune des deux n'a été committée).

Symptôme observé (`openstitch-cli auto-satin-debug --shape trident`, mode Legacy, `--satin-geometry=legacy`) : à `pixel_size = 0,05` mm (défaut CLI), la branche latérale de `trident` échoue (« colonne refusée : saut de largeur incohérent entre stations finales #1/2 »), ce qui fait échouer la validation de jonction (« jonction 1 incomplète, 2/3 branches disponibles ») et refuse **toute la région**. À `pixel_size = 0,1` mm (convention des fixtures de test), la même forme réussit (88,1 % de couverture).

Deux hypothèses testées, toutes deux invalidées par instrumentation directe (dump des largeurs de station réelles, `OS_DEBUG_STATIONS`, retiré avant commit) :

1. *Pointe effilée mal exemptée près du bout OUVERT.* Correctif tenté : n'appliquer le contrôle de saut de largeur qu'au tiers médian de l'axe (mêmes bornes que `representative_station_width`). **Invalidé** : la branche fautive ne compte que 5 stations après amputation de jonction, un tiers médian trop étroit pour exempter l'intervalle fautif — aucun changement observé après correctif.
2. *Amputation de jonction qui s'arrête trop tôt faute de palier stable.* Correctif tenté : étendre `trim_unstable_junction_tail` pour continuer d'amputer tant que la frontière reste incohérente avec sa voisine immédiate (même seuil que la validation séparée). **Invalidé** : l'intervalle fautif (stations #1/#2, largeurs 881 µm puis 4393 µm) se situe près du bout **OUVERT**, pas du bout de jonction — `trim_unstable_junction_tail` n'amputant QUE côté jonction par construction, aucune amputation supplémentaire ne pouvait l'atteindre. Largeurs réelles observées sur cette branche : 807,5 → 881,1 → **4393,0** → 7606,2 → 9423,7 µm — un saut brutal et isolé juste après la pointe, puis une remontée globalement cohérente vers la confluence.

Cause probable (non creusée plus loin, cf. portée ci-dessous) : la station à 4393 µm correspond à la station d'axe #23, signalée ailleurs comme « interpolée » (échec isolé de `cross_section` comblé par interpolation linéaire, § *Génération partielle sur formes concaves*). Sur cette branche courte et proche d'une confluence très asymétrique, la section transversale calculée depuis la seule tangente locale peut basculer brutalement de « mesure de cette branche seule » à « balayage de la branche voisine bien plus large », dès la station suivant la pointe — un phénomène de bord plausible pour une géométrie aussi contrastée, mais dont le mécanisme exact dans `cross_section` n'a pas été tracé en détail.

Conclusion — pas de correctif nécessaire, car pas de perte de couverture réelle. Relecture de `libs/autodigitize/src/autodigitize.cpp` :

1. `autodigitize.cpp` utilise le mode **Parametric** par défaut (`geometry_mode = SatinGeometryMode::Parametric`, ligne 189), jamais **Legacy** — alors que le CLI `auto-satin-debug` testé ci-dessus utilise **Legacy** par défaut (`--satin-geometry=legacy`). Le symptôme observé n'a donc jamais été observé dans le chemin réellement emprunté par la numérisation automatique.
2. Même le mécanisme de refus en cascade (« jonction incohérente ») qui transforme un échec de branche isolé en refus de LA RÉGION ENTIÈRE est spécifique à **Legacy** : le mode **Parametric** n'a « aucune ancre centrale commune, aucun sommet de bissectrice... » (§ *Jonctions : recouvrement local, sans ancre* plus haut) — un échec de branche y resterait individuel, sans faire échouer les deux autres.
3. Et si malgré tout `build_satin_columns` refusait la totalité (`columns` ET `parametric_columns` vides), `autodigitize.cpp` (ligne 300-315) retombe automatiquement sur un remplissage **tatami de la région entière** dès que `sectionCount == 0` — aucune zone sans point, juste un changement de type de point (satin → tatami).

Trois filets indépendants (mode par défaut différent de celui testé, refus non cascadié en mode **Parametric**, repli tatami région entière en dernier recours) protègent donc la couverture livrée à l'utilisateur, même dans le pire des cas où cette fragilité de construction de colonne **Legacy** se manifeste. **Rien à corriger côté couverture** ; la fragilité elle-même (refus d'une branche courte proche d'une confluence très asymétrique, à certaines résolutions de raster, en mode **Legacy**) reste une piste d'amélioration future légitime de robustesse — pas urgente, et hors du périmètre de cet audit de couverture.

Balayage du corpus de formes (mode × `pixel_size`) : deux défauts réels trouvés et corrigés (2026-08-13)

État : Présent, corrigé. `geometry::inset_path_set` (`libs/geometry/src/offset.cpp`) et le câblage `--coverage-svg` du CLI (`apps/cli/main.cpp`).

Utilisation directe de l'analyseur pour balayer les 15 formes du corpus `auto_satin::shapes`, en **Legacy** ET **Parametric**, à `pixel_size` 0,05 et 0,1 mm (60 exécutions, `openstitch-cli auto-satin-debug --coverage-svg`) — pas encore automatisé en test, exploration manuelle guidée par les chiffres bruts. Deux défauts RÉELS trouvés, tous deux corrigés séance tenante, vérifiés sans régression (517 cas ctest, Debug+Release) :

1. **`geometry::inset_path_set` ne réoriente pas les trous avant Clipper2** (contrairement à `boolean.cpp`, qui le fait déjà pour `subtract_polygons` et pour `intersect_polygons/difference_polygons` ajoutés par ce lot). `InflatePaths` offsette chaque contour par rapport à SA PROPRE orientation : un trou de même orientation que son extérieur (au lieu de l'opposée) grandit ou rétrécit à l'envers lors d'une érosion. Repéré sur `ring` (`shapes.cpp`, dont le trou et l'extérieur sont générés par le même parcours d'angle croissant de `circle()`) : couverture brute correcte (99,99 %, les opérations d'union/différence QUE cet audit a ajoutées se réorientent déjà correctement) mais **couverture du cœur mesurée à 71,18 %** au lieu d'un ~100 % attendu — rien d'autre dans le pipeline Auto-Satin n'étant sensible à l'orientation d'un trou, ce défaut latent était invisible avant que `core_coverage_ratio` (qui appelle `inset_path_set` sur la région cible) ne l'expose. Corrigé par le même motif `oriented_as` que `boolean.cpp` (dupliqué localement, ADR-005) ; `shapes.cpp` corrigé en complément (trou de `ring` réorienté à la construction) pour que la fixture soit correcte par construction, pas seulement tolérée par une réorientation défensive en aval. Nouveau test dédié (`tests/unit/geometry/test_offset.cpp`, anneau carré dont le trou partage délibérément l'orientation

de l'extérieur) vérifiant l'aire nette après érosion. Après correctif : `ring` couverture cœur 100,00 %, PASSED.

2. Le câblage `--coverage-svg` du CLI lisait `parametric_columns` sur le seul indicateur

`--satin-geometry` demandé, pas sur la liste RÉELLEMENT peuplée par `build_satin_columns` — alors que certaines formes (anneaux, cas refusés en Parametric) retombent automatiquement sur `columns` (Legacy) À L'INTÉRIEUR de `build_satin_columns` même quand Parametric est demandé (§ *Objets satin paramétriques* plus haut). Symptôme : `ring --satin-geometry=parametric` rapportait 0 colonne, 0 % de couverture, alors que des colonnes existaient bel et bien (côté `columns`, jamais lu). Corrigé en testant `!parametric_columns.empty()` plutôt que l'indicateur demandé — même motif que `autodigitize.cpp` (`useParametric`, déjà correct). Bug de câblage CLI, jamais de défaut Auto-Satin lui-même ; le bloc pré-existant de génération du SVG `--output-svg` (`parametric_to_svg`) partage la même faiblesse mais n'a pas été touché ici (hors périmètre de cet audit, à corriger séparément).

Piste non investiguée, potentiellement plus significative que les deux ci-dessus : `notch` (bande à encoche en V profonde) montre un écart systématique et notable entre modes — couverture brute **~86-87 % en Legacy contre ~77-78 % en Parametric**, alors que Parametric est le mode utilisé par défaut dans `autodigitize.cpp` (contrairement au cas `trident` ci-dessus, aucun filet ne compense ici puisque Parametric produit bien une colonne, juste avec une couverture dégradée). Zone manquante la plus grande mesurée jusqu'à 39,4 mm² (22 % de la cible) selon la résolution de raster. Hypothèse de départ, non vérifiée : l'ajustement Bézier à paires structurantes éparses (`select_structural_indices/fit_both_rails`) capturerait moins fidèlement une encoche concave profonde que l'échantillonnage dense de Legacy. À investiguer en profondeur dans une prochaine session (tracer `fit_both_rails`/l'erreur d'ajustement autour de l'encoche, comme fait ici pour `trident/pixel_size`) avant toute tentative de correctif.

Diagnostic sur image réelle (`tentabrode.png`) : le repli tatami ne protège pas toutes les régions (2026-08-13)

État : Présent, diagnostic seul (aucun correctif) — test permanent `TEST_CASE("DIAGNOSTIC TEMPORAIRE couverture satin (tentabrode)")` (`tests/integration/test_pipeline.cpp`), toujours en échec (`CHECK(false)`), même convention que les diagnostics GISTRE existants.

Applique l'analyseur à la sortie réelle d'`autodigitize::auto_digitize` sur `tests/fixtures/tentabrode.png` (pas une forme synthétique du corpus), en regroupant par `source_region` (`document::VectorObject::source_region`) pour combiner chaque vecteur satin avec son éventuel repli tatami — le filet décrit plus haut (§ *Repli tatami sur une branche auto-satin rejetée*) ne se déclenche QUE sur le signal structurel `network.debug.graph.edges.size() > sectionCount` (branche entière rejetée), jamais sur une couverture partielle d'une branche par ailleurs acceptée.

Sur les 14 régions satin-portantes de cette image :

- **Couverture satin seule (sans repli), agrégée sur toute l'image : 21,3 %**. Ce chiffre seul est trompeur — il ignore le filet tatami — mais confirme que l'écart entre colonnes générées et surface cible n'est pas anecdotique sur une image réelle.
- **Couverture réelle (satin + repli tatami combiné par région), agrégée : 97,3 %**. Le filet comble donc l'essentiel de l'écart, comme conçu.
- **Mais 9 des 14 régions restent sous le seuil de 99,5 % même après combinaison**, dont deux sévèrement et **sans aucun repli** (le signal structurel qui déclenche le filet n'a pas été franchi) : un vecteur à **46,6 %** et un autre à **59,3 %** de couverture réelle.

Ce résultat corrobore et QUANTIFIE, sur une image réelle, la même catégorie de défaut que la piste `notch`/Parametric non investiguée ci-dessus (écart de couverture sur une branche acceptée, pas un rejet structurel) : le filet tatami actuel protège contre le cas « branche entièrement refusée » mais pas contre le cas « branche acceptée avec une géométrie de colonnes qui ne couvre pas fidèlement sa région », qui semble être la situation la plus courante en pratique sur des formes réelles bruitées. C'est le constat central qui motive la refonte d'architecture décrite ci-dessous (§ *Décomposition guidée par squelette*) : plutôt que d'élargir encore le filet de repli, traiter la décomposition d'une région branchée en sous-régions satinables comme une étape de planification à part entière, en amont du générateur actuel.

Suite (2026-08-14) : ce constat a depuis motivé le plan SGSD en entier (§ ci-dessous, 9 phases livrées) puis son branchement en production dans `autodigitize.cpp` (§ *Intégration production*, tout en bas de ce chapitre). Ce même diagnostic, réexécuté après le branchement, mesure une couverture réelle agrégée de **98,9 %** (contre 97,3 % avant) — un gain net, mais plus modeste et plus mitigé région par région que sur le corpus synthétique propre (§ *Outil CLI sgdsd-debug*, gains de +4,6 à +7,4 points) : sur cette image réelle bruitée, SGSD s'appuie plus souvent sur le filet tatami plutôt que sur une couverture satin propre, plutôt que d'améliorer la couverture satin elle-même comme sur les formes synthétiques. Détails dans la section d'intégration.

Visualisation de debug (`coverage_to_svg`)

État : Présent · Câblé dans `openstitch-cli auto-satin-debug --coverage-svg`.

`satin_coverage::coverage_to_svg(target, columns, report)` superpose, mêmes conventions que `auto_satin::debug_export` (millimètres, Y inversé) : la couverture en vert translucide, les zones manquantes en rouge (une par composante connexe, `fill-rule="evenodd"` pour respecter leurs propres trous), le débordement en orange, le contour de la cible en gris, et pour chaque colonne ses deux rails (bleu/orange) et ses stations structurales (points — rouges aux stations atteintes par un saut, cf. `SatinStation::jump_before`). Un commentaire d'en-tête résume le rapport en texte (aires, ratios, nombre de régions manquantes), lisible sans rendu graphique, plus une ligne par région manquante (aire, part de la cible, rayon de trou, centroïde).

`openstitch-cli auto-satin-debug --shape <forme> --coverage-svg <fichier>` calcule la couverture des colonnes réellement construites (Legacy ou Parametric selon `--satin-geometry`) et écrit à la fois le diagnostic texte sur la sortie standard et ce SVG. Vérifié en conditions réelles sur `trident` : à la résolution de raster par défaut (0,05 mm/px), le mode Legacy REFUSE entièrement la forme (« jonction 1 incomplète, 2/3 branches disponibles ») — le SVG rend alors 100 % de la cible en rouge, cohérent avec zéro colonne produite ; à 0,1 mm/px, 88,14 % de couverture avec quatre composantes manquantes (la plus grande, 18,05 mm², est le noyau de jonction déjà documenté). Cette sensibilité de `trident` à la résolution de raster en mode Legacy n'était pas documentée avant cet outil ; investiguée en profondeur ci-dessous (§ *Diagnostic « trident »/pixel_size*) — conclusion : sans impact réel sur la couverture livrée à l'utilisateur, grâce aux filets déjà en place en amont de ce chemin CLI brut.

Testé (`tests/unit/satin_coverage/test_coverage.cpp`) : SVG bien formé (`<svg>...</svg>`), couleurs de remplissage vert/rouge et rails bleu/orange effectivement présents pour un cas de couverture partielle connu.

Décomposition guidée par squelette (SGSD) : planification globale au-dessus de l'Auto-Satin (2026-08-13)

Motivation : une forme = une colonne est une hypothèse trop restrictive

État : En cours — phase 1 sur 9 livrée (planification topologique pure, aucune génération de géométrie).

`build_satin_columns` (§ *Colonnes automatiques par squelette* plus haut) traite déjà une région branchée en construisant une colonne par arête du squelette, mais TOUJOURS par rapport au contour ORIGINAL non découpé : les sections transversales de chaque branche sont mesurées contre la même `geometry::PathSet` entière, jamais contre une sous-région dédiée. Les jonctions sont donc résolues a posteriori par des heuristiques locales (`trim_unstable_junction_tail`, `StableBranchEnd/JunctionSeparator` en Legacy, recouvrement borné en Parametric) qui amputent ou recouvrent, mais ne peuvent jamais réellement replanifier la forme. Le diagnostic `tentabrode.png` ci-dessus montre l'impact réel de cette limite : des régions acceptées (pas rejetées) dont la couverture reste significativement incomplète, sans qu'aucun filet ne s'en aperçoive.

L'objectif de la SGSD (*Skeleton-Guided Satin Decomposition*) est d'introduire un étage de PLANIFICATION avant l'Auto-Satin existant, qui devient un solveur LOCAL : découper une région branchée en un petit nombre de sous-régions simples (chacune proche d'un chemin squelette `endpoint – endpoint`, sans jonction interne), construire une colonne satin par sous-région avec le générateur actuel INCHANGÉ,

mesurer la couverture de chaque sous-région et de l'assemblage avec `satin_coverage` (déjà générateur-agnostique, § plus haut), puis fusionner/router. L'Auto-Satin existant n'est ni supprimé ni modifié : il est réutilisé tel quel comme générateur d'une région simple, et comme oracle de qualité pour comparer des découpages candidats.

Déploiement en 9 phases, chacune testable et committée séparément, sans jamais toucher au générateur de rails existant avant que la décomposition elle-même soit validée :

1. **Graphe de squelette + appariement de branches (diagnostic seul).**
2. **Décomposition du graphe en `SatinPath`** (chemins topologiques maximaux) — livrée avec la phase 1, § *Phase 1* ci-dessous.
3. **Génération de candidats de coupe + découpage réel du polygone en `SatinRegion`.**
4. **`SatinabilityAnalyzer`** (la région candidate est-elle satinable ?) — livrée, § *Phase 4* ci-dessous.
5. **Auto-Satin par région + `Coverage Analyzer` comme oracle de sélection** — livrée, § *Phase 5* ci-dessous.
6. **Recherche à faisceau (*beam search*) sur les découpages candidats** — livrée (portée réduite à une décision de coupe à la fois, § *Phase 6* ci-dessous), aucune recherche combinatoire multi-jonctions.
7. **Passe de fusion (*merge*) post-découpage pour minimiser le nombre de segments** — livrée (une seule passe, pas de fusion en cascade, § *Phase 7* ci-dessous).
8. **Recouvrements (*overlaps*) entre régions adjacentes + génération de géométrie de couture** — livrée (paires directement adjacentes uniquement, § *Phase 8* ci-dessous).
9. **Routage multi-colonnes (continuité directe / travel / jump / trim)** — livrée, pont vers le moteur `stitch_generation` existant (§ *Phase 9* ci-dessous), toutes les phases du plan sont désormais couvertes.

Phase 1 : réutilisation du `SkeletonGraph` existant + appariement de branches (`libs/satin_planning`)

État : Présent, testé, aucune régression.

`libs/satin_planning/include/openstitch/satin_planning/branch_pairing.hpp` + `src/branch_pairing.cpp` (nouvelle bibliothèque, ne dépend que de `openstitch::core` et `openstitch::auto_satin` — jamais l'inverse, la couche `auto_satin` reste indépendante de la planification).

Le graphe de squelette voulu par la SGSD (nœuds typés `Endpoint/Junction`, arêtes portant une centerline et des rayons locaux) EXISTAIT DÉJÀ presque intégralement avant cette session :

`auto_satin::SkeletonGraph/SkeletonNode/SkeletonEdge`

(`libs/auto_satin/include/openstitch/auto_satin/skeleton_graph.hpp`), construit par `build_skeleton_graph` et élagué par `prune_graph` (`graph_cleanup.hpp`), tous deux déjà exposés via `auto_satin::analyze_region(region, params).debug.graph` (graphe élagué, déterministe). La phase 1 ne réimplémente donc RIEN de cette partie — elle consomme directement ce graphe existant. Ce qui manquait réellement : une façon de décider, à chaque jonction, quelles branches se prolongent naturellement l'une l'autre plutôt que de traiter chaque arête indépendamment.

Coût de continuation (`continuation_cost`, `ContinuationCostWeights` — poids `angle=1.0`, `width=0.6`, `curvature=0.4`, tout centralisé, aucun coefficient dispersé) entre deux arêtes incidentes à une même jonction :

- `angle_cost = (1 + cos θ) / 2` où θ est l'angle entre les tangentes SORTANTES des deux branches (moyenne pondérée par longueur sur une fenêtre de `tangent_window_um = 1,5 mm` depuis la jonction, cohérent avec `GraphCleanupParameters::minimum_branch_length`) : 0 pour une continuation en ligne droite (tangentes opposées), 1 pour un repli sur soi-même.
- `width_cost = |log(rayonA / rayonB)|` (rayon local moyen sur la même fenêtre) — différence RELATIVE plutôt qu'absolue, pour qu'une transition 4,1 mm→4,0 mm coûte presque rien alors qu'une transition 1 mm→8 mm coûte cher.

- `curvature_cost` = stabilité de la tangente sur la fenêtre (écart entre le premier et le dernier segment échantillonné) — pénalise une branche qui tourne déjà fortement près de la jonction.

Résolution d'une jonction (`pair_branches_at_junction`) : toutes les paires d'arêtes incidentes sont testées (`C` (degré, 2) candidats, triés par coût croissant) et la moins coûteuse devient la continuation principale (`selected_pair`) ; les autres arêtes restent `detached` — chacune devient l'extrémité d'un `SatinPath` secondaire. Pour un degré 3 (`T`, `Y`), c'est un choix entre 3 appariements. Pour un degré ≥ 4 (croix), stratégie délibérément simple : une seule continuation principale, toutes les autres branches détachées individuellement — les appariements multiples simultanés (ex. $A \leftrightarrow C$ ET $B \leftrightarrow D$) sont reportés à une phase ultérieure car ils peuvent produire des colonnes qui se croisent géométriquement au centre de la jonction, ce que la phase 1 (aucune géométrie générée) ne peut pas encore vérifier.

Décomposition complète (`decompose_into_paths`) : parcourt tous les nœuds `Junction`, construit la carte des continuations retenues, puis assemble des `SatinPath` maximaux en suivant les appariements de proche en proche — partition EXACTE des arêtes du graphe (chaque arête apparaît dans exactement un chemin, vérifié par test sur tout le corpus de formes). Un pont `Jonction-Jonction` (le cas `h`, § plus haut) qui continue naturellement aux DEUX bouts fusionne les deux montants et le pont en un seul chemin, exactement la même intuition que la conversion pont→colonne déjà en place côté générateur. `format_decomposition_report` produit un rendu texte structuré (jonction par jonction, candidats avec leurs coûts, sélection, chemins résultants) pour le debug — aucun `fprintf` direct dans la bibliothèque.

Aucune géométrie n'est générée à cette phase : ni découpage de polygone, ni appel à `build_satin_columns`. C'est une étape de planification topologique pure, décorrélée du générateur, exactement comme demandé.

Testé (`tests/unit/satin_planning/test_branch_pairing.cpp`, 12 cas) :

- Cas synthétique fait main (jonction en `T` à trois branches, coûts calculés à la main) vérifiant numériquement les trois formules de coût.
- Formes réelles du corpus `auto_satin::shapes` passées par le VRAI pipeline de rasterisation/squelettisation (`analyze_region`) : `rectangle` (0 jonction, 1 chemin), `t` (1 jonction degré 3, 2 chemins), `cross` (1 jonction degré 4, 1 continuation + 2 détachées, 3 chemins), `h` (2 jonctions, le pont détaché aux deux bouts confirmé par introspection du graphe, 3 chemins), `trident` (1 jonction degré 3, 2 chemins).
- Propriété de partition (chaque arête dans exactement un chemin) vérifiée génériquement sur 12 formes du corpus.
- Rendu `format_decomposition_report` non vide et contenant les marqueurs attendus.

Phase 3 : candidats de coupe + découpage réel du polygone (`libs/satin_planning/region_split`)

État : Présent, testé, aucune régression — v1 purement géométrique (pas encore d'oracle Auto-Satin, cf. phase 5). `libs/satin_planning/include/openstitch/satin_planning/region_split.hpp` + `src/region_split.cpp`.

Cette phase transforme chaque `SatinPath` (chemin topologique, phase 1) en un véritable sous-polygone (`SatinRegion`), en réutilisant tel quel l'outil de coupe manuelle déjà existant (`geometry::cut_path_set`, § *Ligne de coupe manuelle* plus haut) plutôt que d'écrire un nouveau moteur de découpe — la seule brique manquante était de choisir AUTOMATIQUEMENT où et dans quelle direction couper.

Un point important sur `cut_path_set` : ce n'est jamais une entaille locale — la coupe est toujours prolongée en une ligne complète qui traverse toute la boîte englobante de la région, quels que soient les points fournis (pensé pour un geste utilisateur qui vise la jonction sans forcément atteindre les bords exacts). Un candidat de coupe automatique doit donc être validé géométriquement avant d'être accepté, pas seulement calculé : la ligne peut en principe traverser une zone sans rapport si la forme n'est pas localement simple près de la jonction.

Recherche de candidats (`generate_cut_candidates`) : pour une branche détachée (jonction + arête, directement issues de `JunctionPairingReport::detached`), balaie des distances croissantes depuis la

jonction (`search_min_um=300 µm` à `search_max_um=1500 µm` par pas de `150 µm`, §11 spec SGSD — jamais exactement au pixel de jonction). À chaque distance, calcule le point et la tangente locale sur la centerline, construit la ligne de coupe selon la normale, et appelle `cut_path_set`. Rejette immédiatement (§13 spec SGSD, sous-ensemble géométrique — l'oracle Auto-Satin complet est phase 5) :

- un résultat qui n'a pas EXACTEMENT 2 morceaux (ligne qui a traversé une zone sans rapport) ;
- un fragment plus petit que `min_piece_area_mm2` (`0,3 mm²` par défaut).

Le premier candidat valide (le plus proche de la jonction) est retenu — un choix délibérément simple pour cette v1 géométrique pure ; la sélection par score global (§13 complet, oracle de couverture) est reportée à la phase 5/6, une fois `SatinabilityAnalyzer` et l'intégration Auto-Satin par région en place.

Découpage complet (`split_region`) : traite chaque événement de détachement (une jonction, une arête de son `detached`) dans un ordre déterministe (jonction puis arête croissantes), en localisant à chaque fois le morceau COURANT du pool qui contient le point de jonction (test point-dans-polygone par ray casting, implémenté localement — les chemins manipulés ici sont toujours polygonaux, jamais des courbes de contrôle) avant d'y appliquer la coupe retenue. Le morceau découpé est remplacé par les deux résultants dans le pool. **Aucun traitement spécial n'est nécessaire pour une branche à deux bouts de jonction** (le pont d'un « H ») : le second événement de détachement retrouve naturellement, dans le pool, le morceau laissé par le premier — la même mécanique générique s'applique. Assignment finale : chaque `SatinPath` reçoit le morceau du pool qui contient un point sonde intérieur à son propre tracé (un nœud strictement interne au chemin s'il y en a un, sinon le milieu de ses deux extrémités — jamais un point exactement sur une coupe). Une branche qu'aucune coupe valide ne sépare reste fusionnée avec son morceau parent, reportée dans `unresolved_paths` plutôt que silencieusement ignorée.

Testé (`tests/unit/satin_planning/test_region_split.cpp`, 4 cas) contre le VRAI pipeline de rasterisation/squelettisation, pas de fixtures synthétiques faites main pour cette phase — la géométrie réelle du contour est indispensable ici (contrairement à la phase 1, purement topologique) :

- `rectangle` (0 jonction) : aucune coupe tentée, la région ressort inchangée.
- `t` : une coupe à la distance minimale testée (`300 µm`) isole le pied (`100,44 mm²`) de la barre (`238,84 mm²`) sur les 9 candidats testés, tous valides et croissants avec la distance — confirme que le balayage se comporte de façon monotone et prévisible.
- Propriété générale sur `t`, `y`, `cross`, `h`, `trident` : **tous les chemins de ces cinq formes sont intégralement résolus** (`unresolved_paths` vide dans les cinq cas), y compris `h` (les deux coupes successives du pont fonctionnent sans logique dédiée) et `cross` (les deux branches détachées de l'unique jonction degré 4 sont isolées l'une après l'autre du même morceau restant). Aire cumulée des régions résolues toujours inférieure à l'aire d'origine (une coupe ne peut que retirer une fine bande, jamais en ajouter), `path_index` assignés uniques.
- Rendu `format_region_split_report` non vide et contenant les marqueurs attendus.

Aucune géométrie satin n'est générée à cette phase non plus — c'est un découpage de polygone pur, décorrélé du générateur.

Mise à jour (phase 4, voir ci-dessous) : la validation purement géométrique ci-dessus s'est révélée insuffisante et a été renforcée, pas remplacée — `generate_cut_candidates` intègre désormais un garde-fou de satinabilité (§ Phase 4) qui a directement corrigé un défaut réel trouvé sur `t`, `y` et `cross` : une coupe jugée valide par les seuls critères géométriques (exactement 2 morceaux, aucun fragment trop petit) pouvait en réalité trancher AUSSI la branche voisine près d'une confluence où plusieurs branches se chevauchent géométriquement, laissant une jonction résiduelle bien réelle dans le morceau censé être isolé.

Phase 4 : `SatinabilityAnalyzer` — et un défaut réel trouvé et corrigé dans la phase 3 (`libs/satin_planning/region_satinability`)

État : Présent, testé, aucune régression.

`libs/satin_planning/include/openstitch/satin_planning/region_satinability.hpp` et `src/region_satinability.cpp`.

Bonne surprise, même schéma qu'aux phases précédentes : la spec SGSD (§14) demande de « créer une vraie fonction `analyzeSatinability` » examinant le nombre de jonctions/extrémités du squelette, la largeur, la variation de largeur, etc. — exactement ce que `auto_satin::evaluate_satinability / auto_satin::analyze_region` font déjà (§ *Analyse de satinabilité* plus haut). Rien à réimplémenter : `check_region_satinability(region, params)` fait simplement tourner le VRAI pipeline existant (rasterisation → distance → squelette → graphe → `evaluate_satinability`) sur une `SatinRegion` déjà découpée, exactement comme le ferait `build_satin_columns` en interne, et classe le verdict `ready_for_generation` selon que le statut résultant est `Suitable/SuitableWithWarnings`. `check_all_regions` applique ça à tout un `RegionSplitReport` (phase 3), en reportant aussi automatiquement dans `not_ready` les chemins jamais isolés (`unresolved_paths`) — pas de verdict sans région, mais pas d'omission silencieuse non plus.

Défaut réel trouvé en testant bout-en-bout (décompose + découpe + vérifie) sur le corpus branché : chaque `SatinRegion` fraîchement découpée à la phase 3 était réanalysée isolément, et le résultat était systématiquement `RequiresDecomposition` (une jonction résiduelle) pour `t`, `y`, `cross` ET le pont d'un « H » — jamais `Suitable` comme attendu pour une branche qui vient d'être proprement détachée. Diagnostic en trois temps (instrumentation temporaire, retirée une fois la cause confirmée, même méthode que l'audit `trident/pixel_size`) :

1. **D'abord écarté : artefact de rasterisation.** Un balayage de `pixel_size` de 50 à 6,25 µm sur la région coupable ne change RIEN au résultat (`junctions=1` à toutes les résolutions) — élimine l'hypothèse d'un escalier de pixellisation au bord de la coupe (contrairement au cas `trident/pixel_size` documenté plus haut, qui LUI dépendait bien de la résolution).
2. **Cause confirmée : la coupe par défaut (300–1500 µm de la jonction) tombe encore dans la zone où les empreintes de deux branches voisines se chevauchent.** Sur `t`, la branche du pied (2,5 mm de demi-largeur) est testée à des distances qui restent À L'INTÉRIEUR de l'emprise en X de la barre horizontale (elles se croisent exactement à la jonction) : la ligne de coupe, bien que géométriquement valide au sens « exactement 2 morceaux, aucun fragment trop petit », tranche EN RÉALITÉ un peu de la barre en même temps que le pied, laissant un résidu détectable comme une vraie troisième branche. Balayage direct de la distance de coupe (300 µm à 7 mm) : le morceau isolé reste branché (`RequiresDecomposition`) jusqu'à 2000 µm inclus, et devient proprement `Suitable` (0 jonction) à partir de 2600 µm — la marge par défaut de la phase 3 (max 1500 µm) n'atteignait jamais cette distance.
3. **Corrigé :** `CutCandidateParams` élargit la plage de recherche par défaut (`search_max_um` 1500→4000 µm, `search_step_um` 150→300 µm) ET `generate_cut_candidates` ajoute un garde-fou — quand le bout distal de la branche détachée est une VRAIE extrémité (pas une autre jonction), le morceau isolé par chaque candidat est réanalysé avec `auto_satin::analyze_region` et le candidat est rejeté si une jonction résiduelle subsiste, avant même de le proposer comme sélection. Le cas d'un bout de jonction (le pont d'un « H », § plus haut) est délibérément EXCLU de ce garde-fou : ce chemin a besoin de DEUX coupes successives, et le vérifier après la première le rejetterait à tort — il est encore légitimement branché à ce stade-là. Documenté comme limite connue plutôt que « corrigé » : une validation correcte du pont demanderait de ne réappliquer le garde-fou qu'après la DERNIÈRE coupe d'un chemin à deux bouts de jonction, une information que `generate_cut_candidates` (qui ne traite qu'un événement de détachement à la fois, sans mémoire du chemin complet) n'a pas.

Vérifié après correctif (`tests/unit/satin_planning/test_region_satinability.cpp`, 5 cas) :

- rectangle : déjà simple, prête sans découpe.
- `t`, `y`, `cross` (branches détachées se terminant toutes par une vraie extrémité) : **100 % des régions ressortent propres** (`junction_count == 0`, `ready_for_generation`) — le défaut ci-dessus est bien corrigé pour ces trois formes.
- `h` : les deux montants (jamais coupés qu'à une seule extrémité chacun) ressortent propres ; le pont (les deux bouts sur une jonction) reste honnêtement classé `not_ready` — limite connue, pas un échec silencieux.
- `trident` : la branche latérale, étroite et courte, peut ne trouver AUCUNE distance de coupe qui échappe complètement à l'emprise de sa voisine dans la plage testée — le garde-fou rejette alors

TOUTES les distances candidates plutôt que d'accepter une coupe défectueuse, et `split_region` reporte honnêtement le chemin en `unresolved_paths` plutôt que de produire une région silencieusement fausse. Vérifié : quand une région EST résolue, elle est toujours effectivement propre (`junction_count == 0`).

Le coût de ce garde-fou est un `analyze_region` complet (rasterisation + squelettisation) par candidat testé sur une branche à extrémité réelle — pas gratuit, mais borné (13 candidats maximum par branche avec les paramètres par défaut) et seulement pendant la planification, jamais pendant la génération de points elle-même.

Phase 5 : Auto-Satin réel par région + Coverage Analyzer comme oracle complet (`libs/satin_planning/region_oracle`)

État : Présent, testé, aucune régression.

`libs/satin_planning/include/openstitch/satin_planning/region_oracle.hpp` et `src/region_oracle.cpp`.

Contrairement à la phase 4 (qui ne vérifie que le STATUT de satinabilité avant toute génération), cette phase construit réellement les colonnes satin sur chaque `SatinRegion` (`auto_satin::build_satin_columns`, mode `Parametric` pour reproduire le comportement de production d'`autodigitize.cpp` — repli automatique vers `Legacy` déjà géré en interne) puis mesure la couverture obtenue avec `satin_coverage::analyze_satin_coverage` — exactement l'oracle demandé par la spec SGSD (§15) : « la preuve finale reste, est-ce que l'Auto-Satin généré couvre réellement correctement la région ? ». `satin_coverage` reste totalement indépendant du décomposeur (ne connaît que des rails/barreaux génériques, jamais un type `auto_satin` ou `satin_planning`) — même garantie qu'à sa création (§ *Satin Coverage Analyzer* plus haut).

`evaluate_region_generation(region, genParams, coverageConfig, density)` convertit la sortie de `build_satin_columns` (`parametric_columns` si non vide, sinon `columns` — même sélection que `autodigitize.cpp`/le CLI, même défaut corrigé cette session, § *Balayage du corpus de formes*) en `satin_coverage::SatinColumnInput`, puis appelle l'analyseur. `evaluate_decomposition_generation` l'applique à tout un `RegionSplitReport` et agrège une couverture globale (aire couverte / aire cible sur toutes les régions).

Résultats mesurés sur le corpus branché (mode `Parametric`, seuils par défaut de `satin_coverage`, dont `min_core_coverage = 99,5 %`) :

- `t, y, cross` : chaque région (toutes propres depuis le correctif phase 4) atteint **97–99 % de couverture brute/cœur** — cohérent avec la limite déjà documentée des résidus de noyau de jonction/bouts (§ *Vérifié*, section *Satin Coverage Analyzer*) : aucune de ces régions ne franchit le seuil strict de 99,5 % (`passed = false` partout), mais ce n'est PAS un défaut nouveau — c'est la même limite connue de l'Auto-Satin actuel, simplement mesurée ici région par région plutôt que sur la forme entière.
- `h` : les deux montants (jamais coupés qu'à une seule extrémité chacun, propres dès la phase 4) couvrent aussi **~98 %** — mais **le pont (encore branché après découpe, not_ready en phase 4) chute à ~70 %** de couverture. L'oracle phase 5 confirme AVEC DES CHIFFRES ce que la phase 4 ne pouvait que signaler qualitativement (statut `RequiresDecomposition`) : un écart net et mesurable, pas juste un statut binaire.
- `trident` : la branche isolée avec succès couvre **~98 %** ; la branche non résolue (§ phase 4, aucune coupe n'échappe à sa voisine) reste honnêtement absente du rapport de couverture plutôt que représentée par un chiffre inventé.

Ce résultat borne précisément le périmètre encore à couvrir par les phases suivantes : la phase 6 (recherche à faisceau) pourra comparer plusieurs découpages candidats en utilisant CET oracle comme fonction de score ; la phase 7 (fusion) pourra détecter qu'une paire de régions adjacentes aux couvertures individuellement bonnes redeviendrait encore meilleure fusionnée (ou, pour le pont du H, qu'une meilleure position de coupe/fusion est nécessaire avant que sa couverture ne devienne acceptable) ; aucune des deux n'est encore implémentée à ce stade.

Testé (tests/unit/satin_planning/test_region_oracle.cpp, 4 cas) :

- rectangle : couverture > 90 % sans aucune décomposition.
- Boucle complète sur t, y, cross, h, trident : chaque région effectivement construite produit un rapport de couverture exploitable (aire cible non nulle), que le seuil strict soit franchi ou non.
- h dédié : vérifie numériquement que le pont couvre nettement moins bien (< 85 %) que le meilleur montant (> 95 %) — non-régression directe sur la découverte ci-dessus.
- Rendu format_generation_report non vide et contenant les marqueurs attendus.

Phase 6 : recherche à faisceau guidée par l'oracle (libs/satin_planning/beam_search)

État : Présent, testé, aucune régression — portée réduite à une décision de coupe à la fois (voir plus bas).

libs/satin_planning/include/openstitch/satin_planning/beam_search.hpp et
src/beam_search.cpp.

Jusqu'ici, split_region retenait toujours le premier candidat géométriquement/topologiquement valide (le plus proche de la jonction, § Phase 3/4) — un choix arbitraire parmi plusieurs qui passaient déjà les garde-fous. La phase 6 remplace ce choix par une vraie décision de qualité, en utilisant l'oracle de la phase 5 pour ARBITRER entre plusieurs candidats plutôt que d'en accepter un au hasard.

Portée volontairement réduite par rapport à la spec (§16) : une recherche à faisceau complète explorerait des combinaisons de coupes sur PLUSIEURS jonctions simultanément (la spec avertit elle-même du risque d'explosion combinatoire). Cette phase se limite à une recherche locale, une décision de coupe à la fois — un vrai beamWidth (candidats évalués, meilleur retenu), mais appliqué indépendamment à chaque événement de détachement plutôt qu'à l'arbre complet des décompositions possibles. Une recherche multi-jonctions arborescente reste un travail futur, explicitement hors périmètre ici.

Mécanisme : region_split.hpp gagne un point d'injection, CutCandidateSelector (std::function<optional<size_t>(PathSet, vector<CutCandidate>)>), stocké dans CutCandidateParams::selector — vide par défaut (comportement historique inchangé, aucune régression pour les appelants existants). Ce point d'injection permet à beam_search de brancher une stratégie de sélection SANS que region_split.hpp/.cpp (qui reste un découpeur de polygone pur) n'acquière de dépendance vers region_oracle.hpp — c'est beam_search qui dépend des deux, jamais l'inverse, la même discipline de couches que partout ailleurs dans libs/satin_planning.

OracleGuidedSelector (operator() compatible CutCandidateSelector, à injecter via std::ref) matérialise, pour chaque événement de détachement, jusqu'à beam_width candidats déjà valides (les plus proches de la jonction d'abord — le pré-filtrage géométrique/satinabilité de la phase 3/4 s'applique toujours en amont, ce sélecteur ne peut jamais choisir un candidat que ces garde-fous ont déjà rejeté), construit réellement des colonnes satin sur chacun et mesure sa couverture (oracle phase 5 complet), puis retient celui dont la couverture brute est la meilleure. Chaque décision est journalisée (decisions()) pour le debug.

Résultat mesuré sur le pont du « H » (le cas le plus dégradé identifié en phase 5, ~70 % de couverture) : avec beam_width = 3 et mode Parametric, sa couverture passe de **69,9 % à 92,3 %** — un gain net de plus de 20 points, obtenu SANS modifier le générateur de rails ni la logique de découpage elle-même, seulement en arbitrant, parmi des positions de coupe déjà valides, laquelle produit réellement le meilleur résultat une fois générée. Les deux montants (déjà propres à ~98 %) restent globalement stables (97,6 % et 98,4 %, les positions de coupe ayant légèrement bougé en conséquence).

Coût : jusqu'à beam_width générations + mesures complètes PAR événement de détachement, en plus des analyses de satinabilité déjà faites par generate_cut_candidates — réservé à un usage hors ligne (planification, CLI de debug, tests), jamais au chemin interactif.

Testé (tests/unit/satin_planning/test_beam_search.cpp, 4 cas) :

- t : résout toujours la branche, journalise une décision (au plus beam_width candidats évalués).
- Boucle sur t, y, cross, h, trident : le nombre de régions résolues avec le sélecteur guidé par l'oracle n'est jamais inférieur au comportement par défaut (le sélecteur ne peut que choisir MIEUX parmi les

candidats déjà valides, jamais en révéler de nouveaux).

- `h` : deux décisions journalisées (les deux bouts du pont), au plus `beam_width` candidats chacune.
- `h`, non-régression numérique dédiée : confirme le gain de couverture du pont (< 80 % avant, > 85 % après, écart > 10 points) — verrouille la découverte ci-dessus.

Phase 7 : passe de fusion post-découpage (`libs/satin_planning/merge_pass`)

État : Présent, testé, aucune régression — une seule passe, pas de fusion en cascade (voir plus bas).

`libs/satin_planning/include/openstitch/satin_planning/merge_pass.hpp` et

`src/merge_pass.cpp`, plus une extension de `region_split.hpp/.cpp`

(`RegionSplitReport::merge_candidates`, `MergeCandidate`).

Repérer les candidats de fusion sans reconstruction géométrique approximative — c'est le point délicat de cette phase. Une paire de régions adjacentes pourrait sembler se recombinaison par une simple union `Clipper2`, mais la bande retirée par chaque coupe (`cut_width`, 20 µm par défaut) laisserait un interstice entre les deux morceaux : une union classique ne les refusionnerait PAS en un seul contour extérieur. La solution retenue évite le problème plutôt que de le contourner : `split_region` (phase 3) connaît déjà, à l'instant précis de chaque coupe, la géométrie EXACTE du morceau juste avant qu'elle ne soit appliquée — c'est littéralement l'union des deux résultats, sans aucun interstice. Il suffisait de la conserver. `split_region` trace maintenant, pour chaque coupe réussie, un petit arbre de filiation (quel morceau vient d'où) et expose `merge_candidates` : une entrée par coupe dont les DEUX résultats sont restés des feuilles jusqu'à la fin du découpage (jamais redécoupés par un événement ultérieur — cf. `cross`, où une seule des deux coupes qualifie, l'autre ayant son « reste » redécoupé ensuite). Chaque candidat porte directement la géométrie pré-coupe capturée en mémoire, prête à être réanalysée sans recalcul.

Décision de fusion (`evaluate_merge_pass`) : pour chaque candidat, construit réellement (oracle phase 5) les deux régions séparées ET la région fusionnée, compare la couverture obtenue (moyenne pondérée par aire des deux séparées, contre la couverture unique de la version fusionnée), et recommande la fusion si l'écart ne dépasse pas une tolérance configurable (`coverage_tolerance`, 2 points par défaut) — un seul segment vaut une petite perte de couverture, l'inverse (fusionner malgré une dégradation importante) non.

Portée volontairement réduite, même discipline que la phase 6 : chaque candidat est évalué INDÉPENDAMMENT, sans fusion en cascade (une région fusionnée n'est jamais elle-même retestée contre un troisième voisin). La spec (§16/§18) avertit explicitement du risque d'explosion combinatoire d'une passe itérative « jusqu'à stabilité » sur l'ensemble du graphe de décomposition — reportée à un travail futur.

Résultat vérifié sur `t` : la fusion est correctement **rejetée** (couverture séparée 97,68 %, couverture fusionnée 90,32 % — la version fusionnée réexpose exactement la complexité de jonction que la phase 3 avait résolue en coupant, un écart de 7,4 points bien au-delà de la tolérance). C'est la preuve que la passe ne réduit pas aveuglément le nombre de segments : elle mesure réellement, via le même oracle qu'ailleurs dans le plan SGSD, et ne fusionne QUE quand la qualité le justifie.

Testé (`tests/unit/satin_planning/test_merge_pass.cpp`, 4 cas) :

- `t` : exactement un candidat de fusion exposé, dont la géométrie fusionnée a une aire égale à l'aire d'origine (preuve indirecte qu'aucun interstice de coupe ne subsiste — la reconstruction est exacte, pas une union approximative).
- `cross` : sur les deux coupes, une seule expose ses deux enfants comme feuilles finales (l'autre a son « reste » redécoupé par l'événement suivant) — exactement un candidat de fusion, comme attendu de l'arbre de décision décrit ci-dessus.
- `t` : décision de fusion exploitable dans les deux sens (le test ne préjuge pas du sens de la recommandation, seulement de la cohérence des mesures).
- Rendu `format_merge_pass_report` non vide et contenant les marqueurs attendus.

Phase 8 : recouvrements entre régions adjacentes (`libs/satin_planning/overlap`)

État : Présent, testé, aucune régression — limité aux paires directement adjacentes (même restriction qu'à la phase 7, voir plus bas). `libs/satin_planning/include/openstitch/satin_planning/overlap.hpp` et `src/overlap.cpp`.

Le problème (§19 spec SGSD) : deux régions séparées exactement sur une ligne de coupe laisseraient, une fois cousues indépendamment, un interstice physique visible — la bande retirée par `cut_width` (20 µm par défaut). Il faut une petite extension de chaque région vers l'autre, mais SANS modifier les régions structurelles utilisées pour toute mesure de couverture (phases 4 à 7) : une géométrie dédiée à la broderie, distincte.

Méthode, entièrement construite sur des primitives déjà existantes — aucune nouvelle opération géométrique bas niveau n'a été nécessaire. Dilater une région de `overlap_distance` est un simple appel à `geometry::inset_path_set` avec un delta NÉGATIF (l'outil existant d'érosion/dilatation, déjà utilisé pour les sous-couches). Reste à garantir que l'extension « reste dans la shape originale » (§19) sans reconstruire une ligne de coupe déplacée : la région dilatée est recadrée (`geometry::intersect_polygons`) directement dans `MergeCandidate::merged_region` — la géométrie EXACTE d'avant coupe, déjà capturée sans interstice à la phase 7. Deux appels à des primitives existantes suffisent donc à produire une extension géométriquement correcte et bornée.

Résultat vérifié sur t (`overlap_distance` = 300 µm par défaut) : chaque région élargie est strictement plus grande que l'originale, jamais au-delà de la géométrie fusionnée de référence, et surtout — la preuve centrale de cette phase — **les deux régions élargies se chevauchent réellement désormais** (aire d'intersection mesurée : 2,95 mm²), là où les régions structurelles d'origine ne se touchaient pas (séparées par `cut_width`). L'interstice est fermé.

Limite connue, même restriction qu'à la phase 7 et pour la même raison : ne couvre que les paires DIRECTEMENT adjacentes dont les deux côtés sont restés des feuilles jusqu'à la fin du découpage (`RegionSplitReport::merge_candidates`, réutilisé tel quel — c'est déjà exactement l'ensemble des paires dont la géométrie exacte d'avant coupe est connue). Une couture plus profonde dans l'arbre de décision (un côté redécoupé ensuite, ex. le pont d'un « H » vis-à-vis d'un montant) n'a pas encore de mécanisme de recouvrement — déterminer l'adjacence finale au-delà d'une seule coupe reste un travail futur. La densité de points dans la zone de recouvrement (§19 : « ne pas créer une surdensité énorme ») est hors périmètre de cette phase, purement géométrique — à traiter quand `stitch_generation` consommera cette géométrie dédiée.

Testé (`tests/unit/satin_planning/test_overlap.cpp`, 3 cas) :

- `t` : aire élargie strictement supérieure à l'originale pour les deux régions, jamais au-delà de la géométrie fusionnée, ET intersection non vide entre les deux régions élargies (l'invariant central : l'interstice est bien fermé).
- `cross` : un recouvrement généré par paire directement adjacente, en correspondance exacte avec `merge_candidates`.
- Rendu `format_overlap_report` non vide et contenant les marqueurs attendus.

Phase 9 : routage multi-régions (`libs/satin_planning/region_routing`)

État : Présent, testé, aucune régression — pont vers le moteur de routage existant, aucune logique de routage réimplémentée. Toutes les phases du plan SGSD (1 à 9) sont désormais livrées.

`libs/satin_planning/include/openstitch/satin_planning/region_routing.hpp` et `src/region_routing.cpp`.

Encore une bonne surprise, le schéma qui revient tout au long de ce chantier : le moteur de routage que la spec réclame (§20 : continuité directe / travel run / jump, le trim comme conséquence du choix plutôt qu'une contrainte imposée au générateur) **existait déjà**, complet et testé —

`stitch_generation::route_columns/RoutePlan` (Lot 6, § *Routage multi-colonnes* plus haut). Son vocabulaire diffère légèrement de celui de la spec (`ConnectorKind::Underpath` pour un trajet caché plutôt que « travel run », `Jump` qui porte déjà la coupe — donc le « trim » — dans sa propre définition) mais

couvre exactement le même problème : ordonner et orienter des colonnes pour minimiser le déplacement, en préférant un trajet caché quand il est topologiquement justifié, un saut sinon. Rien à réimplémenter : la phase 9 est un pont, pas un nouveau moteur.

Le pont (`route_regions`) construit réellement une colonne satin par `SatinRegion` résolue (même conversion que l'oracle phase 5), la réduit à ses deux extrémités (centre des barreaux d'about, `stitch_generation::RouteColumn`) et confie l'ensemble à `route_columns`.

Limite connue, documentée plutôt que masquée : `RouteColumn::start_junction/ end_junction` ne sont pas renseignés. Dans le cas normal (une seule région non décomposée produisant plusieurs colonnes), ces champs portent un identifiant de jonction PARTAGÉ entre colonnes, ce qui autorise `route_columns` à valider un trajet caché avec une portée généreuse (`underpath_max`, 8 mm). Ici, chaque `SatinRegion` est réanalysée INDÉPENDAMMENT par `build_satin_columns` (son propre appel interne à `analyze_region`), avec sa PROPRE numérotation locale de jonctions — non comparable d'une région à l'autre. `route_columns` retombe donc sur son seuil de quasi-contact le plus strict (`underpath_max_without_junction`, 1,5 mm) pour toutes les liaisons entre régions SGSD : sûr (jamais de trajet caché injustifié à travers du vide), mais plus conservateur qu'il ne pourrait l'être — l'adjacence RÉELLE entre régions est pourtant déjà connue avec certitude via `RegionSplitReport::merge_candidates` (phase 7). Relier cette connaissance à la numérotation de jonctions attendue par `route_columns` reste un travail futur, qui débloquerait des trajets cachés à la portée normale précisément là où on sait déjà que deux régions étaient adjacentes avant découpage.

Testé (`tests/unit/satin_planning/test_region_routing.cpp`, 3 cas) :

- `rectangle` : une seule région, un seul pas de départ (`ConnectorKind::Start`), aucun saut.
- `t` : les deux régions sont routées, le plan contient exactement un pas de départ et le reste en trajets cachés/sauts, chaque chemin de la décomposition représenté exactement une fois (aucune région omise ni dupliquée dans le plan).
- Rendu `format_region_routing_report` non vide et contenant les marqueurs attendus.

Outil CLI `sgsd-debug` : brancher le pipeline complet, mesurer l'effet réel (2026-08-13)

État : Présent. `openstitch-cli sgsd-debug --shape <forme> [--pixel-size mm] [--beam-width n]` (`apps/cli/main.cpp`, `run_sgsd_debug`).

Les 9 phases du plan SGSD n'étaient jusqu'ici exercées que par des tests unitaires — aucun point d'entrée ne permettait de faire tourner le pipeline complet sur une forme et de voir, concrètement, si la décomposition change quoi que ce soit pour un utilisateur. `sgsd-debug` fait exactement ça, suivant le même modèle que `auto-satin-debug` (outil de diagnostic, aucune géométrie satin définitive écrite) : `analyze_region` → `decompose_into_paths` → `split_region` (avec un `OracleGuidedSelector`, phase 6, actif par défaut à `--beam-width 3`) → `evaluate_decomposition_generation` (phase 5) → `evaluate_merge_pass` (phase 7) → `generate_overlaps` (phase 8) → `route_regions` (phase 9), en imprimant le rapport texte de CHAQUE phase (les mêmes `format_*` déjà testés unitairement, rien de nouveau à faire confiance ici) puis une comparaison chiffrée : couverture agrégée du pipeline SGSD contre un appel direct à `build_satin_columns` sur la forme entière, non décomposée (l'approche historique).

Résultat balayé sur tout le corpus de formes

Forme	SGSD	Direct	Écart
<code>rectangle, capsule, ribbon, s, notch</code>	= Direct	—	0 (aucune décomposition nécessaire, comportement identique)
<code>y</code>	97,9 %	92,6 %	+5,3
<code>t</code>	97,7 %	90,3 %	+7,4
<code>cross</code>	97,8 %	90,6 %	+7,2
<code>h</code>	94,9 %	89,5 %	+5,4
<code>trident</code>	95,1 %	90,5 %	+4,6
<code>wide, tiny, pinch, circle</code>	refus (identique)	refus	0 (SGSD hérite fidèlement du refus de <code>build_satin_columns</code> par région, aucun faux négatif introduit)
<code>r,ing</code>	0 % (0 région)	100 %	région connue, voir ci-dessous

Sur toutes les formes BRANCHÉES du corpus historique, la décomposition apporte un gain de couverture réel et mesurable (+4,6 à +7,4 points) par rapport à l'appel direct non décomposé — la validation concrète, chiffrée, que le chantier SGSD visait depuis le début. Sur les formes déjà simples ou déjà refusées, SGSD se comporte de façon strictement identique à l'existant : aucune régression silencieuse introduite par la décomposition elle-même.

Défaut réel trouvé et corrigé : `probe_point` sur un chemin sans nœud interne (`notch`)

En balayant le corpus avec cet outil, `notch` (bande à encoche concave profonde, § *Diagnostic sur formes concaves* — SANS AUCUNE jonction, un seul arc de squelette) ressortait à **0 région résolue sur 1** — un chemin pourtant trivialement résolu (aucune coupe nécessaire) rapporté comme « jamais isolé ». Cause : `probe_point` (utilisée par `split_region` pour retrouver, en fin de découpage, quel morceau du pool appartient à quel `SatinPath`) retombait, faute de nœud interne (un seul arc, donc deux nœuds seulement), sur le MILIEU DROIT entre les deux extrémités du chemin — une corde qui sort de la forme dès que le tracé courbe suffisamment autour d'une concavité profonde, faisant échouer l'assignation d'un chemin par ailleurs parfaitement valide.

Corrigé en préférant, dans tous les cas, un point pris DIRECTEMENT sur la centerline de l'arc médian du chemin — toujours intérieur à la forme par construction (c'est l'axe médian lui-même), quelle que soit la courbure ; le repli « milieu des extrémités » ne reste qu'un dernier recours théorique. Vérifié après correctif : `notch` ressort à 76,6 % de couverture, identique à l'appel direct (aucune décomposition n'était de toute façon nécessaire). Non-régression dédiée (`tests/unit/satin_planning/test_region_split.cpp`, « `notch -- chemin unique sans jonction résolu malgré une forte courbure` ») ; la propriété générale du corpus a aussi été renforcée pour exiger `unresolved_paths` vide sur toute forme sans jonction (`s`, `ribbon`, `wide` ajoutés au balayage général en plus de `notch`).

Limite réelle découverte, non corrigée : les anneaux (`ring`)

Le balayage a aussi révélé un vrai trou de couverture fonctionnelle du pipeline SGSD, distinct du défaut ci-dessus (pas un bug de bookkeeping, une lacune de conception) : **`ring` (anneau) n'est pas décomposé du tout** (`decompose_into_paths` renvoie zéro chemin) alors que l'appel direct le gère parfaitement via un chemin spécial dédié (`build_annular_sections`, § *Colonnes automatiques par squelette*). La raison structurelle : le squelette d'un anneau est une boucle fermée sans aucune vraie extrémité (`Endpoint`) ni jonction détectable par les règles actuelles de classification de `SkeletonGraph` — exactement le même mécanisme dégénéré déjà identifié pour `circle` lors de l'inspection initiale du code existant (§ *Première mission*), mais jamais quantifié jusqu'ici. **Documenté comme limite connue plutôt que masqué** : `sgsd-debug --shape ring` rapporte honnêtement 0 région et 0 % plutôt qu'un chiffre trompeur. Traiter la topologie annulaire dans `decompose_into_paths` (détecter un cycle fermé et le traiter comme un cas spécial, à l'image de `build_annular_sections` côté générateur) reste un travail futur.

Intégration production : SGSD dans `autodigitize.cpp` (2026-08-14)

État : Présent, seul chemin disponible (§ section suivante — le bascule `AutoOptions::use_sgsd_decomposition` mentionné ci-dessous a depuis été supprimé).
`libs/autodigitize/src/autodigitize.cpp` (`try_sgsd_decomposition` à l'origine).

Le CLI `sgsd-debug` (§ ci-dessus) a permis de MESURER le gain SGSD, mais rien ne l'utilisait encore pour de vrai : `autodigitize.cpp` appelait toujours `build_satin_columns` directement sur la région entière. Cette étape branche enfin le pipeline sur le chemin réellement emprunté par l'application (numérisation automatique depuis une image).

Où et comment

Sur une région dont le squelette est réellement branché (`analyze_region(main).debug.graph.junction_count() > 0`), tente d'abord `decompose_into_paths` → `split_region` (recherche à faisceau active, `beam_width = 3`) → `build_satin_columns` RÉEL sur chaque `SatinRegion` résolue. Repli **intégral** sur l'appel direct historique dès que SGSD ne résout rien (`split.regions` vide — c'est exactement le gate qui protège les anneaux, § *Limite réelle découverte* ci-dessus : leur squelette n'a pas de jonction, donc SGSD n'est même pas tenté, et le chemin spécial `build_annular_sections` reste seul aux commandes). Les deux chemins (direct et SGSD) convergent ensuite dans le MÊME code d'émission de sections et le MÊME mécanisme de repli tatami structurel déjà en place — seule la manière dont les colonnes sont produites diffère, jamais le format de sortie ni les garanties de couverture.

Un vrai défaut trouvé en le branchant, corrigé avant d'aller plus loin

Faire tourner la suite de tests existante (`tests/unit/autodigitize`) contre ce nouveau chemin par défaut a immédiatement révélé un cas réel non couvert par les tests SGSD précédents (tous basés sur le corpus de formes propres, jamais sur le pipeline de rejet complet d'`autodigitize`) : quand SGSD isole une branche localement trop large en sa propre `SatinRegion`, cette sous-région — n'ayant plus la jonction voisine — n'est plus `RequiresDecomposition` mais passe par le chemin « colonne unique » de `build_satin_columns`, qui NE POUSSE PAS le même message « colonne refusée » que la boucle par arête du chemin direct en cas d'échec. Résultat : le remplissage de repli comblait bien la zone (aucun trou physique), mais `AutoResult::warnings` restait complètement VIDE — silence total sur la raison du repli, brisant la garantie explicite « jamais silencieux » du contrat documenté sur ce champ. Corrigé en poussant explicitement un message (incluant `built.refusal` s'il est renseigné) chaque fois qu'une sous-région SGSD ne produit aucune colonne.

Résultat mesuré

Sur le CORPUS SYNTHÉTIQUE (formes propres), l'intégration en production préserve exactement les gains déjà mesurés par `sgsd-debug` (+4,6 à +7,4 points sur les formes branchées, comportement strictement identique aux formes déjà simples ou refusées).

Sur une IMAGE RÉELLE (`tentabrode.png`, § diagnostic ci-dessus), le même diagnostic permanent, réexécuté après ce branchement, montre un résultat positif mais plus mesuré : couverture réelle agrégée (satin + repli tatami) **98,9 %**, contre 97,3 % avant l'intégration — un gain net, mais SGSD s'y appuie plus fréquemment sur le filet tatami que sur une couverture satin directement meilleure, contrairement au corpus synthétique propre. Cause probable, cohérente avec les limites déjà documentées (`notch`, branches étroites du `trident`) : les régions vectorisées depuis une vraie image sont nettement plus bruitées que les formes procédurales du corpus de test, et la recherche de coupe (phase 3, plage 300–4000 µm) échoue plus souvent à isoler proprement une branche courte ou irrégulière — auquel cas SGSD reporte honnêtement le chemin en `unresolved_paths` plutôt que de forcer une coupe défectueuse (§ phase 3/4), et le filet tatami prend le relais. La garantie de couverture reste intacte (aucune régression, gain net mesuré) ; améliorer la robustesse de la recherche de coupe sur des squelettes bruités reste un travail futur.

Testé

`tests/unit/autodigitize/test_autodigitize.cpp` : à l'introduction (avec bascule encore présente), le test historique du « réseau en T » (validant la topologie de jonction partagée du chemin direct) était conservé avec `use_sgsd_decomposition = false` explicite, à côté d'un nouveau test couvrant le comportement SGSD sur la même forme (2 sections indépendantes au lieu de 3, `topology` absent — état déjà documenté comme « colonne indépendante » dans `document::SatinParams`) ; le test de non-silence sur une lettre réelle trop large (`tests/unit/autodigitize, fixture GISTRE`) confirme le correctif ci-dessus. Depuis la suppression du bascule (§ section suivante), le test « réseau en T » côté direct pur a été retiré ici — cette garantie reste couverte indépendamment par `tests/unit/auto_satin/test_columns.cpp`, qui exerce `build_satin_columns` directement. Suite complète sans régression sur les 4 diagnostics permanents déjà connus.

SGSD exclusif : suppression du bascule + intégration UI desktop (2026-08-14)

État : Présent. SGSD n'est désormais plus une option activable/désactivable côté application : `AutoOptions::use_sgsd_decomposition` (§ section précédente) a été **supprimé** de `libs/autodigitize/include/openstitch/autodigitize/autodigitize.hpp`. Sur une région dont le squelette est branché, la décomposition guidée par squelette est désormais le SEUL chemin emprunté par l'application — plus d'échappatoire côté `AutoOptions`. L'appel direct historique à `build_satin_columns` sur la région entière reste utilisé, mais uniquement en INTERNE comme filet de sécurité structurel (région non branchée dès le départ, ou anneau — squelette en boucle fermée sans jonction détectable) : jamais une zone laissée sans le moindre point.

Point d'entrée unique partagé

Toute la logique auparavant privée à `autodigitize.cpp` (`try_sgsd_decomposition + sections_from_result`) a été extraite en une fonction publique, `autodigitize::build_satin_sections`, déclarée dans `autodigitize.hpp` aux côtés de `satin_params_from_column` :

```
[[nodiscard]] SatinBuildReport build_satin_sections(
    const geometry::PathSet& region,
    const auto_satin::SatinColumnsParameters& genParams,
    Micrometers density, Micrometers pullCompensation,
    bool centerUnderlay, Micrometers maxWidth,
    const std::string& warningLabel = {});
```

`SatinBuildReport` regroupe les sections construites (`BuiltSatinSection`, géométrie + `document::SatinParams`), un indicateur `used_sgsd`, un signal `structural_gap` (zone potentiellement non couverte, pour un éventuel repli tatami côté appellant), les avertissements accumulés, l'analyse de satinabilité de la région ENTIÈRE (`whole_region_report`, toujours calculée en amont, utile pour un aperçu même quand SGSD décompose ensuite en sous-régions) et un message de refus le plus pertinent si rien n'a pu être construit.

Ce point d'entrée est désormais utilisé par LES DEUX consommateurs de satin automatique de l'application, garantissant les mêmes garanties de couverture partout plutôt que de dupliquer la logique de décomposition + repli :

- `auto_digitize()` (`libs/autodigitize/src/autodigitize.cpp`), déjà branché depuis la section précédente ;
- trois des quatre points d'appel satin manuel de `apps/desktop/main_window.cpp` (§ ci-dessous).

Migration de l'UI desktop

`apps/desktop/main_window.cpp` appelait `auto_satin::build_satin_columns` directement à quatre endroits. Trois sont migrés vers `build_satin_sections` :

- **`createSatinObject()`** (« Colonne satin ») : la géométrie des rails ne dépend pas de la densité/compensation/sous-couche choisies dans le dialogue (uniquement des réglages de remplissage des points, appliqués en aval) — `build_satin_sections` est donc appelée UNE SEULE FOIS, avant le dialogue, avec les valeurs par défaut de `document::SatinParams`, puis les champs `density/pull_compensation/center_underlay` de chaque section sont réécrits avec les valeurs choisies par l'utilisateur une fois le dialogue validé — évite de relancer la recherche à faisceau (potentiellement coûteuse) une seconde fois pour la même forme. Les avertissements SGSD sont désormais remontés via `warnAboutSkippedAutoSatinBranches` (déjà utilisé après la segmentation IA), auparavant silencieux sur ce chemin.
- **Outil ligne de coupe** (`createSatinObjectWithCutLine`) : le dialogue étant affiché AVANT la boucle sur les morceaux coupés, la densité est déjà connue — chaque morceau appelle `build_satin_sections` directement avec les valeurs finales. Un morceau ne produisant aucune section

reste ignoré (comportement inchangé), et les avertissements de tous les morceaux sont accumulés puis remontés en une seule fois en fin d'opération.

- **autoConvertToSatin()** (aperçu + confirmation avant conversion) : l'aperçu de satinabilité (statut, largeurs, nombre de branches) utilise désormais `whole_region_report` — l'analyse de la région ENTIÈRE, calculée une seule fois en amont même quand SGSD décompose ensuite en plusieurs sous-régions — plutôt que `SatinColumnsResult::report`, qui ne portait que sur l'appel direct.

setStitchType() (cas 2, conversion de type d'un objet existant) reste **INCHANGÉ**, intentionnellement : cette action exige EXACTEMENT une colonne unique (modèle « un objet, un type »), un cas explicitement hors du champ de la décomposition SGSD — commentaire déjà présent dans le code avant ce chantier. Elle continue d'appeler `build_satin_columns` directement.

Tests

`tests/unit/autodigitize/test_autodigitize.cpp` : le test « réseau en T » scindé par bascule (§ section précédente) n'était plus exprimable une fois `use_sgsd_decomposition` supprimé — la variante « SGSD désactivé » a été retirée (garantie du moteur direct seul déjà couverte indépendamment par `tests/unit/auto_satin/test_columns.cpp`), la variante SGSD renommée pour ne plus référencer un bascule qui n'existe plus. Suite complète (Debug + Release) sans régression sur les 4 diagnostics permanents déjà connus. Aucun nouveau test desktop ajouté pour la migration UI : le comportement géométrique exercé (`build_satin_sections`) est déjà couvert côté `autodigitize`, et les trois fonctions migrées n'ont pas de suite Qt dédiée préexistante à étendre.

Défaut réel : une sous-région SGSD acceptée peut laisser un reliquat massif non couvert (2026-08-14)

État : Corrigé. Signalé par une capture d'écran utilisateur montrant une zone visiblement sans le moindre point (ni satin, ni repli tatami) sur une grande partie d'une forme réelle (une lettre avec contre-poinçon — un trou proche d'une jonction de branches).

Root cause

`build_satin_sections` (§ *Intégration production* ci-dessus) déterminait `structural_gap` — le signal qui déclenche le remplissage tatami de repli — à partir de signaux purement STRUCTURELS : un chemin SGSD non isolé (`unresolved_paths`), ou une sous-région ayant produit **zéro** colonne. Ce signal ne détecte que l'absence totale de colonne sur une branche — jamais une colonne **acceptée** (au moins une colonne produite, statut `SuitableWithWarnings` ou `RequiresDecomposition`) dont la géométrie n'approxime qu'une fraction de la surface de sa propre sous-région.

Rejoué avec les coordonnées exactes exportées par l'utilisateur

(`tests/unit/satin_planning/test_region_split.cpp`, nouveau test de régression) : la région (60,8 mm², un trou/boucle + 3 jonctions) se décompose en 4 chemins SGSD, **tous isolés, tous acceptés** (`regions=4`, `non_isoles=0`, chaque sous-région produit au moins une colonne) — succès structurel complet, donc `structural_gap` restait `false`. Pourtant, la sous-région contenant la boucle (21,2 mm², topologie proche d'un anneau) est si mal approximée par un unique ruban satin que le reliquat géométrique réel (région moins bandes réellement produites, mesuré par `geometry::subtract_polygons`) atteint **31,5 mm² sur 60,8 mm² (52 %)** — sans qu'aucun signal structurel n'ait jamais existé pour le détecter. C'est la même famille de limite que celle déjà mesurée sur `tentabrode.png` (§ *Intégration production* : « une branche **acceptée** dont la géométrie de colonne ne couvre pas fidèlement sa région »), mais ici démontrée **après** décomposition SGSD, pas seulement sur l'appel direct pré-SGSD.

Correctif

`structural_gap` est désormais calculé une seule fois, à la fin de `build_satin_sections`, à partir d'une mesure géométrique RÉELLE plutôt que d'un proxy structurel : `geometry::subtract_polygons(region, bandes des sections produites)`, avec un seuil mixte fixe **et** proportionnel ($\max(1 \text{ mm}^2, 3 \% \text{ de l'aire totale})$) pour tolérer le reliquat NATUREL d'une pointe/jonction (quelques mm^2 même quand tout réussit, § *Intégration production*) sans le confondre avec un vrai trou structurel, quelle que soit la taille de la région. Ce calcul généralise et remplace les deux anciens signaux (arêtes du squelette vs sections produites côté direct ; chemins non isolés/refusés côté SGSD) : un chemin jamais isolé ou une sous-région refusée n'ont, par construction, aucune bande dans le calcul, donc leur surface apparaît déjà dans le reliquat mesuré.

Point de calibration important, trouvé en cours de correctif : la première version utilisait `shrink_strips_for_cutout` (le même rétrécissement de 0,4 mm par bande que la découpe du remplissage de repli lui-même, pour faire légèrement déborder le repli DANS la bande satin plutôt que de laisser un interstice) pour la MESURE du reliquat — ce qui gonflait artificiellement le reliquat mesuré d'une valeur proportionnelle au périmètre total des bandes, au point de déclencher un repli même sur le réseau en T et l'anneau fin du corpus synthétique (couverture déjà mesurée à 97,7 % et 100 % respectivement, § *Outil CLI sgds-debug*) — un faux positif immédiat sur les deux tests existants. Corrigé en mesurant le reliquat sur les bandes à leur taille RÉELLE, non rétrécies ; le rétrécissement reste réservé, comme avant, à la découpe du polygone de repli lui-même côté appelant (`auto_digitize`).

Résultat mesuré

Sur le réseau en T (corpus synthétique, jusqu'ici considéré « sans repli nécessaire » alors que sa couverture SGSD mesurée était déjà de 97,7 %, pas 100 %) : le reliquat naturel près de l'ancienne jonction est désormais correctement détecté et comblé par un remplissage tatami — comportement attendu, test mis à jour en conséquence (`tests/unit/autodigitize/test_autodigitize.cpp`). Sur l'anneau fin (couverture 100 % mesurée) : aucun repli ne se déclenche, confirmant l'absence de faux positif sur un cas réellement complet. Sur la forme utilisateur ayant motivé ce correctif : 31,5 mm^2 désormais comblés en tatami plutôt que silencieusement absents.

Tests

- `tests/unit/satin_planning/test_region_split.cpp` — nouveau test (« région réelle avec boucle -- SGSD accepte chaque sous-région mais laisse un reliquat massif non couvert ») jouant les coordonnées exactes de la forme utilisateur : vérifie le succès structurel complet (`regions=4`, `non_isoles=0`, chaque sous-région produit ≥ 1 colonne) **et** le reliquat géométrique réel ($> 30 \% \text{ de l'aire totale}$) malgré ce succès — la preuve que le signal structurel seul est insuffisant.
- `tests/unit/autodigitize/test_autodigitize.cpp` — le test « réseau en T » mis à jour pour vérifier explicitement l'apparition du remplissage de repli (`result->vectors.size() == 2`, un objet tatami présent) plutôt que son absence.
- Suite complète (Debug + Release) sans régression sur les 4 diagnostics permanents déjà connus.

Refonte du contrat satin : le planner récursif `create_satin_plan` (2026-08-14)

État : Présent. Changement de paradigme, pas une nouvelle optimisation locale : jusqu'ici, une région trop complexe pouvait se voir répondre « pas directement satinable » comme résultat FINAL, avec un repli tatami silencieux ou une heuristique naïve dégradée en substitut. Le contrat devient désormais : **une intention SATIN explicite (utilisateur ou classification automatique) doit toujours déclencher une recherche de partition satinable, jamais un refus direct** — `RequiresDecomposition` (et tout statut de satinabilité) redevient une information INTERNE au planner, jamais une réponse finale à l'utilisateur.

Le planner : `satin_planning::create_satin_plan`

Nouveau module (`libs/satin_planning/include/openstitch/satin_planning/satin_plan.hpp` + `src/satin_plan.cpp`), point d'entrée unique désormais partagé par TOUS les chemins de création satin de l'application (auto-numérisation, création manuelle, ligne de coupe, conversion de type) via `autodigitize::build_satin_sections`, qui n'est plus qu'un mince adaptateur document au-dessus de lui.


```
SatinPlan create_satin_plan(const geometry::PathSet& source,
                           const SatinPlanConfig& config = {});
```

Algorithme, réellement récursif (contrairement à l'ancienne décomposition SGSD à une seule passe, § sections précédentes) :

1. **Essaie toujours le solveur local d'abord** (`auto_satin:: build_satin_columns`, inchangé — jamais patché, exactement l'esprit du chantier SGSD original) sur la région telle quelle.
2. **Mesure la couverture réellement obtenue** (`satin_coverage:: analyze_satin_coverage`, le même oracle indépendant que les phases 5 à 7 de SGSD) — jamais une règle théorique sur la topologie du squelette (nombre de jonctions, largeur agrégée...). Si la couverture passe (`SatinCoverageReport::passed`) ou que la profondeur maximale de récursion est atteinte, la région est acceptée telle quelle.
3. **Si non, décompose** (réutilise `decompose_into_paths + split_region + OracleGuidedSelector`, exactement l'infrastructure SGSD existante, jamais réimplémentée) et **replanifie récursivement CHAQUE sous-région** — c'est ici la vraie nouveauté : une région fille encore médiocre peut à son tour être redécoupée, au lieu de s'arrêter après une seule passe.
4. Si la décomposition ne produit rien de mieux (aucune jonction, ou aucune coupe valide), le meilleur effort local est conservé tel quel plutôt que perdu.
5. **Réparation de résidu** (une fois l'arbre de récursion épuisé) : mesure la couverture AGRÉGÉE sur la région SOURCE entière et tente de replanifier chaque composante manquante significative comme une nouvelle région à part entière — jusqu'à `max_residual_repair_rounds` passes.

```
plan(region):
    candidate = try_local_satin(region)
    if candidate.couverture suffisante OU profondeur max atteinte:
        return candidate
    decompositions = decompose(region)
    if rien de mieux:
        return candidate (meilleur effort)
    return concat(plan(sous-région) pour chaque sous-région)

# une fois la racine entièrement planifiée :
reliquat = couverture_agreee(source) - union(toutes les régions acceptées)
pour chaque composante significative du reliquat :
    tenter plan(composante) -- vraie tentative recursive, jamais un remplissage automatique
```

Ce que `create_satin_plan` ne fait JAMAIS

Ce module ne connaît même pas `TatamiParams` — il ne produit QUE de la géométrie satin + un résidu explicite (`SatinPlan::unresolved_residual`, une liste de `geometry::PathSet` brute, jamais transformée). Aucune décision de repli n'est prise ici : c'est strictement à l'appelant (§ section suivante) de décider quoi faire d'un résidu non nul — jamais une substitution automatique et silencieuse de l'intention SATIN.

Trois défauts réels trouvés en construisant le planner

1. Filtrer un reliquat individuellement négligeable peut masquer un vrai trou agrégé. Première version : une composante manquante sous un seuil de signifiante (aire, rayon de trou) était simplement écartée du résidu rapporté. Sur une forme réelle (lettre T d'un vrai logo, empattement large), ce filtrage a d'abord masqué un trou de **384,8 mm² sur 392,3 mm² source (98 %)** composé de nombreuses petites composantes individuellement sous le seuil. Corrigé en séparant les deux décisions que ce seuil confondait : il ne décide plus JAMAIS de ce qui est *rapporté* (`unresolved_residual` reste exhaustif, toute composante manquante y apparaît), seulement de ce qui mérite une *tentative de réparation individuelle* coûteuse (éviter de relancer `build_satin_columns` et une mesure de couverture complète sur chaque micro-résidu d'arrondi). Le filtrage de signifiante pour la décision finale (« faut-il alerter l'utilisateur ? ») vit désormais uniquement chez l'appelant (`autodigitize::build_satin_sections`, § section suivante), sur la somme totale du résidu — jamais composante par composante.

2. Une couverture invérifiable n'est pas une réussite. `analyze_satin_coverage` ne peut échouer (une vraie erreur, jamais un simple score bas) que sur une région cible dégénérée (moins de 3 sommets) — un artefact occasionnel d'un découpage ou d'une réparation de résidu. `build_satin_columns` construit parfois quand même des colonnes (souvent quasi vides) sur une telle région dégénérée. Accepter cela sans distinction d'une vraie réussite mesurée fabriquerait une « réussite » invérifiable en silence : sur la même forme réelle, un plan se déclarait complet avec 4 régions acceptées alors que la couverture agrégée RÉELLE n'était que de **1,9 %**. Corrigé en routant explicitement ce cas vers le résidu plutôt que vers une acceptation aveugle — seule la profondeur de récursion maximale (dernier recours documenté, pas un raccourci général) accepte encore un résultat non vérifié.

3. Une couverture mesurée mais dérisoire n'est pas non plus une réussite. Trouvé en rejouant le diagnostic permanent `tentabrode.png` (une vraie image, § chapitre *Analyse et validation*) après les deux premiers correctifs : la couverture agrégée RÉELLE s'est effondrée à **16,0 %** (contre 98,9 % avant ce chantier), alors que le plan se déclarait sans résidu significatif. Cause : sur une région sans aucune jonction interne (un gros blob rond, aucune décomposition possible par construction), le « meilleur effort local » — une colonne unique mesurée à 1,2 % de couverture réelle — était accepté SANS aucun seuil minimal, simplement parce que sa couverture avait pu être *mesurée* (donc distincte du défaut n°2 ci-dessus) et qu'aucune décomposition n'était possible pour faire mieux. Corrigé en ajoutant `SatinPlanConfig::min_fallback_coverage_ratio` (50 % par défaut) : un « meilleur effort » en dessous de ce seuil est désormais rapporté comme résidu au lieu d'être accepté tel quel — que ce soit au dernier niveau de récursion ou après un échec de décomposition. Ce seuil s'applique de la même façon à la boucle de réparation de résidu (§ ci-dessus), qui aurait sinon pu « réparer » un trou en y empilant plusieurs colonnes toutes individuellement dérisoires sans jamais s'apercevoir que la couverture agrégée restait mauvaise.

Intégration : `autodigitize::build_satin_sections` devient un adaptateur mince

`libs/autodigitize/include/openstitch/autodigitize/autodigitize.hpp` — `SatinBuildReport` expose désormais `unresolved_residual` (géométrie brute complète, jamais filtrée) et `aggregate_coverage` (`satin_coverage::SatinCoverageReport` sur la région source entière). `structural_gap` reste un raccourci booléen pour les appelants existants, mais recalculé à partir de la somme AGRÉGÉE du résidu (seuil mixte, fixe et proportionnel, même calibration que le correctif précédent) — jamais composante par composante, pour ne pas répéter le premier défaut ci-dessus à ce niveau.

Les deux workflows, distingués comme demandé (§24)

- `auto_digitize()` (auto-numérisation, classification automatique « AutoChoice » — aucun utilisateur interactif à qui proposer un choix) reste la seule voie qui crée encore un remplissage tatami de repli automatique sur un résidu significatif — mais plus jamais silencieusement : un avertissement explicite (« satin incomplet : X % de la région couverte, Y mm² comblés par un remplissage tatami de repli (classification automatique) ») est systématiquement poussé dans `AutoResult::warnings` avant toute création.

- **Les créations satin explicites côté desktop** (`createSatinObject`, l'outil ligne de coupe, `autoConvertToSatin`) ne créent plus JAMAIS de tatami de repli automatiquement. Un nouvel avertisseur partagé, `MainWindow::warnAboutIncompleteSatinCoverage` (`apps/desktop/main_window.cpp/.hpp`), informe l'utilisateur (couverture estimée, surface manquante, options — retoucher la coupe, créer un tatami manuellement, ou accepter le résultat partiel) au lieu de décider à sa place. `autoConvertToSatin()` va plus loin : la couverture estimée apparaît directement dans l'aperçu, AVANT la création (§23 du plan de refonte), pas seulement après coup.
- **`setStitchType()` (conversion de type d'un objet existant, cas satin)** passe désormais par le même point d'entrée unifié (`build_satin_sections`), bénéficiant de la même qualité de solveur récursif — mais reste honnêtement limitée par une contrainte STRUCTURELLE de `SetStitchTypeCommand` (remplace un seul `EmbroideryObject` par un seul jeu de paramètres, pas un lot). Sur une décomposition à plusieurs sections, la plus grande est conservée et l'utilisateur en est explicitement informé (« Cette forme nécessite plusieurs sections... ») plutôt qu'une substitution silencieuse par l'ancienne heuristique naïve dégradée.

Adjacence et recouvrement reliés au planner (§19/§20, 2026-08-14)

`satin_planning::generate_overlaps` (phase 8 du plan SGSD) existait déjà, testé et fonctionnel, depuis une phase antérieure du chantier — mais rien ne l'appelait depuis `create_satin_plan`, et `SatinPlan::adjacency` restait un vecteur systématiquement vide. Corrigé : `decompose_and_recurse` calcule maintenant, à chaque niveau de récursion, l'adjacence directe entre les sous-régions issues d'une même coupe (`RegionSplitReport::merge_candidates`) ET leur recouvrement de couture (`generate_overlaps`), puis réindexe ces paires vers leur position finale dans `SatinPlan::regions` au fur et à mesure que les résultats remontent la récursion (et, séparément, à travers la boucle de réparation de résidu, §17).

Deux nouveaux champs sur `SatinPlan` :

- `adjacency` — paires d'index dans `regions` connues comme directement adjacentes (une même coupe, les deux côtés restés des feuilles).
- `overlaps` — aligné 1:1 sur `adjacency` : la géométrie de couture dédiée (chaque côté élargi vers l'autre, `SatinPlanConfig::overlap_distance`, 300 µm par défaut) — repli sur la région non modifiée si le calcul échoue ou si `SatinPlanConfig::compute_overlaps` est désactivé, jamais un vecteur désaligné ou plus court.

Limite assumée, héritée de `generate_overlaps` lui-même : une paire dont un côté a été redécoupé ultérieurement (donc devenu plusieurs feuilles) n'est pas rapportée — déterminer l'adjacence à travers un redécoupage plus profond reste un travail futur. Le module continue de ne faire QUE calculer cette géométrie : aucune fusion ni génération de points n'y est reliée ici — c'est à un futur consommateur (génération de points, §20) d'en faire usage.

Tests dédiés dans `tests/unit/satin_planning/test_satin_plan.cpp` : vérifient sur le réseau en T que `adjacency/overlaps` sont bien peuplés, alignés en taille, que chaque extension de recouvrement ne rétrécit jamais la région d'origine, et que désactiver `compute_overlaps` laisse `adjacency` intact (deux informations indépendantes) tout en retombant proprement sur la géométrie identité.

Deuxieme famille de coupes candidates : `JunctionSeparator` réutilise (§14, 2026-08-14)

`generate_cut_candidates` ne produisait qu'une seule famille (normale au squelette, distance croissante depuis la jonction) — la cause directe du résultat catastrophique mesuré sur la lettre T réelle (1,9 % de couverture agrégée). Le plan de refonte demandait explicitement de réutiliser les `JunctionSeparatorInfo` déjà calculés par le moteur Legacy (`resolve_junction`, sommet reflex du contour à chaque confluence, ou à défaut intersection de bissectrice) plutôt que de deviner une position par balayage — c'est fait, sans réimplémenter la moindre géométrie de détection d'encoche.

Le principe : avant `split_region`, `decompose_and_recurse` appelle une fois `auto_satin::build_satin_columns` en mode Legacy FORCÉ (indépendamment du mode réellement utilisé pour la génération de rails) sur la région entière, uniquement pour récolter `junction_separators` —

un appel diagnostique, jamais utilisé pour la géométrie satin produite. Pour chaque événement de détachement, `generate_cut_candidates` projette chaque séparateur de la même jonction sur la centerline de l'arête testée (distance d'arc + distance perpendiculaire) ; un séparateur suffisamment proche (`junction_separator_max_perpendicular_um`, 6 mm par défaut — calibré empiriquement sur `t`, où les vrais séparateurs mesurent 2,5 à 3,5 mm depuis la centerline, une tolérance de 2 mm initiale les rejetait tous à tort) devient une distance de coupe candidate **testée en premier**, avant le balayage régulier.

Ordre critique : `OracleGuidedSelector` (phase 6, beam search) n'évalue que les `beam_width` premiers candidats VALIDES du vecteur retourné (3 par défaut) — ajouter les candidats §14 en fin de vecteur les aurait pratiquement condamnés à ne jamais être évalués. Testés en premier, ils concourent en priorité pour le budget du beam search, cohérent avec le fait que ce sont les candidats les mieux motivés géométriquement.

Effet mesuré : sur le corpus synthétique branché (`y/cross/h/trident`), la couverture agrégée est passée à 95–98 % (bien au-delà du seuil de test à 75 %). Sur la lettre T réelle en revanche, aucun changement — le moteur Legacy lui-même échoue à résoudre les mêmes jonctions incohérentes sur cette forme pathologique, donc `junction_separators` reste vide et la famille supplémentaire n'a simplement rien à proposer : dégradation honnête, jamais un crash ni une fabrication.

Effet de bord découvert en testant : le corpus `h/trident/cross` n'a plus jamais besoin d'une deuxième passe de récursion — la première décomposition, désormais meilleure, suffit. Bon signe de qualité, mais cela invalidait la prémisse du test dédié à la PROPRIÉTÉ de récursion (`create_satin_plan : recursion_reelle...`, qui vérifiait `depth > 1` quelque part dans ce corpus) : corrigé en désactivant `use_junction_separator_cuts` dans ce test spécifique — il isole désormais le mécanisme de récursion, pas la qualité d'une famille de coupes particulière.

Nouveau `SatinPlanConfig::use_junction_separator_cuts` (`true` par défaut) — désactivable pour éviter le coût d'un appel Legacy supplémentaire par niveau de récursion, ou pour isoler l'ancienne seule famille en test.

Testé (`tests/unit/satin_planning/test_region_split.cpp`) : la famille apparaît bien en premier et priorise le budget du beam search sur `t` ; un séparateur d'une autre jonction, ou trop éloigné perpendiculairement, est correctement ignoré. Suite complète (`satin_planning`, `autodigitize`, `desktop`) sans régression après le fix de calibration ci-dessus.

Passe de fusion reliée au planner (§18, phase 7 SGSD, 2026-08-14)

`evaluate_merge_pass` existait déjà, testé et fonctionnel, depuis la phase 7 du chantier SGSD — mais comme `generate_overlaps` avant lui, rien ne l'appelait depuis `create_satin_plan`. La sélection de coupe (recherche à faisceau guidée par l'oracle) choisit déjà le découpage jugé le plus favorable au moment de couper, mais ne reconsidérerait jamais après coup si NE PAS couper aurait donné un résultat comparable avec moins de segments (§18 : « le plus petit nombre de segments permettant une couverture et un satin corrects »).

Le principe : dans `decompose_and_recurse`, juste après `split_region`, `evaluate_merge_pass` est appelé sur le rapport de découpage complet. Pour chaque paire recommandée à la fusion, les deux régions ne sont PAS recousues individuellement — une seule récursion est lancée sur la géométrie fusionnée exacte (`MergeCandidate::merged_region`) à leur place. Chaque région issue de `split_region` participe à au plus un `merge_candidates` (celui de la coupe qui l'a créée), donc aucune ambiguïté sur quelles paires fusionner. Nouveau `SatinPlanConfig::use_merge_pass` (`true` par défaut) et `merge_pass_coverage_tolerance` (2 %, même défaut que `MergePassParams`).

Effet mesuré, honnêtement rapporté : sur tout le corpus de formes délibérément branchées (`t/y/cross/h/trident`), §18 ne recommande **aucune** fusion — le nombre de régions final est strictement identique avec ou sans la passe. Attendu : ces formes sont conçues pour nécessiter réellement une découpe, donc défusionner dégraderait la couverture au-delà de la tolérance. Le chemin « fusion appliquée » du wiring restait donc totalement inexercé par le corpus existant — pour le tester honnêtement (plutôt que de le laisser sans preuve), le test dédié force la recommandation via une tolérance de 100 % (`merge_recommended` devient vrai dès que la région fusionnée construit ne serait-ce qu'une colonne,

indépendamment de la comparaison réelle de couverture) : confirme que le nombre de régions diminue réellement (2→1 sur t) quand la recommandation est positive — sans dépendre d'une forme du corpus où §18 gagnerait par hasard.

Testé (tests/unit/satin_planning/test_satin_plan.cpp) : fusion forcée réduit réellement le nombre de régions ; suite complète (satin_planning, autodigitize, desktop) sans régression.

Coupes concavité→concavité et concavité→bord opposé (§14, suite, 2026-08-14)

Les deux familles précédentes (normale au squelette, JunctionSeparator) sont toutes deux ancrées sur un événement de détachement à une JUNCTION du squelette. Une région SANS jonction interne — un seul chemin topologique, comme notch/pinch du corpus (bande entaillée d'un profond V) — n'avait donc **rien** à quoi elles puissent s'accrocher : `decompose_and_recurse` renonçait immédiatement dès que `junction_count() == 0`, même quand le solveur local échouait clairement.

Le principe (nouveau module `libs/satin_planning/concavity_cuts`) : pure géométrie de CONTOUR, sans aucun graphe de squelette. - concavité→concavité : chaque PAIRE de sommets reflex (concaves) du contour extérieur, coupée en ligne droite entre les deux — le cas d'un sablier (deux entailles se faisant face). - concavité→bord opposé : chaque sommet reflex seul, coupé selon la direction perpendiculaire moyenne des deux arêtes qui s'y rejoignent — orientée **empiriquement** vers l'intérieur de la matière (testée point par point via un test point-dans-polygone, jamais supposée depuis le sens de parcours du contour, qui peut différer d'une concavité à l'autre sur une forme non convexe) — le cas d'une entaille unique.

Chaque candidat est filtré par les mêmes règles que les familles précédentes (exactement 2 morceaux, aucun trop petit), puis le meilleur est choisi en construisant réellement les deux moitiés et en mesurant leur couverture combinée (même principe que le beam search phase 6).

Effet mesuré, honnêtement rapporté :

Forme	Avant	Après
pinch (encoche à 0,3mm du bord opposé)	0 région, 0% (échec total)	4 régions, 86,5%
notch (encoche à 1mm du bord opposé)	1 région, 76,6% (repli dégradé)	3 régions, 79,9%

Deux garde-fous trouvés par des régressions réelles en construisant cette famille

(tests/unit/autodigitize/test_autodigitize.cpp) : 1. **Jamais pendant la réparation de résidu** (`fromResidualRepair`) : un fragment de résidu est une géométrie de DÉCOUPE (`Clipper2`, `subtract_polygons` sur la couverture déjà produite), souvent petite et irrégulière par construction — un sommet reflex « parasite » issu de ce bruit géométrique fragmentait le résidu en éclats au lieu de le laisser honnêtement rapporté (le réseau en T voyait son reliquat mesuré augmenter malgré l'« acceptation » de ces éclats). 2. **Jamais sur une région avec un trou** : un anneau a déjà son propre solveur local dédié et excellent (`build_annular_sections`, appelé inconditionnellement dès qu'il y a exactement un trou) — la famille concavité ne considère que le contour EXTÉRIEUR, jamais les trous, donc une coupe rectiligne y ignore complètement la topologie du trou. Sur un anneau réel (segmentation → vectorisation, contour légèrement irrégulier), un sommet reflex marginal du contour tranchait l'anneau en 9 fragments au lieu des 4 sections annulaires propres — préféré inconditionnellement (même règle que le chemin basé sur le squelette : la décomposition l'emporte sans comparaison de qualité dès qu'elle produit quoi que ce soit).

Nouveau `SatinPlanConfig::use_concavity_cuts` (true par défaut), `concavityCutParams`, `concavity_cut_beam_width` (6 par défaut).

Troisième défaut trouvé, en validation manuelle après les deux garde-fous ci-dessus : la famille concavité→concavité génère une paire de candidats par PAIRE de sommets reflex ($O(n^2)$), chacune une vraie coupe `Clipper2` — un contour issu de vectorisation réelle peut compter des dizaines de sommets « reflex » au sens strict mais géométriquement négligeables (quasi rectilignes, bruit de tracé), faisant exploser le temps de calcul sans qu'aucun ne représente une vraie encoche. Corrigé par un filtre d'angle minimal

(`min_reflex_turn_deg`, 15° par défaut) appliqué AVANT toute construction de candidat, plus une borne dure de défense en profondeur (`max_reflex_vertices`, 12 par défaut — garde les concavités les plus prononcées, triées par magnitude de virage).

Testé (`tests/unit/satin_planning/test_concavity_cuts.cpp`, nouveau fichier) : aucune concavité sur un rectangle convexe ; notch n'a qu'une seule concavité réelle (contrairement à l'intuition visuelle — vérifié empiriquement : les « épaules » du V sont convexes, pas concaves, seule la pointe l'est) donc seule la famille bord-opposé s'y applique ; une forme « sablier » construite à la main (deux notches en vis-à-vis) exerce spécifiquement concavité→concavité. Suite complète (`satin_planning`, `autodigitize`, `desktop`) sans régression après les deux garde-fous ci-dessus.

Dialogue à choix multiples pour un satin incomplet (§23, 2026-08-15)

`warnAboutIncompleteSatinCoverage` était une simple `QMessageBox::information` — un résumé en prose des options (« retoucher la coupe, créer un tatami, ou accepter tel quel ») dont AUCUNE n'était réellement actionnable depuis la boîte elle-même. Remplacée par `askAboutIncompleteSatinCoverage`, qui retourne un choix réel (`ContinuePartial` / `UseTatami` / `Cancel`) via une `QMessageBox` à boutons personnalisés (« Continuer avec satin partiel » / « Utiliser tatami pour le reliquat » / « Annuler »), le détail par zone disponible via le bouton « Détails » que Qt ajoute automatiquement (`setDetailedText`) — couvrant « Voir le problème » sans un quatrième bouton dédié.

Changement d'architecture nécessaire : les deux appelants historiques (`createSatinObject`, `createSatinObjectWithCutLine`) committaient déjà l'`AddObjectBatchCommand` AVANT d'afficher l'ancien avertissement — un « Annuler » n'aurait donc pu qu'annuler une commande déjà appliquée. Le choix est désormais demandé AVANT toute création ; `Cancel` retourne sans avoir rien muté ; `UseTatami` construit des paires `VectorObject+EmbroideryObject` supplémentaires (même schéma que le repli automatique d'`autodigitize.cpp`, `appendTatamiFallbackObjects`, jamais réimplémenté différemment) et les inclut dans le MÊME `AddObjectBatchCommand` que les sections satin — un seul geste annulable, satin et repli tatami ensemble.

`autoConvertToSatin()` avait déjà son propre gate Oui/Non pré-crédation avec un aperçu de couverture, mais pas d'option tatami — unifié pour utiliser le même dialogue quand le reliquat est significatif (le Oui/Non simple reste le geste de confirmation le plus léger pour une conversion propre, sans interrompre inutilement l'utilisateur).

Défaut trouvé en écrivant les tests : le helper de test existant (`autoDismissModalDialogs`) appelait `QDialog::accept()` directement plutôt que de cliquer un bouton — invisible pour une `QMessageBox` à boutons `QDialogButtonBox::Ok/Yes` standards (`accept()` en est l'équivalent), mais `askAboutIncompleteSatinCoverage` n'utilise QUE des boutons personnalisés (`addButton(text, role)`) : `clickedButton()` restait `nullptr` après `accept()`, lu à tort comme « Annuler ». Corrigé en cliquant réellement le bouton par défaut (`QMessageBox::defaultButton()->click()`), routant par la vraie machinerie de clic de Qt comme un appui sur Entrée. Nouveau `clickModalDialogButton(widget, texte)` pour choisir explicitement un bouton par son texte à travers plusieurs boîtes modales successives (densité puis §23) dans un test.

Testé (`tests/unit/desktop/test_main_window.cpp`, fixture `buildPinchShapeFixture` — encoche en V sans jonction de squelette, ~86 % de couverture réelle après §14, un reliquat largement significatif) : les trois choix bout en bout depuis `createSatinObject()` — satin seul sans tatami, satin + tatami de repli (un seul geste annulable), et document totalement inchangé sur annulation. Suite complète (61 tests `desktop`, +3) + `Debug/Release` (582 tests, mêmes 4 diagnostics permanents connus, aucune régression).

Recouvrement consommé par la génération de points (§20, 2026-08-16)

`generate_overlaps` (phase 8 SGSD) était relié au planner depuis le 2026-08-14 (`SatinPlan::adjacency/overlaps` peuplés) mais rien n'utilisait encore cette géométrie — les colonnes de chaque `SatinPlanRegion` restaient générées sur la géométrie structurelle seule, laissant l'interstice de coupe entre deux régions voisines visuellement ouvert malgré l'information déjà calculée.

Le principe : après la réparation de résidu (§17) et juste avant la mesure finale d'`aggregate_coverage`, `extend_columns_into_known_overlaps` reconstruit les colonnes de chaque région ayant un voisin direct

connu (`SatinPlan::adjacency`) sur sa géométrie de recouvrement DÉDIÉE (`SatinPlan::overlaps`, jamais la géométrie structurelle — qui reste inchangée, seule référence pour toute mesure de couverture, comme documenté dès la création de ce champ). Garde-fou central : la reconstruction n'est acceptée que si la couverture mesurée sur la région STRUCTURELLE ne se dégrade pas par rapport aux colonnes d'origine — repli automatique et silencieux sur les colonnes d'origine sinon (échec de construction sur la géométrie étendue, ou couverture moindre) — jamais une dégradation acceptée au nom de fermer un interstice.

Effet mesuré, honnêtement rapporté : sur le réseau en T, la couverture d'une des deux régions passe de 98,3 % à 99,8 % (l'autre reste inchangée — son extension n'a simplement rien apporté), la couverture agrégée de 98,3 % à 99,0 %. Une amélioration réelle mais modeste, cohérente avec ce que fait la fonctionnalité : fermer un interstice de `cut_width` (20 µm) déjà petit, pas résoudre un déficit de couverture majeur (ça, c'est le rôle des quatre familles de coupes et de la réparation de résidu).

Nouveau `SatinPlanConfig::extend_columns_into_overlap` (true par défaut, sans effet si `compute_overlaps` est désactivé).

Testé (`tests/unit/satin_planning/test_satin_plan.cpp`) : garde-fou — chaque région couvre au moins autant avec l'extension qu'sans elle, jamais moins ; et effet réel — la couverture agrégée s'améliore réellement sur le réseau en T (pas seulement « ne se dégrade pas »), preuve que le recouvrement est réellement consommé et non un no-op. Suite complète (`satin_planning`, `autodigitize`, `desktop`) + `Debug/Release` sans régression.

Ce qui reste hors périmètre (limites connues, honnêtement documentées)

(État au 2026-08-16 : aucune limite logicielle connue à cette date. Voir la section « Durcissement du contrat du SatinPlanner » plus bas, datée du 2026-08-17, pour un audit délibérément adverse qui en a trouvé plusieurs — la philosophie de documentation change à partir de là : une liste vide n'est plus le but recherché, cf. §26 de cette mission.)

Champ `EmbroideryIntent` et regroupement du panneau d'objets (§21/§24, 2026-08-16)

La distinction `AutoChoice/ForcedUserChoice` (§24) restait implicite dans le CODE (quel chemin d'appel a créé l'objet), jamais un concept explicite du modèle de document — et les sections d'un même plan satin multi-régions apparaissaient en liste plate dans le panneau d'objets (§21), sans regroupement visuel.

Modèle de données : nouveau `document::EmbroideryIntent` (`AutoChoice / ForcedUserChoice`) sur `EmbroideryObject::intent`, `AutoChoice` par défaut (comportement historique, absent d'un `.osp` antérieur $\Rightarrow$ `AutoChoice`, jamais une supposition différente — champ purement additif, aucun bump de `kSchemaVersion` nécessaire, même convention que `overrides/topology`). Fixé à chaque site de construction : `AutoChoice` dans `autodigitize.cpp` (classification automatique, sans utilisateur interactif à qui proposer un choix) ; `ForcedUserChoice` dans les trois créations satin manuelles côté desktop (`createSatinObject`, `createSatinObjectWithCutLine`, `autoConvertToSatin`), le repli tatami explicite du dialogue §23 (`appendTatamiFallbackObjects`), et les créations manuelles de contour/tatami (`createRunningStitchObject/createTatamiObject`). `SetStitchTypeCommand` marque `ForcedUserChoice` à `apply()` (toute conversion de type via cette commande est une action explicite de l'utilisateur, menu contextuel ou inspecteur) et restaure l'intention précédente à `revert()` — `undo/redo` exacts, comme `params_`.

Regroupement du panneau d'objets : `DocumentPanel::objectsList_` migré de `QListWidget` à `QTreeWidget` — les `EmbroideryObject` partageant le même `source_vector` (les sections d'un plan satin) sont regroupées sous un nœud parent, mais SEULEMENT quand il y en a plus d'une ; un objet seul reste un simple item de premier niveau, visuellement identique à l'ancienne liste plate. API publique (`refresh/syncSelection/signaux`) inchangée — migration contenue entièrement dans `document_panel.hpp/.cpp`.

Défaut trouvé en écrivant les tests : `source_vector.value == 0` est la valeur *invalid* de `ObjectId` (`Id::valid()`, `ids.hpp`) — grouper naïvement par `source_vector` aurait mélangé à tort tous les objets qui n'en ont jamais reçu (ex. les deux objets indépendants de la fixture historique de `test_document_panel.cpp`, tous deux à `source_vector` par défaut). Corrigé en ne comptant/regroupant QUE les `source_vector` valides.

Testé (tests/unit/desktop/test_document_panel.cpp, 4 nouveaux cas ; tests/unit/project_io/test_intent_persistence.cpp, nouveau fichier ; tests/unit/commands/test_undo_stack.cpp) : regroupement correct (3 sections plus 1 objet autonome ⇒ 1 groupe plus 1 item de premier niveau, jamais 4 items plats) ; sélectionner une section groupée émet son propre id ; sélectionner le nœud de groupe lui-même n'émet rien (aucun ObjectId propre) ; syncSelection retrouve un enfant groupé en descendant dans l'arbre ; round-trip exact de intent (API publique) et absence ⇒ AutoChoice (fichier forgé à la main) ; SetStitchTypeCommand marque/restaure intent sur undo/redo. Suite complète (desktop, project_io, commands, autodigitize) + Debug/Release (585 tests, mêmes 4 diagnostics permanents connus, aucune régression).

Coupes polygonales (§14, dernier élément, 2026-08-16)

Dernière limite honnêtement documentée du plan de refonte (§14) : les quatre familles de coupe candidates (normale au squelette, JunctionSeparator, concavité→concavité, concavité→bord opposé) ne suivent qu'une ligne DROITE entre leurs deux extrémités — aucune ne gère une coupe en plusieurs segments (un « coude »). Contrairement aux trois familles précédentes, chacune motivée par un cas RÉEL en échec dans le corpus de test, **aucune forme du corpus connu ne déclenche ce besoin à ce jour** : les coupes polygonales ont donc été implémentées et validées sur une forme synthétique construite spécifiquement pour exercer honnêtement le mécanisme, pas sur une régression de corpus. Travail repris à ce stade précis après plusieurs tours explicites de validation de la portée (l'équipe a confirmé vouloir ce dernier élément malgré l'absence de cas réel connu).

Le principe : quand la coupe DROITE entre deux concavités (famille concavité→concavité) échoue parce que son point médian tombe hors de la région — la ligne « coupe dans le vide » plutôt qu'à travers la matière —, find_elbow_waypoint cherche un point de passage intérieur (recherche bornée, 12 pas le long de la perpendiculaire au segment direct depuis son milieu, deux sens) tel que les deux segments obtenus (concavité→coude, coude→concavité) restent chacun entièrement dans la région (vérifié par échantillonnage : le point de passage et le milieu de chaque sous-segment). Si un tel point existe, try_polyline construit et mesure la coupe en coude exactement comme try_segment le fait pour une ligne droite (mêmes règles d'acceptation : deux morceaux exactement, aucun trop petit) ; sinon, comportement inchangé (rejet « point médian hors de la région »). Désactivable via ConcavityCutParams::try_elbow_cuts pour isoler le comportement lignes-droites seul (comparaison A/B, non-régression).

Trois défauts géométriques réels trouvés en construisant la forme de preuve (aucun n'était visible sur le corpus existant, puisque cette famille n'y était jamais exercée) :

1. *Coin non coupé côté extérieur du coude* — les deux quadrilatères de coupe (un par segment) ne se touchaient qu'en un seul point exact (le coude), laissant un triangle de matière intacte du côté convexe du virage dès que l'angle n'était pas parfaitement rectiligne. Corrigé par un petit carré centré au point de coude (make_square, 3× la demi-largeur de coupe) qui referme ce coin quel que soit l'angle, sans mitre à calculer.
2. *Extension trop longue aux deux extrémités* — les segments d'entrée/sortie étaient prolongés très au-delà de la région (même marge que la famille concavité→bord opposé, 1 m) dans la direction du segment ENTRANT ; pour un coude, cette direction n'a aucun rapport avec la géométrie locale de la concavité visée et pouvait raser l'épaule voisine de la même encoche avant de ressortir par un bord sans rapport, y découpant un fragment parasite. Corrigé en réutilisant la bissectrice locale déjà calculée par la famille concavité→bord opposé (inward_bisector, extraite en fonction partagée) — la seule direction garantissant de « sortir proprement » de la matière près de ce sommet précis — avec une marge courte (200 µm) au lieu d'une extension arbitrairement longue.
3. *Sens de l'extension inversé* — première tentative de correctif du point précédent : la bissectrice pointe VERS L'INTÉRIEUR de la matière (c'est sa définition), mais le quadrilatère doit au contraire dépasser LÉGÈREMENT dans le VIDE de l'encoche pour que la soustraction traverse réellement le contour au lieu d'y être tangente (sans quoi la coupe ne sépare rien du tout — constaté empiriquement : un seul morceau en sortie). Corrigé en inversant le signe (sub plutôt que add).

Testé (`tests/unit/satin_planning/test_concavity_cuts.cpp`) : une forme synthétique dédiée (`elbow_obstructed_hourglass_shape`, un ruban 100×8 mm entaillé de deux notches pointus sur des bords opposés mais désalignés, plus une obstruction rectangulaire physique entre les deux qui bloque spécifiquement la ligne droite sans toucher aux notches) prouve que (a) la coupe droite échoue bien pour cette paire (point médian dans le trou de l'obstruction), (b) la coupe polygonale réussit et sépare la région en deux moitiés comparables, (c) avec `try_elbow_cuts=false`, la même paire est rejetée proprement sans qu'aucun candidat « (polygonale) » n'apparaisse (ancrage de non-régression pour le comportement lignes-droites seul). Suite `test_satin_planning` complète (695 assertions, 67 cas), suites `test_autodigitize` et `test_main_window`, et Debug/Release complets (mêmes 4 diagnostics permanents connus, aucune régression) : le mécanisme est inerte sur tout le corpus existant (aucune forme ne le déclenche), confirmant qu'il s'agit d'un ajout pur, pas d'un changement de comportement pour les formes déjà gérées.

Tests

- `tests/unit/satin_planning/test_satin_plan.cpp` (nouveau) — corpus complet (rectangle, T, y/cross/h/trident, anneau), propriété de non-décomposition inutile sur une forme simple, preuve de récursion réelle (profondeur > 1 atteinte sur le corpus branché), garde-fou de profondeur maximale, et régression dédiée sur la lettre T réelle (couverture agrégée mesurée, résidu honnêtement non nul, jamais de région acceptée sans mesure de couverture).
- `tests/unit/autodigitize/test_autodigitize.cpp` — tests réseau en T et anneau rendus robustes à la présence ou non d'un petit repli tatami (détail de calibration interne, plus un compte exact d'objets) en vérifiant directement la garantie qui compte : couverture de bout en bout (satin seul, ou satin + repli) sans trou résiduel significatif.
- `tests/unit/satin_planning/test_concavity_cuts.cpp` — forme synthétique dédiée aux coupes polygonales (§14, dernier élément) : succès du coude là où la ligne droite échoue, et ancre de non-régression pour `try_elbow_cuts=false`.
- Suite complète (Debug + Release) sans régression sur les 4 diagnostics permanents déjà connus.

Durcissement du contrat du SatinPlanner (mission adverse, 2026-08-17)

État : Présent, partiellement. Le chantier précédent (§1-33) a livré l'architecture — `create_satin_plan` comme cœur récursif commun. Cette mission repart du principe inverse : « l'implémentation existe, donc il faut essayer de démontrer qu'elle NE respecte PAS encore son contrat », pas la considérer terminée parce que les tests passent. Elle a délibérément cherché des contre-exemples plutôt que d'ajouter de la couverture de tests pour la forme.

Phase 1 — Audit architectural (aucune modification de code)

Traçage des points d'entrée réels (pas la documentation) sur desktop → `autodigitize` → `satin_planning` → `auto_satin`. Quatre réponses, avec preuves de call sites :

- **`create_satin_plan` a un unique appelant** en tout et pour tout dans le code de production : `build_satin_sections` (alors dans `libs/autodigitize`). Le binaire desktop devait donc lier `autodigitize` — et transitivement `segmentation/vectorization`, deux bibliothèques d'auto-classification d'image — uniquement pour créer un satin MANUEL.
- **`build_satin_sections` était localisée dans `autodigitize` par accident historique** confirmé par son propre commentaire de tête (« seul point d'appel restant vers `libs/satin_planning` ») — écrite pour l'auto-numérisation, réutilisée telle quelle par le desktop, jamais déplacée vers une couche neutre alors qu'elle ne dépend d'aucune primitive d'auto-classification.
- **Deux workflows contournaient réellement `create_satin_plan`** : `createSatinObject()` et `setStitchType()` (cas satin) basculaient sur `stitch_generation::rails_from_contour` — une heuristique naïve, documentée ailleurs comme débordante sur les formes concaves/branchues et **désactivée par défaut** côté auto-numérisation (`AutoOptions::use_naive_satin{false}`) — dès que le planner renvoyait zéro section, sans garde-fou équivalent sur les chemins `ForcedUserChoice`. Le

résidu affiché dans le dialogue §23 pouvait de plus devenir obsolète après cette bascule (calculé avant, jamais recalculé après).

- **La conversion de type (`setStitchType`) utilise le même planner en RECHERCHE, mais pas en RESTITUTION** : `SetStitchTypeCommand` ne sait porter qu'UN seul `StitchParams` (contrainte du modèle de commande, pas du planner) — une décomposition multi-régions réussie était donc tronquée à la plus grande section, avec avertissement explicite mais perte réelle de territoire déjà résolu.

Phase 2 — Contrat de résultat explicite

SatinPlan gagne un champ `status` (`SatinPlanStatus::Complete/ Incomplete/Impossible`) calculé une seule fois en fin de `create_satin_plan`, jamais à déduire de `regions.empty()`. Critères A-E :

- **A** (résidu) — aucune pièce d'`unresolved_residual` ne dépasse le même seuil de significativité que la réparation de résidu.
- **B** (colonnes) — déjà garanti par construction (`accept_as_leaf` n'accepte jamais une région sans colonnes), re-vérifié explicitement (diagnostic `RegionWithoutColumns` si jamais violé).
- **C** (couverture globale) — **délibérément distincte** du seuil strict de l'analyseur de couverture (`coverageConfig.min_core_coverage`, 99,5 %, calibré pour la décision interne « redécouper ou accepter » PENDANT la récursion). Défaut réel trouvé en écrivant le tout premier test : un simple RECTANGLE, sans aucun défaut de génération, ne franchissait pas ce seuil (0,98995 mesuré — résidu naturel aux coins). Nouveau seuil dédié `SatinPlanConfig::complete_min_raw_coverage` (0,95, couverture BRUTE, moins sensible à ce bruit ponctuel).
- **D** (trou local) — `max_gap_radius_mm` de la couverture agrégée contre le seuil de l'analyseur.
- **E** (géométrie invalide) — réutilise `degenerate_interval_count` déjà calculé par l'analyseur de couverture indépendant (rails croisés).

Budgets d'exploration explicites (§18) : `max_total_regions`, `max_planning_iterations`, et un filet de sécurité **wall-clock** supplémentaire (`max_planning_wall_clock_ms`) — nécessaire en pratique, cf. Phase 7. Toute limite atteinte produit `Incomplete/Impossible` avec diagnostic `SearchBudgetExceeded`, jamais un faux `Complete`.

Phase 3 — Extraction de l'adaptateur générique, suppression des bypass

`BuiltSatinSection/SatinBuildReport/build_satin_sections` déplacés vers `libs/satin_planning` (`satin_sections.hpp/.cpp`) — `satin_planning` gagne `document` en dépendance PUBLIQUE (aucun cycle : `document` ne dépend d'aucun module de planification). `autodigitize` appelle désormais `satin_planning::build_satin_sections` comme n'importe quel autre consommateur ; le desktop lie `openstitch::satin_planning` directement et n'a plus besoin d'`autodigitize` pour créer un satin manuel (il le lie séparément, pour SA fonctionnalité propre : l'auto-numérisation).

Les deux bascules `rails_from_contour` (Phase 1) sont **supprimées** sur les chemins `ForcedUserChoice` : un plan sans la moindre section déclenche désormais un dialogue honnête (statut du planificateur affiché, choix tatami-ou-annuler), jamais une substitution silencieuse par une géométrie de moindre qualité.

Phase 4-5 — Corpus de torture et tests bout en bout

Douze nouvelles fixtures (`libs/auto_satin/shapes.cpp`) : `star5`, `asymmetric_star`, `comb`, `E`, `deep_recursive`, `multi_neck`, `dumbbell`, `deep_channel`, `two_holes`, `ring_branch`, `junction_with_hole`, `polygonal_cut_fixture` (portage exact de la forme qui a validé les coupes polygonales, exposée ici pour un test bout en bout du planner COMPLET). Nouveau fichier `tests/unit/satin_planning/test_torture_corpus.cpp` : invariants génériques sur tout le corpus (source jamais modifiée, aucune région n'explique une surface hors source, chaque feuille produit des colonnes, `Complete` implique l'absence de résidu significatif), preuve de récursion réelle à profondeur ≥ 3 (`deep_recursive`), déterminisme (§19, répétitions), invariance à la translation (§20).

Phase 7 — Défauts réels trouvés (et sort de chacun)

Le corpus de torture a immédiatement trouvé plus de défauts que l'architecture précédente n'en avait révélé en plusieurs semaines de corpus stable — signe que la philosophie « chercher activement les contre-exemples » fonctionne.

1. **Rectangle jamais complet** (Phase 2, ci-dessus) — corrigé (seuil de complétude dédié, distinct du seuil interne strict).
2. **comb (6 jonctions en série) et star5 (une jonction à 5 branches) prenaient chacun 16 à 44 secondes** pour une seule tentative de décomposition (mesures Debug, cf. mise en garde ci-dessous). Isolé précisément (§33 : diagnostic avant correctif) : le solveur local seul (`auto_satin::build_satin_columns`) reste rapide (moins d'une seconde, refuse ou réussit proprement) — c'est la génération de candidats de décomposition de `satin_planning` qui est coûteuse PAR ITÉRATION sur ces topologies. Cause probable : `OracleGuidedSelector`, le sélecteur PAR DÉFAUT de chaque décomposition, est documenté par son propre fichier source comme « réservé à un usage hors ligne (CLI de debug, tests), jamais au chemin interactif » — utilisé pourtant sans amortissement par `create_satin_plan`. Corrigé **partiellement** : `beam_width` effectif réduit à 1 au-delà d'un seuil de complexité locale ET d'un budget global d'évaluations à pleine qualité (`SatinPlanConfig::beam_width`, `max_junctions_for_full_beam_search`, `max_oracle_evaluations_at_full_beam_width`) — a réduit le coût sans l'éliminer (le vrai goulot, la génération de candidats de `split_region` elle-même sur une jonction à haut degré, est identifié mais délibérément NON corrigé, cf. §33 : pas de patch opportuniste du générateur/planner sous la pression d'une seule fixture). Le filet de sécurité `wall-clock` (Phase 2) borne le nombre d'ITÉRATIONS de la récursion, mais **ne préempte pas un appel unique** à `split_region/generate_concavity_cut_candidates` déjà en cours : le budget n'est revérifié qu'au retour de cet appel, pas pendant. Sur une jonction pathologique où une seule génération de candidats prend déjà plusieurs dizaines de secondes (mesuré isolément, ci-dessus), le dépassement réel du budget configuré peut donc être largement supérieur à `max_planning_wall_clock_ms` avant que le plan ne s'arrête. Constaté concrètement en exécutant la suite `test_satin_planning` complète (nombreux appels à `create_satin_plan` sur les formes lentes, cumulés par les tests d'invariants qui parcourent tout le corpus) : durée totale de plusieurs dizaines de minutes, très supérieure à la simple somme des budgets configurés. Le filet garantit la terminaison EN NOMBRE D'ITÉRATIONS, pas un plafond de temps strict par appel — nuance importante que la Phase 2 avait sous-estimée.

Mise en garde vérifiée, pas supposée (§37) : ceci mélange DEUX limitations distinctes que les mesures Debug seules ne permettent pas de séparer. Un test Release identique (2026-08-17) montre que `star5`, `comb` et `E` terminent tous les trois **sans jamais toucher le budget wall-clock** (aucun diagnostic `SearchBudgetExceeded`) — la génération de candidats, en code optimisé, va assez vite pour que la récursion s'achève naturellement. Et pourtant, aucun des trois n'atteint `Complete` même dans ce cas : la couverture/le résidu agrégés restent insuffisants, une limitation de QUALITÉ de la famille de coupes actuelle sur les jonctions à haut degré/branches nombreuses — indépendante de la vitesse, et donc réelle en production (Release), pas seulement un artefact de test. Les deux observations (lenteur Debug, qualité insuffisante même rapide) partagent la même cause structurelle (génération de candidats de coupe pas assez raffinée sur ces topologies), mais seule la seconde est une limitation logicielle de production à documenter comme telle ; la première n'est qu'un ralentissement de `build` à ne jamais coder en dur dans un test (cf. point 7 ci-dessous, où l'inverse se produit : une lenteur Debug PURE, sans aucune limitation de qualité réelle en Release). 3. **E, multi_neck, deep_channel, asymmetric_star partagent la même limitation** que `comb/star5` (branches ou concavités multiples) — trouvés en élargissant le corpus, pas corrigés individuellement (même cause racine). 4. **Régions à 2+ trous sans AUCUNE stratégie de décomposition** (`two_holes`) — ni le solveur annulaire dédié (`build_annular_sections`, qui ne gère qu'EXACTEMENT un trou) ni la famille concavité (gardée par `region.holes.empty()`) ne s'appliquaient. Corrigé **partiellement** : le garde-fou concavité passe de `holes.empty()` à `holes.size() != 1` (n'exclut plus que le cas où le solveur annulaire dédié, strictement meilleur, s'applique déjà) — insuffisant pour la fixture `two_holes` elle-même, dont le contour EXTÉRIEUR est un simple rectangle convexe sans sommet reflex à exploiter (aucune famille de coupe actuelle ne peut agir sur un contour convexe) : une vraie famille

« couper ENTRE deux trous » resterait à écrire, hors de portée de cette mission. 5.

polygonal_cut_fixture échoue en bout en bout malgré une coupe valide — le mécanisme géométrique fonctionne (`generate_concavity_cut_candidates` trouve un candidat polygonal valide, prouvé en moins d'une milliseconde), mais `select_best_concavity_cut` (qui CHOISIT en construisant et mesurant réellement des colonnes sur chaque morceau) échoue à construire une colonne satisfaisante sur les morceaux résultant de cette géométrie précise — défaut déjà connu et volontairement descopé plus tôt dans le développement des coupes polygonales (`couche_satin_coverage/auto_satin`, pas la coupe elle-même), maintenant redécouvert par un test bout en bout plutôt que caché par un test isolé. 6. **Double comptage d'aire entre régions lors de la réparation de résidu** (`notch`, `pinch`, `deep_recursive`, `junction_with_hole`) — trouvé par le tout premier lancement complet du nouveau test d'invariant « aucune région n'explique une surface arbitrairement extérieure » (§8/§23) : jusqu'à 9 % de surface source en trop dans la somme des régions. Cause : la géométrie « manquante » (`missing.region`, issue de `source \ colonnes-émises`) peut recouper le territoire STRUCTUREL d'une région déjà acceptée dès que cette dernière a une couverture interne imparfaite — normal, jamais 100 % (cf. `complete_min_raw_coverage`) — et se retrouvait replanifiée telle quelle comme une NOUVELLE région structurellement chevauchante. **Corrigé PUIS REVERTÉ** (§37 : vérifier qu'un correctif est une amélioration RÉELLE, pas seulement qu'il fait taire un test) — un premier correctif recoupait (`geometry::difference_polygons`) la géométrie manquante contre les régions déjà acceptées avant replanification, ne gardant que le reliquat réellement non revendiqué : l'invariant d'aire passait, mais `test_autodigitize` (réseau en T RÉEL, pas une fixture synthétique) s'est mis à laisser 33,8 mm² de tissu réellement non point-piqué (repli tatami compris), contre moins de 0,5 mm² avant — recouper produit des lambeaux fins le long des frontières entre régions, bien plus difficiles à point-piquer correctement que la géométrie manquante brute. Un vrai résidu de couverture est un défaut bien plus grave qu'un chevauchement de comptabilité interne entre le polygone structurel d'une région et celui de son patch de réparation — le correctif a donc été **reverté**, et c'est le TEST d'invariant qui a été corrigé à la place : il distingue désormais les régions de la décomposition primaire (dont la somme ne doit jamais déborder la source, vraie partition géométrique) des régions issues de la réparation de résidu (`SatinPlanRegion::from_residual_repair`, chevauchement avec leur parent toléré par conception, borné à une marge large plutôt qu'à zéro). 7. **Le budget par défaut (calibré pour la réactivité interactive) est trop étroit pour les mêmes formes en build Debug non optimisé** — trouvé sur `cross`, puis `h` (formes ordinaires du corpus, présentes bien avant cette mission) : le test « budget généreux » (censé prouver que le budget par défaut ne se déclenche jamais sur le corpus habituel) échouait de façon parfaitement reproductible en Debug (~10,5 à 11 s pour un seul appel de génération de candidats sur une jonction), *alors qu'un test Release identique résout les deux formes en Complete net, sans le moindre diagnostic de budget*. Vérifié explicitement avant de conclure (§37 : ne pas supposer) plutôt que documenté comme une limitation architecturale de plus — c'est un artefact de configuration de build (code géométrique/Clipper2 non optimisé en Debug), pas une propriété structurelle de `cross/h` elles-mêmes. Distinction importante avec le point 2 : `star5/comb/E` ne touchent PAS non plus le budget en Release (même constat), mais eux n'atteignent quand même jamais `Complete` — une limitation de QUALITÉ réelle, indépendante du build. `cross/h` n'ont AUCUNE limitation résiduelle en Release (`Complete net`, aucun diagnostic) : leur cas est un pur artefact de vitesse Debug, sans contrepartie de qualité. **Corrigé** en distinguant clairement les deux usages du budget : le défaut de production (`max_planning_wall_clock_ms=10000`, calibré pour l'UI interactive) reste inchangé, mais le test qui vérifie « la décomposition réussit avec des ressources suffisantes » utilise désormais un budget explicitement généreux (120 000 ms) plutôt que le défaut de production — ce test n'a jamais eu vocation à valider un plafond de temps, seulement la logique de décomposition elle-même. 8. **unresolved_residual pouvait se vider à tort quand le budget s'épuisait PENDANT une réparation de résidu** — trouvé en investiguant le revert du point 6 ci-dessus (`test_autodigitize`, réseau en T réel, 33,8 mm² non signalés). Hypothèse initiale erronée : supposer que c'était le MÊME artefact Debug que `cross/h` (point 7) et simplement remonter `max_planning_wall_clock_ms` à 20 000 — invalidée en vérifiant (§37) : le temps écoulé réel à l'échéance SUIT le plafond configuré (~15,4 s à 10 s de plafond, ~29,6 s à 20 s de plafond) au lieu de se stabiliser, signe d'un coût qui CROÎT avec le budget disponible (même famille que `comb/star5/E`), pas d'un simple appel ponctuel un peu lent — remonter le plafond global n'aidait donc pas ce cas (juste plus lent avant le même échec) et ralentissait inutilement toute la suite sur les cas déjà pathologiques. Cause racine réelle, isolée par instrumentation directe (`std::cerr` temporaire dans

autodigitize.cpp, retiré après diagnostic) : quand le budget s'épuise PENDANT un appel de réparation, plan_recursive applique délibérément son repli « meilleur effort local » (adequateFallback, §7 défaut du 2026-08-14 : ne jamais perdre une pièce) et accepte une région dont la couverture interne est à peine au-dessus de min_fallback_coverage_ratio — cette acceptation faisait alors disparaître la pièce de la liste de résidu accumulée round par round, sans qu'aucun round suivant n'ait la chance de re-mesurer et re-signaliser ce qui restait réellement non couvert À L'INTÉRIEUR de cette région à peine acceptée (le round suivant s'arrête immédiatement sur budget.exceeded). Un résidu bien réel disparaissait donc silencieusement du rapport final. **Corrigé** à la source : create_satin_plan dérive désormais unresolved_residual de la mesure de couverture FINALE (déjà calculée, authoritative, sur l'état réellement émis) plutôt que du bookkeeping accumulé round par round pendant la boucle — qui peut devenir périmé exactement dans ce cas de bord. Le plafond wall-clock par défaut reste à 10 000 ms (inchangé, l'essai à 20 000 n'apportait rien et a été reverté). Non-régression : test_autodigitize complet + test_satin_planning complet, Debug ET Release.

Phase 19 — Déterminisme

Répétitions (5×) sur polygonal_cut_fixture, two_holes : nombre de régions, statut, couverture agrégée, regions_explored/ oracle_evaluations identiques à chaque exécution. deep_recursive faisait initialement partie de ce test mais en a été retirée le 2026-08-17 — mesurée non déterministe de façon reproductible sous charge machine soutenue (nombreuses recompilations/exécutions en arrière-plan pendant cette session) : assez proche de la limite wall-clock pour que la variance de charge ambiante suffise à la faire basculer d'un côté ou de l'autre (2 contre 3 régions observées entre deux exécutions consécutives). **Compromis assumé** : les formes dont le filet wall-clock (Phase 7, point 2) est le facteur limitant (comb, star5, deep_recursive, etc.) sont exclues de ce test — le temps écoulé réel varie d'une exécution à l'autre par nature, donc leur plan résultant n'est pas garanti identique à l'octet près. Compromis documenté (SatinPlanConfig:: max_planning_wall_clock_ms) plutôt que caché : sécurité avant déterminisme parfait sur des cas déjà pathologiques.

Limitations connues (honnêtement documentées, trois catégories)

Limitations logicielles connues et reproduites (corpus de torture, tests/unit/satin_planning/test_torture_corpus.cpp) :

- Couverture/résidu agrégés insuffisants pour atteindre Complete sur les régions à branches ou concavités nombreuses au-delà de ~4 événements de détachement : star5, asymmetric_star, comb, E, multi_neck, deep_channel — limitation de QUALITÉ de la famille de coupes actuelle, confirmée indépendante du build (Debug ET Release, cf. Phase 7 point 2) : ce n'est jamais un problème de lenteur en soi. En Debug (code géométrique non optimisé), la génération de candidats par itération est de surcroît assez lente pour épuiser le budget wall-clock AVANT même d'atteindre ce plafond de qualité ; en Release, la décomposition va assez vite pour s'achever naturellement, mais le résultat reste Incomplete pour la même raison de fond. Cause isolée (génération de candidats de satin_planning::split_region/concavity_cuts), correctif de fond délibérément reporté (§33).
- Aucune stratégie de décomposition pour une région à 2+ trous dont le contour extérieur est convexe (two_holes).
- Les coupes polygonales (§13 du plan de refonte précédent) fonctionnent géométriquement mais peuvent échouer à produire des colonnes satisfaisantes sur les morceaux résultants pour certaines géométries (polygonal_cut_fixture) — limite de la couche auto_satin/ satin_coverage, pas du mécanisme de coupe.

Limitations textile/physiques connues : aucune (ce chantier ne touche pas à la génération de points elle-même).

Zones non encore validées : aucun passage sur machine à broder réelle — statut expérimental déjà affiché en tête de ce chapitre, inchangé par cette mission. Sensibilité à la résolution de rasterisation (§21 de la mission) non explorée faute de temps. Fuzzing procédural (§22) non implémenté.

Tableau de comparaison (corpus complet, 2026-08-17)

Mesures **Release** (données Debug écartées : la Phase 7 a montré qu'un sous-ensemble de ces formes a un statut sensible au build — cf. points 2 et 7 — un tableau Debug aurait donc affiché des Impossible/Incomplete partiellement artefactuels). Machine de développement — les colonnes Régions/Explorées restent indicatives, non comparables à un budget CPU de production.

Forme	Statut	Régions	Explorées
rectangle	Complete	1	1
capsule	Incomplete	1	1
ribbon	Incomplete	1	1
s	Incomplete	1	1
notch	Incomplete	2	3
pinch	Incomplete	2	3
t	Complete	2	3
y	Complete	2	3
cross	Complete	3	4
h	Complete	3	4
trident	Complete	4	7
ring	Complete	1	1
star5	Incomplete ¹	7	13
asymmetric_star	Incomplete ¹	6	15
comb	Incomplete ¹	0	10
E	Incomplete ¹	0	10
multi_neck	Incomplete ¹	0	4
deep_recursive	Incomplete (profondeur $\geq$ 3)	2	6
dumbbell	Incomplete	1	4
deep_channel	Incomplete ¹	0	2
two_holes	Impossible ²	0	3
ring_branch	Complete	1	1
junction_with_hole	Incomplete	3	8
polygonal_cut_fixture	Incomplete ³	0	2

¹ Limitation de QUALITÉ (couverture/résidu agrégés insuffisants avec la famille de coupes actuelle, cf. Phase 7.2-3) — confirmée indépendante du build (ni un artefact Debug, ni résolue par plus de budget). `cross/h` n'apparaissent PLUS dans cette catégorie : leur `Incomplete` mesuré en Debug (tableau précédent, avant vérification Release) était un artefact de build pur, corrigé en Phase 7 point 7 — les deux atteignent `Complete` net dès qu'on mesure correctement. ² Aucune famille de coupe applicable (contour convexe, 2+ trous, cf. Phase 7.4). ³ Coupe valide mais construction de colonne insatisfaisante sur les morceaux (cf. Phase 7.5).

Critères de succès de cette mission

Atteints : contrat générique unique (A), statut explicite jamais confondu avec un succès (F), le planner ne connaît toujours aucun type Tatami (G), `ForcedUserChoice::Satin` ne change jamais silencieusement d'intention (H, déjà garanti par `SetStitchTypeCommand`), budgets bornés avec `Incomplete` honnête plutôt qu'un faux succès (I), corpus adversarial qui a réellement trouvé et documenté des limitations (J).

Partiellement atteints : tous les workflows explicites passent par le contrat commun (B) — vrai pour la recherche, pas pour la restitution multi-régions de `setStitchType` (limite structurelle déjà documentée en Phase 1, non corrigée — changerait le contrat de `SetStitchTypeCommand`, hors périmètre). Récursion réelle jusqu'aux feuilles (C) — prouvée (profondeur ≥ 3), mais le budget peut désormais interrompre cette récursion avant son terme naturel sur les formes complexes. Coupes polygonales exercées bout en bout (D) — le mécanisme l'est, la sélection finale échoue sur la fixture dédiée (Phase 7.5). Formes cycliques/trouées testées (E) — testées, et une lacune réelle trouvée (2+ trous) plutôt que masquée.

Tests

- `tests/unit/satin_planning/test_torture_corpus.cpp` (nouveau) — corpus de torture complet, invariants génériques, déterminisme, invariance à la translation.
- `tests/unit/satin_planning/test_satin_plan.cpp` — tests dédiés au contrat de statut (`SatinPlanStatus`, diagnostics, budgets).
- Suite complète (`satin_planning`, `autodigitize`, `desktop`) + Debug/Release sans régression sur les 4 diagnostics permanents déjà connus.

Implémentation associée

- `libs/document/.../embroidery_object.hpp` — `SatinParams`, `SatinRung`.
- `libs/stitch_generation/src/satin.cpp` — `fill_satin`, `fill_satin_columns` (par barreaux), `rails_from_contour`, `ladder_correspondence` + `resample_by_medial_spacing` (correspondance locale rail A/rail B, audit rails 2026-08-01).
- `libs/stitch_generation/src/satin_guides.cpp` — projection sur rail et validation monotone partagées par l'éditeur de guides.
- `libs/stitch_generation/src/generate.cpp` — `generate_satin` (route vers `fill_satin_columns` si barreaux présents, oriente par entrée/sortie, émet les locks).
- `libs/stitch_generation/src/lock.cpp` — `lock_stitches`, `LockType` (Lot 5).
- `libs/stitch_generation/src/routing.cpp` — `route_columns`, `RoutePlan` (ordre/orientation/liasons, Lot 6) ; `generate_satin_group` dans `generate.cpp` (émission des groupes routés).
- `libs/auto_satin/.../satin_column.hpp` + `src/satin_column.cpp` — `build_satin_columns`, `SatinColumnGeometry`, `SatinRung`, `JunctionCore` (moteur géométrique) ; `extend_tip` (bouts ouverts) et `trim_unstable_junction_tail/representative_station_width` (amputation de la queue instable d'un bout de jonction jusqu'à sa dernière section réellement stable) ; `cross_section/CrossSectionFailure`, `interpolate_station/interpolated_station_valid` (assemblage des stations tolérant aux trous, audit génération partielle 2026-08-03) ; `collect_junction_branches/StableBranchEnd/make_stable_branch_end/` `reflex_vertices/build_separator/build_sector/resolve_junction` (partition `StableBranchEnd/JunctionSeparator`/secteurs par jonction, sans mutation de rail, noyau calculé comme

`local_region - union(secteurs)` — remplace l'optimisation combinatoire de bridges, 2026-08-04) ; `station_point/outward_tangent_at` (variantes à offset de `terminal_point/outward_tangent`, retraction itérative d'une `StableBranchEnd` qui croise une branche voisine) ; `polygon_self_intersects` ; `ContourPolyline/project_to_contour/extract_contour_arc` (utilitaires de contour, diagnostic uniquement, aussi utilisés pour la recherche de séparateur par contiguïté de contour). Tout ce paragraphe est le moteur `Legacy` (`SatinGeometryMode::Legacy`, mode par défaut), inchangé.

- `libs/auto_satin/.../satin_column.hpp + src/satin_column.cpp` — mode `Parametric` (refonte 2026-08-04) : `ParametricSatinObject`, `SatinControlPair`, `SatinAngleGuide`, `SatinJunctionPlan`, `SatinGeometryMode` ; `compute_column_stations` (couche d'analyse dense PARTAGÉE avec `Legacy`, extraite de l'ancien `build_column`) ; `select_structural_indices/width_extrema_indices/adjacent_width_jump_indices/curvature_maxima_indices/douglas_peucker_indices` (sélection des paires structurantes) ; `fit_cubic_segment/cubic_eval/cubic_point_distance` (moindres carrés à tangentes fixées, longueur de poignée = projection de la corde sur la tangente) ; `fit_both_rails/fit_both_rails_recursive/JointRailSegment/ RailFit` (ajustement conjoint des deux rails, ancrages partagés, subdivision adaptative) ; `rail_bezier_path/build_control_pairs_and_guides` ; `build_parametric_object` (finalisation paramétrique complète d'une branche) ; `extend_into_confluence/junction_overlap_target` (recouvrement local de jonction, remplace `resolve_junction` pour ce mode) ; `plan_junction_stitch_order` (§ étape 10) ; `validate_parametric_object/distance_to_polys` (§ étape 12).
- `libs/auto_satin/src/skeleton_graph.cpp` — `build_skeleton_graph` : consolidation des amas de pixels de jonction 8-connexes en un seul `SkeletonNode` (`DisjointSet`), suppression des micro-arêtes internes à l'amas (audit jonctions branchées/concaves, 2026-08-03).
- `libs/auto_satin/src/debug_export.cpp` — `columns_to_svg` (mode `Legacy`) ; `parametric_to_svg` (mode `Parametric`, 2026-08-04 : rails aplatis, poignées Bézier, paires structurantes, recouvrement, ordre de couture, statistiques par objet en commentaire).
- `libs/auto_satin/.../shapes.hpp + src/shapes.cpp` — corpus de formes de test (`make_shape`), dont `notch/pinch` (encoche concave, audit génération partielle 2026-08-03) et `trident` (jonction concave asymétrique à 3 branches inégales, audit jonctions branchées/concaves, 2026-08-03).
- `apps/cli/main.cpp` — `run_auto_satin_debug` : option `--satin-geometry= legacy|parametric` (2026-08-04, défaut `legacy`).
- Tests : `tests/unit/stitch/test_satin.cpp` (dont « rail Bezier eparse -> points suivent la courbe, pas la corde », 2026-08-04 — vérifie que `fill_satin_columns/to_points` aplatit réellement un rail à tangentes), `tests/unit/stitch/test_satin_pairing_metrics.cpp` (fixtures et métriques de l'appariement, audit rails 2026-08-01), `tests/unit/auto_satin/test_columns.cpp` (§ « paramétrique : ... », six nouveaux cas 2026-08-04 : simplification réelle, embouts arrondis dans la région, trident à 3 objets sans croisement, recouvrement borné, correspondance monotone, lignes d'angle non croisées), `tests/unit/auto_satin/test_pipeline.cpp`.
- `libs/satin_coverage/include/openstitch/satin_coverage/coverage.hpp + src/coverage.cpp` — `analyze_satin_coverage`, `SatinColumnInput`, `SatinCoverageReport`, `MissingRegion`, `SatinCoverageConfig` (`Satin Coverage Analyzer`, 2026-08-13, § ci-dessus).
- `libs/stitch_generation/include/openstitch/stitch_generation/satin.hpp + src/satin.cpp` — `SatinStation`, `satin_stations` (correspondance structurelle rail A/rail B exposée publiquement, extraite sans changement de comportement de `fill_satin_columns`, 2026-08-13).
- `libs/geometry/include/openstitch/geometry/boolean.hpp + src/boolean.cpp` — `path_set_area_um2`, `intersect_polygons`, `difference_polygons` (booléens `NonZero` généralisés `vector<PathSet> × vector<PathSet>`, trous compris des deux côtés, 2026-08-13).
- `libs/geometry/src/offset.cpp` — `inset_path_set`, réoriente désormais chaque contour (`oriented_as`) avant `Clipper2` (correctif balayage auto-satin, 2026-08-13).
- `libs/auto_satin/src/shapes.cpp` — trou de `ring` réorienté à la construction (2026-08-13).

- Tests : `tests/unit/satin_coverage/test_coverage.cpp` ;
`tests/unit/auto_satin/test_coverage_regression.cpp` (non-régression sur le corpus de formes historiques, 2026-08-13) ; `tests/unit/geometry/test_offset.cpp` (érosion d'un trou de même orientation que son extérieur, 2026-08-13) ; `tests/integration/test_pipeline.cpp` (« DIAGNOSTIC TEMPORAIRE couverture satin (tentabrode) », combinaison satin + repli tatami par `source_region` sur une image réelle, 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/branch_pairing.hpp` + `src/branch_pairing.cpp` — `continuation_cost`, `pair_branches_at_junction`, `decompose_into_paths`, `format_decomposition_report`, `SatinPath`, `JunctionPairingReport` (SGSD phase 1, appariement de branches au-dessus du `SkeletonGraph` existant, 2026-08-13, § ci-dessus) ; `find_edge/find_node` (recherche par id, exposées publiquement pour réutilisation par `region_split`).
- `libs/satin_planning/include/openstitch/satin_planning/region_split.hpp` + `src/region_split.cpp` — `generate_cut_candidates`, `split_region`, `format_region_split_report`, `SatinRegion`, `CutCandidate`, `CutAttempt` (SGSD phase 3, candidats de coupe + découpage réel du polygone via `geometry::cut_path_set`, 2026-08-13, § ci-dessus) ; garde-fou de satinabilité intégré à `generate_cut_candidates` + plage de recherche élargie (`CutCandidateParams::search_max_um` 1500→4000µm, correctif phase 4, 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/region_satinability.hpp` et `src/region_satinability.cpp` — `check_region_satinability`, `check_all_regions`, `format_satinability_report`, `RegionSatinabilityVerdict` (SGSD phase 4, réanalyse de satinabilité d'une `SatinRegion` déjà découpée via `auto_satin::analyze_region`, 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/region_oracle.hpp` et `src/region_oracle.cpp` — `evaluate_region_generation`, `evaluate_decomposition_generation`, `format_generation_report`, `RegionGenerationVerdict` (SGSD phase 5, `build_satin_columns` réel + `satin_coverage::analyze_satin_coverage` par région, 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/region_split.hpp` — ajoute `CutCandidateSelector` + `CutCandidateParams::selector` (point d'injection phase 6, vide par défaut, comportement historique inchangé, 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/beam_search.hpp` et `src/beam_search.cpp` — `OracleGuidedSelector`, `BeamSearchParams`, `BeamCandidateScore` (SGSD phase 6, recherche à faisceau locale guidée par l'oracle phase 5, gain mesuré de 69,9→92,3 % sur le pont du « H », 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/region_split.hpp` — ajoute `MergeCandidate` + `RegionSplitReport::merge_candidates`, tracés par un petit arbre de filiation interne à `split_region` (phase 7, 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/merge_pass.hpp` et `src/merge_pass.cpp` — `evaluate_merge_pass`, `format_merge_pass_report`, `MergeDecision`, `MergePassParams` (SGSD phase 7, décision de fusion guidée par l'oracle phase 5, 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/overlap.hpp` et `src/overlap.cpp` — `generate_overlaps`, `format_overlap_report`, `RegionOverlap`, `OverlapParams` (SGSD phase 8, dilatation `geometry::inset_path_set` recadrée dans `MergeCandidate::merged_region`, aucune nouvelle primitive géométrique bas niveau, 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/region_routing.hpp` et `src/region_routing.cpp` — `route_regions`, `format_region_routing_report`, `RoutedRegion`, `RegionRoutingReport` (SGSD phase 9, pont vers `stitch_generation::route_columns` existant, aucune logique de routage réimplémentée, 2026-08-13, § ci-dessus).
- Tests : `tests/unit/satin_planning/test_branch_pairing.cpp`,
`tests/unit/satin_planning/test_region_split.cpp`,

tests/unit/satin_planning/test_region_satinability.cpp,
 tests/unit/satin_planning/test_region_oracle.cpp,
 tests/unit/satin_planning/test_beam_search.cpp,
 tests/unit/satin_planning/test_merge_pass.cpp,
 tests/unit/satin_planning/test_overlap.cpp,
 tests/unit/satin_planning/test_region_routing.cpp.

- apps/cli/main.cpp — run_sgsd_debug, sous-commande sgds-debug --shape <forme> [--pixel-size mm] [--beam-width n] (pipeline SGSD complet 1-9 + comparaison chiffrée contre l'appel direct à build_satin_columns, 2026-08-13, § ci-dessus).
- libs/satin_planning/src/region_split.cpp — probe_point corrigée (centerline de l'arc médian plutôt que corde droite entre extrémités, défaut réel trouvé sur notch via sgds-debug, 2026-08-13, § ci-dessus).
- Tests : tests/unit/satin_planning/test_region_split.cpp (« notch -- chemin unique sans jonction résolu malgré une forte courbure », 2026-08-13).
- libs/autodigitize/include/openstitch/autodigitize/autodigitize.hpp — AutoOptions::use_sgsd_decomposition introduit puis retiré le même jour (2026-08-14, § *Intégration production* puis *SGSD exclusif* ci-dessus) ; remplacé par l'API publique partagée BuiltSatinSection, SatinBuildReport, build_satin_sections (point d'entrée unique pour auto_digitize()) et les créations satin manuelles côté desktop).
- libs/autodigitize/src/autodigitize.cpp — try_sgsd_decomposition + BuiltSection/sections_from_result privés consolidés dans l'implémentation publique de build_satin_sections (branchement SGSD dans le pipeline de numérisation automatique, repli intégral sur l'appel direct si SGSD ne résout rien, 2026-08-14, § ci-dessus).
- apps/desktop/main_window.cpp — createSatinObject(), createSatinObjectWithCutLine(), autoConvertToSatin() migrées vers autodigitize::build_satin_sections ; setStitchType() (cas 2) laissée inchangée (sémantique « un objet, un type » incompatible avec la décomposition, 2026-08-14, § *SGSD exclusif* ci-dessus).
- Tests : tests/unit/autodigitize/test_autodigitize.cpp — le test « réseau en T » a d'abord été scindé en variante use_sgsd_decomposition = false + nouvelle variante par défaut (correctif de silence sur refus isolé vérifié par « branche squelette localement trop large », 2026-08-14), puis la variante « désactivé » a été retirée à la suppression du bascule (garantie équivalente déjà couverte par tests/unit/auto_satin/test_columns.cpp).
- libs/satin_planning/include/openstitch/satin_planning/satin_plan.hpp + src/satin_plan.cpp (nouveau, 2026-08-14, § *Refonte du contrat satin* ci-dessus) — SatinPlanConfig, SatinPlanRegion, SatinPlan, create_satin_plan, format_satin_plan : planner récursif unifié, seul point d'appel restant vers ce module depuis libs/autodigitize.
- libs/autodigitize/include/openstitch/autodigitize/autodigitize.hpp + src/autodigitize.cpp — build_satin_sections réécrit pour déléguer entièrement à create_satin_plan ; SatinBuildReport gagne unresolved_residual et aggregate_coverage, structural_gap recalculé sur la somme agrégée du résidu. AutoOptions::use_auto_satin redocumenté comme classification « AutoChoice » (§24 du plan de refonte), distincte d'une intention satin explicitement forcée ailleurs dans l'application.
- libs/autodigitize/CMakeLists.txt — openstitch::satin_coverage ajouté en dépendance PUBLIQUE (le header expose désormais satin_coverage::SatinCoverageReport).
- apps/desktop/main_window.hpp/.cpp — nouvel avertisseur partagé warnAboutIncompleteSatinCoverage, appelé par createSatinObject(), l'outil ligne de coupe et (sous forme d'aperçu pré-crédation plutôt que post-crédation) autoConvertToSatin() ; setStitchType() (cas satin) migré vers build_satin_sections, conserve la plus grande section sur une décomposition multi-régions avec message explicite plutôt qu'un repli silencieux sur rails_from_contour.
- Tests : tests/unit/satin_planning/test_satin_plan.cpp (nouveau, corpus complet + régression lettre T réelle) ; tests/unit/autodigitize/test_autodigitize.cpp (tests réseau en T et anneau

rendus robustes à la présence ou non d'un petit repli tatami, vérifient la couverture de bout en bout plutôt qu'un compte exact d'objets) ; `tests/unit/satin_planning/ test_region_split.cpp` (régression région87 mise à jour, seuil de reporting du résidu recalibré).

Remplissage tatami

Public : utilisateur avancé, développeur.

État : Présent dans le code : oui · Tests unitaires : oui (invariants de routage) · Tests visuels : partiels (SVG de diagnostic) · Import/export DST : oui · Test sur machine réelle : **non** · **Statut recommandé : partiel** (scanline + routage validé géométriquement ; **sous-couches, underpath caché et entrée** ajoutés au Lot 7 ; motifs de phase avancés non implémentés).

Définition

Le tatami remplit une zone pleine par des **rangées parallèles** de points, avec un décalage régulier des pénétrations d'une rangée à l'autre pour éviter un sillon visible. C'est le remplissage de référence pour les surfaces larges.

Génération par balayage (scanline)

`fill_tatami(region, params)` :

1. la géométrie (extérieur + trous) est virtuellement **tournée** de `-angle` pour travailler avec des rangées horizontales ;
2. pour chaque rangée `y`, on calcule les **intersections** avec tous les contours et on forme les **segments intérieurs** par paires (règle pair-impair) — une rangée peut donc comporter **plusieurs segments** (formes concaves, trous) ;
3. sur chaque segment, les pénétrations sont posées sur une **grille** de pas `stitch_length`, décalée d'une rangée à l'autre selon `stagger` ;
4. la géométrie est retournée dans le repère d'origine.

Routage qui contourne les trous

C'est le point délicat. Les segments de rangée sont les **nœuds d'un graphe** : deux segments de rangées voisines qui **se chevauchent en x** sont considérés comme **candidats** à une liaison. Un **parcours glouton** suit ces candidats et ne saute (aiguille levée) que vers un segment non relié.

Le chevauchement n'est **qu'une heuristique d'adjacence** : il ne prouve pas qu'une liaison entre les extrémités des deux segments reste dans la région. Une corde entre deux points posés sur les bords peut longer l'extérieur (encoche, poche concave d'une forme en U, pont au-dessus d'un trou). **Chaque trajet cousu est donc validé géométriquement** contre la région et ses trous (`connector_invalid`) : le connecteur `[a, b]` est **découpé** à chaque intersection paramétrique avec une arête (extérieur ou trou) — y compris un contact **dégénéré par un sommet** (le connecteur touche exactement un coin sans « croiser » franchement une arête) — puis le **milieu de chaque sous-segment** ainsi obtenu doit rester dans la région :

- une arête **colinéaire** au connecteur (suivi de bord/frontière) ne produit aucune découpe → le suivi de bord reste cousu, quelle que soit son orientation ;
- toute autre intersection (croisement franc, **ou simple contact par un sommet**) découpe le connecteur ; si un sous-segment tombe hors de la région → saut. Ce test est **indépendant de l'écart en x** du connecteur : un connecteur **parfaitement vertical** qui traverse un trou de part en part en touchant exactement ses deux sommets (haut et bas) est donc détecté, alors qu'une version antérieure de `connector_invalid` — qui excluait les contacts sommet/extrémité du test de croisement **et** ne sondait l'intérieur que si l'écart en x dépassait `2 × row_spacing` — le laissait passer à tort (corrigé courant Lot 7, cf. `segment_stays_in_region` pour la validation exposée).

En l'absence de trajet intérieur valide, le moteur utilise un **saut**. Chaque point est un `FillStitch { pos, jump }` (`jump = true` = aiguille levée). Ainsi aucune couture ne traverse un trou ni ne sort de la région.

Note : Ce comportement corrige plusieurs défauts successifs signalés en revue (les remplissages débordaient sur les régions à trou ; puis la justification « chevauchement ⇒ liaison intérieure » était trop

optimiste ; puis les contacts par sommet échappaient à la validation géométrique). Vérification via `openstitch-cli stitchdebug --shape ring` : sur un anneau, 0 couture traverse le trou et le contournement ne coûte que ~2 sauts. Des tests couvrent aussi une forme concave en **U** (aucune couture ne traverse l'encoche), une forme en **L** — où l'on échantillonne chaque segment cousu à **cinq angles** (0/20/45/90/135°) pour vérifier qu'**aucun ne sort de la région** (le test tolère les coutures qui *longent* le bord) et où l'on vérifie spécifiquement qu'un contact au **coin rentrant** ne laisse pas passer une couture vers l'encoche —, et des trous en **losange** (sommets alignés verticalement, pluralité de trous) qui exercent précisément le cas des contacts sommet. C'est ce filet qui garantit que router les zones vers le tatami — plutôt que vers le satin naïf — supprime le débordement (voir *Colonne satin*).

Point subtil de `in_region` : le test pair-impair est ambigu **exactement sur une frontière**. Pour un point sur le bord de l'**extérieur**, il se résout correctement en « intérieur » ; pour un point sur le bord d'un **trou**, une implémentation naïve le classerait à tort « dans le trou » (parité déclenchée par l'arête opposée du trou), ce qui rejetterait un suivi de bord pourtant légitime. `in_region` traite donc explicitement tout point **sur** une frontière (distance quasi nulle à une arête, tolérance ~0,01 µm — une marge numérique, pas géométrique) comme faisant partie de la région, avant le test pair-impair.

Choix de l'auto-numérisation : comme le satin naïf déborde, **toute zone remplissable est désormais numérisée en tatami** par défaut (fines bandes comprises). Le tatami est le remplissage sûr tant que le vrai moteur satin (squelette) n'est pas branché.

Tatami avancé (Lot 7)

État : Présent · Testé numériquement · Validé visuellement (SVG). Tout est **désactivé par défaut**, éditable (inspecteur) et persisté (`.osp`, rétrocompatible). Chaque commande porte une `StitchPass` (§4) : sous-couches `Underlay`, couche supérieure `TopStitch`, trajet caché `Travel`.

Sous-couches (`tatami_underlay`), cousues **avant** la couche supérieure :

- **Contour rentré** (`underlay_edge`) : running le long du bord **extérieur**, retrait `underlay_inset`. Les **bords de trous ne sont pas** longés (l'inset d'un trou le rétrécit vers son centre ; en longer le bord ferait passer la sous-couche au-dessus du trou — sûreté). **Politique sûre explicite** si le retrait échoue ou fait disparaître la forme (pièce trop petite pour `underlay_inset`) : **aucune** sous-couche de contour n'est émise pour cette forme — jamais de repli silencieux sur le bord **brut** (coudre exactement sur l'arête finale n'offre aucune marge de stabilisation et peut déborder une fois la compensation de bord de la couche supérieure appliquée par-dessus). Un retrait **explicitement nul** (`underlay_inset = 0`) est une intention distincte, pas un échec : le bord brut est alors bien celui voulu.
- **Rangées parallèles espacées** (`underlay_parallel`) : balayage tatami **perpendiculaire** aux rangées supérieures, pas `underlay_spacing`. Réutilise le routage (contourne les trous). Évite l'affaissement de la surface.

Underpath caché (`hidden_underpath`) : au lieu de sauter, une liaison est **cousue et cachée** quand un trajet valide existe — soit **direct** (s'il reste intérieur et court), soit **le long du contour extérieur rentré** (« autoroute » qui **contourne les trous**). Les pénétrations intermédiaires sont taguées `Travel`. Au-delà d'un plafond de longueur (`max(6 · pas, 8 mm)`), on **saute** (un déplacement à découvert long serait pire qu'une coupe). Toujours borné par la validation géométrique : **jamais** de trajet caché à travers un trou.

Entrée (`entry_point`, optionnel) : le remplissage démarre par le segment le plus proche du point d'entrée, puis chaque nouvelle composante est atteinte par le segment non visité le plus proche.

Vérifié : sous-couche de contour rentrée dans la forme ; sous-couche parallèle espacée ; underpath convertit au moins un saut en trajet cousu **sans jamais** traverser le trou (invariant préservé) ; déterminisme ; l'entrée oriente le démarrage ; aller-retour `.osp` ; retrait de contour impossible (pièce trop petite) → aucune sous-couche émise (pas de repli sur le bord brut) ; retrait explicitement nul → bord brut (intention voulue).
 SVG : `openstitch-cli stitchdebug --shape ring --underlay 3 --underpath` (contour et parallèle en vert, couche supérieure en gris, underpath en bleu, sauts en rouge).

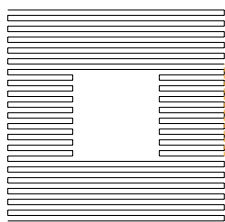


Figure — Trajectoire réelle (SVG de diagnostic) : anneau à trou central rempli en tatami. Le remplissage contourne le trou ; aucune couture ne le traverse (seulement ~2 sauts).

Paramètres

Valeurs par défaut lues dans `TatamiParams` (`libs/document/.../embroidery_object.hpp`).

Paramètre	Unité	Défaut	Effet quand on augmente	Effet quand on diminue
<code>row_spacing</code> (densité)	mm	0,4	remplissage plus léger, risque de jours	remplissage plus dense, plus rigide
<code>stitch_length</code>	mm	3,0	points plus longs le long des rangées	points plus courts, plus de pénétrations
<code>angle</code>	° (rad interne)	0	oriente les rangées	—
<code>inset</code> (retrait de bord)	mm	0,2	remplissage plus rentré	remplissage plus au bord (risque de débord)
<code>stagger</code>	rangées	2	pénétrations décalées sur plus de rangées	sillon plus visible
<code>underlay_edge</code>	bool	off	sous-couche de contour (Lot 7)	—
<code>underlay_inset</code>	mm	0,6	contour de sous-couche plus rentré (échec/disparition → aucune sous-couche émise, jamais le bord brut ; 0 = bord brut voulu)	contour plus proche du bord final
<code>underlay_parallel</code>	bool	off	sous-couche perpendiculaire espacée (Lot 7)	—
<code>underlay_spacing</code>	mm	2,0	rangées de sous-couche plus espacées	plus denses
<code>hidden_underpath</code>	bool	off	coud et cache les liaisons courtes (Lot 7)	plus de sauts
<code>entry_point</code>	µm	—	démarre le remplissage près de ce point (Lot 7)	—

Le paramètre principal de **densité** est `row_spacing` (entre rangées) — à ne pas confondre avec `stitch_length` (le long d'une rangée). Le retrait de bord est appliqué **en amont** (offset intérieur du polygone) par `generate_tatami`. Tous ces paramètres, y compris `underlay_inset` et `underlay_spacing`, sont éditables dans l'inspecteur (`PropertiesPanel`), avec undo/redo générique (`SetStitchParamsCommand`).

Orientation éditable (angle des fils)

L'angle se règle à la création, puis se **modifie après coup** : soit numériquement (menu *Broderie ■ Orientation du remplissage...*, ou clic droit sur la forme), soit **directement dans la scène** en faisant glisser une **poignée de rotation** (un axe bleu au centre du remplissage). Le nouvel angle est appliqué au relâchement via une commande annulable (`SetFillAngleCommand`) ; les points sont régénérés (ADR-014). L'angle est modulo 180° (orientation d'une droite).

Validation physique : non effectuée. Ces valeurs sont des points de départ logiciels, pas des réglages éprouvés sur machine.

Limitation réelle (au-delà du Lot 7, cf. ci-dessus qui **est** implémenté) : les **motifs de phase avancés** (staggering non uniforme, densité variable) restent **prévus, non implémentés**.

Implémentation associée

- `libs/document/.../embroidery_object.hpp` — `TatamiParams`.
- `libs/stitch_generation/include/openstitch/stitch_generation/tatami.hpp` — `FillStitch`, `fill_tatami`, `tatami_underlay`, `segment_stays_in_region` (validation géométrique exposée pour test).
- `libs/stitch_generation/src/tatami.cpp` — `scanline`, graphe de routage, `connector_invalid` (validation géométrique des trajets cousus, découpe paramétrique robuste aux contacts sommet), `in_region` (frontière traitée comme intérieure, tolérance numérique).
- `libs/stitch_generation/src/generate.cpp` — `generate_tatami`, `emit_fill`.
- `libs/geometry/src/offset.cpp` — `inset_path_set` (retrait de bord).
- `apps/desktop/properties_panel.cpp` — inspecteur : tous les champs `TatamiParams`, y compris `underlay_inset/underlay_spacing`.
- Tests : `tests/unit/stitch/test_tatami.cpp` (invariants : aucune couture dans le trou ; aucune couture hors région sur une forme en L à tous les angles ; contacts sommet — trou en losange, multi-trous, coin rentrant d'un L ; politique sûre de `tatami_underlay` sur retrait impossible/nul), `tests/unit/geometry/test_offset.cpp`.
- `libs/commands/.../project_commands.hpp` — `SetFillAngleCommand` (orientation), `ConvertFillsToTatamiCommand`, `SetStitchTypeCommand`.

Retouche des points et de la géométrie

Public : utilisateur avancé, développeur.

Ce qui est éditable aujourd'hui

Niveau	Retouche disponible	État
Image	pile d'opérations non destructive	Implémenté
Segmentation	fusion, suppression, recoloration	Implémenté
Vecteurs	déplacement de nœuds	Implémenté
Objets de broderie	paramètres (longueur, densité, angle, type)	Implémenté
Objets de broderie	changer le type (contour/tatami/satin), clic droit	Implémenté
Objets de broderie	orientation des fils (poignée dans la scène)	Implémenté
Ordre de couture	monter/descendre, verrouiller	Implémenté
Points générés	déplacement d'un point (mode d'édition dédié), indicateurs Clean/ManuallyEdited/Dirty, abandon des retouches	Partiel (Lot 8.2 — déplacement seul ; Stitch↔Jump et coupe de fil restent cœur seul, sans UI, cf. Lot 8.3)

Régénération vs édition manuelle

Les points sont **régénérés** à chaque modification du document (fonction pure), puis patchés par les retouches manuelles éventuelles de l'objet (`stitch_generation::effective_sequence`, ADR-014). Le **cœur** (Lot 8.0/8.1) implémente le modèle complet : déplacer un point, convertir Stitch↔Jump, ajouter/retirer une coupe de fil, abandonner les retouches d'un objet — quatre commandes annulables (`MoveStitchPointCommand`, `SetStitchPointTypeCommand`, `SetStitchTrimCommand`, `DiscardOverridesCommand`) et une persistance `.osp` (schéma v3). Le **Lot 8.2** expose le **déplacement** dans le canevas (mode d'édition dédié, poignées, indicateurs Clean/Dirty/ManuallyEdited, abandon des retouches) ; `SetStitchPointTypeCommand`/`SetStitchTrimCommand` restent, pour l'instant, uniquement accessibles par usage programmatique (CLI, tests) — prévu au sous-lot 8.3 (voir `docs/lot8-manual-editing-design.md`).

Cas particulier : une séquence **importée d'un DST** est traitée comme la vérité (le DST n'a pas d'objets), elle n'est donc pas régénérée tant qu'aucune image n'est chargée.

Édition de nœuds vectoriels

Sélectionnez un objet vectoriel : ses nœuds s'affichent en poignées. Un déplacement passe par `MoveNodeCommand` (annulable) et **régénère** les points de tout objet de broderie qui suit cet objet.

Limitation : l'ajout/suppression de nœuds, la conversion anguleux/lisse et l'édition de tangentes ne sont pas exposés (voir *Vectorisation*).

Changer le type de points (clic droit)

Un **clic droit** sur une forme ouvre un menu contextuel *Type de points* : Contour cousu / Remplissage tatami / Colonne satin (le type courant est coché). Le changement passe par `SetStitchTypeCommand` (annulable) ; pour le satin, les rails sont reconstruits depuis le contour source (`rails_from_contour`). Le menu donne aussi accès à l'orientation (si tatami) et aux calques.

Orientation des fils dans la scène

Quand un remplissage tatami est sélectionné, une **poignée de rotation** (axe bleu) s'affiche à son centre. La faire glisser réoriente les fils ; l'angle est validé au relâchement (`SetFillAngleCommand`, annulable). Détail d'ergonomie important : la commande est **différée** d'un tour de boucle d'événements, car `refreshImage()` reconstruit la scène et détruirait la poignée pendant son propre événement souris (le même soin s'applique aux poignées de nœuds).

Mode d'édition des points (Lot 8.2)

Un objet de broderie sélectionné et non `Dirty` peut entrer en mode « Éditer les points » (menu Broderie, raccourci **E**, ou barre contextuelle) : mode **exclusif**, lié à l'unique objet actif à l'activation (changer de sélection, charger un autre projet, ou faire passer l'objet à `Dirty` en fait sortir proprement, sans confirmation). Chaque point de couture réel (passe `TopStitch`, type `Stitch` — ni saut, ni sous-couche) affiche une poignée dédiée (couleur distincte des poignées de nœuds vectoriels) ; la glisser construit une `MoveStitchPointCommand` unique au relâchement (différée d'un tour de boucle d'événements, comme la poignée de rotation ci-dessus — même raison). Aucune commande n'est créée si le point n'a pas bougé.

Au-delà d'un seuil de points déplaçables (2000, coût mémoire/événements d'un `QGraphicsItem` par poignée), l'activation est refusée avec un message explicite en barre de statut plutôt que de laisser le mode actif sans aucune poignée visible ; réduire la densité ou la longueur de point de l'objet permet de repasser sous le seuil.

Un objet retouché (`ManuallyEdited`) affiche un indicateur (↖) dans le panneau Document, la barre contextuelle et l'inspecteur, avec un bouton « Abandonner les retouches » (confirmation, `DiscardOverridesCommand` annulable). Un objet `Dirty` (géométrie source changée depuis les dernières retouches, cf. docs/lot8-manual-editing-design.md §1) affiche un avertissement (■) à la place : ses retouches ne sont plus appliquées et le mode d'édition lui reste fermé tant qu'elles n'ont pas été abandonnées.

Implémentation associée

- `apps/desktop/node_handle.hpp` — poignée de nœud (réutilisée pour la rotation et pour les points de couture, Lot 8.2).
- `apps/desktop/main_window.cpp` — sélection, poignées, menu contextuel, poignée de rotation, mode d'édition des points (`onStitchEditModeToggled`, `discardOverrides`, gating dans `updateActions`).
- `apps/desktop/canvas_view.cpp` — signal `canvasContextMenu` (clic droit).
- `apps/desktop/document_panel.cpp`, `apps/desktop/properties_panel.cpp` — indicateurs `Clean/ManuallyEdited/Dirty` (Lot 8.2).
- `libs/commands/.../project_commands.hpp` — `MoveNodeCommand`, `SetFillAngleCommand`, `SetStitchTypeCommand`, `ConvertFillsToTatamiCommand`, et (Lot 8.1, exposées dans l'UI pour le déplacement seul depuis le Lot 8.2) `MoveStitchPointCommand`, `SetStitchPointTypeCommand`, `SetStitchTrimCommand`, `DiscardOverridesCommand`.
- `libs/stitch_generation/include/.../overrides.hpp` — `effective_sequence`, `apply_manual_overrides`, `raw_slice`, `fingerprint`, `classify_edit_state`, et (Lot 8.2) `is_movable_point`, `edit_view`, `classify_all_edit_states`, `refresh_context` (un seul `generate_sequence` interne pour la séquence effective, les états par objet et la vue d'édition d'une cible, réutilisé par `MainWindow::refreshImage` à chaque rafraîchissement).

- `libs/document/include/.../embroidery_object.hpp` — `StitchOverride`, `StitchPointType`, champs `overrides/edited_fingerprint/edited_point_count`.

Palettes et fils

Public : utilisateur avancé, mainteneur. État : **Prévu / partiel**.

Ce qui existe

- Chaque objet de broderie porte une **couleur RGB** (`std::array<uint8_t, 3>`), reprise de la région d'origine.
- Un **changement de couleur** est inséré automatiquement entre deux objets consécutifs de couleurs différentes lors de la génération.
- La recoloration d'une région est disponible (choix d'une couleur libre).

Ce qui n'existe pas encore

Limitation : il n'y a **pas** de véritable **palette de fils** :

- pas de base de fils par fabricant/gamme/référence ;
- pas de recherche du fil le plus proche par distance perceptuelle ;
- pas d'association objet ↔ bobine/aiguille ;
- pas d'import/export de palette.

La bibliothèque `thread_palette` évoquée dans l'étude de cadrage **n'est pas présente** dans `libs/`. La couleur reste un simple RGB attaché à l'objet.

Conséquence pour le DST

Le format DST ne stocke de toute façon **pas** les couleurs réelles, seulement des arrêts « changement de fil ». L'ordre des couleurs est porté par le document `.osp`, pas par le DST (voir *Format DST*).

Implémentation associée

- `libs/document/.../embroidery_object.hpp` — champ `rgb`.
- `libs/stitch_generation/src/generate.cpp` — insertion des `ColorChange`.
- `libs/segmentation/src/segmentation.cpp` — `recolor_region`.
- *Information non déterminée* : aucune source `thread_palette` dans le dépôt.

Simulation de couture

Public : utilisateur. État : **Implémenté** (base).

But

Rejouer visuellement la couture pour repérer les sauts, l'ordre et la trajectoire de l'aiguille avant l'export.

Fonctionnement

La barre de simulation apparaît dès qu'une séquence de points existe. Elle offre :

- un bouton **Lecture / Pause** ;
- un **curseur** qui révèle la couture jusqu'à un index de point ;
- un **compteur** « point courant / total » ;
- un **repère d'aiguille** (cercle rouge) à la position courante.

Pendant la lecture, un minuteur (~60 pas/seconde) avance l'index proportionnellement à la taille du motif. Seule la **couche des points** est reconstruite à chaque image (l'image de fond et les vecteurs ne sont pas recalculés), pour rester fluide.

Limitation : la simulation est **point par point** via le curseur. Le réglage de vitesse explicite, l'avance objet-par-objet ou couleur-par-couleur, et l'affichage de l'objet actif ne sont **pas** exposés (le modèle en garde l'information : chaque `StitchCommand` porte son `ObjectId` source).

Implémentation associée

- `apps/desktop/main_window.cpp` — `buildSimulationToolbar`, `toggleSimulation`, `onSimTick`, `onSimSliderMoved`, `renderStitches`.
- `libs/stitch/include/openstitch/stitch/sequence.hpp` — `StitchSequence`, `StitchCommand` (avec `source`).

Analyse et validation

Public : utilisateur avancé, développeur. État : **Implémenté** (règles de base).

But

Détecter, avant l'export, les problèmes courants de broderie sans jamais appliquer de correction automatique silencieuse.

Règles détectées

`analyze(sequence, options)` renvoie une liste de `Finding` triés par gravité :

Catégorie	Condition (par défaut)	Gravité
vide	aucun point	Erreur
point-court	segment cousu < 0,5 mm	Avertissement
point-long	segment cousu > 7 mm	Avertissement
saut-long	déplacement > 30 mm	Avertissement
hors-cadre	point hors du cadre (si fourni)	Erreur
trop-de-points	> 100 000 points	Avertissement

Chaque problème porte une **gravité** (Info/Warning/Error), un **message**, une **localisation** et l'**objet** concerné. Un plafond par catégorie évite l'inondation ; le résultat est **déterministe**.

point-long fantôme juste après un saut (2026-08-12)

Défaut trouvé en usage réel (export debug utilisateur, objet satin) : la règle `point-long` comparait chaque point cousu à `prevStitch`, la position du DERNIER point cousu — sans jamais tenir compte d'un `Jump` intercalé entre les deux. Un `Jump` lève l'aiguille : le fil n'est plus continu, donc la distance entre le point cousu juste avant le saut et celui juste après n'a AUCUN sens en tant que « longueur de point cousu ». Or `emit_polyline` (§ Génération de points) fait systématiquement suivre un `Jump` d'un `Stitch` à la position d'atterrissage (point de « pinning », distance nulle) : la comparaison portait donc en réalité sur la longueur du SAUT lui-même, rejouée comme un second avertissement `point-long` trompeur en plus du `saut-long` déjà émis pour le même saut — quasi systématique dès qu'un saut dépassait 7 mm (le seuil `point-long`), qui est bien plus bas que le seuil `saut-long` (30 mm).

Corrigé en réinitialisant `hasPrevStitch` sur tout `Jump` : la continuité du fil (et donc la comparaison `point-court/point-long`) ne traverse plus un saut. Vérifié : `tests/unit/stitch_analysis/test_analyze.cpp` (absence de `point-long/point-court` fantôme après un saut, y compris à distance d'atterrissage nulle).

Dans l'interface

Analyse → **Analyser le motif** (F5) remplit le dock *Analyse* ; un double-clic sur un problème centre la vue sur sa localisation. Le cadre utilisé est le cadre courant (100 × 100 mm par défaut).

Limitation : l'analyse **spatiale de densité** (carte de chaleur, superposition de couches, satin trop large/étroit dédié, alignement excessif des pénétrations) et les **corrections automatiques proposées** sont **prévues**, non implémentées.

Implémentation associée

- `libs/stitch_analysis/include/openstitch/stitch_analysis/analyze.hpp` — Finding, Severity, AnalysisOptions.
- `libs/stitch_analysis/src/analyze.cpp` — analyze.
- `apps/desktop/main_window.cpp` — buildAnalysisPanel, runAnalysis.
- Tests : `tests/unit/stitch_analysis/test_analyze.cpp`.

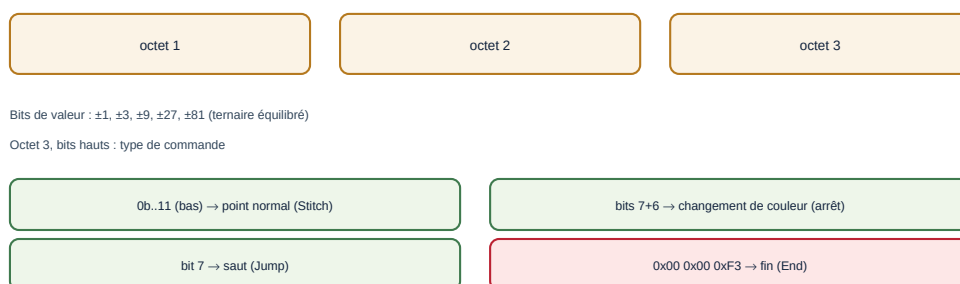
Partie II

Formats, architecture et développement

Format DST (Tajima)

Public : utilisateur avancé, développeur, mainteneur. État : **Implémenté** (encodeur + décodeur maison, testés en aller-retour).

Enregistrement DST : 3 octets = déplacement (dx, dy) en unités de 0,1 mm



En-tête ASCII de 512 octets (LA, ST, CO, $\pm X$, $\pm Y$, AX, AY) calculé après le corps.

Figure — Structure d'un enregistrement DST de 3 octets et des commandes.

Rôle du format

Le DST (Tajima) est le format d'échange le plus répandu pour la broderie machine. C'est un fichier **binaire** contenant un en-tête ASCII puis une suite de **déplacements relatifs** de l'aiguille. Le module `formats` implémente son encodage et son décodage **sans dépendance externe** (le format est documenté publiquement ; la bibliothèque `libembroidery` n'a servi que de référence croisée).

Unités

Le pas du DST est **0,1 mm**. En interne, le logiciel travaille en **micromètres** ($1 \mu\text{m} = 1/1000 \text{ mm}$) ; la conversion vers le DST se fait par division par 100, et la quantification n'intervient **qu'à l'export**.

En-tête (512 octets ASCII)

Champs terminés par CR, complétés d'espaces jusqu'à 512 octets. Calculés **après** le corps (donc cohérents avec les octets réellement produits) :

Champ	Signification
LA :	nom du motif (16 caractères)
ST :	nombre d'enregistrements
CO :	nombre de changements de couleur
+X: -X: +Y: -Y:	étendue du motif (unités 0,1 mm)
AX: AY:	position finale relative au départ
MX: MY: PD:	multi-motif (valeurs par défaut)

Corps : enregistrements de 3 octets

Chaque enregistrement encode un déplacement (dx, dy) dans $[-121, +121]$ unités de 0,1 mm, par bits de valeurs $\pm 1, \pm 3, \pm 9, \pm 27, \pm 81$ (ternaire équilibré) répartis sur les trois octets. Les bits hauts de l'octet 3 donnent le **type** :

Commande interne	Encodage DST
Stitch (point)	bits bas 11, point normal
Jump (saut)	bit 7 de l'octet 3
ColorChange / Stop	bits 7 + 6 de l'octet 3
Trim (coupe)	convention : N sauts de delta nul (défaut 3)
End (fin)	0x00 0x00 0xF3

Encodage (sans dérive)

encode_dst quantifie les **positions absolues** au pas de 0,1 mm puis en dérive les deltas : $\text{delta}_i = \text{round}(\text{pos}_i/100) - \text{round}(\text{pos}_{i-1}/100)$. Cela borne l'erreur à $\pm 50 \mu\text{m}$ **sans dérive cumulative**. Tout déplacement dépassant ± 121 est **subdivisé** en sauts intermédiaires. La sortie est **déterministe**.

Décodage (tolérant)

decode_dst lit l'en-tête de façon **laxiste** (la vérité est le corps), décode chaque enregistrement, et réinterprète une rafale de sauts nuls en Trim. La lecture **s'arrête au marqueur de fin 00 00 F3** et **ignore tout octet suivant** : certains logiciels (p. ex. **Hatch**) ajoutent un octet de fin DOS 0x1A — voire du padding — après le marqueur, ce qui ne doit pas faire échouer l'import. Les cas d'erreur **couverts par les tests** (fichier vide, tronqué, sans marqueur de fin, octets en trop après F3) renvoient une erreur structurée (ou décodent proprement) au lieu de provoquer un arrêt du programme.

Exemple interprété (motif minimal)

Un carré de 5 mm : après l'en-tête, une commande Jump amène au premier coin, puis quatre points Stitch de 5 mm parcourent les côtés, et l'enregistrement 0x00 0x00 0xF3 termine. Sur ce motif, +X: et +Y: valent 50 (5 mm en unités de 0,1 mm) et CO: vaut 0.

Limites et informations perdues

Avertissement : Un DST **ne conserve pas** les objets éditables de haut niveau (contours, remplissages, colonnes satin, paramètres). Il ne stocke pas non plus les **couleurs réelles** (seulement des arrêts). Après un export DST, seul le **projet .osp** permet de retrouver la géométrie et les paramètres.

Tests

- **Aller-retour** : séquence → encode → decode → séquence' (mêmes types, positions à $\pm 50 \mu\text{m}$) — dont un test sur **tous les deltas** de -121 à $+121$.
- **Subdivision** d'un déplacement $> 12,1 \text{ mm}$.
- **Fichiers invalides** (vide, tronqué, sans fin) refusés proprement.
- **Déterminisme** octet à octet.

Implémentation associée

- `libs/formats/include/openstitch/formats/dst.hpp` — API.
- `libs/formats/src/dst.cpp` — `encode_dst`, `decode_dst`, `write_dst_file`, `read_dst_file`.
- `apps/cli/main.cpp` — `stats`, `dst2svg`.
- Tests : `tests/unit/formats/test_dst.cpp`.

Format de projet .osp

Public : développeur, mainteneur. État : **Implémenté**.

Nature

Un projet .osp est une **archive ZIP** (via minizip-ng) contenant :

Entrée	Contenu
project.json	tout le document, en JSON versionné
original.png	l'image source, encodée PNG (sans perte)
segmentation.u32	la carte des labels, en binaire little-endian (si présente)

Le JSON et le binaire sont séparés pour ne pas alourdir le JSON (la carte des labels peut faire plusieurs mégaoctets).

Contenu de project.json

- `schemaVersion` (entier) et un objet document ;
- `mmPerPx` (résolution de travail) et `objectIdLast` (compteur d'ids) ;
- `canvas` : taille du cadre de broderie (`width`, `height` en μm). **Optionnel et rétrocompatible** : un projet antérieur sans ce champ retombe sur 100×100 mm ;
- `ops` : la pile d'opérations d'image (chaque opération porte son type) ;
- `segmentation` : dimensions et régions (`id`, `rgb`, nombre de pixels ; les labels sont dans `segmentation.u32`) ;
- `vectorObjects` : `id`, nom, couleur, visibilité, région source, paths (chemins avec nœuds et tangentes optionnelles) ;
- `embroideryObjects` : `id`, nom, couleur, visibilité, vecteur source, et params (variant `running` | `tatami` | `satin`). Le satin porte ses deux rails et, depuis le schéma v2, ses **barreaux** (`rungs` : liste de segments `{ax, ay, bx, by}` en μm), ses réglages de finition (points courts, split, terminaisons), de sous-couche/compensation, et de **fixation/entrée-sortie** (`lockStart/lockEnd`, `lockLength`, `lockPasses`, `entryPoint/exitPoint`) — tous **optionnels** et rétrocompatibles (clés absentes → valeurs par défaut). Une section issue d'un réseau multi-rail peut aussi porter `topology` : `sectionIndex`, `sectionCount` et les identifiants optionnels `startJunction/endJunction`. Ces identifiants sont locaux au réseau défini par `sourceVector`; l'absence de `topology` désigne un satin isolé/historique. Le `tatami` porte de même ses réglages avancés (Lot 7) : `underlayEdge`, `underlayParallel`, `underlayInset`, `underlaySpacing`, `hiddenUnderpath` et `entryPoint` — optionnels et rétrocompatibles. Depuis le schéma v3, un objet peut aussi porter ses **retouches manuelles** (Lot 8.1, ADR-014) : `overrides` (**tableau** JSON obligatoire de `{index, pos?, type?, trimAfter}` — `index` désigne une position dans la vue brute de l'objet et doit être **unique** dans le tableau, `pos` un déplacement `{x, y}` en μm , `type` `"stitch"` ou `"jump"` ; chaque entrée doit porter au moins une modification effective — `pos`, `type`, ou `trimAfter: true`), `editedFingerprint` (empreinte FNV-1a 64 bits de la vue brute au moment de la dernière édition, entier **exact**, jamais passé par un double) et `editedPointCount` — **obligatoires et explicites** dès que `overrides` n'est pas vide (aucune valeur zéro implicite). Absents, ou `overrides` vide (métadonnées éventuellement présentes mais alors ignorées) → objet `Clean` (comportement actuel inchangé).

Versionnement et validation

schemaVersion vaut **3** (v1 → v2 : cadre canvas et barreaux satin runs ; v2 → v3 : retouches manuelles overrides/editedFingerprint/ editedPointCount par objet de broderie, Lot 8.1). La lecture est **rétrocompatible** : un fichier v1 ou v2 se charge (cadre 100×100 par défaut si absent, aucun barreau, aucune retouche → état Clean). Une version **supérieure** à celle du binaire est refusée proprement (UnsupportedFormat) ; un JSON invalide, une topologie satin incohérente (sectionCount == 0 ou sectionIndex >= sectionCount), une valeur hors bornes (index négatif, non entier, ou au-delà de numeric_limits<size_t>::max()), coordonnée au-delà d'un int32, compteur au-delà d'un uint32, type de point inconnu), un overrides qui n'est pas un tableau, un index en double, une entrée sans modification effective, des métadonnées editedFingerprint/editedPointCount manquantes pour un tableau non vide, ou une carte de labels incohérente renvoie une erreur utile (InvalidFile), jamais une troncature silencieuse.

Sauvegarde atomique

save_project écrit d'abord un fichier temporaire .osp.tmp puis le **renomme** : un plantage pendant l'écriture ne corrompt jamais le fichier existant.

Aller-retour

Le test d'intégration vérifie que save puis load restitue exactement l'image (PNG sans perte), les opérations, les labels de segmentation, les tangentes de nœuds et les paramètres des trois types de points.

Limitation : la **sauvegarde automatique**, la **récupération après crash** et les **migrations** entre versions de schéma sont **prévues**, non implémentées. Le cache des points générés n'est pas stocké (il est recalculé au chargement).

Implémentation associée

- libs/project_io/include/openstitch/project_io/project_io.hpp — save_project, load_project, kSchemaVersion.
- libs/project_io/src/json_serialize.cpp — sérialisation du document.
- libs/project_io/src/archive.cpp — lecture/écriture ZIP (minizip-ng encapsulé).
- libs/project_io/src/project_io.cpp — orchestration, écriture atomique.
- Tests : tests/unit/project_io/test_roundtrip.cpp, tests/unit/project_io/test_overrides_persistence.cpp (retouches v3, migration v1/v2, topologie satin optionnelle/rétrocompatible, validation stricte).

Architecture logicielle

Public : développeur, mainteneur.

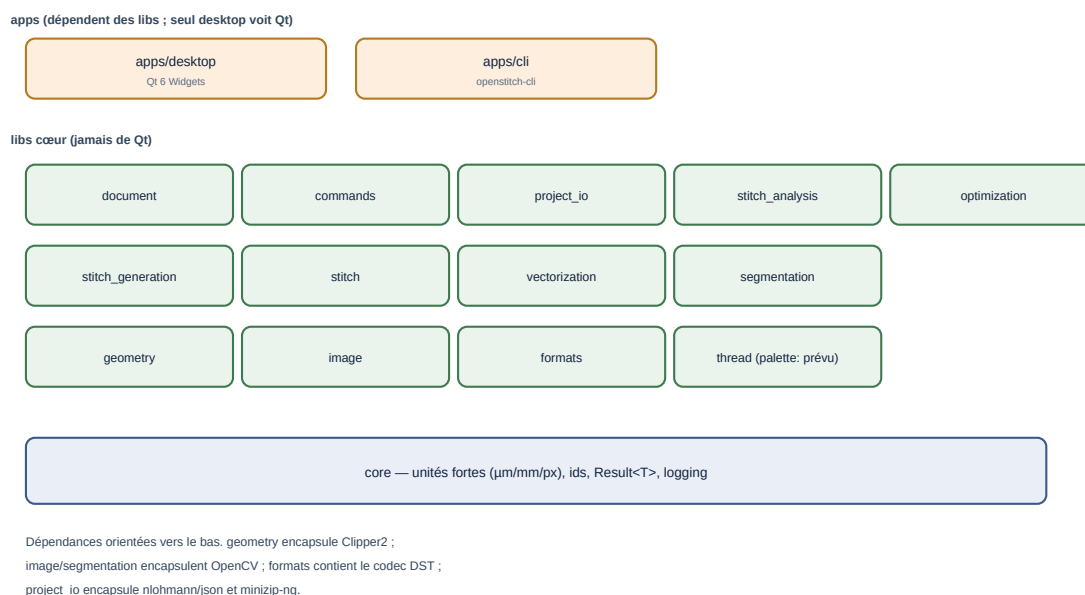


Figure — Les couches : applications, libs cœur, socle core.

Principes

1. **Le cœur ne dépend pas de Qt.** Seule apps/desktop voit Qt ; tout le reste se compile sans interface (vérifié par la CLI et par un build Linux du cœur).
2. **Dépendances orientées vers le bas :** apps → libs, et entre libs un graphe **acyclique**.
3. **Une lib = une cible CMake** openstitch::- 4. **Encapsulation des tiers :** Clipper2 dans geometry, OpenCV dans image/segmentation/vectorization, le codec DST dans formats, json et minizip dans project_io.

Couches

- **Applications :** apps/desktop (Qt), apps/cli.
- **Métier / génération :** document, commands, project_io, stitch_generation, stitch, stitch_analysis, optimization, autodigitize, auto_satin, formats.
- **Traitement :** image, segmentation, vectorization, geometry.
- **Socle :** core (unités, ids, Result, logging).

Séparation cœur / interface

L'application ne contient **aucune logique métier** : elle câble des actions, affiche, et recalcule l'aperçu. Le chargement d'image, le placement physique, les transformations, la génération de points, l'analyse, l'ordre et les formats vivent tous dans les libs. La règle est vérifiable : la CLI utilise les mêmes modules sans Qt.

Modèle de document et rendu

Le document `::Project` est la source de vérité (voir *Modèle de données*). Les **points** ne sont pas stockés : ils sont recalculés par `generate_sequence`, puis patchés par les **retouches manuelles** de l'objet le cas échéant (Lot 8.1, ADR-014). `stitch_generation::effective_sequence(project)` est le **point d'entrée unique** que tout consommateur de production (aperçu, export, analyse, simulation) doit utiliser — il enchaîne les deux passes ; `generate_sequence` reste appelable séparément (implémentation interne, tests, générateurs synthétiques). Une garde CTest structurelle (`tests/check_no_raw_sequence_bypass.cmake`) échoue si un nouveau site de production appelle `generate_sequence` directement sans annotation explicite. Le rendu (côté desktop) est organisé en **deux couches** persistantes — base (image/vecteurs/régions) et points — pour ne reconstruire que le nécessaire, notamment pendant la simulation.

Commandes et undo/redo

Toute mutation du document passe par une **commande** (`ICommand`) empilée dans un `UndoStack`. C'est le patron *Command* : chaque commande sait s'appliquer et se révoquer exactement.

Tâches asynchrones

Limitation : il n'y a **pas** d'infrastructure de tâches asynchrones (pool de threads, annulation, progression) implémentée. Les traitements lourds (segmentation, remplissage) s'exécutent de façon **synchrone** (avec un curseur d'attente). Les fonctions de génération sont **pures** sur des instantanés, donc compatibles avec une exécution en arrière-plan future.

Diagrammes de séquence

- Import d'une image : voir *Introduction* et la figure du chapitre.
- Génération des points : voir *Génération de points*.
- Export DST : voir *Format DST*.

Séquence — import d'une image

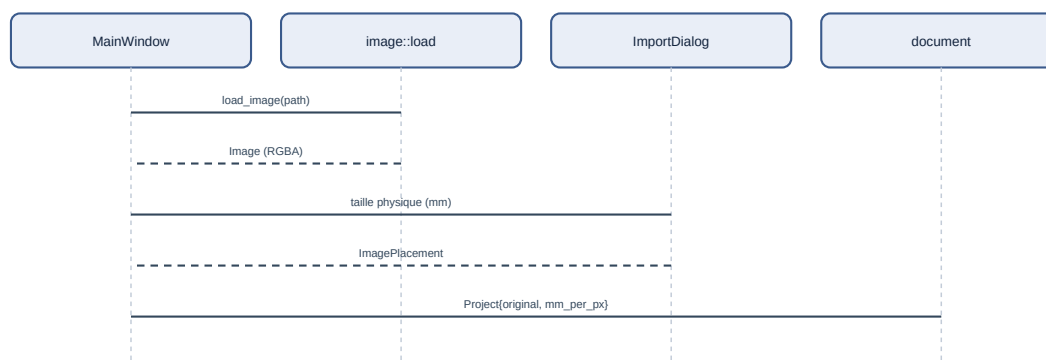


Figure — Import d'une image (du menu au document).

Implémentation associée

- `CMakeLists.txt` (racine), `libs/*/CMakeLists.txt` — cibles et dépendances.
- `apps/desktop/main_window.cpp` — câblage interface.
- `libs/commands/` — `ICommand`, `UndoStack`.
- `docs/phase0/02-architecture.md` — étude d'architecture d'origine.

Référence des modules

Public : développeur. Ce chapitre aide à trouver où modifier une fonctionnalité. Chaque module est une cible `openstitch::<nom>` sous `libs/`.

Tableau de synthèse

Module	Responsabilité	Dépendances	Tests
<code>core</code>	unités fortes, ids, <code>Result</code> , logging	—	<code>tests/unit/core</code>
<code>geometry</code>	chemins, simplification, booléens/offsets, longueur d'arc	<code>core</code> (+Clipper2)	<code>tests/unit/geometry</code>
<code>image</code>	chargement, prétraitement non destructif	<code>core</code> (+OpenCV)	<code>tests/unit/image</code>
<code>segmentation</code>	quantification CIELAB, régions connexes	<code>core</code> , <code>image</code> (+OpenCV)	<code>tests/unit/segmentation</code>
<code>vectorization</code>	régions → contours vectoriels	<code>core</code> , <code>geometry</code> , <code>segmentation</code>	<code>tests/unit/vectorization</code>
<code>document</code>	modèle métier (projet, objets)	<code>core</code> , <code>geometry</code> , <code>image</code> , <code>segmentation</code>	via <code>commands/project_io</code>
<code>stitch</code>	commandes machine, statistiques	<code>core</code>	<code>tests/unit/stitch</code>
<code>stitch_generation</code>	running / tatami / satin	<code>core</code> , <code>geometry</code> , <code>stitch</code> , <code>document</code>	<code>tests/unit/stitch</code>
<code>stitch_analysis</code>	règles de validation	<code>core</code> , <code>stitch</code>	<code>tests/unit/stitch_analysis</code>
<code>optimization</code>	ordre de couture	<code>core</code>	<code>tests/unit/optimization</code>
<code>autodigitize</code>	<code>image</code> → objets éditables	<code>vectorization</code> , <code>stitch_generation</code>	<code>tests/unit/autodigitize</code>
<code>auto_satin</code>	squelette → satinabilité → rails/barreaux multi-sections	<code>core</code> , <code>geometry</code> (+OpenCV)	<code>tests/unit/auto_satin</code>
<code>satin_coverage</code>	mesure la surface géométrique couverte par des colonnes satin	<code>core</code> , <code>geometry</code> , <code>stitch_generation</code>	<code>tests/unit/satin_coverage</code>
<code>commands</code>	undo/redo	<code>document</code>	<code>tests/unit/commands</code>
<code>formats</code>	codec DST, SVG diagnostic	<code>core</code> , <code>stitch</code>	<code>tests/unit/formats</code>
<code>project_io</code>	format <code>.osp</code>	<code>core</code> , <code>document</code> , <code>image</code>	<code>tests/unit/project_io</code>

Points d'extension

- **Ajouter un type de point** : nouvelle alternative de `StitchParams` (document), un générateur dans `stitch_generation`, une branche `std::visit` dans `generate.cpp`, un dialogue et une action dans `apps/desktop`.
- **Ajouter un format d'export** : une fonction dans `formats` (encapsulée derrière une interface interne), une sous-commande CLI et une action Fichier.
- **Ajouter une règle d'analyse** : une catégorie dans `analyze.cpp`.
- **Ajouter une opération d'image** : une alternative de `ImageOp` (image), un cas dans `apply_op`, une action menu.
- **Ajouter une commande annulable** : une classe `ICommand` dans `commands`.

Où trouver quoi (raccourci)

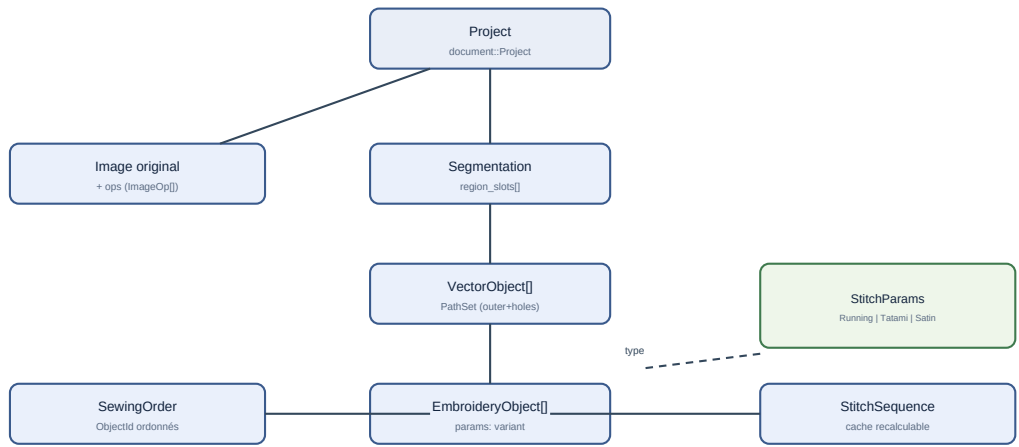
Je veux modifier...	Fichier
l'échantillonnage des points	<code>libs/stitch_generation/src/running_stitch.cpp</code> , <code>polyline.cpp</code>
le remplissage/routage tatami	<code>libs/stitch_generation/src/tatami.cpp</code>
la colonne satin	<code>libs/stitch_generation/src/satin.cpp</code>
le satin par squelette et ses sections	<code>libs/auto_satin/src/</code>
la mesure de couverture géométrique du satin	<code>libs/satin_coverage/src/coverage.cpp</code>
le choix de type auto (tatami/satin)	<code>libs/autodigitize/src/autodigitize.cpp</code>
l'édition (type, orientation, filtres, calques)	<code>apps/desktop/main_window.cpp</code>
l'encodage DST	<code>libs/formats/src/dst.cpp</code>
le format projet	<code>libs/project_io/src/</code>
les menus	<code>apps/desktop/main_window.cpp</code>
les unités	<code>libs/core/include/openstitch/core/units.hpp</code>

Implémentation associée

Voir les chapitres thématiques pour le détail de chaque module.

Modèle de données

Public : développeur.



La géométrie éditable (VectorObject) et les points générés (StitchSequence) sont séparés : régénérer ne détruit pas la source.

Figure — Relations principales du document.

Unités et coordonnées

Type	Fichier	Rôle	Unité
Micrometers	core/units.hpp	coordonnée interne (int32)	µm
Millimeters	core/units.hpp	affichage / paramètres UI	mm (double)
Pixels	core/units.hpp	domaine image	px (double)
Angle	core/units.hpp	angles	radians
Vec2um	core/units.hpp	point 2D	µm

Invariant : **aucune coordonnée n'est un double nu** ; l'unité interne est le micromètre entier (déterminisme, pas de dérive), Y vers le haut. La conversion vers/depuis les pixels exige une résolution explicite (mm/px).

Identifiants

ObjectId, RegionId, ColorId, ThreadId sont des types **forts distincts** (Id<Tag>), générés par un IdGenerator<T> monotone (jamais réutilisés). Fichier : core/ids.hpp.

Document

document::Project (fichier document/project.hpp) contient :

Champ	Type	Rôle
original	Image::Image	image source étendue
mm_per_px	Millimeters	résolution de travail
canvas	Canvas	carte de broderie (origine/hauteur, offset (0-100 mm))
ops	vector<ImageOp>	pile de traitements
segmentation	api::Image::Segmentation	carte de régions
vector_objects	vector<VectorObject>	géométrie éditables
embroidery_objects	vector<EmbroideryObject>	objets de broderie (ordre = ordre de couture)
object_id_gen	IdGenerator<ObjectId>	compteur partagé

Types géométriques

- `geometry::PathNode` : `pos` (Vec2um), `type` (Corner/Smooth), `tan_in/tan_out` optionnelles.
- `geometry::Path` : `nœuds` + `closed`.
- `geometry::PathSet` : `outer` + `holes`.
- `geometry::Polyline` : polyligne aplatie (travail par longueur d'arc).

Points et commandes

- `stitch::CommandType` : `Stitch`, `Jump`, `Trim`, `ColorChange`, `Stop`, `End`.
- `stitch::StitchCommand` : `pos` (Vec2um absolue), `type`, `source` (ObjectId).
- `stitch::StitchSequence` : `vector<StitchCommand>`.
- `stitch::StitchStats` : compteurs, longueur de fil, bornes.

Sérialisation

Voir *Format de projet* : tout ce qui précède se sérialise en JSON (+ PNG + binaire), sauf les points générés (recalculés).

Éléments non présents dans le modèle

Limitation : `profil machine`, `cadre paramétrable riche` (formes, marges), `fil/palette`, `avertissement persistant` et `commande undo sérialisée` ne sont pas (ou peu) présents. Le `Canvas` porte la **taille du cadre**, réglable et persistée, mais reste un simple rectangle.

Implémentation associée

- `libs/core/include/openstitch/core/{units,ids,error}.hpp`
- `libs/document/include/openstitch/document/{project,vector_object,embroidery_object,canvas}.hpp`
- `libs/geometry/include/openstitch/geometry/{path,polyline}.hpp`
- `libs/stitch/include/openstitch/stitch/sequence.hpp`

Algorithmes

Public : développeur. Ce chapitre décrit les algorithmes **réellement** utilisés, avec un pseudo-code reflétant l'implémentation (et non une copie du C++).

Quantification (segmentation)

- **Objectif** : réduire l'image à N couleurs perceptuellement cohérentes.
- **Principe** : conversion RGB→CIELAB, k-means déterministe sur un échantillon, affectation au centre le plus proche.
- **Complexité** : $\sim O(\text{pixels} \cdot N)$ pour l'affectation.
- **Cas limites** : image entièrement transparente (refus), N hors bornes.

```
segment(image, N):
    opaques = pixels d'alpha >= 128 en Lab
    centres = kmeans(échantillon(opaques), N, graine fixe)
    pour chaque pixel opaque: label = argmin_c ||Lab(pixel) - centres[c]||
    régions = composantes_connexes(label) # 4-connexité, par couleur
    absorber les régions < taille_min dans la voisine majoritaire
```

Extraction et simplification de contours

- **Contours** : Suzuki-Abe (cv::findContours, RETR_CCOMP).
- **Simplification** : Douglas-Peucker, tolérance **adaptive** ($\leq \text{périmètre}/16$) pour préserver les petits trous.
- **Nettoyage** : union Clipper2 (règle pair-impair) → hiérarchie extérieur/trous.

Aplatissement de Bézier + longueur d'arc

```
flatten(path, tol):
    pour chaque segment à tangentes: subdiviser (De Casteljau) tant que
        distance(contrôles, corde) > tol
    fusionner les points identiques

resample_run(points, cible):
    L = longueur(points)
    n = max(1, ceil(L / cible)) # parts égales, <= cible, sans résidu
    renvoyer points aux abscisses i*L/n, i=0..n
```

Running stitch

```
run_stitch(path, cfg):
    poly = flatten(path, cfg.tol)
    coins = sommets d'angle de rotation > seuil (+ extrémités si ouvert)
    pour chaque tronçon entre coins: concat(resample_run(tronçon, cible))
```

- **Complexité** : $O(\text{points})$.
- **Cas limites** : chemin vide/trop court → avertissement, pas de crash.

Tatami (scanline + routage)

```
fill_tatami(région, params):
    tourner la géométrie de -angle
    pour chaque rangée y: segments = intervalles intérieurs (pair-impair)
    graphe: segments de rangées voisines qui se chevauchent en x sont adjacents
    parcours glouton: suivre une arête (couture) ; sinon sauter (déplacement)
    retourner dans le repère d'origine
```

- **Garantie** : aucune couture ne traverse un trou (adjacence $\Rightarrow$ bande intérieure).
- **Complexité** : $\sim O(\text{segments}^2)$ au pire pour l'adjacence locale par rangée (voisins uniquement), acceptable aux tailles usuelles.

Satin

```
fill_satin(railA, railB, cfg):
    pour u de 0 à 1 par pas dérivé de la densité:
        L = point(railA, u*longueurA) ; R = point(railB, u*longueurB)
        appliquer compensation (écarter L et R le long de LR)
        émettre zigzag alterné L,R
    sous-couche centrale = point droit sur (L+R)/2
```

Encodage / décodage DST

```
encode(seq):
    pour chaque commande: delta = round(pos/100) - round(prev/100)
        subdiviser si |delta| > 121 (sauts)
        émettre 3 octets (ternaire équilibré + bits de type)
    en-tête calculé après le corps
```

- **Propriété** : déterministe, erreur $\leq 50 \mu\text{m}$, sans dérive.

Ordre de couture

```
optimize_order(items, stratégie):
    items libres (non verrouillés) réarrangés:
        ByColor: regrouper par couleur (ordre d'apparition)
        ByProximity: plus proche voisin
        ColorThenProximity: groupes de couleur puis proximité
    réinsérer dans les emplacements libres, verrous à leur place
    coût = distance de déplacement + 50000 * changements de couleur
```

Implémentation associée

Voir chaque chapitre thématique et docs/[stitch-engine-research.md](#) pour les références (Douglas-Peucker, scanline fill, k-means, ré-échantillonnage par longueur d'arc).

Compilation et développement

Public : développeur, mainteneur.

Chaîne de build

- **CMake** (≥ 3.27) avec `CMakePresets.json`.
- **MSVC** (Visual Studio 2022+) sous Windows ; générateur par défaut du preset.
- **vcpkg** en mode manifeste (`vcpkg.json`, baseline verrouillée).
- **Qt 6.8** hors `vcpkg` (variable `QT_ROOT`).

Presets

Preset	Cible
<code>msvc</code> (configure)	Windows, application complète
<code>msvc-debug</code> / <code>msvc-release</code> (build/test)	Debug / Release
<code>linux-core</code>	Cœur + CLI sans Qt (garde-fou de portabilité)

Commandes

```
$env:VCPKG_ROOT = "C:\dev\vcpkg"
$env:QT_ROOT    = "C:\Qt\6.8.3\msvc2022_64"

cmake --preset msvc           # configuration (premier run long : vcpkg)
cmake --build --preset msvc-debug # compilation Debug
ctest --preset msvc-debug      # tests
cmake --build --preset msvc-release # Release
```

Options CMake

- `OPENSTITCH_BUILD_DESKTOP` (ON) — construit l'application Qt.
- `OPENSTITCH_BUILD_TESTS` (ON) — construit les tests (Catch2).
- Cible d'interface `openstitch::warnings:/W4 /permissive- /utf-8` (MSVC), `-Wall -Wextra -Wconversion` (GCC/Clang), liée en PRIVATE par chaque cible.

Déploiement des DLL (Windows)

L'application lie dynamiquement Qt et OpenCV. `windeployqt` copie les DLL Qt à côté de l'exécutable (étape `POST_BUILD`) ; `vcpkg` copie ses propres DLL.

Linux (cœur uniquement)

Le preset `linux-core` compile les libs cœur et la CLI **sans Qt** — c'est un garde-fou pour éviter toute dépendance Windows/Qt involontaire dans le cœur. L'interface graphique n'est pas construite sous Linux à ce stade.

Erreurs fréquentes

- « Qt6 introuvable » → vérifier `QT_ROOT`.
- Erreur de toolchain `vcpkg` → vérifier `VCPKG_ROOT` et le bootstrap.
- Compilation OpenCV très longue → normal au premier configure (cache ensuite).

- Édition de lien de l'exé échouée (LNK1168) → l'application est en cours d'exécution ; la fermer.

Intégration continue

Un workflow GitHub Actions (`.github/workflows/ci.yml`) est présent : build MSVC Debug/Release + tests + artefacts, build Linux du cœur, et vérification du formatage. *Information non déterminée* : aucun dépôt distant n'étant déclaré, la CI n'a pas encore été exécutée.

Implémentation associée

- `CMakeLists.txt`, `CMakePresets.json`, `vcpkg.json`.
- `.github/workflows/ci.yml`, `.clang-format`.
- `docs/build-windows.md`.

Guide du développeur

Public : développeur contributeur.

Conventions

- **C++23**, `RAII`, `std::optional`/`std::variant`/`std::span`/`std::filesystem`, `std::expected` (via `Result<T>`).
- **Types forts** pour les unités : jamais de coordonnée en double nu.
- **Const-correctness**, warnings élevés (`/W4 /permissive-`).
- **Formatage** : `.clang-format` (base LLVM, 4 espaces, 100 colonnes).
- **En-tête de licence** : `// SPDX-License-Identifier: Apache-2.0`.

Pièges connus (spécifiques à ce dépôt)

Avertissement : Quelques écueils rencontrés lors du développement :

- les **noms de tests Catch2** doivent être en **ASCII** (les accents ne survivent pas à l'encodage console Windows lors de la découverte des tests) ;
- l'en-tête `spdlog` est `stdout_color_sinks.h` ;
- un membre nommé `slots` est interdit (macro Qt) — utiliser un autre nom ;
- déclarer les classes Qt en **forward declaration au scope global** ;
- pas de `M_PI` sous MSVC → utiliser `std::numbers::pi` (`<numbers>`) ;
- cibles `vcpkg` exactes : `Clipper2::Clipper2`, `MINIZIP::minizip-ng`.

Ajouter une fonctionnalité (recettes)

- **Nouveau type de point** : voir *Référence des modules* → *Points d'extension*.
- **Nouvelle opération d'image** : une alternative de `ImageOp`, un cas dans `apply_op`, un test, une action menu.
- **Nouvelle commande annulable** : dériver `ICommand` (`apply/revert/name`), l'exécuter via `UndoStack::execute`. Toute mutation du document doit passer par une commande.

Undo/redo

`UndoStack` conserve deux piles (undo/redo). Une nouvelle commande vide la pile redo. Les commandes de segmentation mémorisent les **indices** de pixels modifiés pour une annulation exacte et légère.

Gestion des unités

Toujours convertir aux frontières : pixels→μm à l'import (résolution explicite), μm→mm pour l'affichage, μm→0,1 mm à l'export DST. Les algorithmes internes peuvent utiliser des `double`, mais les entrées/sorties des modules sont en types forts.

Multithreading

Il n'y a pas encore d'infrastructure asynchrone. Écrire les nouveaux traitements comme des **fonctions pures** sur des instantanés, pour rester compatible avec un futur passage en tâche de fond.

Implémentation associée

- `.clang-format`, `CMakeLists.txt` (cible `openstitch::warnings`).
- `libs/commands/` — `ICommand`, `UndoStack`, commandes du projet.
- `docs/phase0/` — décisions d'architecture (ADR).

Tests

Public : développeur, mainteneur.

Structure

- **Framework (cœur, cli)** : Catch2 v3 (via vcpkg), intégré à CTest.
- **Framework (UI desktop)** : Qt6::Test (QTest/QSignalSpy), intégré à CTest, toujours exécuté **headless** (QT_QPA_PLATFORM=offscreen) — voir Tests UI (Qt, desktop).
- **Tests unitaires** : tests/unit/<lib>/test_*.cpp (un exécutable par lib).
- **Test d'intégration** : tests/integration/test_pipeline.cpp (chaîne complète).
- **Golden (SVG de diagnostic)** : tests/golden/stitch-generation/.
- **Garde structurelle CI** : tests/check_no_raw_sequence_bypass.cmake (invquée via add_test, cf. Lot 8.1) — échoue si un site de production hors libs/stitch-generation/ appelle generate_sequence() directement sans annotation raw-sequence-ok:, contournant les retouches manuelles.
- Total au dernier passage vérifié : **398 tests CTest**, 100 % réussis.

Encadré de traçabilité (dernier passage vérifié)

Un simple « 398/398 » devient vite périmé ; voici le contexte exact du dernier passage vérifié manuellement. Régénérez ces valeurs avant toute publication.

Élément	Valeur
Commit (état du code testé)	e4ceaa2 — formes dessinees a la main : forme libre + correctif Maj=cercle
Compilateur	MSVC toolset 14.50 (Visual Studio 2026)
CMake	4.4.0-rc3
Configurations	Debug et Release
Résultat CTest	398 / 398 réussis
Tests désactivés	0
Fichiers de tests d'intégration	1 (tests/integration/test_pipeline.cpp)
Suites Qt (UI desktop)	5 exécutables CTest (tests/unit/desktop/), 54 fonctions de test QTest
Tests sur machine réelle	0
Couverture de code	non mesurée

Note : chaque TEST_CASE Catch2 est enregistré séparément par catch_discover_tests. Côté Qt, chaque exécutable QTest est un seul test CTest (add_test) qui contient plusieurs fonctions. Le nombre d'**assertions** est supérieur au nombre de tests CTest (398, garde structurelle CI incluse).

Exécution

```
ctest --preset msvc-debug          # tous les tests
ctest --preset msvc-debug -R tatami # filtre par nom
build\msvc\tests\unit\stitch\Debug\test_stitch.exe "[nom]" # un exécutable
```

Correspondance modules ↔ tests

Module	Tests
core	tests/unit/core/test_units.cpp, test_ids.cpp
geometry	test_simplify.cpp, test_clean.cpp, test_offset.cpp, test_polyline.cpp, test_primitives.cpp
image	test_image_load.cpp, test_ops.cpp
segmentation	test_segmentation.cpp
vectorization	test_vectorize.cpp
stitch / stitch_generation	test_stats.cpp, test_running_stitch.cpp, test_generate.cpp, test_tatami.cpp, test_satin.cpp, test_satin_pairing_metrics.cpp, test_routing.cpp, test_overrides.cpp
autodigitize	test_autodigitize.cpp
auto_satin	tests/unit/auto_satin/test_pipeline.cpp, test_columns.cpp
stitch_analysis	test_analyze.cpp
optimization	test_order.cpp
commands	test_undo_stack.cpp
formats	test_dst.cpp, test_svg.cpp
project_io	test_roundtrip.cpp, test_overrides_persistence.cpp
desktop (UI, Qt)	tests/unit/desktop/test_canvas_view.cpp, test_node_handle.cpp, test_document_panel.cpp, test_properties_panel.cpp, test_main_window.cpp
intégration	tests/integration/test_pipeline.cpp

Tests UI (Qt, desktop)

Fondation ajoutée pour vérifier réellement l'interface (et pas seulement le cœur) : la sélection au canevas, le déplacement d'un nœud, la synchronisation canevas/liste/inspecteur, sans jamais comparer de pixels ni dépendre d'un écran.

Architecture de testabilité

apps/desktop/CMakeLists.txt sépare les sources en une bibliothèque statique `openstitch_desktop_widgets` (tout sauf `main.cpp`), liée à la fois par l'exécutable `openstitch` et par les tests. Pour rendre `MainWindow` elle-même instanciable en test (deuxième tranche, voir plus bas), trois seams minimaux s'y ajoutent, sans changer son comportement en production : `QSettings()` par défaut (au lieu d'une organisation/application codées en dur) lit l'organisation/application déjà posées sur `QCoreApplication` dans `main.cpp` — un test peut donc les rediriger vers un fichier `.ini` temporaire avant de construire `MainWindow`, sans jamais toucher au registre Windows réel ;

`MainWindow::applyLoadedProject(Project)` (**privée**, appelée depuis `loadProject()` et réutilisée telle quelle) applique un projet déjà construit sans passer par `QFileDialog` — accessible aux tests via `friend class MainWindowTest` (déclaré dans `main_window.hpp`, `MainWindowTest` vit dans `openstitch::desktop`), pas via une méthode publique ajoutée uniquement pour les tests (revue corrective, voir plus bas) ; `QAction::setObjectName(...)` sur une poignée d'actions (`action_undo`, `action_redo`, `action_deleteRegion`, `action_createStitch`) pour les retrouver par `findChild` sans dépendre du texte traduit. Aucun membre n'est rendu public au-delà de l'API de production existante. Chaque suite est un exécutable `QTest` (`QTEST_MAIN`), exécuté par `CTest` avec `QT_QPA_PLATFORM=offscreen` (propriété `ENVIRONMENT`) et le dossier bin de Qt ajouté au `PATH` (propriété `ENVIRONMENT_MODIFICATION` — Qt n'est pas déployé à côté de chaque exécutable de test comme il l'est pour `openstitch` via `windeployqt`). Les objets métier non enregistrés comme type Qt (`ObjectId`, `RegionId`, `StitchParams`) sont observés par connexion directe (lambda) plutôt que par `QSignalSpy`, pour ne pas avoir à leur ajouter `Q_DECLARE_METATYPE` (le cœur ne dépend jamais de Qt) ; `QSignalSpy` reste utilisé pour les signaux à types Qt natifs (`QPointF`, `QRectF`).

Matrice couverte

Composant	Scénario testé	Résultat métier vérifié
CanvasView	zoom avant/arrière, <code>fitCanvas</code>	échelle (<code>pixelsPerMm</code>) change dans le bon sens, <code>viewChanged</code> émis
CanvasView	clic hors mode recadrage	<code>canvasClickedMm</code> émis avec la position scène exacte
CanvasView	mode recadrage actif	clic simple n'émet plus <code>canvasClickedMm</code>
CanvasView	glisser-rectangle en mode recadrage	<code>cropSelectedMm</code> émis avec le rectangle réellement dessiné (régression, voir plus bas)
NodeHandleItem	glisser une poignée	callbacks <code>onMoved/onReleased</code> reçoivent la position scène exacte, l'item a bougé
NodeHandleItem	clic sans glisser	<code>onReleased</code> appelé une fois, position inchangée
DocumentPanel	<code>refresh(project)</code>	listes Objets/Régions peuplées, id stocké (<code>Qt::UserRole</code>) correct
DocumentPanel	sélectionner une ligne Objets	<code>embroiderySelected</code> émis avec le bon <code>ObjectId</code>
DocumentPanel	<code>syncSelection</code> (retour canevas -> panneau)	ne réémet aucun signal (évite la boucle de sélection)
PropertiesPanel	<code>showEmbroidery</code>	formulaire peuplé aux bonnes valeurs, aucun <code>paramsEdited</code> pendant la construction
PropertiesPanel	éditer un champ du formulaire	<code>paramsEdited</code> émis avec le champ modifié à jour, les autres champs préservés
PropertiesPanel	changer de sélection (<code>showInfo</code> après <code>showEmbroidery</code>)	l'ancien formulaire est bien détruit (pas de contrôles fantômes)
MainWindow	clic canevas sur un objet vectoriel	<code>DocumentPanel</code> (liste Objets) et <code>PropertiesPanel</code> (spin boxes) se synchronisent sur le point de contour rattaché ; <code>action_createStitch</code> s'active
MainWindow	sélection région (liste) puis sélection objet (canevas)	<code>action_deleteRegion/action_createStitch</code> s'activent et se désactivent en opposition, selon la priorité de <code>resolveSelectedEmbroidery/onCanvasClicked</code>
MainWindow	suppression d'une région (<code>action_deleteRegion</code> , sans dialogue) puis undo/redo (<code>action_undo/action_redo</code>)	la liste Régions de <code>DocumentPanel</code> reflète la suppression et sa restauration (pas seulement <code>undoStack/_le</code> modèle) ; les actions annuler/rétablir s'activent en conséquence
MainWindow	sélection d'une broderie dans un projet, puis chargement d'un second projet (<code>applyLoadedProject</code>) réutilisant le même <code>ObjectId</code> de broderie (régression, voir plus bas)	ni <code>DocumentPanel</code> (sélection liste Objets) ni <code>PropertiesPanel</code> (formulaire) ne montrent la broderie du projet précédent tant qu'aucune sélection explicite n'a été faite dans le nouveau

`MoveNodeCommand` (la commande undo/redo réellement appliquée quand une poignée est relâchée) est déjà couverte côté cœur, sans Qt, par `tests/unit/commands/test_undo_stack.cpp` (cas « `AddVectorObject` et `MoveNode` ») — non dupliqué ici.

Revue corrective de la deuxième tranche (2026-08-01)

Deux défauts trouvés indépendamment sur les commits `d49e660/37ffc27`, corrigés sans toucher au reste du périmètre de la deuxième tranche :

1. **Seam de test dans l'API publique de production.** `MainWindow::loadProjectForTests(Project)` avait été ajouté **public** uniquement pour les tests, ce qui contredit la règle du projet (aucune API de production élargie pour le seul bénéfice des tests). Corrigé : `applyLoadedProject` (déjà privée) reste privée ; `MainWindowTest` est forward-déclarée dans `main_window.hpp` et déclarée `friend` de `MainWindow`, et vit dans `openstitch::desktop` (comme `MainWindow` elle-même) pour que le `friend` s'applique. Aucune donnée interne supplémentaire n'est exposée ; `loadProjectForTests` a été supprimée.
2. **Fuite de sélection de broderie entre deux projets.** `applyLoadedProject` réinitialisait `selectedRegion_` et `selectedObject_` mais oubliait `selectedEmbroidery_` — défaut déjà présent avant la deuxième tranche (queue de l'ancien `loadProject()`, seulement déplacée telle quelle dans `applyLoadedProject`). Si le nouveau projet réutilise le même `ObjectId` qu'une broderie sélectionnée dans l'ancien (cas réaliste : les projets de test allouent des ID déterministes à partir d'un compteur remis à zéro), `resolveSelectedEmbroidery()` la retrouvait silencieusement dans le nouveau document — l'inspecteur, la sélection dans `DocumentPanel` et la barre contextuelle affichaient alors une broderie jamais choisie explicitement par l'utilisateur. Corrigé par un `selectedEmbroidery_.reset()` ajouté à côté des deux autres réinitialisations. Test de régression : `embroiderySelectionDoesNotLeakAcrossProjectLoadWithReusedId` — vérifié en retirant temporairement la ligne corrigée : le test échoue bien sans le correctif (`objectsList(*docPanel)->currentRow()` reste à 0 et le formulaire de l'inspecteur reste peuplé au lieu de revenir à « Aucune sélection »).

Audit ciblé du reste de l'état de session réinitialisé par `applyLoadedProject` : `undoStack_.clear()`, `sequence_.reset()` et `sequenceImported_ = false` étaient déjà corrects. Les filtres d'affichage temporaires (`hiddenColors_`, `showType_`, `minAreaMm2_`) et `contextSig_` persistent délibérément d'un chargement à l'autre (préférences d'affichage de session, pas des données de document) ; aucun défaut démontré à leur sujet, donc non modifiés.

Bug découvert et corrigé par ce lot

Le test du glisser-rectangle a révélé que `CanvasView` utilisait `fromScenePoint/toScenePoint` du signal `QGraphicsView::rubberBandChanged`, qui accusent un retard d'une étape sur `viewportRect` (constaté en forçant un glisser à seulement deux événements de déplacement espacés) : le rectangle final capturait la position du glisser **précédent**, pas la position réelle au relâchement. Invisible en usage réel (le pointeur génère des événements quasi continus, l'écart est sub-pixel), mais faux pour un glisser rapide à peu d'événements (trackpad, remote desktop). Correctif : reconverter `viewportRect` lui-même (`mapToScene(viewportRect).boundingRect()`), toujours à jour. Test de régression : `cropModeRubberBandEmitsCropSelectedMm`.

Obstacles de testabilité identifiés

- **MainWindow reste un god-object** : constructeur de ~250 lignes construisant menus/docks/barres d'outils/panneaux en bloc, logique de sélection/filtre/simulation imbriquée dans des lambdas privées (`objectPassesFilter`, `embroideryCentroid...`). La deuxième tranche (voir ci-dessous) n'a **pas** décomposé la classe — elle a percé trois seams minimaux (`QSettings` injectable, `applyLoadedProject` accessible par `friend class MainWindowTest`, `objectName` sur quelques actions) qui suffisent à instancier `MainWindow` et à observer son état sans dialogue modal. Toute action qui ouvre une boîte de dialogue (`QFileDialog`, `QMessageBox`, `QInputDialog`, `QColorDialog`, `QMenu::exec` — `openImage`, `saveProject`, `segmentImage`, `onCanvasContextMenu...`) reste hors de portée d'un test `QTest` `offscreen` automatisé.
- **Sélection rectangulaire d'objets métier absente** : le `RubberBandDrag` actuel de `CanvasView` ne sert **que** le mode recadrage (`cropSelectedMm` → recadre l'image). Il n'existe aucun mécanisme qui sélectionne les objets vectoriels/de broderie contenus dans un rectangle glissé — fonctionnalité demandée par l'utilisateur mais pas encore codée (hors périmètre de la deuxième tranche, sur consigne explicite). Rien n'a donc pu être testé « avec succès » sur ce point ; c'est une fonctionnalité **manquante**, pas un test manquant.
- **Déplacement global d'un objet absent** : seul le déplacement d'un nœud individuel existe (`NodeHandleItem` → `MoveNodeCommand`, testé). Il n'y a pas de commande ni d'interaction pour translater un objet entier (tous ses nœuds à la fois) ; pas de `MoveObjectCommand` dans `libs/commands` (hors périmètre de la deuxième tranche, sur consigne explicite).

Deuxième tranche (MainWindow — actions/menus, synchronisation, undo/redo)

Les trois points de la roadmap ci-dessous marqués **fait** ont été couverts sans instancier `MainWindow` via un scénario nécessitant un dialogue modal : mutation via `action_deleteRegion` (`RemoveRegionCommand`, aucune boîte de dialogue) sur un document minimal construit directement en mémoire (`applyLoadedProject`, pas de fichier `.osp` ni d'image chargée depuis le disque).

Roadmap (scénarios non automatisables aujourd'hui)

But visé par l'utilisateur, dans l'ordre de valeur/risque :

1. **Sélection rectangulaire d'objets** (prérequis produit avant tout test, toujours non codé) : ajouter un mode canevas dédié (distinct du recadrage) qui émet les `ObjectId/RegionId` sous le rectangle ; puis test `QTest` reprenant le même schéma que `cropModeRubberBandEmitsCropSelectedMm`.
2. **Déplacement global d'un objet** (toujours non codé) : `MoveObjectCommand` (cœur, `Catch2`, translation de tous les nœuds + undo/redo — même schéma que `MoveNodeCommand`), puis interaction canevas (glisser l'intérieur d'un objet sélectionné) testée en `QTest` comme `NodeHandleItem`.
3. **fait — Synchronisation sélection canevas ↔ liste ↔ inspecteur** : clic canevas (`CanvasView::canvasClickedMm` appelé directement, sans simulation souris pixel-exacte — déjà couverte par `test_canvas_view.cpp`) → `MainWindow::onCanvasClicked` → `resolveSelectedEmbroidery` (logique de priorité broderie > vecteur, extraite de la triple duplication `updateContextToolbar/updateInspector/syncDocumentSelection`) → `DocumentPanel::syncSelection` + `PropertiesPanel::showEmbroidery`.
4. **fait — Undo/redo restaure modèle et UI** : le modèle était déjà couvert (`Catch2`, `test_undo_stack.cpp`) ; on vérifie maintenant que la liste `DocumentPanel` (pas seulement `project_undoStack`) se **rafraîchit** après un undo puis un redo déclenchés depuis `action_undo/action_redo`.
5. **fait — Menus/actions selon l'état** (`updateActions`) : `action_createStitch` (dépend de la sélection d'un objet vectoriel) et `action_deleteRegion/région` (dépend de la sélection d'une région **et** d'une segmentation présente) togglent correctement, y compris quand une sélection en chasse une autre (priorité canevas > liste, cf. `onCanvasClicked`).

Types de vérifications notables

- **DST aller-retour** sur tous les deltas de -121 à $+121$, déterminisme octet à octet.
- **Tatami** : invariant « aucune couture ne traverse le trou » d'un anneau, peu de déplacements ; et « aucune couture hors région » sur une forme concave en **L** échantillonnée à cinq angles (filet anti-débordement).
- **Auto-numérisation** : une bande fine devient un **tatami** par défaut (satin naïf désactivé), et un satin uniquement si `use_naive_satin` est activé.
- **Auto-satin géométrique** (`build_satin_columns`) : formes simples produisent une colonne, milieu des barreaux **intérieur à la région**, Y/T → plusieurs sections portant un identifiant de jonction commun, anneau fin → quatre sections formant un cycle, cercle plein/forme large refusés, déterminisme des rails et de la topologie ; SVG dans `tests/golden/auto-satin/`.
- **Extension des bouts ouverts** (`extend_tip`, mission « auto-satin béton ») : un bout arrondi (capsule) atteint le bord réel (longueur bout-à-bout $45\text{ mm} \pm 1\text{ mm}$, contre $\sim 38,8\text{ mm}$ avant) ; un bout carré (rectangle) aussi (40 mm contre $\sim 34,9\text{ mm}$) ; les jonctions d'un réseau Y restent géométriquement identiques bascule activée/désactivée (seuls les bouts ouverts s'allongent) ; aucun barreau dégénéré après extension ; déterminisme ; le bascule `extend_open_ends=false` restaure l'ancien comportement.
- **Ancrage des jonctions** (`trim_and_anchor_junction_end`, mission « auto-satin béton ») : sur un Y symétrique (résolution où les 3 branches survivent), les 6 extrémités de rail touchant la jonction se regroupent en **exactement 3 sommets partagés par exactement 2 rails chacun** ; sur un T (topologie asymétrique, 2 encoches réelles pour 3 branches) **aucune collision** (jamais 3+ rails au même sommet) et aucun barreau dégénéré ; aucune dérive de largeur résiduelle ($> 1,2\times$ la médiane) sur aucune section ; déterminisme ; le bascule `anchor_junction_ends=false` restaure la dérive (régression reproduite pour prouver que le bascule agit réellement).
- **Formes dessinées à la main** (`rectangle_path/ellipse_path/polygon_path/freeform_path`, `tests/unit/geometry/test_primitives.cpp`) : rectangle d'aire exacte quels que soient les coins fournis (ordre indifférent) ; ellipse à 4 nœuds lisses, aire aplatie proche de $\pi \cdot r_x \cdot r_y$ ($< 1\%$ d'écart), cercle quand les côtés sont égaux ; polygone reliant les sommets dans l'ordre, chemin vide si moins de 3 sommets ; forme libre (`freeform_path`) simplifie un tracé bruité (20 points colinéaires par bord d'un carré) en un contour exploitable d'aire correcte, chemin vide si moins de 3 points bruts OU si le tracé simplifié retombe sous 3 sommets (points colinéaires) ; cas dégénérés (largeur/rayon nul) sans crash ; déterminisme. Côté UI (`tests/unit/desktop/test_main_window.cpp`) : les quatre outils créent un `VectorObject` annulable (undo/redo exact) ; un cadre trop petit ne crée rien ; le tracé d'un polygone accumule les sommets (aperçu élastique tenu à jour), se ferme au double-clic (≥ 3 sommets, sinon abandon propre) sans ajouter de sommet fantôme pour la seconde pression du double-clic, et s'annule proprement en changeant d'outil ; le tracé d'une forme libre accumule les points captés pendant le glisser (aperçu tenu à jour), se ferme au relâchement du bouton (< 3 points → message clair, rien créé) et s'annule proprement en changeant d'outil ; **Maj = cercle sur l'ellipse est désormais directement testable** (modificateurs passés en paramètre du signal `boxDrawnMm`, capturés sur l'évènement Qt réel plutôt que relus depuis un état clavier global non fiable en QTest offscreen — correctif de revue, l'ancien test avait dû être écarté à la création de la fonctionnalité).
- **Persistance des réseaux satin** : aller-retour exact de l'index/nombre de sections et des jonctions, ancien satin sans topology encore lisible, index hors réseau refusé proprement.
- **Appariement rail gauche/rail droit (audit rails, 2026-08-01)** : fixtures géométriques dédiées (ruban droit, S, coude 90° , largeur variable, cas combiné) mesurant croisements, monotonie sur les deux rails, angle fil/normale locale, continuité angulaire, régularité de densité, déterminisme — avec comparaison chiffrée à l'ancien algorithme (réplique isolée, test uniquement). Voir `docs/source/satin.md § Correction de l'appariement`.
- **Satin par barreaux** (`fill_satin_columns`) : espacement **médian régulier** (colonne droite), barreaux **traversés exactement**, rails de longueurs différentes, repli sur `fill_satin` si < 2 barreaux, déterminisme.

- **Édition des guides satin** : QTest headless sur le clic de sélection, l'ajout au plus grand intervalle, la suppression avec plancher de deux guides, le glisser d'une extrémité projetée sur son rail, le verrouillage visible d'un guide terminal de jonction et les parcours undo/redo ; tests cœur sur l'espacement minimal, les ordres de guides croissant/décroissant et la détection géométrique d'une jonction malgré des barreaux inversés. La collecte multi-section est testée sur l'isolation par objet source, l'ordre déterministe et le refus d'un réseau incomplet ; la commande coordonnée vérifie undo/redo et l'absence de mutation partielle avec cible obsolète ou dupliquée. Un parcours QTest clique réellement le guide structurel, ajoute un guide interne aux deux branches dans leur intervalle adjacent (fixture asymétrique 0/2/10 mm), puis vérifie l'undo/redo unique sans temporisation ni comparaison de pixels. Deux tests cœur couvrent aussi l'ordre inverse, le guide non structurel et l'intervalle trop court.
- **Groupe de guides liés — édition atomique** (`satin_linked_guides`, `move_satin_guide_group`, `RemoveSatinGuidesCommand`) : tests cœur sur l'énumération triée et déterministe d'un groupe, l'isolation par réseau (même `link_id` numérique réutilisé dans un `source_vector` différent, aucune fuite), le refus d'un `link_id` dupliqué dans une même section (donnée corrompue), le rejet total hors de l'intervalle admissible (marge de densité), le rejet d'une section glissée hors du groupe, le rejet d'un guide qui a perdu l'adjacence à sa jonction (topologie modifiée entretemps), le rejet d'un réseau incomplet, de membres liés à des jonctions différentes et d'une progression croisée entre rails, ainsi que le calcul exact du delta normalisé partagé sur deux sections aux géométries de rail **différentes** (longueurs 10 mm/20 mm) — vérifie que le résultat de chaque section vient bien de sa propre géométrie (2 mm vs 6 mm) et non d'une coordonnée recopiée. Côté commandes : suppression de groupe tout-ou-rien avec undo/redo exact, rejet d'une cible obsolète/dupliquée/vide, réinsertion correcte de plusieurs guides dans un même objet (ordre descendant à la suppression puis ascendant à la restauration), et un test de revue corrective vérifiant qu'un document modifié entre `apply()` et `revert()` (section ayant perdu sa géométrie satin) interdit une restauration partielle — la première section reste elle aussi non restaurée même si sa propre cible était encore valide. Côté UI : Maj+glisser une extrémité d'un guide lié déplace les deux sections d'un coup (même résultat exact dans les deux, géométrie identique) en une seule commande annulable, supprimer un guide lié supprime le groupe entier en une seule commande, et un glisser SANS Maj sur le même guide reste local (seule la section glissée bouge) — non-régression du comportement d'édition d'angle pré-existant. Un Maj+clic sans mouvement ne crée aucune commande undo. Aucune temporisation, aucune comparaison de pixels.
- **Satin — finitions (Lot 3)** : points courts (inset modifie le rail intérieur, remove réduit les pénétrations), split (staggered ≠ ligne centrale, jitter déterministe), terminaisons (taper réduit la largeur au bout sans l'annuler) ; aller-retour `.osp` des modes.
- **Satin — sous-couches + compensation (Lot 4)** : center/edge/zigzag = 4 passes distinctes ordonnées ; pull élargit **un seul côté** (asymétrique) ; push étend le bout ; le générateur tague les sous-couches `Underlay` et le satin `TopStitch` ; aller-retour `.osp`.
- **Satin — entrée/sortie + lock (Lot 5)** : `lock_stitches` (None → vide, les autres progressent dans l'axe de couture et restent bornés) ; les locks sont émis en passe `Lock` en **exactement deux groupes** (début/fin) ; le point d'entrée fait démarrer la couture à la bonne extrémité ; aller-retour `.osp` des champs Lot 5 (lock, entrée/sortie).
- **Satin — routage multi-colonnes (Lot 6)** : `route_columns` (liste vide → plan vide, une colonne → départ sans saut, réordonnancement minimisant le déplacement, orientation par l'extrémité proche, liaison courte → trajet caché / longue → saut) ; une jonction commune admissible prime sur une extrémité plus proche sans relation topologique, tandis qu'une jonction trop distante est ignorée ; le test de génération vérifie que `SatinParams.topology` atteint le plan. Un groupe adjacent n'émet qu'un saut initial (le reste cousu en passe `Travel`), un groupe éloigné **ou sans jonction commune** conserve ses sauts.
- **Routage — trajet caché non validé (correctif, revue « auto-satin béton » (suite))** : avant correction, seule la distance (`underpath_max`, 8 mm) décidait d'un trajet caché, sans jamais vérifier `step.junction` — deux colonnes proches (5 mm) mais **sans aucune jonction commune** auraient été cousues en trajet caché à travers un espace non garanti couvert. Quatre tests verrouillent le correctif (seuil à deux paliers, `underpath_max` si `step.junction` a une valeur,

`underpath_max_without_junction` sinon) : liaison à 5 mm sans jonction → **saut** (avant : trajet caché, défaut) ; quasi-contact à 1 mm sans jonction → trajet caché toujours toléré (seuil strict non pénalisant) ; la même liaison à 5 mm **avec** jonction commune validée reste un trajet caché jusqu'à 8 mm (jonction non pénalisée par le nouveau seuil strict) ; au niveau génération, deux sections à 5 mm sans jonction ne produisent plus aucun point Travel (0 trajet caché, 2 sauts). Aucun test préexistant modifié : tous leurs écarts étaient déjà soit quasi-contact, soit jonction-justifiés, soit très supérieurs à 8 mm.

- **Audit satin adversarial (2026-08-02)** : 11 nouveaux tests verrouillant les 8 défauts trouvés par 4 audits indépendants du pipeline complet.

- `tests/unit/auto_satin/test_pipeline.cpp` : jonction d'une cross correctement détectée à degré 4 réel dans le graphe élagué (comptage d'arêtes incidentes, pas seulement le type déclaré du nœud) ; réseau y conserve ses 3 branches, aucune arête `from == to` (auparavant : une branche entière disparaissait) ; nœud isolé d'un circle (squelette réduit à un point) apparaît dans `removed_branches` au lieu de disparaître silencieusement.

- `tests/unit/auto_satin/test_columns.cpp` : nouvelle fixture "h" (`shapes.cpp` — deux barres verticales reliées par un pont horizontal, 2 jonctions dont une arête Jonction-Jonction) — le pont est bien converti en colonne (5 colonnes au total), aucune collision aux deux jonctions, aucun barreau dégénéré ; jonction à 4 branches (cross, désormais correctement détectée) — aucune collision de sommet reflex entre les 4 colonnes.

- `tests/unit/stitch/test_satin.cpp` : `default_rungs` produit au moins deux barreaux sur une colonne simple (vide sur des rails dégénérés) et débloquent un réglage ignoré par `fill_satin` (`cap_end = Tapered` sans effet sans barreaux, effectif avec) ; rétraction `push_start` demandée à -50 mm reste bornée près de son origine (pas d'auto-croisement) ; lock sur une colonne de 0,3 mm avec `lock_length` par défaut (0,8 mm) reste dans la largeur réelle, quel que soit le type.

- `tests/unit/stitch/test_routing.cpp` : un écart brut de 900 µm (sous le seuil sans jonction) devient un saut une fois le `push_end` (rétraction de 700 µm) réellement appliqué pris en compte (écart réel 1,6 mm, au-dessus du seuil) — plus aucun trajet caché fondé sur un point non réellement cousu. Aucun test préexistant modifié dans cette revue (seules des fixtures/tests nouveaux) — voir `docs/source/satin.md` pour le détail de chaque défaut et son correctif.

- **Tatami avancé (Lot 7)** : sous-couche de contour rentrée dans la forme ; sous-couche parallèle espacée ; underpath caché convertit au moins un saut en trajet cousu **sans jamais** traverser le trou (invariant préservé) et ne l'augmente jamais ; déterminisme ; le point d'entrée oriente le démarrage ; le générateur tague la sous-couche `Underlay` et le remplissage `TopStitch` ; aller-retour `.osp` des champs Lot 7.

- **Retouches manuelles — cœur pur (Lot 8.0, corrigé)** : fingerprint (FNV-1a 64 bits) stable pour une vue brute identique, sensible à la position, au type de commande, à la passe et à l'ordre pris séparément ; `raw_slice` reconstruit correctement un objet sur une séquence contiguë et sur une séquence entrelacée (synthétique, et via un routage satin réel à trajet caché) ; `apply_manual_overrides` valide chaque champ d'un override indépendamment sur les entrées `TopStitch` : `moved_to` exige une cible `Stitch`, `forced_type` et `trim_after` acceptent une cible `Stitch` ou `Jump` (`Stitch↔Jump` bidirectionnel — correctif : `is_overridable_entry` n'acceptait auparavant qu'une cible `Stitch`, rendant `Jump→Stitch` impossible malgré le cadrage et le rapport initial de Lot 8.0 ; aucun test ne couvrait cette direction), un champ invalide pour sa cible est ignoré sans rejeter les autres champs du même override ; laisse un objet `Dirty` strictement inchangé (par empreinte **et**, indépendamment, par compteur de points), isole chaque objet retouché des autres, résout déterministement les doublons d'overrides (dernière entrée du vecteur, en bloc), les overrides vides et les index hors bornes ; déterminisme du résultat (séquence patchée et liste d'objets `Dirty` triée) vérifié sur deux exécutions indépendantes. Aucune commande `undo/redo`, aucune persistance `.osp`, aucune UI dans ce sous-lot (cf. `docs/lot8-manual-editing-design.md`).

- **Retouches manuelles — commandes, persistance, point d'entrée unique (Lot 8.1)** : les 4 commandes (`MoveStitchPointCommand`, `SetStitchPointTypeCommand`, `SetStitchTrimCommand`, `DiscardOverridesCommand`) valident `ObjectId/base_index/état` avant toute mutation, refusent un objet déjà `Dirty` sans y toucher, restaurent exactement l'entrée précédente au `revert` (y compris la

transition `Clean` → `ManuallyEdited` et son annulation), et partagent une seule entrée d'`overrides` quand deux commandes ciblent le même `base_index` (chaque `revert` ne restaure que son propre champ). `SetStitchTrimCommand(false)` sur un index sans override existant est un no-op exact (aucune entrée vide créée, aucune transition `Clean` → `ManuallyEdited` fantôme) ; si elle efface le dernier champ effectif d'une entrée préexistante (`trim_after` était son seul champ posé), l'entrée est retirée plutôt que laissée vide, et `revert` la réinsère exactement (`end_stitch_edit` gère ce cas). Persistance `.osp v3` : `overrides/editedFingerprint/editedPointCount` survivent à un aller-retour exact, y compris un `editedFingerprint` **au-delà de 2³¹** (aucune perte de bits via `double` — `nlohmann` conserve un entier positif en `number_unsigned`) ; un fichier `v1/v2` sans ces champs charge un objet `Clean` ; un index négatif/non entier ou au-delà de `numeric_limits<size_t>::max()`, une coordonnée au-delà d'un `int32`, un compteur au-delà d'un `uint32`, un type de point inconnu ou une entrée sans `index` renvoient une erreur `InvalidFile`, jamais une troncature silencieuse ou un plantage. **Revue corrective (2026-08-01)** : `overrides` doit être un tableau JSON (objet/scalaire refusé) ; un tableau non vide exige la présence **explicite** d'`editedFingerprint/editedPointCount` (plus de valeur zéro implicite sur un document malformé) ; un tableau vide accompagné de ces mêmes clés est normalisé en `Clean` sans erreur (métadonnées orphelines ignorées, non significatives quand `overrides` est vide) ; un `index` en double dans le tableau est refusé (`InvalidFile`) au lieu d'être fusionné champ à champ (le cadrage initial l'autorisait, décision revue — aucun producteur réel n'écrit jamais deux entrées pour le même index, cf. `docs/lot8-manual-editing-design.md §4`) ; une entrée sans aucune modification effective (ni pos, ni type, ni `trimAfter: true`) est également refusée. `effective_sequence` (enchaîne `generate_sequence` puis `apply_manual_overrides`) est vérifiée identique au brut pour un objet `Clean`, patchée pour un objet `ManuallyEdited`, déterministe sur deux appels indépendants, et propage sans plantage une erreur de génération (projet sans objet visible). Une garde CTest structurelle (`check_no_raw_sequence_bypass`) fait échouer la suite si un nouveau site de production appelle `generate_sequence()` sans passer par `effective_sequence()`. Toujours aucune UI dans ce sous-lot.

- **Retouches manuelles — mode d'édition des points, UI desktop (Lot 8.2)** : côté cœur, `is_movable_point` (prédicat d'éligibilité exporté, plus de duplication dans l'UI), `edit_view/classify_all_edit_states` (état dérivé
- vue brute/empreinte/compteur pour un objet) et `refresh_context` (un seul `generate_sequence` interne pour produire à la fois la séquence effective — contenu **identique** à `effective_sequence`, mêmes deux passes —, les états par objet et la vue d'édition d'une cible optionnelle) sont vérifiés cohérents entre eux et propagent sans plantage une erreur de génération. Côté UI (QTest, offscreen, événements souris réels sur poignées visibles à deux niveaux de zoom, `QSignalSpy`) : activation/désactivation du mode selon la sélection et l'état `Dirty` (aucun second signal `toggled` émis par une sortie forcée, garde-fou vérifié via `QSignalSpy`) ; glisser une poignée exécute **une seule** `MoveStitchPointCommand` (undo ramène `canUndo()` à faux) dont l'annulation/le rétablissement restaurent exactement la même entrée d'override ; un clic sans déplacement ne crée aucune commande ; un changement de document pendant la fenêtre où la commande différée (`QTimer::singleShot(0)`, nécessaire pour ne pas détruire sa propre poignée) est en file d'attente fait abandonner cette commande sans muter le nouveau projet (régression du compteur `documentGeneration_`) ; abandonner les retouches d'un objet `Dirty` (confirmation auto-acceptée) est annulable et restaure exactement l'état antérieur ; un objet dont le nombre de points déplaçables dépasse le seuil d'affichage refuse proprement l'activation, avec message explicite en barre de statut, plutôt que de laisser le mode actif sans aucune poignée.
- **Tatami — correctif contacts sommet** : `segment_stays_in_region` détecte un connecteur **parfaitement vertical** traversant un trou en losange de part en part en touchant exactement ses deux sommets (défaut qu'une version antérieure laissait passer, faute de sonder l'intérieur pour un petit écart en x) ; même vérification avec **deux trous** (le couloir entre eux reste cousable) et sur le **coin rentrant** d'une forme en L (contact sommet vers l'encoche rejeté) ; non-régression : un suivi de bord colinéaire reste cousu ; `fill_tatami` sur un trou en losange ne produit aucun point cousu à l'intérieur.
- **Tatami — politique sûre de `tatami_underlay`** : un retrait de contour (`underlay_inset`) impossible (pièce trop petite) ne produit **aucune** sous-couche de contour (jamais de repli silencieux sur le bord brut) ; un retrait **explicitement nul** longe bien le bord brut (intention distincte).

- **Running** : espacement par longueur d'arc (cercle), coins préservés, courbes Bézier suivies, résultats déterministes.
- **Undo/redo** : « undo total = état initial », restauration exacte des labels ; aller-retour exact des commandes d'objet (`SetFillAngle`, `SetStitchType`, `SetStitchParams`, `ConvertFillsToTatami`, `SetCanvas`).
- **Projet** : aller-retour complet (image, ops, **cadre**, segmentation, tangentes, params).

Ajouter un test

Ajoutez un `TEST_CASE` (nom **ASCII**) dans le `test_*.cpp` du module, ou un nouveau fichier référencé dans le `CMakeLists.txt` du dossier de tests. Les golden SVG ne sont **jamais** réécrits par les tests : ils se régénèrent explicitement via `openstitch-cli stitchdebug`.

Pour l'UI desktop : ajoutez une fonction de test dans une suite existante de `tests/unit/desktop/`, ou un nouveau fichier + `openstitch_add_qt_test(...)` dans son `CMakeLists.txt`. Chaque test doit vérifier un résultat métier observable (modèle, sélection, enabled/checked, valeur d'un signal) — pas seulement l'absence de crash — et rester déterministe (pas de `sleep`, pas de comparaison de pixels, pas de dépendance à la résolution/au thème).

Implémentation associée

- `tests/` (arborescence complète), `tests/unit/*/CMakeLists.txt`.
- `tests/unit/desktop/` — tests Qt (QTest) de la couche UI.
- `apps/desktop/CMakeLists.txt` — bibliothèque `openstitch_desktop_widgets`.
- `CMakeLists.txt` (racine) — `enable_testing`, `Catch`.

Guide de contribution

Public : contributeur.

Flux

1. Cloner le dépôt et créer une **branche** dédiée (le développement se fait sur main en local ; ne pas committer directement sans branche pour une fonctionnalité).
2. Compiler et exécuter les tests (`ctest --preset msvc-debug`).
3. Ajouter/mettre à jour les **tests** couvrant la modification.
4. Formater (`clang-format`) et vérifier les warnings.
5. Rédiger un message de commit clair (voir ci-dessous).

Messages de commit

Le dépôt utilise des messages descriptifs en français, terminés par une ligne de co-auteur. Exemple observé :

```
Corrige le débordement des remplissages hors des regions
<description détaillée>
Co-Authored-By: ...
```

Style C++

- Respecter `clang-format` et les warnings élevés.
- Types forts pour les unités ; pas de logique métier dans les widgets Qt.
- Toute mutation du document via une **commande** (undo/redo).
- En-tête SPDX Apache-2.0 en tête de fichier.

Ajouter une dépendance

- La déclarer dans `vcpkg.json` (ou documenter l'installation si hors `vcpkg`).
- **Vérifier la licence** : compatible Apache-2.0, permissive de préférence. **Aucun code GPL** ne peut être copié dans ce dépôt (voir *Licences*).
- L'encapsuler derrière une interface interne (ne pas exposer ses types).
- La documenter dans `THIRD_PARTY_LICENSES.md`.

Signaler un bug / proposer une fonctionnalité

Information non déterminée dans le dépôt : aucun gestionnaire d'issues public n'est configuré (dépôt local). En interne, documentez le problème avec un cas minimal reproductible et, si possible, un test qui échoue.

Documentation

Toute fonctionnalité visible doit être reflétée dans `docs/source/` puis le PDF régénéré (voir `docs/README.md`). Les affirmations doivent rester **vérifiables** dans le code (section « Implémentation associée »).

Implémentation associée

- `clang-format`, `THIRD_PARTY_LICENSES.md`, `LICENSE`.
- `docs/README.md` — régénération de la documentation.

Dépannage

Public : utilisateur, développeur. Pour chaque problème : symptôme, cause probable, diagnostic, solution.

Compilation

Symptôme	Cause probable	Solution
« Qt6 introuvable »	QT_ROOT absent/incorrect	Pointer sur <code>... \msvc2022_64</code> , reconfigurer
Erreur toolchain vcpkg	VCPKG_ROOT absent, bootstrap manquant	Définir la variable, relancer <code>bootstrap-vcpkg.bat</code>
DLL manquante au lancement	déploiement Qt/OpenCV incomplet	Lancer depuis le dossier Debug/Release (windeployqt a copié les DLL)
LNK1168 à l'édition de lien	l'application tourne encore	Fermer <code>openstitch.exe</code> puis recompiler
OpenCV très longue à compiler	premier configure vcpkg	Normal ; les runs suivants utilisent le cache

Usage

Symptôme	Cause probable	Diagnostic	Solution
Image non chargée	format/fichier corrompu	<code>openstitch-cli info fichier</code>	Réexporter l'image en PNG
Segmentation « incorrecte »	trop/peu de couleurs, petites régions	carte des régions	Ajuster N et la taille min, fusionner/supprimer
Aucun point généré	objet sans géométrie, tous invisibles	barre d'état / Statistiques	Vérifier qu'un objet vectoriel source existe et est visible
Satin impossible	rails non constructibles / forme non allongée	message à la création	Préférer un tatami
Remplissage qui déborde	trou perdu à la vectorisation	zoom sur la zone	Corrigé : tolérance adaptative + routage ; re-numériser
Motif hors cadre	taille physique trop grande	règles / cadre rouge	Réduire la taille à l'import, ou déplacer
Points trop longs	longueur de point élevée	Analyse (F5)	Réduire <code>stitch_length</code>
Export DST refusé	séquence vide	message	Générer des points d'abord
Fichier DST « vide »	aucun objet visible	<code>openstitch-cli stats</code>	Vérifier les objets et l'ordre
Couleurs incorrectes	pas de palette de fils	—	Fonctionnalité prévue ; RGB par objet en attendant
Projet impossible à ouvrir	<code>.osp</code> corrompu / version inconnue	message d'erreur	Vérifier le fichier ; version de schéma supportée = 1
Performances faibles	très gros motif	—	Corrigé : rendu deux couches, pastilles limitées
Crash	entrée inattendue	—	Aucune entrée utilisateur ne devrait faire planter ; signaler avec un cas minimal

Logs

Le logger (spdlog) écrit sur la **sortie standard/erreur** de la console. Lancer la CLI ou l'application depuis un terminal pour voir les messages. *Information non déterminée* : aucun fichier de log persistant n'est écrit par défaut.

Implémentation associée

- `libs/core/src/log.cpp` — initialisation du logger.
- `libs/*/src/*` — les erreurs renvoyées via `Result<T>` avec un message montrable.

Limitations et état des fonctionnalités

Public : tous. Ce chapitre distingue explicitement l'état de chaque fonctionnalité, vérifié dans le code.

Tableau des fonctionnalités

Functionalité	État	Interface	Module	Tests	Limitations
Import PNG/JPEG/BMP/TIFF	Implémenté	Filetier	image	oui	—
Prototypage non destructif	Implémenté	Image	image	oui	pas de resize
Segmentation CIE L*a*b* et régions	Implémenté	Segmentation	segmentation	oui	4-couleurs
Édition de régions	Implémenté	Segmentation	segmentation	oui	—
Vecteurisation	Implémenté	Segmentation	vectorization	oui	—
Formes dessinées à la main (rectangle/cercle/polygone/ligne libre)	Présent - testé	palette d'outils	geometry/desktop	QTest	pas de capture d'écran manuelle possible dans cet environnement de dev
Édition de résultats	Partiel	canvas	document	—	déplacement stat
Édition d'objets brodee	Implémenté	Inspecteur/outil droit	commands	oui	changer le type et tous les paramètres après création - orientation à la poignée
Interface (thème, panneau, workflow, wordview)	Implémenté	desktop	desktop	(vue)	thème dark/light + désable, inspecteur, panneau Document, workflow, état d'accueil, persistance UI (QSettings)
Paire de crochets/broche	Implémenté	Broderie	stitch_generation	oui	—
Tatami	Présent - testé - SVG	Broderie	stitch_generation	oui	scanline + routeur ; sous-couches (contour + parallèle, underpath caché, entrée (Lot 1) ; orientation éditée
Satin (génération par barreau)	Présent - testé - SVG	Broderie / inspecteur	stitch_generation	oui	FILL_SATIN, columns sections, placement perpendicular, guides sélectionnables (couteaux/supprimables/déplacables avec endroind, points courts / saut / terminations (Lot 3), sous-couches [contour/edge/trim]) + compensation multi-pass (Lot 4, passes distinctes), lock + entrelacé/sortie (Lot 5), multiple multi-colonnes (Lot 6)
Module de passes	Présent - testé	(générateur)	stitch	oui	SEITCHPASS par commande (UndermyTopStitchTravelLockManual) affichage/toggle par point dans l'UI à voir
Auto-satin (paire-barreau)	Présent - testé - SVG	Auto + Broderie ■ Convertir en satin	auto_satin	oui	formes simples, YTT multi-actions, crée 4 à 6 branches (jonction degré 4 connectement détectée) et anneau fin en 4 sections bouts ouverts étendus jusqu'au bord net + bouts de jonction ancrés sur les segments reflex du contour (mission « auto-satin béton ») ; pont 2-jonction-2jonction (ex. un "H") converti en colonne + carcé plethiforme large refusés
Classification auto des régions	Expérimental	Segmentation	autodizig	oui	bandes fines → motif topologique par défaut (use_auto_sat:n) ; reflex → béton ; module natif obsolète
Filtres d'affichage / calques	Implémenté	Affichage	desktop	(vue)	affichage seulement (couleur, type, taille) image/rgbnvsw/broderie)
Ordre de couture	Implémenté	dock	optimization	oui	2-pot non implémenté
Analyse	Implémenté	Analyse	stitch_analysis	oui	pas de carte de dessin
Simulation	Implémenté	barre	desktop	—	pas de réglage de vitesse
Export/Import DST	Implémenté	Fichier/CLI	formats	oui	limites du format
Export SVG diagnostic	Implémenté	CLI	formats	oui	—
Format projet .osp	Implémenté	Fichier	project_io	oui	sauve + modifié + garde à la barre/au pas d'autosave
Cadre de broderie	Implémenté	Affichage	document	oui	taille réglable et perpendic ; rectangle simple (pas de profil/forme)
Palette de fils	Non implémenté	—	(thread_palette absent)	—	RGB par objet uniquement
Édition manuelle des points	Partiel (Lot 8.2)	canvas	desktop/commands/QTest	—	déplacement d'un point + undo/redo ; stitchJump/Trim UI restera à faire
Remplissages courbes/angles/arcs	Non implémenté	—	—	—	phivus
Profilis machine/cadres ancrés	Non implémenté	—	—	—	cadre = rectangle simple (taille réglable)
Compensation dictionnelle	Partiel	—	—	—	satin uniquement
Tachis asynchrones	Non implémenté	—	—	—	traitements synchrones

Dette technique connue

- Le compte CTest courant est **398** en Debug et Release ; éviter de figer ce nombre dans les pages d'introduction sans le mettre à jour avec la CI.
- Le satin dispose désormais d'un moteur géométrique par **squelette** (`auto_satin::build_satin_columns`, Lot 1) et d'une **génération par barreaux** (`fill_satin_columns`, Lot 2), points courts / split / terminaisons (Lot 3) et **sous-couches (center/edge/zigzag) + compensation pull/push** (Lot 4, passes distinctes), **points d'entrée/sortie + points de fixation (lock)** (Lot 5) et **routage multi-colonnes** (Lot 6 : ordre/orientation + trajets cachés). Le **tatami** reçoit ses **sous-couches (contour + parallèle)**, **l'underpath caché** et **le point d'entrée** (Lot 7). Le satin **auto naïf** reste désactivé ; le vrai moteur topologique est désormais celui essayé par défaut dans `autodigitize`.
- Les sections d'un réseau satin portent maintenant des indices et identifiants de jonction explicites, déterministes et persistés. Le routage privilégie ces jonctions quand l'écart reste compatible avec un trajet caché. La collecte déterministe des extrémités, la transaction multi-section atomique et l'ajout UI d'un guide interne dans chaque branche incidente sont présents. Les guides ainsi liés (`link_id`, scopé par (`source_vector`, `link_id`)) se déplacent et se suppriment désormais en **groupe atomique** : Maj+glisser une extrémité déplace toute l'identité logique (delta normalisé partagé, jamais de coordonnée/angle recopié entre sections) et supprimer un guide lié supprime le groupe entier, chacun via une seule commande `undo/redo` tout-ou-rien (`move_satin_guide_group`, `RemoveSatinGuidesCommand`). Un glisser SANS Maj reste un geste local (édition d'angle propre à une section, toujours possible et volontairement non propagé). Un groupe incomplet, associé à des identifiants de jonction différents ou dont la progression se croise entre les deux rails est refusé en bloc. Un Maj+clic sans déplacement ne crée aucune commande d'historique. Les guides terminaux de jonction restent verrouillés (ni déplaçables ni supprimables). Un guide lié qui perdrait son adjacence à sa jonction (un autre guide inséré entre les deux) refuse le geste de groupe plutôt que de deviner un intervalle. De plus, un retour textile vers une jonction reste rectiligne au lieu de retracer le centre d'une branche.
- **Correctif Lot 7** : `connector_invalid` ignorait les contacts sommet/extrémité et ne sondait l'intérieur d'un connecteur que si l'écart en x dépassait $2 \times \text{row_spacing}$, laissant passer un connecteur quasi vertical traversant un trou de part en part en touchant exactement ses sommets. Corrigé par une découpe paramétrique du segment à chaque intersection (`segment_stays_in_region` exposé pour test). `tatami_underlay` retombait aussi silencieusement sur le bord **brut** si le retrait de contour échouait/disparaissait ; politique sûre désormais : aucune sous-couche de contour dans ce cas. `underlay_inset` et `underlay_spacing` sont exposés dans l'inspecteur (`PropertiesPanel`).
- **Correctif « auto-satin béton » — extension des bouts ouverts** : un bout de colonne sans jonction s'arrêtait, par artefact générique de l'aminçissement (Zhang-Suen), sensiblement avant le bord réel — un embout arrondi restait entièrement hors couture (~11 % de la longueur d'une capsule de test), un bout carré retractait aussi (~2,55 mm). Corrigé (`extend_tip`, activé par défaut) : marche le long de la tangente sortante en ré-évaluant des sections transversales réelles tant qu'elles rétrécissent, fermeture par bissection (largeur plancher non nulle). Voir `docs/source/satin.md § Extension des bouts ouverts`, sources (brevet Pulse Microsystems, littérature d'élagage de squelette) citées dans ce même paragraphe.
- **Correctif « auto-satin béton » — ancrage des jonctions** : le défaut inverse du précédent. La section transversale d'une branche, calculée depuis sa seule tangente, dérive nettement en approchant d'une jonction (jusqu'à ~9,2 mm mesuré sur "y" contre ~5,0 mm nominal) car le rayon balaie le bourrelet de la confluence, pas la ceinture réelle de la branche — les sections d'un Y ne se rejoignaient pas au même point (jusqu'à ~5 mm d'écart). Corrigé (`trim_and_anchor_junction_end`, activé par défaut) : amputation de la queue instable (croissance > 10 % par rapport à la station voisine) puis ancrage indépendant de chaque rail sur le sommet reflex (concave) du contour le plus proche, avec exclusion mutuelle si les deux rails d'une branche convoient le même sommet (défaut réel trouvé sur "T", corrigé avant commit). Voir `docs/source/satin.md § Ancrage des jonctions sur les sommets reflex du contour`. La limite alors non corrigée (topologie de squelette incorrecte sur la forme "croix", nœud de degré 2 au lieu de degré 4) a depuis été corrigée — voir le correctif « audit satin adversarial » plus bas.

- **Correctif « auto-satin béton » (suite) — trajet caché non validé** : `route_columns` décidait un trajet caché (`ConnectorKind: :Underpath`, fil caché sous la broderie, sans coupe) sur la seule distance (`gap ≤ underpath_max`, 8 mm), sans jamais regarder si `step.junction` — déjà calculé juste au-dessus pour ce même pas — avait une valeur : deux colonnes satin géométriquement proches mais **sans aucun lien topologique réel** (deux lettres rapprochées, une forme en C dont les deux bouts se frôlent sans être reliés) pouvaient donc être cousues en ligne droite à travers un espace dont rien ne garantissait qu'il était couvert de tissu. Corrigé par un seuil à deux paliers : `underpath_max` (8 mm, inchangé) ne s'applique plus que si `step.junction` a une valeur — désormais fiable grâce à l'ancrage exact des jonctions ci-dessus — sinon un nouveau seuil, bien plus strict, `underpath_max_without_junction` (1,5 mm) ne tolère qu'un quasi-contact (arrondi de rasterisation, extrémités coïncidentes) ; au-delà, la liaison redevient un saut. Voir `docs/source/satin.md § Routage multi-colonnes`.
- **Audit satin adversarial (2026-08-02)** : revue systématique de tout le pipeline satin (squelette, colonnes, remplissage, guides, routage, lock), demandée explicitement par l'utilisateur (« vraiment check et recheck »). Quatre audits indépendants ont trouvé 8 défauts réels, tous corrigés et verrouillés par test :
- **skeleton_graph.cpp** : le traçage d'arête s'arrêtait sur le premier voisin rencontré au lieu de prioriser un nœud sur tout le 8-voisinage — cause racine du bug "croix" (degré 2 au lieu de 4) **et** d'un second défaut non documenté (une branche entière de "y" disparaissait avec une arête parasite en boucle sur la jonction, faute d'exclure le pixel d'origine de la trace, pas seulement le précédent). Les deux corrigés.
- **graph_cleanup.cpp** violait son propre contrat (« ne supprime rien silencieusement ») : un nœud sans aucune arête (isolé dès le départ) disparaissait sans apparaître dans `removed`. Corrigé.
- **satin_column.cpp** : une arête reliant deux jonctions (le pont d'un "H") n'était jamais convertie en colonne — zone non cousue, aucun avertissement. Corrigé (toutes les arêtes sont désormais essayées).
- **satin_column.cpp** : l'amputation de la queue instable à l'ancrage d'une jonction comparait chaque station à sa seule voisine immédiate (+10 %), insuffisant si le bourrelet décroît progressivement sur plusieurs stations (mesuré : +72 % cumulé, chaque écart local restant sous 10 %). Corrigé par un critère de décroissance stricte.
- **satin.cpp** : tout satin créé manuellement (sans barreaux) retombait sur `fill_satin`, qui n'implémente qu'un sous-ensemble de `SatinConfig` — terminaisons/split/sous-couches de bord/compensation asymétrique restaient silencieusement sans effet malgré l'inspecteur. Corrigé par `default_rungs` (barreaux par défaut, même correspondance ladder), appliqué à la création manuelle (`createSatinObject`).
- **satin.cpp** : `push_start/push_end` n'avaient aucune borne (contrairement à `pull_max`) — une rétraction excessive croisait le premier fil de la colonne sur lui-même. Corrigé (borné à la station voisine).
- **lock.cpp** : la longueur du lock n'était jamais comparée à la distance réelle disponible — sur une colonne fine, l'aiguille piquait hors de la matière cousue. Corrigé.
- **generate.cpp** : la décision de routage (trajet caché/saut) se fondait sur le milieu brut des barreaux, pas sur le point réellement cousu une fois `push_start/push_end` appliqué — pouvait réintroduire le défaut du correctif précédent par un chemin différent. Corrigé. Détails et preuves de test pour chacun dans `docs/source/satin.md`. **398 tests CTest** verts Debug/Release (383 avant + 11).
- **Formes dessinées à la main (rectangle/ellipse/polygone/forme libre)** : demande utilisateur en cours de mission « auto-satin béton » — jusqu'ici, la seule façon d'obtenir un `VectorObject` était de vectoriser une région depuis une image importée, comme dans un logiciel de digitalisation classique (Hatch et équivalents) qui permet aussi de dessiner directement une forme de base. Quatre outils dans la palette (rectangle glissé, ellipse glissée avec Maj = cercle, polygone à clics successifs fermé par double-clic, **forme libre/lasso à glisser continu, simplifiée par Douglas-Peucker à la fermeture**) créent un `VectorObject` sans région source, immédiatement sélectionné : l'utilisateur enchaîne avec les actions **Créer un...** existantes, sans nouveau chemin côté broderie. Voir `docs/source/vectorization.md § Formes dessinées à la main`. Testé par QTest (création, undo/redo, cadre dégénéré, tracé de polygone/forme libre, annulation) ; **pas de validation visuelle manuelle** dans cet environnement de développement (capture d'écran de l'application non disponible).

- **Finition « formes dessinées à la main » (2026-08-02)**, demandée explicitement par l'utilisateur (« finis d'implém celle existante » + ajout d'une forme libre). Deux volets : 1. **Défaut trouvé par revue — Maj = cercle jamais réellement testé bout en bout.** La contrainte cercle (ellipse + Maj) interrogeait `QGuiApplication::keyboardModifiers()` (état clavier global) au moment de traiter la fin du glisser — correct en usage réel, mais impossible à couvrir par un test QTest offscreen (état clavier non fiable à rejouer), donc jamais vérifié par un test qui aurait pu échouer si le comportement se cassait. Corrigé : les modificateurs sont capturés directement sur l'évènement Qt qui termine le glisser (`CanvasView::mouseReleaseEvent`) et transmis par le signal `boxDrawnMm` — plus de dépendance à un état global, testable directement en passant `Qt::ShiftModifier`. 2. **Nouvel outil « forme libre » (lasso, raccourci L)** : glisser continu capté point par point (`CanvasView::freeformPointMm/freeformStrokeFinished`), simplifié à la fermeture par la même fonction Douglas-Peucker que la vectorisation d'image (`freeform_path`, réutilise `simplify()` — aucun nouvel algorithme de correspondance). Tolérance de 0,3 mm (même ordre de grandeur que la densité satin par défaut) : lisse le tremblement de la souris sans effacer l'intention du tracé. Moins de 3 sommets après simplification → aucun objet créé, message clair (au lieu d'un échec silencieux, vérifié même quand le tracé brut a suffisamment de points bruts mais s'avère colinéaire). Voir `docs/source/vectorization.md` § *Formes dessinées à la main* pour le détail. **398 tests CTest** verts Debug/Release (398 avant + 4 nouveaux `TEST_CASE Catch2` pour `freeform_path` ; les 4 nouvelles fonctions QTest côté desktop s'ajoutent dans le même exécutable `test_main_window`, déjà compté comme un seul test CTest, sans changer ce total).
- **Correction de l'appariement rail gauche/rail droit (2026-08-01)** : l'ancien appariement (`fill_satin` sans barreaux, et l'interpolation intra-intervalle de `fill_satin_columns`) associait les deux rails par la même fraction d'abscisse curviligne appliquée indépendamment à chacun — faux dès que les rails divergent en longueur/courbure (virage, coude, largeur variable), causant éventails, quasi-croisements et densité irrégulière sur un ruban courbe ou anguleux. Remplacé par une correspondance locale monotone (« ladder », diagonale la plus courte, garde-fou anti-croisement, $O(n)$ par intervalle) — voir `docs/source/satin.md` § *Correction de l'appariement*. Limite assumée : au sommet d'un coude C0 franc (sans congé), un seul fil absorbe nécessairement une déviation angulaire importante (mitre/congé = `ShortStitchMode`, hors périmètre). Fixtures et métriques dédiées : `tests/unit/stitch/test_satin_pairing_metrics.cpp` (13 tests).
- **Revue corrective ladder correspondance, audit adverse (2026-08-01)** : fixtures délibérément défavorables (rails tête-bêche, échantillonnage très asymétrique/segments nuls, longueurs très différentes + largeur quasi nulle, épingle à cheveux $\sim 170^\circ$, barreaux désordonnés/dupliqués). Deux défauts réels corrigés : rails fournis tête-bêche (précondition non vérifiée, nœud papillon en $O(n^2)$) → détection + ré-orientation interne automatique ; barreaux dépendants de l'ordre du vecteur d'entrée (un barreau en tête mais loin le long de la colonne faisait rejeter silencieusement tous les suivants, repli sans barreaux) → tri par position projetée avant filtrage, plus fusion des barreaux quasi-dupliqués (source d'un croisement isolé près d'un virage serré, faute de garde-fou entre deux intervalles voisins). Voir `docs/source/satin.md` § *Revue corrective (audit adverse)*. Aucun chemin de production actuel n'atteint le cas tête-bêche (`rails_from_contour/build_satin_columns` garantissent déjà le même sens). L'UI édite désormais les barreaux, mais maintient leur progression monotone et leur écart minimal ; les imports désordonnés restent acceptés et triés à la génération. Ces gardes défensives restent nécessaires à la fonction de bibliothèque publique.
- Aucune validation sur machine à broder réelle.

Niveaux de validation

« Implémenté » signifie ici *présent et testé logiciellement* — pas *prêt pour la production*. Pour un logiciel de broderie, il faut distinguer :

Niveau	Signification	État du projet
Présent	le code existe	oui (pipeline complet)
Testé numériquement	les invariants logiciels passent	oui (217 tests)
Validé visuellement	les trajectoires paraissent cohérentes	partiel (SVG de diagnostic, aperçu)
Validé sur simulateur	vérifié dans un visualiseur tiers	non
Validé physiquement	broderies réelles examinées	non

Un générateur peut passer 217 tests sans produire une bonne broderie : les tests vérifient des **invariants** (pas de couture dans un trou, espacement par longueur d'arc, aller-retour DST exact...), pas la **qualité textile**.

Statut de validation

Le logiciel constitue un **prototype fonctionnel** couvrant l'ensemble du pipeline principal. Ses composants sont **testés logiciellement** et vérifiés **visuellement** (SVG de diagnostic), mais la **qualité de broderie produite n'est pas encore validée sur machine réelle**. Ne pas considérer les résultats comme prêts pour la production sans essais machine.

Implémentation associée

Voir chaque chapitre thématique et l'annexe *Audit du dépôt*.

Roadmap

Public : mainteneur, contributeur. Cette roadmap est construite à partir du code existant, des documents de conception (docs/phase0/, docs/stitch-engine-*.md) et des fonctionnalités manquantes identifiées.

Aucune date n'est annoncée.

Court terme (dette et robustesse)

Priorité immédiate après le Lot 8.2 — qualité satin et orientation guidée : le pipeline utilise maintenant le moteur topologique et représente les Y/T et anneaux comme un réseau de sections `SatinParams` rétrocompatibles. Les fixtures bande/T/anneau/tentabrode verrouillent cette base. Continuer à durcir les arcs très asymétriques (progression bilatérale et métriques de stagnation), puis ajouter plusieurs guides de direction éditables par zone satin : chaque guide relie les deux rails et impose une orientation locale, interpolée continûment entre guides. **Premier incrément livré** : affichage des guides, déplacement indépendant des deux extrémités avec projection/validation monotone, undo/redo et test Qt de glisser. **Deuxième incrément livré** : sélection explicite sur le canevas, ajout dans le plus grand intervalle admissible, suppression avec plancher de deux guides, ordre de routage préservé et parcours QTest complet. **Troisième incrément livré** : chaque section persiste son index, le nombre de sections et ses identifiants de jonction déterministes, avec lecture des anciens `.osp` inchangée. **Quatrième incrément livré** : le routage maximise les transitions par jonction admissible avant de minimiser la distance et ignore une jonction géométriquement incohérente. **Cinquième incrément livré** : les guides terminaux porteurs d'une jonction sont détectés par stations projetées et verrouillés dans le cœur et l'UI afin qu'une édition locale ne raccourcisse pas la section. **Sixième incrément livré** : découverte déterministe et stricte de toutes les extrémités partageant une jonction, plus commande multi-objet tout-ou-rien avec undo/redo unique. **Septième incrément livré** : sélectionner un guide structurel puis ajouter un guide créé, en une transaction, un guide interne dans l'intervalle directement adjacent de chaque section incidente ; si une seule section est invalide, rien n'est modifié. Restent la propagation géométrique coordonnée des angles et déplacements, puis les retours textiles suivant le réseau plutôt qu'un segment direct. Ne jamais accepter silencieusement une gerbe ou un croisement comme satin valide.

Refonte avancée du **satin** (§12 de l'étude) : correspondance par sections, barreaux de direction, densité perpendiculaire au fil, gestion des points courts dans les virages, split stitch pour les colonnes larges, terminaisons.

- Refonte avancée du **tatami** (§15) : underpath **caché** au lieu de sauts, points d'entrée/sortie imposés, sous-couches, motifs de phase.
- Séparer la **normalisation machine** de l'encodeur DST (préparer PES/JEF/EXP).

Moyen terme (fonctionnalités)

- **Palette de fils** (`thread_palette`) : base fabricants, distance perceptuelle, association objet↔bobine, import/export.
- **Édition manuelle des points** (déplacer, convertir en saut, ajouter une coupe) avec états `Clean/Dirty/ManuallyEdited`.
- **Sous-couches** paramétrables (contour, zigzag) pour satin et tatami.
- **Compensation** directionnelle et profils tissu/stabilisateur/fil.
- **Édition de nœuds** complète (ajout/suppression, tangentes, anguleux/lisse).

Long terme (avancé)

- **Auto-satin** : poursuivre au-delà du réseau de sections maintenant branché dans l'auto-numérisation (jonctions textiles avancées, réseaux à plusieurs trous, validation machine, guides d'orientation par section). Les formes refusées retombent sur le tatami ; le satin naïf reste désactivé.

- **Remplissages avancés** : concentrique, spirale, radial, guidé (champ de direction), motifs.
- **Autres formats** : PES, JEF, EXP, VP3.
- **Tâches asynchrones** (annulation, progression) et **carte de densité**.
- **Profils machine/cadres**, sauvegarde automatique, migrations de projet.
- Portage **macOS**, distribution de binaires, publication publique.

Sources

- docs/phase0/08-roadmap-adr.md — roadmap d'origine (13 phases).
- docs/stitch-engine-audit.md — défauts et priorités du moteur de points.

Implémentation associée

Les fonctionnalités « Non implémenté » du chapitre *Limitations* constituent la liste de travail. Chacune indique le module cible.

Glossaire

Terme français, terme anglais utilisé dans le code, définition, et type/classe associé lorsqu'il existe.

Français	Anglais (code)	Définition	Type / fichier
Point	stitch	Pénétration d'aiguille cousant le fil	<code>CommandType::Stitch</code>
Saut	jump	Déplacement de l'aiguille sans couture	<code>CommandType::Jump</code>
Coupe-fil	trim	Coupe du fil	<code>CommandType::Trim</code>
Changement de couleur	color change	Arrêt pour changer de fil	<code>CommandType::ColorChange</code>
Arrêt	stop	Arrêt machine	<code>CommandType::Stop</code>
Fin	end	Fin du motif	<code>CommandType::End</code>
Point droit / courant	running stitch	Points le long d'un chemin	<code>run_stitch</code>
Point triple	bean stitch	Chaque segment cousu 3x	<code>RepeatMode::BeanStitch</code>
Point arrière	backstitch	Progression avec recouvrement	<code>RepeatMode::Backstitch</code>
Remplissage	fill	Couture d'une surface	<code>fill_tatami</code>
Tatami	tatami	Remplissage par rangées parallèles	<code>TatamiParams</code>
Colonne satin	satin column	Zigzag entre deux rails	<code>SatinParams, fill_satin</code>
Rail	rail	Bord d'une colonne satin	<code>SatinParams::rail_a/b</code>
Sous-couche	underlay	Couche cousue avant la couche visible	<code>center_underlay</code>
Compensation de tirage	pull compensation	Élargissement pour compenser la traction	<code>pull_compensation</code>
Densité	density / row_spacing	Espacement des fils/rangées	<code>density, row_spacing</code>
Longueur de point	stitch length	Longueur d'un point	<code>stitch_length</code>
Angle	angle	Orientation du remplissage	<code>Angle</code>
Contour	contour / outer	Bord d'une région	<code>PathSet::outer</code>
Trou	hole	Contour intérieur	<code>PathSet::holes</code>
Chemin	path	Polyligne/courbe	<code>geometry::Path</code>
Région	region	Zone de couleur connexe	<code>segmentation::Region</code>
Objet vectoriel	vector object	Contours éditables	<code>document::VectorObject</code>
Objet de broderie	embroidery object	Intention de couture	<code>document::EmbroideryObject</code>
Séquence de points	stitch sequence	Commandes machine	<code>stitch::StitchSequence</code>
Cadre / tambour	hoop / canvas	Zone de broderie	<code>document::Canvas</code>
Palette	palette	Ensemble de couleurs/fils	<i>(prévu)</i>
Fil	thread	Fil de broderie	<i>(prévu)</i>
Ordre de couture	sewing order	Ordre des objets	<code>optimization, SewingOrder</code>
Micromètre	micrometer	Unité interne (1/1000 mm)	<code>Micrometers</code>
Déplacement (interne)	travel	Liaison non cousue d'un remplissage	<code>FillStitch::travel</code>

Licences

Public : mainteneur, contributeur.

Licence du logiciel

OpenStitch Studio est distribué sous **Apache-2.0** (permissive, avec clause de brevets). Fichier : `LICENSE`.
Chaque fichier source porte // `SPDX-License-Identifier: Apache-2.0`.

Conséquence importante, formulée avec prudence : intégrer directement du code sous licence **copyleft** (comme la GPL) dans ce projet imposerait vraisemblablement de distribuer l'ensemble dérivé sous une licence compatible avec cette licence copyleft, ce qui empêcherait probablement de conserver le tout **uniquement** sous Apache-2.0. La compatibilité exacte dépend de la version de la licence et de la manière dont le code est combiné.

En pratique, pour **conserver la distribution du projet sous Apache-2.0**, aucun code sous licence copyleft incompatible avec cet objectif ne doit être intégré directement. Les **idées algorithmiques** peuvent être réimplémentées indépendamment à partir de sources publiques, sans reprise de code ni de structure expressive protégée. **Toute réutilisation de code tiers doit faire l'objet d'une vérification de licence spécifique.** Voir `docs/stitch-engine-research.md`.

Dépendances externes

Bibliothèque	Version	Rôle	Licence	Intégration	URL
fmt	via vcpkg	formatage	MIT	vcpkg	github.com/fmtlib/fmt
spdlog	via vcpkg	journalisation	MIT	vcpkg	github.com/gabime/spdlog
CLI11	via vcpkg	args CLI	BSD-3-Clause	vcpkg	github.com/CLIUtils/CLI11
OpenCV (core/imgproc/imgcodecs)	via vcpkg	image	Apache-2.0	vcpkg	opencv.org
libpng / libjpeg-turbo / libtiff / zlib	transitives	codecs	zlib / IJG / libtiff / zlib	via OpenCV	—
Clipper2	via vcpkg	booléens/offsets	BSL-1.0	vcpkg (encapsulée)	github.com/AngusJohnson/Clipper2
nlohmann/json	via vcpkg	JSON projet	MIT	vcpkg (encapsulée)	github.com/nlohmann/json
minizip-ng	via vcpkg	ZIP projet	zlib	vcpkg (encapsulée)	github.com/zlib-ng/minizip-ng
Catch2 v3	via vcpkg	tests	BSL-1.0	vcpkg (dev)	github.com/catchorg/Catch2
Qt 6.8 LTS (Widgets/Gui/Core)	binaires officiels	interface	LGPL-3.0	liaison dynamique	qt.io

Les versions exactes sont verrouillées par la **baseline vcpkg** (`vcpkg.json`).

Qt et la LGPL-3.0

Qt est utilisé en **liaison dynamique** avec les DLL officielles non modifiées : elles sont remplaçables par l'utilisateur, aucune source Qt n'est modifiée, et seuls des modules LGPL sont utilisés (pas de module GPL-only ni commercial). Ce mode est compatible avec la distribution d'un logiciel Apache-2.0.

Licences de la chaîne documentaire

La documentation est générée avec `python-markdown` (BSD), `xhtml2pdf` (Apache-2.0) et `reportlab` (BSD), `svglib` (LGPL, utilisée comme outil externe, non liée au produit). Ces outils ne sont pas distribués avec le logiciel.

Implémentation associée

- `LICENSE, THIRD_PARTY_LICENSES.md`.
- `docs/phase0/03-bibliotheques-licences.md` (analyse d'origine, ADR-002).
- `docs/stitch-engine-research.md` (point juridique Ink/Stitch).

Dépendances externes

Depuis `vcpkg.json` et `apps/desktop/CMakeLists.txt` : `fmt`, `spdlog`, `cli11`, `clipper2`, `nlohmann-json`, `minizip-ng`, `opencv4` (features core, png, jpeg, tiff), `catch2` (tests) ; **Qt 6.8** hors `vcpkg` (binaires officiels, variable `QT_ROOT`). Voir le chapitre *Licences*.

Principaux espaces de noms et types

- Espace racine `openstitch` ; sous-espaces `geometry`, `image`, `segmentation`, `vectorization`, `document`, `stitch`, `stitch_generation`, `stitch_analysis`, `optimization`, `autodigitize`, `commands`, `formats`, `project_io`, `desktop`.
- Unités fortes : `Micrometers`, `Millimeters`, `Pixels`, `Angle`, `Vec2um` (`libs/core/include/openstitch/core/units.hpp`).
- Identifiants : `ObjectId`, `RegionId`, `ColorId`, `ThreadId`, `IdGenerator<T>` (`libs/core/include/openstitch/core/ids.hpp`).
- Erreurs : `Result<T> = std::expected<T, Error>` (`.../core/error.hpp`).
- Document : `Project`, `VectorObject`, `EmbroideryObject`, `StitchParams` (variant `RunningStitchParams` | `TatamiParams` | `SatinParams`), `Canvas` (`libs/document/include/openstitch/document/`).
- Points : `StitchCommand`, `CommandType`, `StitchSequence`, `StitchStats` (`libs/stitch/include/openstitch/stitch/sequence.hpp`).

Formats pris en charge

Format	Import	Export	Module
PNG, JPEG, BMP, TIFF	Oui	— (export projet uniquement)	<code>image</code> (OpenCV)
DST (Tajima)	Oui	Oui	<code>formats/src/dst.cpp</code>
SVG (diagnostic)	—	Oui	<code>formats/src/svg.cpp</code>
<code>.osp</code> (projet)	Oui	Oui	<code>project_io</code>

Interface graphique réellement implémentée

Menus construits dans `apps/desktop/main_window.cpp` (`buildMenus`, `buildAnalysisPanel`, `buildSimulationToolbar`, `buildOrderPanel`) : Fichier, Édition, Image, Segmentation, Broderie, Affichage, Analyse, plus un panneau d'ordre de couture (dock), un panneau d'analyse (dock) et une barre de simulation. Le détail est donné dans le *Guide utilisateur détaillé*.

Types d'objets de broderie

`RunningStitchParams` (point droit/double/triple), `TatamiParams` (remplissage), `SatinParams` (colonne satin), portés par le variant `StitchParams` (`libs/document/include/openstitch/document/embroidery_object.hpp`).

Algorithmes présents (vérifiés dans le code)

Conversion RGB↔CIELAB et k-means (segmentation), composantes connexes 4-conn (segmentation), extraction de contours Suzuki-Abe (vectorization), simplification Douglas-Peucker (geometry/simplify.cpp), union/offset via Clipper2 (geometry/clean.cpp, geometry/offset.cpp), aplatissement Bézier adaptatif + longueur d'arc (geometry/polyline.cpp), running stitch par longueur d'arc avec découpe aux coins (stitch_generation/running_stitch.cpp), tatami scanline + routage par graphe (stitch_generation/tatami.cpp), satin à deux rails (stitch_generation/satin.cpp), règles d'analyse (stitch_analysis/analyze.cpp), ordre de couture (optimization/order.cpp), codec DST (formats/dst.cpp).

Tests présents

25 fichiers test_*.cpp sous tests/unit/<lib>/ et un test d'intégration tests/integration/test_pipeline.cpp. Total : **161 tests** au dernier passage (ctest). Framework : Catch2 v3. Voir le chapitre Tests.

Exemples et ressources

- SVG de diagnostic de référence : tests/golden/stitch-generation/ (formes de running stitch et remplissage anneau).
- Aucun jeu d'images d'exemple versionné à la racine examples/ à ce jour (*Information non déterminée dans le dépôt* : dossier examples/ non peuplé).

Fonctionnalités expérimentales / partielles / prévues

- **Prévu (architecture mentionnée, non implémenté)** : palette de fils réelle (thread_palette), profils de machine/cadres avancés, remplissages courbes / radiaux / spirale / motifs, sous-couches tatami, édition manuelle des points.
- **Partiel** : classification auto des régions (heuristique de largeur) — essaie désormais le vrai moteur auto_satin par squelette puis retombe sur le **tatami** ; le satin **automatique** naïf reste désactivé par défaut (use_naive_satin). Rails, barreaux et décomposition multi-sections sont présents. Compensation (satin uniquement) ; routage tatami (graphe + validation géométrique, underpath caché non implémenté — les liaisons hors-région sont des sauts).
- **Expérimental** : openstitch-cli stitchdebug / auto-satin-debug (outils de diagnostic du moteur).

Marqueurs, incohérences

- Aucun FIXME critique repéré ; le README.md mentionne un compte de tests d'un jalon antérieur alors que le compte réel est **161** — incohérence mineure, à resynchroniser dans le README.
- Documents de conception : docs/phase0/ (étude de cadrage, 14 ADR), docs/stitch-engine-audit.md et docs/stitch-engine-research.md (refonte du moteur de points).

Éléments non déterminés dans le dépôt

- URL de dépôt public : aucune (dépôt local uniquement).
- Fichier de logo : aucun logo bitmap/vectoriel dédié trouvé ; la couverture du PDF n'en utilise donc pas.
- Couverture de code chiffrée : non configurée.